

```

window.__MANIFEST__ = {"files": {"city": ["city.wasm"], "shas": ["shas.p0.wasm"],
"use strict";
const __M = {};
__M['math'] = (() => {
// Minimal 3D math library (column-major matrices, like WebGL)
class V3 {
  constructor(x = 0, y = 0, z = 0) { this.x = x; this.y = y; this.z = z; }
  set(x, y, z) { this.x = x; this.y = y; this.z = z; return this; }
  copy(v) { this.x = v.x; this.y = v.y; this.z = v.z; return this; }
  clone() { return new V3(this.x, this.y, this.z); }
  add(v) { this.x += v.x; this.y += v.y; this.z += v.z; return this; }
  sub(v) { this.x -= v.x; this.y -= v.y; this.z -= v.z; return this; }
  addS(v, s) { this.x += v.x * s; this.y += v.y * s; this.z += v.z * s; return this; }
  scale(s) { this.x *= s; this.y *= s; this.z *= s; return this; }
  dot(v) { return this.x * v.x + this.y * v.y + this.z * v.z; }
  cross(v) { const x = this.y * v.z - this.z * v.y, y = this.z * v.x - this.x * v.z, z = this.x * v.y - this.y * v.x; return new V3(x, y, z); }
  crossVecs(a, b) { const x = a.y * b.z - a.z * b.y, y = a.z * b.x - a.x * b.z, z = a.x * b.y - a.y * b.x; return new V3(x, y, z); }
  subVecs(a, b) { this.x = a.x - b.x; this.y = a.y - b.y; this.z = a.z - b.z; return this; }
  addVecs(a, b) { this.x = a.x + b.x; this.y = a.y + b.y; this.z = a.z + b.z; return this; }
  len() { return Math.hypot(this.x, this.y, this.z); }
  lenSq() { return this.x * this.x + this.y * this.y + this.z * this.z; }
  norm() { const l = this.len(); if (l > 1e-12) this.scale(1 / l); return this; }
  lerp(v, t) { this.x += (v.x - this.x) * t; this.y += (v.y - this.y) * t; this.z += (v.z - this.z) * t; return this; }
  dist(v) { return Math.hypot(this.x - v.x, this.y - v.y, this.z - v.z); }
  applyQ(q) { // rotate by quaternion
    const x = this.x, y = this.y, z = this.z, qx = q.x, qy = q.y, qz = q.z, qw = q.w;
    const ix = qw * x + qy * z - qz * y, iy = qw * y + qz * x - qx * z, iz = qw * z + qx * y - qy * x;
    this.x = ix * qw + qy * -qx + qz * -qy - iz * -qz;
    this.y = iy * qw + qz * -qx + qx * -qy - ix * -qz;
    this.z = iz * qw + qx * -qy + qy * -qx - iz * -qx;
    return this;
  }
  applyInvQ(q) { _q.set(-q.x, -q.y, -q.z, q.w); return this.applyQ(_q); }
  applyM4(m) {
    const x = this.x, y = this.y, z = this.z;
    this.x = m[0] * x + m[4] * y + m[8] * z + m[12];
    this.y = m[1] * x + m[5] * y + m[9] * z + m[13];
    this.z = m[2] * x + m[6] * y + m[10] * z + m[14];
    return this;
  }
  applyM4Dir(m) {
    const x = this.x, y = this.y, z = this.z;

```

```

    this.x = m[0] * x + m[4] * y + m[8] * z;
    this.y = m[1] * x + m[5] * y + m[9] * z;
    this.z = m[2] * x + m[6] * y + m[10] * z;
    return this;
}
}

class Q {
    constructor(x = 0, y = 0, z = 0, w = 1) { this.x = x; this.y = y; this.z = z; t
    set(x, y, z, w) { this.x = x; this.y = y; this.z = z; this.w = w; return this;
    copy(q) { this.x = q.x; this.y = q.y; this.z = q.z; this.w = q.w; return this;
    clone() { return new Q(this.x, this.y, this.z, this.w); }
    identity() { return this.set(0, 0, 0, 1); }
    setAxisAngle(a, ang) { const s = Math.sin(ang / 2); this.x = a.x * s; this.y =
    setEuler(x, y, z) { // order YXZ (yaw, pitch, roll)
        const c1 = Math.cos(x / 2), c2 = Math.cos(y / 2), c3 = Math.cos(z / 2), s1 =
        this.x = s1 * c2 * c3 + c1 * s2 * s3; this.y = c1 * s2 * c3 - s1 * c2 * s3; t
        return this;
    }
    mul(b) { return this.mulQs(this, b); }
    premul(a) { return this.mulQs(a, this); }
    mulQs(a, b) {
        const ax = a.x, ay = a.y, az = a.z, aw = a.w, bx = b.x, by = b.y, bz = b.z, b
        this.x = ax * bw + aw * bx + ay * bz - az * by;
        this.y = ay * bw + aw * by + az * bx - ax * bz;
        this.z = az * bw + aw * bz + ax * by - ay * bx;
        this.w = aw * bw - ax * bx - ay * by - az * bz;
        return this;
    }
    conj() { this.x = -this.x; this.y = -this.y; this.z = -this.z; return this; }
    norm() { const l = Math.hypot(this.x, this.y, this.z, this.w) || 1; this.x /= l
    slerp(qb, t) {
        if (t === 0) return this; if (t === 1) return this.copy(qb);
        const x = this.x, y = this.y, z = this.z, w = this.w;
        let cos = w * qb.w + x * qb.x + y * qb.y + z * qb.z;
        let bx = qb.x, by = qb.y, bz = qb.z, bw = qb.w;
        if (cos < 0) { cos = -cos; bx = -bx; by = -by; bz = -bz; bw = -bw; }
        if (cos > 0.9995) { this.x = x + (bx - x) * t; this.y = y + (by - y) * t; thi
        const th = Math.acos(cos), s = Math.sin(th), ra = Math.sin((1 - t) * th) / s,
        this.x = x * ra + bx * rb; this.y = y * ra + by * rb; this.z = z * ra + bz *
        return this;
    }
    // integrate angular velocity w over dt
    integrate(w, dt) {
        const hx = w.x * dt * 0.5, hy = w.y * dt * 0.5, hz = w.z * dt * 0.5;
        const x = this.x, y = this.y, z = this.z, qw = this.w;

```

```

    this.x += hx * qw + hy * z - hz * y;
    this.y += hy * qw + hz * x - hx * z;
    this.z += hz * qw + hx * y - hy * x;
    this.w += -hx * x - hy * y - hz * z;
    return this.norm();
}
setFromUnitVectors(a, b) {
    let r = a.dot(b) + 1;
    if (r < 1e-6) { r = 0; if (Math.abs(a.x) > Math.abs(a.z)) this.set(-a.y, a.x,
    else this.set(a.y * b.z - a.z * b.y, a.z * b.x - a.x * b.z, a.x * b.y - a.y *
    return this.norm();
    }
}
const _q = new Q();

const M4 = {
    create() { const m = new Float32Array(16); m[0] = m[5] = m[10] = m[15] = 1; ret
    identity(m) { m.fill(0); m[0] = m[5] = m[10] = m[15] = 1; return m; },
    copy(o, m) { o.set(m); return o; },
    compose(o, p, q, s) {
        const x = q.x, y = q.y, z = q.z, w = q.w, x2 = x + x, y2 = y + y, z2 = z + z;
        const xx = x * x2, xy = x * y2, xz = x * z2, yy = y * y2, yz = y * z2, zz = z
        const sx = s ? s.x : 1, sy = s ? s.y : 1, sz = s ? s.z : 1;
        o[0] = (1 - (yy + zz)) * sx; o[1] = (xy + wz) * sx; o[2] = (xz - wy) * sx; o[
        o[4] = (xy - wz) * sy; o[5] = (1 - (xx + zz)) * sy; o[6] = (yz + wx) * sy; o[
        o[8] = (xz + wy) * sz; o[9] = (yz - wx) * sz; o[10] = (1 - (xx + yy)) * sz; o
        o[12] = p.x; o[13] = p.y; o[14] = p.z; o[15] = 1;
        return o;
    },
    decompose(m, p, q, s) {
        let sx = Math.hypot(m[0], m[1], m[2]); const sy = Math.hypot(m[4], m[5], m[6]
        const det = M4.det3(m); if (det < 0) sx = -sx;
        p.set(m[12], m[13], m[14]); if (s) s.set(sx, sy, sz);
        const m11 = m[0] / sx, m12 = m[4] / sy, m13 = m[8] / sz, m21 = m[1] / sx, m22
        const tr = m11 + m22 + m33;
        if (tr > 0) { const s2 = 0.5 / Math.sqrt(tr + 1); q.set((m32 - m23) * s2, (m1
        else if (m11 > m22 && m11 > m33) { const s2 = 2 * Math.sqrt(1 + m11 - m22 - m
        else if (m22 > m33) { const s2 = 2 * Math.sqrt(1 + m22 - m11 - m33); q.set((m
        else { const s2 = 2 * Math.sqrt(1 + m33 - m11 - m22); q.set((m13 + m31) / s2,
        return m;
    },
    det3(m) { return m[0] * (m[5] * m[10] - m[9] * m[6]) - m[4] * (m[1] * m[10] - m
    mul(o, a, b) {
        const a00 = a[0], a01 = a[1], a02 = a[2], a03 = a[3], a10 = a[4], a11 = a[5],
        a20 = a[8], a21 = a[9], a22 = a[10], a23 = a[11], a30 = a[12], a31 = a[13],
        for (let i = 0; i < 4; i++) {

```

```

    const b0 = b[i * 4], b1 = b[i * 4 + 1], b2 = b[i * 4 + 2], b3 = b[i * 4 + 3]
    o[i * 4] = b0 * a00 + b1 * a10 + b2 * a20 + b3 * a30;
    o[i * 4 + 1] = b0 * a01 + b1 * a11 + b2 * a21 + b3 * a31;
    o[i * 4 + 2] = b0 * a02 + b1 * a12 + b2 * a22 + b3 * a32;
    o[i * 4 + 3] = b0 * a03 + b1 * a13 + b2 * a23 + b3 * a33;
  }
  return o;
},
invert(o, a) {
  const a00 = a[0], a01 = a[1], a02 = a[2], a03 = a[3], a10 = a[4], a11 = a[5],
    a20 = a[8], a21 = a[9], a22 = a[10], a23 = a[11], a30 = a[12], a31 = a[13],
  const b00 = a00 * a11 - a01 * a10, b01 = a00 * a12 - a02 * a10, b02 = a00 * a
    b04 = a01 * a13 - a03 * a11, b05 = a02 * a13 - a03 * a12, b06 = a20 * a31 -
    b08 = a20 * a33 - a23 * a30, b09 = a21 * a32 - a22 * a31, b10 = a21 * a33 -
  let det = b00 * b11 - b01 * b10 + b02 * b09 + b03 * b08 - b04 * b07 + b05 * b
  if (!det) return M4.identity(o);
  det = 1 / det;
  o[0] = (a11 * b11 - a12 * b10 + a13 * b09) * det; o[1] = (a02 * b10 - a01 * b
  o[2] = (a31 * b05 - a32 * b04 + a33 * b03) * det; o[3] = (a22 * b04 - a21 * b
  o[4] = (a12 * b08 - a10 * b11 - a13 * b07) * det; o[5] = (a00 * b11 - a02 * b
  o[6] = (a32 * b02 - a30 * b05 - a33 * b01) * det; o[7] = (a20 * b05 - a22 * b
  o[8] = (a10 * b10 - a11 * b08 + a13 * b06) * det; o[9] = (a01 * b08 - a00 * b
  o[10] = (a30 * b04 - a31 * b02 + a33 * b00) * det; o[11] = (a21 * b02 - a20 *
  o[12] = (a11 * b07 - a10 * b09 - a12 * b06) * det; o[13] = (a00 * b09 - a01 *
  o[14] = (a31 * b01 - a30 * b03 - a32 * b00) * det; o[15] = (a20 * b03 - a21 *
  return o;
},
perspective(o, fovy, aspect, near, far) {
  const f = 1 / Math.tan(fovy / 2), nf = 1 / (near - far);
  o.fill(0); o[0] = f / aspect; o[5] = f; o[10] = (far + near) * nf; o[11] = -1
  return o;
},
ortho(o, l, r, b, t, n, f) {
  o.fill(0); o[0] = 2 / (r - l); o[5] = 2 / (t - b); o[10] = -2 / (f - n);
  o[12] = -(r + l) / (r - l); o[13] = -(t + b) / (t - b); o[14] = -(f + n) / (f
  return o;
},
lookAt(o, eye, target, up) {
  let zx = eye.x - target.x, zy = eye.y - target.y, zz = eye.z - target.z;
  let l = Math.hypot(zx, zy, zz) || 1; zx /= l; zy /= l; zz /= l;
  let xx = up.y * zz - up.z * zy, xy = up.z * zx - up.x * zz, xz = up.x * zy -
  l = Math.hypot(xx, xy, xz) || 1; xx /= l; xy /= l; xz /= l;
  const yx = zy * xz - zz * xy, yy = zz * xx - zx * xz, yz = zx * xy - zy * xx;
  o[0] = xx; o[1] = yx; o[2] = zx; o[3] = 0; o[4] = xy; o[5] = yy; o[6] = zy; o
  o[12] = -(xx * eye.x + xy * eye.y + xz * eye.z); o[13] = -(yx * eye.x + yy *
  return o;
}

```

```

    },
    normalMat3(o9, m) { // inverse-transpose upper 3x3
        const a00 = m[0], a01 = m[1], a02 = m[2], a10 = m[4], a11 = m[5], a12 = m[6],
        const b01 = a22 * a11 - a12 * a21, b11 = -a22 * a10 + a12 * a20, b21 = a21 *
        let det = a00 * b01 + a01 * b11 + a02 * b21; det = det ? 1 / det : 0;
        o9[0] = b01 * det; o9[3] = (-a22 * a01 + a02 * a21) * det; o9[6] = (a12 * a01
        o9[1] = b11 * det; o9[4] = (a22 * a00 - a02 * a20) * det; o9[7] = (-a12 * a00
        o9[2] = b21 * det; o9[5] = (-a21 * a00 + a01 * a20) * det; o9[8] = (a11 * a00
        return o9;
    },
};

const clamp = (v, a, b) => v < a ? a : v > b ? b : v;
const lerp = (a, b, t) => a + (b - a) * t;
const smooth = (cur, target, rate, dt) => cur + (target - cur) * (1 - Math.exp(-r
const damp = smooth;
function angleWrap(a) { while (a > Math.PI) a -= 2 * Math.PI; while (a < -Math.PI

return { V3, Q, M4, clamp, lerp, smooth, damp, angleWrap };
})();
__M['scene'] = (() => {
const { V3, Q, M4 } = __M['math'];

let _id = 0;
class Node {
    constructor(name = '') {
        this.id = ++_id;
        this.name = name;
        this.pos = new V3(); this.rot = new Q(); this.scl = new V3(1, 1, 1);
        this.local = M4.create(); this.world = M4.create();
        this.children = []; this.parent = null;
        this.mesh = null; // {prims:[Prim]}
        this.skin = null; // Skin
        this.visible = true;
        this.castShadow = true; this.receiveShadow = true;
        this.matrixAuto = true;
        this.userData = {};
    }
    add(c) { if (c.parent) c.parent.remove(c); c.parent = this; this.children.push(
    remove(c) { const i = this.children.indexOf(c); if (i >= 0) this.children.splice
    updateWorld(parentWorld) {
        if (this.matrixAuto) M4.compose(this.local, this.pos, this.rot, this.scl);
        if (parentWorld) M4.mul(this.world, parentWorld, this.local); else this.world
        for (const c of this.children) c.updateWorld(this.world);
    }
    traverse(fn) { fn(this); for (const c of this.children) c.traverse(fn); }

```

```

find(name) { let r = null; this.traverse(n => { if (!r && n.name === name) r =
findAll(pred) { const r = []; this.traverse(n => { if (pred(n)) r.push(n); });
worldPos(out = new V3()) { return out.set(this.world[12], this.world[13], this.
clone(deep = true) {
    const n = new Node(this.name);
    n.pos.copy(this.pos); n.rot.copy(this.rot); n.scl.copy(this.scl); n.local.set
    n.mesh = this.mesh; n.visible = this.visible; n.castShadow = this.castShadow;
    n.matrixAuto = this.matrixAuto; n.userData = { ...this.userData };
    n._src = this;
    if (deep) for (const c of this.children) n.add(c.clone(true));
    return n;
}
}

class Skin {
    constructor(joints, ibms) {
        this.joints = joints; // Node[]
        this.ibm = ibms; // Float32Array(16*n)
        this.data = new Float32Array(joints.length * 16);
        this.tex = null; this.dirty = true;
    }
    update() {
        const tmp = Skin._tmp || (Skin._tmp = M4.create());
        const ib = Skin._ib || (Skin._ib = M4.create());
        for (let i = 0; i < this.joints.length; i++) {
            ib.set(this.ibm.subarray(i * 16, i * 16 + 16));
            M4.mul(tmp, this.joints[i].world, ib);
            this.data.set(tmp, i * 16);
        }
        this.dirty = true;
    }
}

class Material {
    constructor(o = {}) {
        this.name = o.name || '';
        this.color = o.color || [1, 1, 1, 1];
        this.map = o.map || null;
        this.roughness = o.roughness ?? 0.8;
        this.metalness = o.metalness ?? 0;
        this.emissive = o.emissive || [0, 0, 0];
        this.alphaMode = o.alphaMode || 'OPAQUE'; // OPAQUE | MASK | BLEND
        this.alphaCutoff = o.alphaCutoff ?? 0.5;
        this.doubleSided = !!o.doubleSided;
        this.unlit = !!o.unlit;
        this.backColor = o.backColor || null; // color used for back faces (car door

```

```

        this.clearcoat = o.clearcoat ?? 0;        // extra glossy layer (car paint)
        this.envBoost = o.envBoost ?? 1;
        this.depthWrite = o.depthWrite ?? true;
        this.vertexColors = !!o.vertexColors;
        this.polygonOffset = o.polygonOffset || 0;
        this.uvScale = o.uvScale || null;
    }
    clone() { const m = new Material(this); m.name = this.name; return m; }
}

return { Node, Skin, Material };
})();
__M['shaders'] = (() => {
// GLSL sources (WebGL2 / GLSL ES 3.00)

const COMMON = /* glsl */`
uniform vec3 uSunDir;        // direction TO the sun
uniform vec3 uSunColor;
uniform vec3 uSkyTop;
uniform vec3 uSkyHorizon;
uniform vec3 uGroundColor;
uniform float uExposure;
uniform float uFogDensity;
uniform float uTime;
uniform float uNight;
// dynamic lights (car headlights / tail lights): pos.xyz + range, dir.xyz + cos(
uniform vec4 uLPos[8];
uniform vec4 uLDir[8];
uniform vec3 uLCol[8];
uniform int uLNum;

vec3 skyColor(vec3 d) {
    float y = d.y;
    float t = pow(clamp(y, 0.0, 1.0), 0.45);
    vec3 c = mix(uSkyHorizon, uSkyTop, t);
    // below horizon: dusty ground haze
    c = mix(c, uSkyHorizon * 0.85 + uGroundColor * 0.15, smoothstep(0.0, -0.25, y))
    float sd = max(dot(d, uSunDir), 0.0);
    c += uSunColor * (0.08 * pow(sd, 8.0) + 0.25 * pow(sd, 64.0));
    return c;
}

vec3 aces(vec3 x) {
    const float a = 2.51, b = 0.03, c = 2.43, d = 0.59, e = 0.14;
    return clamp((x * (a * x + b)) / (x * (c * x + d) + e), 0.0, 1.0);
}

```

```

vec3 finalColor(vec3 c) {
    c = aces(c * uExposure);
    return pow(c, vec3(1.0 / 2.2));
}

vec3 applyFog(vec3 col, vec3 wpos, vec3 camPos) {
    vec3 d = wpos - camPos; float dist = length(d); vec3 dir = d / max(dist, 1e-4);
    float f = 1.0 - exp(-dist * uFogDensity);
    f *= f;
    vec3 fc = skyColor(normalize(vec3(dir.x, max(dir.y, 0.02), dir.z)));
    return mix(col, fc, clamp(f, 0.0, 1.0));
}

`;

const LIT_VS = /* glsl */`#version 300 es
precision highp float;
layout(location=0) in vec3 aPos;
layout(location=1) in vec3 aNrm;
layout(location=2) in vec2 aUv;
#ifdef SKIN
layout(location=3) in vec4 aJoints;
layout(location=4) in vec4 aWeights;
uniform highp sampler2D uBones;
mat4 boneMat(float fi) {
    int i = int(fi) * 4;
    return mat4(texelFetch(uBones, ivec2(i, 0), 0), texelFetch(uBones, ivec2(i + 1,
        texelFetch(uBones, ivec2(i + 2, 0), 0), texelFetch(uBones, ivec2(i
    }
#endif
uniform mat4 uModel;
uniform mat4 uVP;
uniform mat3 uNrmMat;
uniform mat4 uShadowVP;
uniform vec2 uUvScale;
out vec3 vWPos;
out vec3 vNrm;
out vec2 vUv;
out vec4 vShadow;
void main() {
    vec4 p = vec4(aPos, 1.0);
    vec3 n = aNrm;
#ifdef SKIN
    mat4 sk = boneMat(aJoints.x) * aWeights.x + boneMat(aJoints.y) * aWeights.y + b
    vec4 wp = sk * p;
    vNrm = normalize(mat3(sk) * n);
#else
    vec4 wp = uModel * p;

```

```

    vNrm = normalize(uNrmMat * n);
#endif
    vWPos = wp.xyz;
    vUv = aUv * uUvScale;
    vShadow = uShadowVP * wp;
    gl_Position = uVP * wp;
}
`;

```

```

const LIT_FS = /* glsl */`#version 300 es
precision highp float;
precision highp sampler2DShadow;
in vec3 vWPos;
in vec3 vNrm;
in vec2 vUv;
in vec4 vShadow;
uniform vec4 uColor;
uniform sampler2D uMap;
uniform sampler2DShadow uShadowMap;
uniform float uShadowOn;
uniform float uShadowTexel;
uniform float uRough;
uniform float uMetal;
uniform float uClearcoat;
uniform float uEnvBoost;
uniform vec3 uEmissive;
uniform vec3 uCamPos;
uniform vec4 uBackColor; // a>0 => use for back faces
uniform float uAlphaCut;
uniform float uUnlit;
uniform float uDamage; // darkening for dents (not used per-vertex yet)
out vec4 fragColor;
${COMMON}

float shadowAt(vec3 wp, vec3 n) {
    if (uShadowOn < 0.5) return 1.0;
    vec3 s = vShadow.xyz / vShadow.w * 0.5 + 0.5;
    if (s.x < 0.0 || s.x > 1.0 || s.y < 0.0 || s.y > 1.0 || s.z > 1.0) return 1.0;
    float bias = 0.0006;
    float sum = 0.0;
    float t = uShadowTexel;
    sum += texture(uShadowMap, vec3(s.xy + vec2(-t, -t) * 0.75, s.z - bias));
    sum += texture(uShadowMap, vec3(s.xy + vec2( t, -t) * 0.75, s.z - bias));
    sum += texture(uShadowMap, vec3(s.xy + vec2(-t,  t) * 0.75, s.z - bias));
    sum += texture(uShadowMap, vec3(s.xy + vec2( t,  t) * 0.75, s.z - bias));
    sum *= 0.25;
}

```

```

    // fade out at the edges of the shadow area
    vec2 e = smoothstep(vec2(0.0), vec2(0.08), s.xy) * smoothstep(vec2(0.0), vec2(0
    return mix(1.0, sum, e.x * e.y);
}

float D_GGX(float nh, float a) { float a2 = a * a; float d = nh * nh * (a2 - 1.0)

void main() {
    vec4 base = uColor;
#ifdef MAP
    base *= texture(uMap, vUv);
#endif
#ifdef ALPHATEST
    if (base.a < uAlphaCut) discard;
#endif
    vec3 n = normalize(vNrm);
    bool back = !gl_FrontFacing;
    if (back) n = -n;
    if (back && uBackColor.a > 0.0) base.rgb = uBackColor.rgb;
    if (uUnlit > 0.5) { fragColor = vec4(finalColor(base.rgb), base.a); return; }

    vec3 v = normalize(uCamPos - vWPos);
    vec3 l = uSunDir;
    float nl = max(dot(n, l), 0.0);
    float nv = max(dot(n, v), 1e-3);
    vec3 h = normalize(l + v);
    float nh = max(dot(n, h), 0.0);
    float sh = shadowAt(vWPos, n);

#ifdef DEBUGSHADOW
    { vec3 sd = vShadow.xyz / vShadow.w * 0.5 + 0.5; fragColor = vec4(sh, fract(sd.
#endif
    vec3 albedo = base.rgb;
    float rough = clamp(uRough, 0.04, 1.0);
    vec3 F0 = mix(vec3(0.04), albedo, uMetal);
    vec3 diffC = albedo * (1.0 - uMetal);

    // hemisphere ambient + bounce light from the ground
    float hemi = n.y * 0.5 + 0.5;
    vec3 amb = mix(uGroundColor, uSkyTop * 0.9 + uSkyHorizon * 0.35, hemi);
    vec3 col = diffC * amb * 0.95;
    col += diffC * uSunColor * nl * sh;

    // specular (sun)
    float a = rough * rough;
    vec3 F = F0 + (1.0 - F0) * pow(1.0 - max(dot(h, v), 0.0), 5.0);

```

```

float spec = D_GGX(nh, a) * 0.25 / max(nl * nv + 0.25, 0.25);
col += F * spec * uSunColor * nl * sh;

// environment reflection (analytic sky)
vec3 r = reflect(-v, n);
vec3 env = skyColor(normalize(vec3(r.x, max(r.y, -0.2), r.z)));
env = mix(env, amb, rough * rough);
vec3 Fe = F0 + (max(vec3(1.0 - rough), F0) - F0) * pow(1.0 - nv, 5.0);
float envOcc = mix(0.35, 1.0, sh) * clamp(r.y * 2.0 + 0.7, 0.25, 1.0);
col += env * Fe * uEnvBoost * envOcc * (1.0 - rough * 0.6);

// clearcoat layer for car paint
float lum = dot(albedo, vec3(0.3, 0.55, 0.15));
float coat = uClearcoat * smoothstep(0.1, 0.3, lum);
if (coat > 0.0) {
    float Fc = 0.04 + 0.96 * pow(1.0 - nv, 5.0);
    col += coat * (skyColor(normalize(vec3(r.x, max(r.y, -0.2), r.z))) * Fc * env
}
// dynamic lights
for (int i = 0; i < 8; i++) {
    if (i >= uLNum) break;
    vec3 Ld = uLPos[i].xyz - vWPos; float dist = length(Ld); Ld /= max(dist, 1e-3);
    float range = uLPos[i].w; float att = clamp(1.0 - dist / range, 0.0, 1.0); at
    float cone = 1.0;
    if (uLDir[i].w > -0.5) cone = smoothstep(uLDir[i].w, uLDir[i].w + 0.12, dot(-
    float nl2 = max(dot(n, Ld), 0.0);
    vec3 h2 = normalize(Ld + v); float sp2 = D_GGX(max(dot(n, h2), 0.0), max(a, 0
    col += (diffC * nl2 + F0 * sp2 * nl2) * uLCol[i] * att * cone;
}
col += uEmissive;
col = applyFog(col, vWPos, uCamPos);
fragColor = vec4(finalColor(col), base.a);
}
`;

```

```

const DEPTH_VS = /* glsl */`#version 300 es
precision highp float;
layout(location=0) in vec3 aPos;
layout(location=2) in vec2 aUv;
#ifdef SKIN
layout(location=3) in vec4 aJoints;
layout(location=4) in vec4 aWeights;
uniform highp sampler2D uBones;
mat4 boneMat(float fi) {
    int i = int(fi) * 4;
    return mat4(texelFetch(uBones, ivec2(i, 0), 0), texelFetch(uBones, ivec2(i + 1,

```

```

        texelFetch(uBones, ivec2(i + 2, 0), 0), texelFetch(uBones, ivec2(i
    }
#endif
uniform mat4 uModel;
uniform mat4 uVP;
out vec2 vUv;
void main() {
#ifdef SKIN
    mat4 sk = boneMat(aJoints.x) * aWeights.x + boneMat(aJoints.y) * aWeights.y + b
    vec4 wp = sk * vec4(aPos, 1.0);
#else
    vec4 wp = uModel * vec4(aPos, 1.0);
#endif
    vUv = aUv;
    gl_Position = uVP * wp;
}
`;

const DEPTH_FS = /* glsl */`#version 300 es
precision mediump float;
in vec2 vUv;
#ifdef ALPHATEST
uniform sampler2D uMap;
uniform float uAlphaCut;
#endif
out vec4 o;
void main() {
#ifdef ALPHATEST
    if (texture(uMap, vUv).a < uAlphaCut) discard;
#endif
    o = vec4(1.0);
}
`;

const SKY_VS = /* glsl */`#version 300 es
precision highp float;
out vec2 vNdc;
void main() {
    vec2 p = vec2((gl_VertexID << 1) & 2, gl_VertexID & 2) * 2.0 - 1.0;
    vNdc = p;
    gl_Position = vec4(p, 0.99999, 1.0);
}
`;

const SKY_FS = /* glsl */`#version 300 es
precision highp float;
in vec2 vNdc;
uniform mat4 uInvVP;

```

```

uniform vec3 uCamPos;
out vec4 fragColor;
${COMMON}
float hash(vec2 p) { return fract(sin(dot(p, vec2(127.1, 311.7))) * 43758.5453);
float noise(vec2 p) {
    vec2 i = floor(p), f = fract(p); f = f * f * (3.0 - 2.0 * f);
    return mix(mix(hash(i), hash(i + vec2(1, 0))), f.x), mix(hash(i + vec2(0, 1)), h
}
float fbm(vec2 p) { float s = 0.0, a = 0.5; for (int i = 0; i < 5; i++) { s += a
void main() {
    vec4 w = uInvVP * vec4(vNdc, 1.0, 1.0);
    vec3 d = normalize(w.xyz / w.w - uCamPos);
    vec3 c = skyColor(d);
    float sd = max(dot(d, uSunDir), 0.0);
    c += uSunColor * smoothstep(0.9994, 0.9997, sd) * 6.0;
    if (d.y > 0.0) {
        vec2 uv = d.xz / (d.y + 0.08) * 1.6 + vec2(uTime * 0.004, uTime * 0.002);
        float cl = fbm(uv * 1.2);
        cl = smoothstep(0.5, 0.78, cl) * smoothstep(0.0, 0.18, d.y);
        vec3 cc = mix(vec3(0.82, 0.85, 0.9), vec3(1.05), clamp(fbm(uv * 1.2 + uSunDir
        cc += uSunColor * 0.12 * pow(sd, 6.0);
        cc *= mix(1.0, 0.06, uNight);
        c = mix(c, cc * 1.25, cl * 0.9 * mix(1.0, 0.6, uNight));
        if (uNight > 0.0) {
            vec2 sp = d.xz / (d.y + 0.3) * 260.0; vec2 cell = floor(sp); float r = hash
            float star = step(0.992, r) * smoothstep(0.35, 0.0, length(fract(sp) - 0.5)
            c += vec3(star) * uNight * 1.5 * (1.0 - cl) * smoothstep(0.02, 0.2, d.y);
        }
    }
    fragColor = vec4(finalColor(c), 1.0);
}
`;

```

```

const PARTICLE_VS = /* glsl */`#version 300 es
precision highp float;
layout(location=0) in vec2 aCorner;
layout(location=1) in vec4 aPosSize;    // xyz, size
layout(location=2) in vec4 aColor;      // rgb, alpha
layout(location=3) in float aRot;
uniform mat4 uVP;
uniform vec3 uCamRight;
uniform vec3 uCamUp;
out vec2 vUv;
out vec4 vColor;
out vec3 vWPos;
void main() {

```

```

    float c = cos(aRot), s = sin(aRot);
    vec2 q = vec2(aCorner.x * c - aCorner.y * s, aCorner.x * s + aCorner.y * c);
    vec3 wp = aPosSize.xyz + (uCamRight * q.x + uCamUp * q.y) * aPosSize.w;
    vUv = aCorner * 0.5 + 0.5;
    vColor = aColor;
    vWPos = wp;
    gl_Position = uVP * vec4(wp, 1.0);
}
`;

const PARTICLE_FS = /* glsl */`#version 300 es
precision highp float;
in vec2 vUv;
in vec4 vColor;
in vec3 vWPos;
uniform sampler2D uTex;
uniform vec3 uCamPos;
uniform float uAdditive;
out vec4 fragColor;
${COMMON}
void main() {
    vec4 t = texture(uTex, vUv);
    float a = t.a * vColor.a;
    if (a < 0.003) discard;
    vec3 c = vColor.rgb * t.rgb;
    if (uAdditive > 0.5) { fragColor = vec4(finalColor(c) * a, 0.0); return; }
    // smoke is lit by sun + sky
    vec3 lit = c * (uSkyTop * 0.6 + uSkyHorizon * 0.5 + uSunColor * 0.55);
    lit = applyFog(lit, vWPos, uCamPos);
    fragColor = vec4(finalColor(lit), a);
}
`;

const DECAL_VS = /* glsl */`#version 300 es
precision highp float;
layout(location=0) in vec3 aPos;
layout(location=1) in float aAlpha;
uniform mat4 uVP;
out float vA;
out vec3 vWPos;
void main() { vA = aAlpha; vWPos = aPos; gl_Position = uVP * vec4(aPos, 1.0); }
`;

const DECAL_FS = /* glsl */`#version 300 es
precision highp float;
in float vA;
in vec3 vWPos;
uniform vec3 uCamPos;

```

```

uniform vec4 uColor;
out vec4 fragColor;
${COMMON}
void main() {
    fragColor = vec4(finalColor(uColor.rgb * (uSunColor * 0.5 + uSkyTop * 0.6)), vA
}
`;

return { COMMON, LIT_VS, LIT_FS, DEPTH_VS, DEPTH_FS, SKY_VS, SKY_FS, PARTICLE_VS,
})();

__M['glTF'] = (() => {
// Compact GLB loader: meshes, materials, textures, node hierarchy, skins.
const { Node, Skin, Material } = __M['scene'];
const { M4 } = __M['math'];

const CT = { 5120: Int8Array, 5121: Uint8Array, 5122: Int16Array, 5123: Uint16Arr
const NC = { SCALAR: 1, VEC2: 2, VEC3: 3, VEC4: 4, MAT4: 16 };

async function fetchBuffer(url, onProgress) {
    const res = await fetch(url);
    if (!res.ok) throw new Error('HTTP ' + res.status + ' ' + url);
    const total = +res.headers.get('content-length') || 0;
    if (!res.body || !onProgress) return await res.arrayBuffer();
    const reader = res.body.getReader(); const chunks = []; let got = 0;
    for (;;) {
        const { done, value } = await reader.read(); if (done) break;
        chunks.push(value); got += value.length; onProgress(got, total);
    }
    const out = new Uint8Array(got); let o = 0; for (const c of chunks) { out.set(c
    return out.buffer;
}

function loadImage(bytes, mime) {
    return new Promise((resolve, reject) => {
        const blob = new Blob([bytes], { type: mime || 'image/png' });
        const url = URL.createObjectURL(blob);
        const img = new Image();
        img.onload = () => { URL.revokeObjectURL(url); resolve(img); };
        img.onerror = (e) => { URL.revokeObjectURL(url); reject(new Error('image deco
        img.src = url;
    });
}

async function parseGLB(buf, opts = {}) {
    const dv = new DataView(buf);
    if (dv.getUint32(0, true) !== 0x46546C67) throw new Error('not a GLB');

```

```

const j1 = dv.getUint32(12, true);
const json = JSON.parse(new TextDecoder().decode(new Uint8Array(buf, 20, j1)));
let bin = null; let off = 20 + j1;
if (off < buf.byteLength) { const b1 = dv.getUint32(off, true); bin = new Uint8

const bvSlice = (i) => { const v = json.bufferViews[i]; return { bytes: new Uint8

const accCache = new Map();
function accessor(i) {
  if (accCache.has(i)) return accCache.get(i);
  const a = json.accessors[i]; const T = CT[a.componentType]; const n = NC[a.type]
  const { bytes, stride } = bvSlice(a.bufferView);
  const es = T.BYTES_PER_ELEMENT; const ao = a.byteOffset || 0;
  let arr;
  if (stride && stride !== es * n) {
    arr = new T(cnt * n);
    const dvb = new DataView(bytes.buffer, bytes.byteOffset, bytes.byteLength);
    const get = { 1: T === Int8Array ? 'getInt8' : 'getUint8', 2: T === Int16Ar
    for (let k = 0; k < cnt; k++) for (let c = 0; c < n; c++) arr[k * n + c] =
  } else {
    const start = bytes.byteOffset + ao;
    if (start % es === 0) arr = new T(bytes.buffer, start, cnt * n);
    else arr = new T(bytes.buffer.slice(start, start + cnt * n * es));
  }
  // custom dequantization (extras.q = [scale..., offset...])
  const q = a.extras && a.extras.q;
  if (q) {
    const f = new Float32Array(cnt * n);
    for (let k = 0; k < cnt; k++) for (let c = 0; c < n; c++) f[k * n + c] = ar
    arr = f;
  } else if (a.normalized && (a.type !== 'VEC4' || opts.decodeNormalized)) {
    const f = new Float32Array(cnt * n); const mx = T === Int8Array ? 127 : T =
    for (let k = 0; k < f.length; k++) f[k] = Math.max(arr[k] / mx, -1);
    arr = f;
  }
  const r = { data: arr, size: n, count: cnt, normalized: !!a.normalized && !(a
  accCache.set(i, r); return r;
}

// textures
const images = await Promise.all((json.images || []).map(async (im) => {
  const { bytes } = bvSlice(im.bufferView);
  try { return await loadImage(bytes, im.mimeType); } catch (e) { console.warn(
}));
const textures = (json.textures || []).map(t => {
  const s = t.sampler !== undefined ? json.samplers[t.sampler] : {};

```

```

    return { image: images[t.source], wrapS: s.wrapS || 10497, wrapT: s.wrapT ||
});

const materials = (json.materials || []).map(m => {
  const pbr = m.pbrMetallicRoughness || {};
  const sg = m.extensions && m.extensions.KHR_materials_pbrSpecularGlossiness;
  let color = pbr.baseColorFactor ? pbr.baseColorFactor.slice() : [1, 1, 1, 1];
  let map = pbr.baseColorTexture ? textures[pbr.baseColorTexture.index] : null;
  let rough = pbr.roughnessFactor ?? 1, metal = pbr.metallicFactor ?? 1;
  if (sg) {
    map = sg.diffuseTexture ? textures[sg.diffuseTexture.index] : map;
    color = [1, 1, 1, 1]; // Sketchfab exports a black diffuse factor; texture
    rough = 0.75; metal = 0;
  }
  const mat = new Material({
    name: m.name, color, map, roughness: rough, metalness: metal,
    emissive: m.emissiveTexture ? [0, 0, 0] : (m.emissiveFactor || [0, 0, 0]),
    doubleSided: !!m.doubleSided,
  });
  if (m.extras) Object.assign(mat, m.extras);
  return mat;
});

const defMat = new Material({ name: 'default' });

const meshes = (json.meshes || []).map(me => ({
  name: me.name,
  prims: me.primitives.map(p => {
    const at = p.attributes; const attrs = {};
    if (at.POSITION !== undefined) attrs.position = accessor(at.POSITION);
    if (at.NORMAL !== undefined) {
      let nr = accessor(at.NORMAL);
      if (!(nr.data instanceof Float32Array) || nr.size !== 3) {
        const src = nr.data, n = nr.count, f = new Float32Array(n * 3), s = nr;
        const mx = src instanceof Int8Array ? 127 : src instanceof Int16Array ?
        for (let k = 0; k < n; k++) for (let c = 0; c < 3; c++) f[k * 3 + c] =
        nr = { data: f, size: 3, count: n, normalized: false };
      }
      attrs.normal = nr;
    }
    if (at.TEXCOORD_0 !== undefined) attrs.uv = accessor(at.TEXCOORD_0);
    if (at.JOINTS_0 !== undefined) attrs.joints = accessor(at.JOINTS_0);
    if (at.WEIGHTS_0 !== undefined) { const w = accessor(at.WEIGHTS_0); attrs.w
    const index = p.indices !== undefined ? accessor(p.indices).data : null;
    const prim = { attrs, index, material: p.material !== undefined ? materials
    computeBounds(prim);
    return prim;
  }
});

```

```

    })),
  }));

const nodes = json.nodes.map(n => new Node(n.name || ''));
json.nodes.forEach((n, i) => {
  const node = nodes[i];
  if (n.matrix) { M4.decompose(new Float32Array(n.matrix), node.pos, node.rot,
  if (n.translation) node.pos.set(...n.translation);
  if (n.rotation) node.rot.set(...n.rotation);
  if (n.scale) node.scl.set(...n.scale);
  if (n.mesh !== undefined) node.mesh = meshes[n.mesh];
  if (n.extras) node.userData = { ...n.extras };
  for (const c of n.children || []) node.add(nodes[c]);
});
json.nodes.forEach((n, i) => {
  if (n.skin !== undefined) {
    const s = json.skins[n.skin];
    const ibm = s.inverseBindMatrices !== undefined ? new Float32Array(accessor
    nodes[i].skin = new Skin(s.joints.map(j => nodes[j]), ibm);
  }
});
// animations: node TRS channels (LINEAR / STEP)
const animations = (json.animations || []).map(a => {
  let dur = 0;
  const ch = a.channels.map(c => {
    const sm = a.samplers[c.sampler]; const tin = accessor(sm.input), tout = ac
    let out = tout.data; if (!(out instanceof Float32Array)) { const mx = out i
    const times = tin.data instanceof Float32Array ? tin.data : Float32Array.fr
    dur = Math.max(dur, times[times.length - 1] || 0);
    return { node: nodes[c.target.node], path: c.target.path, times, out, size:
  }).filter(c => c.node && c.path !== 'weights');
  return { name: a.name || '', channels: ch, duration: dur };
});
const root = new Node('root');
const sc = json.scenes ? json.scenes[json.scene || 0] : null;
const roots = sc ? sc.nodes : nodes.map((n, i) => i).filter(i => !nodes[i].pare
for (const r of roots) root.add(nodes[r]);
root.userData = (json.asset && json.asset.extras) || {};
root.userData.extras = json.extras || {};
root.updateWorld();
return { root, nodes, materials, textures, json, animations };
}

// sample an animation clip at time t (seconds, looped) onto its nodes
function playAnimation(anim, t) {
  if (!anim || !anim.duration) return; t = t % anim.duration;

```

```

for (const c of anim.channels) {
  const T = c.times; let i = 0; while (i < T.length - 2 && T[i + 1] < t) i++;
  const t0 = T[i], t1 = T[Math.min(i + 1, T.length - 1)]; const u = c.step || t
  const n = c.size, a = i * n, b = Math.min(i + 1, T.length - 1) * n, o = c.out
  if (c.path === 'rotation') { const q = [o[a], o[a + 1], o[a + 2], o[a + 3]],
    else { const v = c.path === 'translation' ? c.node.pos : c.node.scl; v.set(o[
  ]
}

function computeBounds(prim) {
  const p = prim.attrs.position.data;
  let x0 = Infinity, y0 = Infinity, z0 = Infinity, x1 = -Infinity, y1 = -Infinity
  for (let i = 0; i < p.length; i += 3) {
    const x = p[i], y = p[i + 1], z = p[i + 2];
    if (x < x0) x0 = x; if (x > x1) x1 = x; if (y < y0) y0 = y; if (y > y1) y1 =
  }
  prim.min = [x0, y0, z0]; prim.max = [x1, y1, z1];
  prim.center = [(x0 + x1) / 2, (y0 + y1) / 2, (z0 + z1) / 2];
  prim.radius = Math.hypot(x1 - x0, y1 - y0, z1 - z0) / 2;
}

async function loadGLB(url, onProgress, opts) {
  const buf = await fetchBuffer(url, onProgress);
  return parseGLB(buf, opts);
}

return { fetchBuffer, parseGLB, playAnimation, computeBounds, loadGLB };
})();

__M['renderer'] = (() => {
  const { M4, V3 } = __M['math'];
  const S = __M['shaders'];

  class Camera {
    constructor() {
      this.pos = new V3(0, 3, -8); this.target = new V3(0, 1, 0); this.up = new V3(
      this.fov = 60 * Math.PI / 180; this.near = 0.1; this.far = 1500; this.aspect
      this.view = M4.create(); this.proj = M4.create(); this.vp = M4.create(); this
      this.planes = new Float32Array(24);
    }
    update() {
      M4.lookAt(this.view, this.pos, this.target, this.up);
      M4.perspective(this.proj, this.fov, this.aspect, this.near, this.far);
      M4.mul(this.vp, this.proj, this.view);
      M4.invert(this.invVP, this.vp);
      extractPlanes(this.vp, this.planes);
    }
  }

```

```
}
```

```
function extractPlanes(m, p) {
```

```
    const set = (i, a, b, c, d) => { const l = Math.hypot(a, b, c); p[i * 4] = a /  
    set(0, m[3] + m[0], m[7] + m[4], m[11] + m[8], m[15] + m[12]);  
    set(1, m[3] - m[0], m[7] - m[4], m[11] - m[8], m[15] - m[12]);  
    set(2, m[3] + m[1], m[7] + m[5], m[11] + m[9], m[15] + m[13]);  
    set(3, m[3] - m[1], m[7] - m[5], m[11] - m[9], m[15] - m[13]);  
    set(4, m[3] + m[2], m[7] + m[6], m[11] + m[10], m[15] + m[14]);  
    set(5, m[3] - m[2], m[7] - m[6], m[11] - m[10], m[15] - m[14]);
```

```
}
```

```
function sphereVisible(pl, x, y, z, r) {
```

```
    for (let i = 0; i < 6; i++) if (pl[i * 4] * x + pl[i * 4 + 1] * y + pl[i * 4 +  
    return true;
```

```
}
```

```
class Renderer {
```

```
    constructor(canvas) {
```

```
        this.canvas = canvas;
```

```
        const gl = canvas.getContext('webgl2', { antialias: true, alpha: false, power  
        if (!gl) throw new Error('WebGL2 غير مدعوم');
```

```
        this.gl = gl;
```

```
        this.aniso = gl.getExtension('EXT_texture_filter_anisotropic');
```

```
        this.maxAniso = this.aniso ? Math.min(8, gl.getParameter(this.aniso.MAX_TEXTU  
        gl.getExtension('EXT_color_buffer_float');
```

```
        this.programs = new Map();
```

```
        this.pixelRatio = Math.min(window.devicePixelRatio || 1, 2);
```

```
        this.env = {
```

```
            sunDir: new V3(-0.22, 0.8, -0.58).norm(), sunColor: [3.1, 2.85, 2.5],
```

```
            skyTop: [0.22, 0.42, 0.78], skyHorizon: [0.72, 0.8, 0.9], ground: [0.42, 0.
```

```
            exposure: 0.95, fog: 0.0024, time: 0,
```

```
        };
```

```
        this.shadowSize = 2048;
```

```
        this.shadowRange = 55;
```

```
        this.shadowOn = true;
```

```
        this.shadowFocus = new V3();
```

```
        this.initShadow();
```

```
        this.whiteTex = this.makeTex1x1([255, 255, 255, 255]);
```

```
        this.skyProg = this.program(S.SKY_VS, S.SKY_FS, []);
```

```
        this.emptyVao = gl.createVertexArray();
```

```
        this.custom = []; // custom draw callbacks (particles, decals) : fn(renderer,
```

```
        this.stats = { draws: 0, tris: 0 };
```

```
    }
```

```
makeTex1x1(rgba) {
```

```
    const gl = this.gl; const t = gl.createTexture();
```

```

    gl.bindTexture(gl.TEXTURE_2D, t);
    gl.texImage2D(gl.TEXTURE_2D, 0, gl.RGBA, 1, 1, 0, gl.RGBA, gl.UNSIGNED_BYTE,
    return t;
}

initShadow() {
    const gl = this.gl;
    this.shadowTex = gl.createTexture();
    gl.bindTexture(gl.TEXTURE_2D, this.shadowTex);
    gl.texStorage2D(gl.TEXTURE_2D, 1, gl.DEPTH_COMPONENT24, this.shadowSize, this
    gl.texParameteri(gl.TEXTURE_2D, gl.TEXTURE_MIN_FILTER, gl.LINEAR);
    gl.texParameteri(gl.TEXTURE_2D, gl.TEXTURE_MAG_FILTER, gl.LINEAR);
    gl.texParameteri(gl.TEXTURE_2D, gl.TEXTURE_WRAP_S, gl.CLAMP_TO_EDGE);
    gl.texParameteri(gl.TEXTURE_2D, gl.TEXTURE_WRAP_T, gl.CLAMP_TO_EDGE);
    gl.texParameteri(gl.TEXTURE_2D, gl.TEXTURE_COMPARE_MODE, gl.COMPARE_REF_TO_TE
    gl.texParameteri(gl.TEXTURE_2D, gl.TEXTURE_COMPARE_FUNC, gl.LEQUAL);
    this.shadowFbo = gl.createFramebuffer();
    gl.bindFramebuffer(gl.FRAMEBUFFER, this.shadowFbo);
    gl.framebufferTexture2D(gl.FRAMEBUFFER, gl.DEPTH_ATTACHMENT, gl.TEXTURE_2D, t
    gl.drawBuffers([gl.NONE]); gl.readBuffer(gl.NONE);
    gl.bindFramebuffer(gl.FRAMEBUFFER, null);
    this.shadowVP = M4.create(); this.shadowView = M4.create(); this.shadowProj =
}

program(vs, fs, defines) {
    const sid = this._sid || (this._sid = new Map());
    const idOf = (src) => { if (!sid.has(src)) sid.set(src, sid.size); return sid
    const key = idOf(vs) + ':' + idOf(fs) + ':' + defines.join(',');
    let p = this.programs.get(key); if (p) return p;
    const gl = this.gl;
    const hdr = defines.map(d => `#define ${d}\n`).join('');
    const inject = (src) => src.replace('#version 300 es\n', '#version 300 es\n'
    const sh = (type, src) => {
        const s = gl.createShader(type); gl.shaderSource(s, inject(src)); gl.compil
        if (!gl.getShaderParameter(s, gl.COMPILE_STATUS)) { const log = gl.getShade
        return s;
    };
    const prog = gl.createProgram();
    gl.attachShader(prog, sh(gl.VERTEX_SHADER, vs)); gl.attachShader(prog, sh(gl.
    gl.linkProgram(prog);
    if (!gl.getProgramParameter(prog, gl.LINK_STATUS)) throw new Error('link: ' +
    const u = {}; const n = gl.getProgramParameter(prog, gl.ACTIVE_UNIFORMS);
    for (let i = 0; i < n; i++) { const info = gl.getActiveUniform(prog, i); u[in
    p = { prog, u };
    this.programs.set(key, p);
    return p;
}

```

```

}

texture(t) {
  if (!t) return this.whiteTex;
  if (t.gl && t.dirty) { // live canvas (instrument cluster): re-upload
    const gl = this.gl; gl.bindTexture(gl.TEXTURE_2D, t.gl);
    gl.texImage2D(gl.TEXTURE_2D, 0, gl.SRGB8_ALPHA8, gl.RGBA, gl.UNSIGNED_BYTE,
  }
  if (t.gl) return t.gl;
  if (!t.image) return this.whiteTex;
  const gl = this.gl; const tex = gl.createTexture();
  gl.bindTexture(gl.TEXTURE_2D, tex);
  gl.pixelStorei(gl.UNPACK_FLIP_Y_WEBGL, false);
  gl.pixelStorei(gl.UNPACK_PREMULTIPLY_ALPHA_WEBGL, false);
  gl.pixelStorei(gl.UNPACK_COLORSPACE_CONVERSION_WEBGL, gl.NONE);
  gl.texImage2D(gl.TEXTURE_2D, 0, t.srgb === false ? gl.RGBA8 : gl.SRGB8_ALPHA8
  gl.generateMipmap(gl.TEXTURE_2D);
  gl.texParameteri(gl.TEXTURE_2D, gl.TEXTURE_MIN_FILTER, gl.LINEAR_MIPMAP_LINEAR);
  gl.texParameteri(gl.TEXTURE_2D, gl.TEXTURE_MAG_FILTER, gl.LINEAR);
  gl.texParameteri(gl.TEXTURE_2D, gl.TEXTURE_WRAP_S, t.wrapS);
  gl.texParameteri(gl.TEXTURE_2D, gl.TEXTURE_WRAP_T, t.wrapT);
  if (this.aniso) gl.texParameterf(gl.TEXTURE_2D, this.aniso.TEXTURE_MAX_ANISOTROPY,
  t.gl = tex;
  return tex;
}

upload(prim) {
  if (prim.gpu) return prim.gpu;
  if (prim.shareOf) { // same vertex buffers as another prim (per-part split of
    const pg = this.upload(prim.shareOf), gl = this.gl, A = prim.attrs; const v
    const bind = (loc, a) => { if (!a || !pg.bufs[loc]) return; gl.bindBuffer(gl
      const type = d instanceof Float32Array ? gl.FLOAT : d instanceof Uint16Ar
      gl.enableVertexAttribArray(loc); gl.vertexAttribPointer(loc, a.size, type
    bind(0, A.position); if (A.normal) bind(1, A.normal); else gl.vertexAttrib3
    const ib = gl.createBuffer(); gl.bindBuffer(gl.ELEMENT_ARRAY_BUFFER, ib); l
    gl.bufferData(gl.ELEMENT_ARRAY_BUFFER, idx, gl.STATIC_DRAW); gl.bindVertexA
    prim.gpu = { vao, bufs: pg.bufs, ib, itype: idx instanceof Uint32Array ? gl
    return prim.gpu;
  }
  const gl = this.gl; const vao = gl.createVertexArray(); gl.bindVertexArray(va
  const bufs = {};
  const attr = (loc, a, dynamic) => {
    if (!a) return;
    const b = gl.createBuffer(); gl.bindBuffer(gl.ARRAY_BUFFER, b);
    gl.bufferData(gl.ARRAY_BUFFER, a.data, dynamic ? gl.DYNAMIC_DRAW : gl.STATI
    const d = a.data;
  }
}

```

```

    const type = d instanceof Float32Array ? gl.FLOAT : d instanceof Uint16Arra
    gl.enableVertexAttribArray(loc);
    gl.vertexAttribPointer(loc, a.size, type, !!a.normalized, 0, 0);
    bufs[loc] = b;
};

const A = prim.attrs;
attr(0, A.position, prim.dynamic);
if (A.normal) attr(1, A.normal, prim.dynamic); else gl.vertexAttrib3f(1, 0, 1
if (A.uv) attr(2, A.uv); else gl.vertexAttrib2f(2, 0, 0);
if (A.joints) attr(3, A.joints);
if (A.weights) attr(4, A.weights);
let ib = null, itype = 0, count = A.position.count;
if (prim.index) {
    ib = gl.createBuffer(); gl.bindBuffer(gl.ELEMENT_ARRAY_BUFFER, ib);
    let idx = prim.index;
    if (idx instanceof Uint8Array) idx = new Uint16Array(idx);
    gl.bufferData(gl.ELEMENT_ARRAY_BUFFER, idx, gl.STATIC_DRAW);
    itype = idx instanceof Uint32Array ? gl.UNSIGNED_INT : gl.UNSIGNED_SHORT; c
}
gl.bindVertexArray(null);
prim.gpu = { vao, bufs, ib, itype, count };
return prim.gpu;
}

// update dynamic vertex data (deformation)
updatePositions(prim) {
    const g = prim.gpu; if (!g) return; const gl = this.gl;
    gl.bindBuffer(gl.ARRAY_BUFFER, g.bufs[0]); gl.bufferSubData(gl.ARRAY_BUFFER,
    if (g.bufs[1]) { gl.bindBuffer(gl.ARRAY_BUFFER, g.bufs[1]); gl.bufferSubData(
}

boneTex(skin) {
    const gl = this.gl;
    if (!skin.tex) {
        skin.tex = gl.createTexture(); gl.bindTexture(gl.TEXTURE_2D, skin.tex);
        gl.texParameteri(gl.TEXTURE_2D, gl.TEXTURE_MIN_FILTER, gl.NEAREST); gl.texP
        gl.texImage2D(gl.TEXTURE_2D, 0, gl.RGBA32F, skin.joints.length * 4, 1, 0, g
        skin.dirty = false;
    } else if (skin.dirty) {
        gl.bindTexture(gl.TEXTURE_2D, skin.tex);
        gl.texSubImage2D(gl.TEXTURE_2D, 0, 0, 0, skin.joints.length * 4, 1, gl.RGBA
        skin.dirty = false;
    }
    return skin.tex;
}

```

```

resize() {
    const c = this.canvas; const w = Math.round(c.clientWidth * this.pixelRatio),
    if (c.width !== w || c.height !== h) { c.width = w; c.height = h; }
    return [w, h];
}

collect(roots, cam) {
    const list = [];
    const tmp = new V3();
    const visit = (n) => {
        if (!n.visible) return;
        if (n.mesh) {
            const m = n.world; const sc = Math.max(Math.hypot(m[0], m[1], m[2]), Math
            for (const p of n.mesh.prims) {
                if (p.hidden) continue;
                tmp.set(p.center[0], p.center[1], p.center[2]).applyM4(m);
                list.push({ node: n, prim: p, x: tmp.x, y: tmp.y, z: tmp.z, r: p.radius
            }
        }
        for (const c of n.children) visit(c);
    };
    for (const r of roots) visit(r);
    return list;
}

setEnvUniforms(u, cam) {
    const gl = this.gl, e = this.env;
    gl.uniform3f(u.uSunDir, e.sunDir.x, e.sunDir.y, e.sunDir.z);
    gl.uniform3fv(u.uSunColor, e.sunColor); gl.uniform3fv(u.uSkyTop, e.skyTop); g
    gl.uniform3fv(u.uGroundColor, e.ground); gl.uniform1f(u.uExposure, e.exposure
    gl.uniform1f(u.uTime, e.time);
    if (u.uNight) gl.uniform1f(u.uNight, e.night || 0);
    if (u.uLNum) {
        const L = this.lights || [], n = Math.min(8, L.length);
        const P = this._lp || (this._lp = new Float32Array(32)), D = this._ld || (t
        for (let i = 0; i < n; i++) { const l = L[i]; P.set([l.pos.x, l.pos.y, l.po
        gl.uniform1i(u.uLNum, n); if (n) { gl.uniform4fv(u.uLPos, P); gl.uniform4fv
    }
    if (u.uCamPos) gl.uniform3f(u.uCamPos, cam.pos.x, cam.pos.y, cam.pos.z);
}

updateShadowMatrix() {
    const r = this.shadowRange; const e = this.env; const f = this.shadowFocus;
    const eye = new V3().copy(f).addS(e.sunDir, 250);
    M4.lookAt(this.shadowView, eye, f, Math.abs(e.sunDir.y) > 0.99 ? new V3(0, 0,
    // snap to texels to avoid shimmering

```

```

const texel = (2 * r) / this.shadowSize;
const cx = this.shadowView[12], cy = this.shadowView[13];
this.shadowView[12] = Math.round(cx / texel) * texel; this.shadowView[13] = M
M4.ortho(this.shadowProj, -r, r, -r, r, 1, 500);
M4.mul(this.shadowVP, this.shadowProj, this.shadowView);
}

```

```

renderShadows(list) {
  const gl = this.gl;
  this.updateShadowMatrix();
  gl.bindFramebuffer(gl.FRAMEBUFFER, this.shadowFbo);
  gl.viewport(0, 0, this.shadowSize, this.shadowSize);
  gl.clear(gl.DEPTH_BUFFER_BIT);
  gl.enable(gl.DEPTH_TEST); gl.depthMask(true); gl.colorMask(false, false, false, false);
  gl.enable(gl.POLYGON_OFFSET_FILL); gl.polygonOffset(1.5, 3.0);
  gl.disable(gl.CULL_FACE);
  const planes = new Float32Array(24); extractPlanes(this.shadowVP, planes);
  for (const it of list) {
    if (!it.node.castShadow || it.prim.noShadow) continue;
    const mat = it.prim.material; if (mat.alphaMode === 'BLEND') continue;
    if (!sphereVisible(planes, it.x, it.y, it.z, it.r)) continue;
    const prim = it.prim.shadowPrim || it.prim;
    const defs = []; if (it.node.skin) defs.push('SKIN'); if (mat.alphaMode === 'BLEND')
    const p = this.program(S.DEPTH_VS, S.DEPTH_FS, defs);
    gl.useProgram(p.prog);
    gl.uniformMatrix4fv(p.u.uVP, false, this.shadowVP);
    gl.uniformMatrix4fv(p.u.uModel, false, it.node.world);
    if (it.node.skin) { gl.activeTexture(gl.TEXTURE3); gl.bindTexture(gl.TEXTURE3);
    if (defs.includes('ALPHATEST')) { gl.activeTexture(gl.TEXTURE0); gl.bindTexture(gl.TEXTURE0);
    const g = this.upload(prim);
    gl.bindVertexArray(g.vao);
    if (g.ib) gl.drawElements(gl.TRIANGLES, g.count, g.itype, 0); else gl.drawArrays(
  }
  gl.disable(gl.POLYGON_OFFSET_FILL);
  gl.colorMask(true, true, true, true);
  gl.bindFramebuffer(gl.FRAMEBUFFER, null);
}

```

```

render(roots, cam, opts = {}) {
  const gl = this.gl;
  let w, h;
  if (opts.target) { w = opts.target.w; h = opts.target.h; }
  else[w, h] = this.resize();
  cam.aspect = w / h; cam.update();
  const list = this.collect(roots, cam);
  // skins

```

```

for (const it of list) if (it.node.skin && !it._sk) it.node.skin.update();
if (this.shadowOn) this.renderShadows(list);

gl.bindFramebuffer(gl.FRAMEBUFFER, opts.target ? opts.target.fbo : null);
gl.viewport(0, 0, w, h);
gl.depthMask(true);
gl.clearColor(0.5, 0.6, 0.7, 1); gl.clear(gl.COLOR_BUFFER_BIT | gl.DEPTH_BUFFER_BIT);
gl.enable(gl.DEPTH_TEST); gl.depthFunc(gl.LEQUAL);
this.stats.draws = 0; this.stats.tris = 0;

const opaque = [], blend = [];
for (const it of list) {
    if (it.prim.shadowOnly || !sphereVisible(cam.planes, it.x, it.y, it.z, it.r) ||
        (it.prim.material.alphaMode === 'BLEND' ? blend : opaque).push(it);
}
gl.disable(gl.BLEND);
for (const it of opaque) this.drawItem(it, cam);
// sky
if (!opts.noSky) {
    gl.useProgram(this.skyProg.prog); this.setEnvUniforms(this.skyProg.u, cam);
    gl.uniformMatrix4fv(this.skyProg.u.uInvVP, false, cam.invVP);
    gl.depthMask(false); gl.disable(gl.CULL_FACE);
    gl.bindVertexArray(this.emptyVao); gl.drawArrays(gl.TRIANGLES, 0, 3);
    gl.depthMask(true);
}
for (const fn of this.custom) fn(this, cam, 'opaque');
// transparent, back to front
blend.sort((a, b) => (b.x - cam.pos.x) ** 2 + (b.y - cam.pos.y) ** 2 + (b.z - cam.pos.z) ** 2);
gl.enable(gl.BLEND); gl.blendFuncSeparate(gl.SRC_ALPHA, gl.ONE_MINUS_SRC_ALPHA, gl.SRC_ALPHA, gl.ONE_MINUS_SRC_ALPHA);
gl.depthMask(false);
for (const it of blend) this.drawItem(it, cam);
gl.depthMask(true);
for (const fn of this.custom) fn(this, cam, 'transparent');
gl.disable(gl.BLEND);
gl.bindVertexArray(null);
}

drawItem(it, cam) {
    const gl = this.gl; const mat = it.prim.material; const node = it.node;
    const defs = [];
    if (node.skin) defs.push('SKIN');
    if (mat.map) defs.push('MAP');
    if (mat.alphaMode === 'MASK') defs.push('ALPHATEST');
    if (this.debugDefs) defs.push(...this.debugDefs);
    const p = this.program(S.LIT_VS, S.LIT_FS, defs);
    const u = p.u;

```

```

gl.useProgram(p.prog);
this.setEnvUniforms(u, cam);
gl.uniformMatrix4fv(u.uVP, false, cam.vp);
gl.uniformMatrix4fv(u.uModel, false, node.world);
const nm = this._nm || (this._nm = new Float32Array(9)); M4.normalMat3(nm, no
gl.uniformMatrix3fv(u.uNrmMat, false, nm);
gl.uniformMatrix4fv(u.uShadowVP, false, this.shadowVP);
const us = mat.uvScale || [1, 1]; gl.uniform2f(u.uUvScale, us[0], us[1]);
gl.uniform4fv(u.uColor, mat.color);
gl.uniform1f(u.uRough, mat.roughness); gl.uniform1f(u.uMetal, mat.metalness);
gl.uniform1f(u.uClearcoat, mat.clearcoat || 0); gl.uniform1f(u.uEnvBoost, mat
gl.uniform3fv(u.uEmissive, mat.emissive);
gl.uniform4fv(u.uBackColor, mat.backColor || [0, 0, 0, 0]);
gl.uniform1f(u.uAlphaCut, mat.alphaCutoff);
gl.uniform1f(u.uUnlit, mat.unlit ? 1 : 0);
gl.uniform1f(u.uShadowOn, this.shadowOn && node.receiveShadow ? 1 : 0);
gl.uniform1f(u.uShadowTexel, 1 / this.shadowSize);
gl.activeTexture(gl.TEXTURE0); gl.bindTexture(gl.TEXTURE_2D, this.texture(mat
gl.activeTexture(gl.TEXTURE1); gl.bindTexture(gl.TEXTURE_2D, this.shadowTex);
if (node.skin) { gl.activeTexture(gl.TEXTURE3); gl.bindTexture(gl.TEXTURE_2D,
if (mat.doubleSided || node.skin) gl.disable(gl.CULL_FACE); else { gl.enable(
const det = M4.det3(node.world); gl.frontFace(det < 0 ? gl.CW : gl.CCW);
if (mat.polygonOffset) { gl.enable(gl.POLYGON_OFFSET_FILL); gl.polygonOffset(
const g = this.upload(it.prim);
gl.bindVertexArray(g.vao);
if (g.ib) gl.drawElements(gl.TRIANGLES, g.count, g.itype, 0); else gl.drawArr
if (mat.polygonOffset) gl.disable(gl.POLYGON_OFFSET_FILL);
gl.frontFace(gl.CCW);
this.stats.draws++; this.stats.tris += g.count / 3;
}

// Offscreen render target for thumbnails
makeTarget(w, h) {
    const gl = this.gl;
    const fbo = gl.createFramebuffer(); gl.bindFramebuffer(gl.FRAMEBUFFER, fbo);
    const rb = gl.createRenderbuffer(); gl.bindRenderbuffer(gl.RENDERBUFFER, rb);
    gl.renderbufferStorage(gl.RENDERBUFFER, gl.RGBA8, w, h);
    gl.framebufferRenderbuffer(gl.FRAMEBUFFER, gl.COLOR_ATTACHMENT0, gl.RENDERBUF
    const db = gl.createRenderbuffer(); gl.bindRenderbuffer(gl.RENDERBUFFER, db);
    gl.renderbufferStorage(gl.RENDERBUFFER, gl.DEPTH_COMPONENT24, w, h);
    gl.framebufferRenderbuffer(gl.FRAMEBUFFER, gl.DEPTH_ATTACHMENT, gl.RENDERBUFF
    gl.bindFramebuffer(gl.FRAMEBUFFER, null);
    return { fbo, w, h };
}

readTarget(t) {
    const gl = this.gl; const px = new Uint8Array(t.w * t.h * 4);

```

```

        gl.bindFramebuffer(gl.FRAMEBUFFER, t.fbo); gl.readPixels(0, 0, t.w, t.h, gl.R
        const c = document.createElement('canvas'); c.width = t.w; c.height = t.h; co
        const id = ctx.createImageData(t.w, t.h);
        for (let y = 0; y < t.h; y++) id.data.set(px.subarray((t.h - 1 - y) * t.w * 4
        ctx.putImageData(id, 0, 0);
        return c;
    }
}

return { Camera, Renderer };
})();

__M['physics'] = (() => {
// Collision world (static triangle soup in a uniform XZ grid) + rigid body dynam
const { V3, Q } = __M['math'];

class CollisionWorld {
    constructor(cell = 4) {
        this.cell = cell;
        this.tris = []; // flat numbers (9 per tri) during building
        this.meta = []; // material tag per tri
        this.groundY = -0.3; // infinite fallback plane
        this.bounds = 1400; // invisible walls
    }
    addTriangles(pos, idx, matrix, tag = 0) {
        const v = new V3(), a = [];
        const n = pos.length / 3; const wp = new Float32Array(n * 3);
        for (let i = 0; i < n; i++) { v.set(pos[i * 3], pos[i * 3 + 1], pos[i * 3 + 2]
        const cnt = idx ? idx.length : n;
        for (let i = 0; i < cnt; i += 3) {
            const i0 = idx ? idx[i] : i, i1 = idx ? idx[i + 1] : i + 1, i2 = idx ? idx[
            this.tris.push(wp[i0 * 3], wp[i0 * 3 + 1], wp[i0 * 3 + 2], wp[i1 * 3], wp[i
            this.meta.push(tag);
        }
    }
    build() {
        if (!this.tris.length) this.tris.push(0, -1000, 0, 1, -1000, 0, 0, -1000, 1),
        const T = this.T = new Float32Array(this.tris); this.tris = null;
        const nt = T.length / 9; this.nt = nt;
        const N = this.N = new Float32Array(nt * 4); // normal + d
        let minx = Infinity, minz = Infinity, maxx = -Infinity, maxz = -Infinity;
        for (let t = 0; t < nt; t++) {
            const o = t * 9;
            const ex = T[o + 3] - T[o], ey = T[o + 4] - T[o + 1], ez = T[o + 5] - T[o +
            const fx = T[o + 6] - T[o], fy = T[o + 7] - T[o + 1], fz = T[o + 8] - T[o +
            let nx = ey * fz - ez * fy, ny = ez * fx - ex * fz, nz = ex * fy - ey * fx;
            if (10 < 1e-7) { (this.degenerate || (this.degenerate = new Set())).add(t);

```

```

    nx /= 1; ny /= 1; nz /= 1; N[t * 4] = nx; N[t * 4 + 1] = ny; N[t * 4 + 2] =
    for (let k = 0; k < 3; k++) { const x = T[o + k * 3], z = T[o + k * 3 + 2];
}
this.minx = Math.floor(minx) - 1; this.minz = Math.floor(minz) - 1;
this.gw = Math.ceil((maxx - this.minx) / this.cell) + 2; this.gh = Math.ceil(
const counts = new Uint32Array(this.gw * this.gh);
const cellsOf = (t, fn) => {
    const o = t * 9;
    const x0 = Math.min(T[o], T[o + 3], T[o + 6]), x1 = Math.max(T[o], T[o + 3]
    const z0 = Math.min(T[o + 2], T[o + 5], T[o + 8]), z1 = Math.max(T[o + 2],
    const cx0 = Math.floor((x0 - this.minx) / this.cell), cx1 = Math.floor((x1
    const cz0 = Math.floor((z0 - this.minz) / this.cell), cz1 = Math.floor((z1
    for (let cz = cz0; cz <= cz1; cz++) for (let cx = cx0; cx <= cx1; cx++) fn(
};
const dg = this.degenerate || new Set();
for (let t = 0; t < nt; t++) if (!dg.has(t)) cellsOf(t, c => counts[c]++);
const start = this.start = new Uint32Array(this.gw * this.gh + 1);
for (let i = 0; i < counts.length; i++) start[i + 1] = start[i] + counts[i];
const list = this.list = new Uint32Array(start[start.length - 1]);
const fill = new Uint32Array(counts.length);
for (let t = 0; t < nt; t++) if (!dg.has(t)) cellsOf(t, c => { list[start[c]
this.stamp = new Uint32Array(nt); this.q = 1;
this.tagArr = new Uint8Array(this.meta); this.meta = null;
}
// iterate candidate triangles overlapping XZ box
forBox(x0, z0, x1, z1, fn) {
    const q = ++this.q; if (q > 4e9) { this.stamp.fill(0); this.q = 1; }
    let cx0 = Math.floor((x0 - this.minx) / this.cell), cx1 = Math.floor((x1 - th
    let cz0 = Math.floor((z0 - this.minz) / this.cell), cz1 = Math.floor((z1 - th
    cx0 = Math.max(cx0, 0); cz0 = Math.max(cz0, 0); cx1 = Math.min(cx1, this.gw -
    for (let cz = cz0; cz <= cz1; cz++) for (let cx = cx0; cx <= cx1; cx++) {
        const c = cz * this.gw + cx;
        for (let i = this.start[c], e = this.start[c + 1]; i < e; i++) {
            const t = this.list[i]; if (this.stamp[t] === q) continue; this.stamp[t]
        }
    }
}
// Ray cast; returns {t, nx,ny,nz, tag} or null
raycast(o, d, maxT, out = {}) {
    const ex = o.x + d.x * maxT, ez = o.z + d.z * maxT;
    let best = maxT, hit = -1;
    const T = this.T;
    this.forBox(Math.min(o.x, ex), Math.min(o.z, ez), Math.max(o.x, ex), Math.max
        const k = t * 9;
        // Moller-Trumbore
        const elx = T[k + 3] - T[k], ely = T[k + 4] - T[k + 1], elz = T[k + 5] - T[

```

```

    const e2x = T[k + 6] - T[k], e2y = T[k + 7] - T[k + 1], e2z = T[k + 8] - T[
    const px = d.y * e2z - d.z * e2y, py = d.z * e2x - d.x * e2z, pz = d.x * e2
    const det = elx * px + ely * py + elz * pz; if (Math.abs(det) < 1e-9) retur
    const inv = 1 / det;
    const tx = o.x - T[k], ty = o.y - T[k + 1], tz = o.z - T[k + 2];
    const u = (tx * px + ty * py + tz * pz) * inv; if (u < 0 || u > 1) return;
    const qx = ty * elz - tz * ely, qy = tz * elx - tx * elz, qz = tx * ely - t
    const v = (d.x * qx + d.y * qy + d.z * qz) * inv; if (v < 0 || u + v > 1) r
    const tt = (e2x * qx + e2y * qy + e2z * qz) * inv;
    if (tt > 0 && tt < best) { best = tt; hit = t; }
});
// heightfield terrain
if (this.terrain) {
    const tr = this.terrain;
    const f = (t) => (o.y + d.y * t) - tr.h(o.x + d.x * t, o.z + d.z * t);
    let a = 0, b = best, fa = f(a), fb = f(b);
    if (fa > 0 && fb <= 0) {
        for (let i = 0; i < 14; i++) { const m = (a + b) / 2, fm = f(m); if (fm >
        const t = (a + b) / 2; const x = o.x + d.x * t, z = o.z + d.z * t;
        const n = tr.n(x, z);
        out.t = t; out.nx = n[0]; out.ny = n[1]; out.nz = n[2]; out.tag = tr.tag
    }
}
// fallback ground plane
if (!this.terrain && d.y < -1e-6) { const tp = (this.groundY - o.y) / d.y; if
if (hit < 0) return null;
const N = this.N; let nx = N[hit * 4], ny = N[hit * 4 + 1], nz = N[hit * 4 +
if (nx * d.x + ny * d.y + nz * d.z > 0) { nx = -nx; ny = -ny; nz = -nz; }
out.t = best; out.nx = nx; out.ny = ny; out.nz = nz; out.tag = this.tagArr[hi
return out;
}
// sphere vs world; calls fn(nx,ny,nz, depth, px,py,pz, tag) for each penetrati
sphere(c, r, fn) {
    const T = this.T; const r2 = r * r;
    const cp = this._cp || (this._cp = [0, 0, 0]);
    this.forBox(c.x - r, c.z - r, c.x + r, c.z + r, (t) => {
        const k = t * 9;
        // quick plane test
        const N = this.N; const dist = N[t * 4] * c.x + N[t * 4 + 1] * c.y + N[t *
        if (dist > r || dist < -r) return;
        closestPtTri(c.x, c.y, c.z, T, k, cp);
        const dx = c.x - cp[0], dy = c.y - cp[1], dz = c.z - cp[2]; const d2 = dx *
        if (d2 >= r2) return;
        const d = Math.sqrt(d2);
        if (!(d2 >= 0)) return; // NaN guard
        let nx, ny, nz;

```

```

    if (d > 1e-6) { nx = dx / d; ny = dy / d; nz = dz / d; }
    else { nx = N[t * 4]; ny = N[t * 4 + 1]; nz = N[t * 4 + 2]; }
    fn(nx, ny, nz, r - d, cp[0], cp[1], cp[2], this.tagArr[t]);
  });
  // terrain / ground plane + invisible world bounds
  if (this.terrain) {
    const tr = this.terrain; const h = tr.h(c.x, c.z);
    if (c.y - r < h + 0.5) {
      const n = tr.n(c.x, c.z); const dist = (c.y - h) * n[1];
      if (dist < r) fn(n[0], n[1], n[2], r - dist, c.x - n[0] * dist, c.y - n[1]
    }
    } else if (c.y - r < this.groundY) fn(0, 1, 0, this.groundY - (c.y - r), c.x,
    const B = this.bounds, BX = this.boundsX ?? B;
    if (c.x + r > BX) fn(-1, 0, 0, c.x + r - BX, BX, c.y, c.z, 9); if (c.x - r <
    if (c.z + r > B) fn(0, 0, -1, c.z + r - B, c.x, c.y, B, 9); if (c.z - r < -B)
  }
  heightAt(x, z, fromY = 100) {
    if (this.terrain) fromY = Math.max(fromY, this.terrain.h(x, z) + 1.5);
    const r = this.raycast({ x, y: fromY, z }, { x: 0, y: -1, z: 0 }, fromY + 50,
    return r ? fromY - r.t : this.groundY;
  }
}

```

```

function closestPtTri(px, py, pz, T, k, out) {
  const ax = T[k], ay = T[k + 1], az = T[k + 2], bx = T[k + 3], by = T[k + 4], bz
  const abx = bx - ax, aby = by - ay, abz = bz - az, acx = cx - ax, acy = cy - ay
  const apx = px - ax, apy = py - ay, apz = pz - az;
  const d1 = abx * apx + aby * apy + abz * apz, d2 = acx * apx + acy * apy + acz
  if (d1 <= 0 && d2 <= 0) { out[0] = ax; out[1] = ay; out[2] = az; return; }
  const bpx = px - bx, bpy = py - by, bpz = pz - bz;
  const d3 = abx * bpx + aby * bpy + abz * bpz, d4 = acx * bpx + acy * bpy + acz
  if (d3 >= 0 && d4 <= d3) { out[0] = bx; out[1] = by; out[2] = bz; return; }
  const vc = d1 * d4 - d3 * d2;
  if (vc <= 0 && d1 >= 0 && d3 <= 0) { const v = d1 / (d1 - d3); out[0] = ax + ab
  const cpx = px - cx, cpy = py - cy, cpz = pz - cz;
  const d5 = abx * cpx + aby * cpy + abz * cpz, d6 = acx * cpx + acy * cpy + acz
  if (d6 >= 0 && d5 <= d6) { out[0] = cx; out[1] = cy; out[2] = cz; return; }
  const vb = d5 * d2 - d1 * d6;
  if (vb <= 0 && d2 >= 0 && d6 <= 0) { const w = d2 / (d2 - d6); out[0] = ax + ac
  const va = d3 * d6 - d5 * d4;
  if (va <= 0 && (d4 - d3) >= 0 && (d5 - d6) >= 0) { const w = (d4 - d3) / ((d4 -
  const denom = 1 / (va + vb + vc); const v = vb * denom, w = vc * denom;
  out[0] = ax + abx * v + acx * w; out[1] = ay + aby * v + acy * w; out[2] = az +
}

```

```

// 3x3 helpers (row-major arrays of 9)

```

```

function m3FromQ(q, o) {
    const x = q.x, y = q.y, z = q.z, w = q.w;
    o[0] = 1 - 2 * (y * y + z * z); o[1] = 2 * (x * y - z * w); o[2] = 2 * (x * z +
    o[3] = 2 * (x * y + z * w); o[4] = 1 - 2 * (x * x + z * z); o[5] = 2 * (y * z -
    o[6] = 2 * (x * z - y * w); o[7] = 2 * (y * z + x * w); o[8] = 1 - 2 * (x * x +
    return o;
}

class RigidBody {
    constructor(mass, inertia /* V3 diag local */) {
        this.mass = mass; this.invMass = 1 / mass;
        this.invI1 = new V3(1 / inertia.x, 1 / inertia.y, 1 / inertia.z);
        this.pos = new V3(); this.q = new Q(); this.vel = new V3(); this.ang = new V3
        this.R = new Float32Array(9); this.Iw = new Float32Array(9);
        this.force = new V3(); this.torque = new V3();
        this.updateInertia();
    }
    updateInertia() {
        const R = m3FromQ(this.q, this.R), I = this.invI1, o = this.Iw;
        // Iw = R * diag(I) * R^T
        for (let i = 0; i < 3; i++) for (let j = 0; j < 3; j++)
            o[i * 3 + j] = R[i * 3] * I.x * R[j * 3] + R[i * 3 + 1] * I.y * R[j * 3 + 1
        }
    mulIw(v, out) {
        const o = this.Iw; const x = v.x, y = v.y, z = v.z;
        return out.set(o[0] * x + o[1] * y + o[2] * z, o[3] * x + o[4] * y + o[5] * z
        }
    velAt(p, out) { // world point
        const rx = p.x - this.pos.x, ry = p.y - this.pos.y, rz = p.z - this.pos.z; co
        return out.set(this.vel.x + w.y * rz - w.z * ry, this.vel.y + w.z * rx - w.x
        }
    applyImpulse(j, p) {
        this.vel.addS(j, this.invMass);
        const rx = p.x - this.pos.x, ry = p.y - this.pos.y, rz = p.z - this.pos.z;
        const t = _t.set(ry * j.z - rz * j.y, rz * j.x - rx * j.z, rx * j.y - ry * j.
        this.mulIw(t, _t2); this.ang.add(_t2);
    }
    // effective inverse mass along direction n at point p
    invMassAt(p, n) {
        const rx = p.x - this.pos.x, ry = p.y - this.pos.y, rz = p.z - this.pos.z;
        const c = _t.set(ry * n.z - rz * n.y, rz * n.x - rx * n.z, rx * n.y - ry * n.
        this.mulIw(c, _t2);
        const k = _t3.set(_t2.y * rz - _t2.z * ry, _t2.z * rx - _t2.x * rz, _t2.x * r
        return this.invMass + k.dot(n);
    }
    toWorld(local, out) { return out.copy(local).applyQ(this.q).add(this.pos); }
}

```

```

    dirToWorld(local, out) { return out.copy(local).applyQ(this.q); }
    integrate(dt) {
        this.pos.addS(this.vel, dt);
        this.q.integrate(this.ang, dt);
        this.updateInertia();
    }
}

const _t = new V3(), _t2 = new V3(), _t3 = new V3();

return { CollisionWorld, RigidBody };
})();

__M['carparts'] = (() => {
// Procedural car parts: mesh builder, high quality steel wheels, cabin interiors
const { V3, Q, M4 } = __M['math'];
const { Node, Material } = __M['scene'];
const { computeBounds } = __M['glTF'];

// ----- mesh builder -----
class MB {
    constructor() { this.P = []; this.N = []; this.U = []; this.I = []; this.m = M4
    push() { this.stack.push(new Float32Array(this.m)); return this; }
    pop() { this.m = this.stack.pop(); return this; }
    translate(x, y, z) { const t = M4.create(); t[12] = x; t[13] = y; t[14] = z; M4
    rotate(ax, ay, az, ang) { const q = new Q().setAxisAngle(new V3(ax, ay, az).nor
    scale(x, y, z) { const s = M4.create(); s[0] = x; s[5] = y; s[10] = z; M4.mul(t
    v(x, y, z, nx, ny, nz, u, w) {
        const m = this.m; const p = new V3(x, y, z).applyM4(m); const n = new V3(nx,
        this.P.push(p.x, p.y, p.z); this.N.push(n.x, n.y, n.z); this.U.push(u, w); re
    }
    tri(a, b, c) { this.I.push(a, b, c); }
    quad(a, b, c, d) { this.I.push(a, b, c, a, c, d); }
    // axis aligned box centered at (cx,cy,cz)
    box(cx, cy, cz, sx, sy, sz, uvs = 1) {
        const hx = sx / 2, hy = sy / 2, hz = sz / 2;
        const F = [[[1, 0, 0], [0, 0, -1], [0, 1, 0]], [[-1, 0, 0], [0, 0, 1], [0, 1,
        for (const [n, u, w] of F) {
            const c = [cx + n[0] * hx, cy + n[1] * hy, cz + n[2] * hz];
            const ex = [u[0] * hx, u[1] * hy, u[2] * hz], ey = [w[0] * hx, w[1] * hy, w
            const vs = [[-1, -1], [1, -1], [1, 1], [-1, 1]].map(([a, b]) => this.v(c[0]
            this.quad(vs[0], vs[1], vs[2], vs[3]));
        }
        return this;
    }
}

// rounded "pillow" box: a box with bevelled edges (cheap), good for seats and
pillow(cx, cy, cz, sx, sy, sz, r = 0.03) {
    this.box(cx, cy, cz, sx - 2 * r, sy, sz - 2 * r);

```

```

    this.box(cx, cy, cz, sx, sy - 2 * r, sz - 2 * r);
    this.box(cx, cy, cz, sx - 2 * r, sy - 2 * r, sz);
    return this;
}
// lathe around the X axis. profile: [[x, radius], ...] ; v coordinate from 0..
latheX(profile, seg = 48, uRepeat = 1) {
    const base = this.P.length / 3; const n = profile.length;
    let len = 0; const acc = [0]; for (let i = 1; i < n; i++) { len += Math.hypot
    for (let i = 0; i <= seg; i++) {
        const a = i / seg * Math.PI * 2, ca = Math.cos(a), sa = Math.sin(a);
        for (let j = 0; j < n; j++) {
            const [x, r] = profile[j];
            const j0 = Math.max(0, j - 1), j1 = Math.min(n - 1, j + 1);
            const dx = profile[j1][0] - profile[j0][0], dr = profile[j1][1] - profile
            let nx = -dr, nr = dx; const l = Math.hypot(nx, nr) || 1; nx /= l; nr /=
            this.v(x, r * ca, r * sa, nx, nr * ca, nr * sa, i / seg * uRepeat, acc[j]
        }
    }
    for (let i = 0; i < seg; i++) for (let j = 0; j < n - 1; j++) {
        const a = base + i * n + j, b = base + (i + 1) * n + j;
        this.quad(a, b, b + 1, a + 1);
    }
    return this;
}
// flat disc facing +X (or -X if flip) with planar uv
discX(x, r, seg = 48, flip = false, r0 = 0) {
    const base = this.P.length / 3; const s = flip ? -1 : 1;
    for (let i = 0; i <= seg; i++) {
        const a = i / seg * Math.PI * 2, c = Math.cos(a), sn = Math.sin(a);
        this.v(x, r * c, r * sn, s, 0, 0, 0.5 + c * 0.5, 0.5 + sn * 0.5);
        this.v(x, r0 * c, r0 * sn, s, 0, 0, 0.5 + c * 0.5 * r0 / r, 0.5 + sn * 0.5
    }
    for (let i = 0; i < seg; i++) { const a = base + i * 2; if (flip) this.quad(a
    return this;
}
cylY(cx, cy, cz, r, h, seg = 12) {
    const base = this.P.length / 3;
    for (let i = 0; i <= seg; i++) { const a = i / seg * Math.PI * 2, c = Math.co
    for (let i = 0; i < seg; i++) { const a = base + i * 2; this.quad(a, a + 1, a
    return this;
}
torusZ(R, r, seg = 40, sides = 10) { // ring in XY plane
    const base = this.P.length / 3;
    for (let i = 0; i <= seg; i++) {
        const a = i / seg * Math.PI * 2, ca = Math.cos(a), sa = Math.sin(a);
        for (let j = 0; j <= sides; j++) { const b = j / sides * Math.PI * 2, cb =

```

```

    }
    for (let i = 0; i < seg; i++) for (let j = 0; j < sides; j++) { const a = bas
    return this;
    }
    prim(material) {
        const prim = {
            attrs: { position: { data: new Float32Array(this.P), size: 3, count: this.P
            index: this.P.length / 3 > 65535 ? new Uint32Array(this.I) : new Uint16Arra
            };
            computeBounds(prim); return prim;
        }
    }

// ----- procedural textures -----
const texCache = {};
function canvasTex(key, w, h, draw, srgb = true) {
    if (texCache[key]) return texCache[key];
    const c = document.createElement('canvas'); c.width = w; c.height = h; const g
    return (texCache[key] = { image: c, wrapS: 10497, wrapT: 10497, srgb, gl: null
    }
function tyreTexture() {
    return canvasTex('tyre', 512, 256, (g, w, h) => {
        g.fillStyle = '#1b1b1c'; g.fillRect(0, 0, w, h);
        // v runs across the profile: 0..0.3 inner sidewall, 0.3..0.7 tread, 0.7..1 o
        const t0 = h * 0.3, t1 = h * 0.7;
        g.fillStyle = '#232324'; g.fillRect(0, t0, w, t1 - t0);
        // tread blocks (all-terrain pattern)
        g.fillStyle = '#0c0c0d';
        for (let i = 0; i < 32; i++) {
            const x = i * w / 32;
            g.fillRect(x, t0 + 6, 5, (t1 - t0) * 0.42);
            g.fillRect(x + 8, t0 + (t1 - t0) * 0.5, 5, (t1 - t0) * 0.45);
            g.fillRect(x, (t0 + t1) / 2 - 2, w / 32 - 3, 4);
        }
        g.fillStyle = '#101011'; g.fillRect(0, t0 + (t1 - t0) * 0.25, w, 3); g.fillRe
        // sidewall lettering bands
        g.fillStyle = '#2d2d2e';
        for (let i = 0; i < 6; i++) { g.fillRect(i * w / 6 + 10, h * 0.84, 50, 6); g.
        const n = g.getImageData(0, 0, w, h); for (let i = 0; i < n.data.length; i +=
        });
    }
}
function rimTexture(color, alloy) {
    return canvasTex('rim' + color + (alloy ? 'a' : ''), 256, 256, (g, w, h) => {
        const cx = w / 2, cy = h / 2;
        if (alloy) { // 5 twin-spoke alloy
            g.fillStyle = '#16171a'; g.fillRect(0, 0, w, h);

```

```

        g.fillStyle = color; g.beginPath(); g.arc(cx, cy, w * 0.49, 0, Math.PI * 2)
        for (let i = 0; i < 10; i++) { const a = i / 10 * Math.PI * 2 + (i % 2 ? 0.
        g.beginPath(); g.arc(cx, cy, w * 0.14, 0, Math.PI * 2); g.fill();
        g.fillStyle = '#2a2b2e'; g.beginPath(); g.arc(cx, cy, w * 0.07, 0, Math.PI
        return;
    }
    g.fillStyle = color; g.fillRect(0, 0, w, h);
    const grd = g.createRadialGradient(cx, cy, 10, cx, cy, w / 2); grd.addColorSt
    g.fillStyle = grd; g.fillRect(0, 0, w, h);
    // ventilation holes
    g.fillStyle = '#141414';
    for (let i = 0; i < 6; i++) { const a = i / 6 * Math.PI * 2 + 0.26; g.beginPa
    // hub + lug nuts
    g.fillStyle = 'rgba(0,0,0,0.25)'; g.beginPath(); g.arc(cx, cy, w * 0.2, 0, Ma
    g.fillStyle = color; g.beginPath(); g.arc(cx, cy, w * 0.17, 0, Math.PI * 2);
    g.fillStyle = '#9a9a9a'; for (let i = 0; i < 6; i++) { const a = i / 6 * Math
    g.fillStyle = '#2a2a2a'; g.beginPath(); g.arc(cx, cy, w * 0.07, 0, Math.PI *
    g.fillStyle = color; g.beginPath(); g.arc(cx, cy, w * 0.05, 0, Math.PI * 2);
    });
}

// high quality steel wheel, axle along X, centre at origin. side=+1 for left whe
function steelWheel(radius, width, rimColor, side, alloy) {
    const tyreMat = new Material({ name: 'tyre', map: tyreTexture(), roughness: 0.9
    const rimMat = new Material({ name: 'rim', map: rimTexture(rimColor, alloy), ro
    const inner = new Material({ name: 'rimInner', color: [0.18, 0.18, 0.18, 1], ro
    const R = radius, W = width / 2, rr = radius * 0.52;
    const tyre = new MB();
    // tyre profile from inner bead to outer bead (x across width)
    const prof = [];
    const sh = 0.035;
    prof.push([-W * 0.86, rr * 1.02], [-W, rr * 1.12], [-W * 1.04, R * 0.8], [-W, R
    prof.push([W * 0.72, R], [W * 0.9, R - sh * 0.35], [W, R - sh * 1.4], [W * 1.04
    tyre.latheX(prof, 56, 4);
    const rim = new MB();
    // barrel + dish (dished steel wheel: face recessed)
    const face = side > 0 ? 1 : -1;
    rim.latheX([-W * 0.86, rr * 1.03], [-W * 0.6, rr * 0.98], [W * 0.6, rr * 0.98]
    rim.discX(face * W * 0.35, rr * 0.98, 40, face < 0);
    const innerMB = new MB(); innerMB.discX(-face * W * 0.3, rr * 0.97, 24, face >
    const n = new Node('proc_wheel');
    n.mesh = { prims: [tyre.prim(tyreMat), rim.prim(rimMat), innerMB.prim(inner)] }
    return n;
}

// ----- cabin interior -----

```

```

// per-car themes (colours + dash layout) modelled on photos of the real cars
const THEMES = {
  lc: { top: [0.16, 0.15, 0.14], low: [0.46, 0.43, 0.38], accent: [0.32, 0.1, 0.0
  hilux: { top: [0.05, 0.05, 0.05], low: [0.09, 0.09, 0.09], accent: [0.62, 0.3,
  changan: { top: [0.035, 0.035, 0.04], low: [0.06, 0.06, 0.065], accent: [0.62,
  elantra: { top: [0.06, 0.06, 0.065], low: [0.1, 0.1, 0.105], accent: [1.0, 0.25
  camry: { top: [0.05, 0.05, 0.055], low: [0.12, 0.115, 0.11], accent: [0.55, 0.5
  avanza: { top: [0.13, 0.13, 0.135], low: [0.4, 0.4, 0.39], accent: [0.62, 0.63,
};

function screenTexture(style) {
  return canvasTex('screen_' + style, 512, 256, (g, w, h) => {
    const gr = g.createLinearGradient(0, 0, w, h); gr.addColorStop(0, '#0b2140');
    g.fillStyle = '#123a6b'; g.fillRect(0, 0, w, 26); g.fillStyle = '#bfe0ff'; g.
    // map-ish / media tiles
    g.fillStyle = '#18324f'; g.fillRect(16, 40, w * 0.55, h - 60);
    g.strokeStyle = '#3f86d6'; g.lineWidth = 4; g.beginPath(); g.moveTo(30, h - 4
    for (let i = 0; i < 3; i++) { g.fillStyle = ['#1f5fa8', '#2a7d4f', '#9a3b2f']
  });
}

function woodTexture() {
  return canvasTex('wood', 256, 64, (g, w, h) => {
    g.fillStyle = '#5a2210'; g.fillRect(0, 0, w, h);
    for (let i = 0; i < 40; i++) { g.strokeStyle = `rgba(${30 + Math.random() * 4
  });
}

function extrudeX(mb, prof, x0, x1, bulge = 0) {
  // profile [[z,y],...] swept from x0 to x1 (with optional forward bulge in the
  const nx = 12, base = mb.P.length / 3, n = prof.length;
  for (let i = 0; i <= nx; i++) {
    const x = x0 + (x1 - x0) * i / nx, t = i / nx, bz = bulge * Math.sin(t * Math
    for (let j = 0; j < n; j++) {
      const [z, y] = prof[j]; const j0 = Math.max(0, j - 1), j1 = Math.min(n - 1,
      const dz = prof[j1][0] - prof[j0][0], dy = prof[j1][1] - prof[j0][1]; let n
      if (nz > 0) { nz = -nz; ny = -ny; }
      mb.v(x, y, z + bz, 0, ny, nz, t, j / (n - 1));
    }
  }

  for (let i = 0; i < nx; i++) for (let j = 0; j < n - 1; j++) { const a = base +
}

function buildInterior(o) {
  const T = THEMES[o.theme] || THEMES.changan;
  const lin = (c) => c.map(v => Math.pow(v, 2.2));
  const M = (name, c, r = 0.75, extra = {}) => new Material({ name, color: [...li
  const matTop = M('dashTop', T.top, 0.8), matLow = M('dashLow', T.low, 0.7), car
  const fabric = M('seat', o.fabric ? o.fabric.slice(0, 3) : T.seat, 0.92), fabri
  const liner = M('liner', T.liner, 0.95, { doubleSided: true });

```

```

const chrome = new Material({ name: 'chrome', color: [...lin(T.trim), 1], rough
const accent = T.wood ? new Material({ name: 'wood', map: woodTexture(), color:
    : T.accentGlow ? new Material({ name: 'accent', color: [...T.accent, 1], unli
const black = M('black', [0.02, 0.02, 0.02], 0.5, { doubleSided: true });
const screenM = new Material({ name: 'screen', map: screenTexture(o.theme), unl
const root = new Node('interior');
const B = { top: new MB(), low: new MB(), seat: new MB(), seat2: new MB(), floo
const { xs, floorY, seatTop, seatZ, halfW } = o;
const dF = o.dashFront, dT = o.dashTop, dD = o.dashDepth || 0.34;
const wZ = o.windZ ?? dF + 0.05, wY = o.windY ?? dT + 0.04;
const hw = halfW - 0.02, back = dF - dD; // back = the face of the
// ---- dashboard: one swept profile from the windscreen down to the floor (clo
const flat = T.shape === 'flat';
const upper = [[wZ + 0.03, wY - 0.01], [(wZ + back) / 2 + 0.05, dT + 0.015], [b
const lower = flat ? [[back, dT - 0.05], [back - 0.01, dT - 0.32], [back + 0.03
    : [[back, dT - 0.05], [back + 0.03, dT - 0.2], [back + 0.1, dT - 0.3], [back
extrudeX(B.top, upper, -hw, hw, 0.02); extrudeX(B.low, lower, -hw, hw, 0.03);
const faceZ = (y) => { for (let i = 0; i < lower.length - 1; i++) { const [z0,
// firewall behind it all (nothing to see through)
B.low.box(0, (floorY + wY) / 2, Math.max(dF, wZ) + 0.07, hw * 2, wY - floorY +
// accent band across the dash face (wood on the LC, orange trim on the Hilux,
const ay = dT - 0.13, ax0 = T.wood || o.theme === 'hilux' ? -hw + 0.05 : -hw +
B.accent.box((ax0 + ax1) / 2, ay, back - 0.012, ax1 - ax0, T.accentGlow ? 0.012
if (T.wood) { B.accent.box(xs + 0.3, ay, back - 0.012, 0.18, 0.05, 0.02); }
// vents
const vent = (x, y, style) => {
    if (style === 'round') { B.chrome.push().translate(x, y, back - 0.01).torusZ(
    else if (style === 'hex') { B.black.box(x, y, back - 0.01, 0.16, 0.07, 0.02);
    else if (style === 'slim') { B.black.box(x, y, back - 0.01, 0.2, 0.035, 0.02)
    else { B.black.box(x, y, back - 0.01, 0.14, 0.07, 0.02); for (let k = -1; k <
};
const vy = dT - 0.12;
vent(hw - 0.1, vy, T.vents); vent(-hw + 0.1, vy, T.vents);
if (T.vents === 'round') { vent(0.1, vy, 'round'); vent(-0.1, vy, 'round'); } e
// centre screen / radio
const scr = (x, y, z, w_, h_, tilt = -0.12) => { B.screen.push().translate(x, y
if (T.screen === 'float') { B.black.box(0, dT + 0.08, back + 0.06, 0.34, 0.15,
else if (T.screen === 'panorama') { /* screen + cluster share one wide glass pa
else if (T.screen === 'din') { const z = faceZ(dT - 0.2); B.chrome.box(0, dT -
else if (T.screen === 'stack') { const z = faceZ(dT - 0.22); B.chrome.box(0, dT
else { const z = faceZ(dT - 0.17); B.chrome.box(0, dT - 0.17, z - 0.008, 0.28,
// climate knobs
for (const k of (o.theme === 'elantra' ? [-0.09, 0, 0.09] : [-0.07, 0.07])) { c
// glovebox line and door cards
B.black.box(-xs, dT - 0.3, back + 0.03, 0.4, 0.006, 0.02);
let cz0 = o.rearWall ?? seatZ - 0.7, cz1 = back + 0.05;

```

```

const DB = o.doorBox && (Math.min(cz1, o.doorBox.z1) - Math.max(cz0, o.doorBox.
// door cards live on the doors themselves (they swing open with the door; noth
const DC = { 1: { low: new MB(), top: new MB(), accent: new MB(), chrome: new M
for (const sx of [1, -1]) {
    const x = sx * (DB ? Math.min(halfW + 0.005, DB.x) : halfW + 0.005), D = DC[s
    D.low.box(x, floorY + 0.28, (cz0 + cz1) / 2, 0.03, 0.5, cz1 - cz0);
    D.top.box(x - sx * 0.03, floorY + 0.55, (cz0 + cz1) / 2 - 0.1, 0.06, 0.05, 0.
    D.accent.box(x - sx * 0.018, floorY + 0.64, (cz0 + cz1) / 2, 0.012, T.accentG
    D.chrome.box(x - sx * 0.02, floorY + 0.5, cz1 - 0.2, 0.012, 0.03, 0.1);
}
// floor and centre console (tunnel, gear lever, handbrake)
const fz0 = (o.rearWall ?? seatZ - 0.9) + 0.02, fz1 = Math.max(dF, wZ) + 0.05;
B.floor.box(0, floorY - 0.02, (fz0 + fz1) / 2, halfW * 2, 0.04, fz1 - fz0);
B.low.box(0, floorY + 0.1, (seatZ - 0.2 + back + 0.15) / 2, 0.24, 0.2, back + 0
B.top.box(0, floorY + 0.205, (seatZ - 0.2 + back + 0.05) / 2, 0.22, 0.01, back
B.chrome.box(0, floorY + 0.33, seatZ + 0.35, 0.022, 0.24, 0.022);
B.black.pillow(0, floorY + 0.46, seatZ + 0.35, 0.05, 0.07, 0.05, 0.02);
B.black.pillow(0.1, floorY + 0.26, seatZ + 0.05, 0.05, 0.05, 0.28, 0.015);
// seats (with headrests, side bolsters and a contrasting centre panel)
const seat = (sx, sz, w_ = 0.5) => {
    const cushionH = seatTop - floorY;
    B.seat.pillow(sx, floorY + cushionH / 2, sz, w_, cushionH, 0.5, 0.04);
    B.seat2.box(sx, seatTop + 0.003, sz + 0.02, w_ * 0.5, 0.01, 0.36);
    B.seat.push().translate(sx, seatTop + 0.32, sz - 0.27).rotate(1, 0, 0, -0.22)
    B.seat.pillow(0, 0, 0, w_, 0.64, 0.13, 0.04);
    B.seat2.box(0, 0, 0.066, w_ * 0.5, 0.5, 0.01);
    B.seat.pillow(w_ * 0.4, -0.04, 0.05, 0.09, 0.46, 0.11, 0.03); B.seat.pillow(-
    B.seat.pillow(0, 0.41, -0.01, 0.26, 0.18, 0.11, 0.04); B.chrome.box(0.07, 0.3
    B.seat.pop();
};
seat(xs, seatZ); seat(-xs, seatZ);
if (o.rearSeat !== undefined) { const cushionH = seatTop - floorY; B.seat.pillo
// instrument cluster (hood on top of the dash behind the wheel)
const cl = clusterCanvas(o.cluster || 'analog2');
const gauges = new Material({ name: 'cluster', map: cl.tex, unlit: true, double
const bz = back + 0.1, by = dT + 0.045;
if (T.screen === 'panorama') {
    // Elantra: one wide glass panel for cluster + screen, angled towards the dri
    const pw = 0.72, px = xs - 0.14;
    B.black.push().translate(px, by + 0.04, bz - 0.02).rotate(0, 1, 0, 0.12).rota
    B.gauge.push().translate(px + 0.17, by + 0.04, bz - 0.04).rotate(0, 1, 0, 0.1
    { const a = B.gauge.v(-0.2, -0.07, 0, 0, 0, -1, 1, 1), b = B.gauge.v(0.2, -0.
    B.gauge.pop();
    B.screen.push().translate(px - 0.2, by + 0.04, bz - 0.04).rotate(0, 1, 0, 0.1
    { const a = B.screen.v(-0.15, -0.07, 0, 0, 0, -1, 1, 1), b = B.screen.v(0.15,
    B.screen.pop();

```

```

    } else {
        // slim curved visor over the gauges (like the real cars), open towards the d
        B.top.push().translate(xs, by, bz);
        { const prof = []; for (let i = 0; i <= 8; i++) { const a = i / 8 * Math.PI *
        B.top.box(0, -0.075, 0.05, 0.46, 0.02, 0.2); B.top.pop();
        B.gauge.push().translate(xs, by, bz - 0.04).rotate(1, 0, 0, -0.25);
        { const a = B.gauge.v(-0.2, -0.065, 0, 0, 0, -1, 1, 1), b = B.gauge.v(0.2, -0
        B.gauge.pop();
    }
    // interior rear-view mirror, hung just behind the windscreen glass
    if (o.linerY) { const my = o.mirrorY ?? o.linerY - 0.12, mz = o.mirrorZ ?? (wZ
    // sun visors
    if (o.linerY) for (const sx of [xs, -xs]) B.roof.box(sx, o.linerY - 0.025, o.li
    // steering column
    const wz = o.wheel[0], wy = o.wheel[1];
    { const cz = (wz + back) / 2 + 0.05, cy = (wy + dT - 0.2) / 2; const len = Math
    // head liner & rear cab wall
    B.roof.box(0, o.linerY, (o.linerZ[0] + o.linerZ[1]) / 2, o.linerHalfW * 2, 0.02
    if (o.rearWall !== undefined) B.roof.box(0, (floorY + o.linerY) / 2, o.rearWall
    const mats = { top: matTop, low: matLow, seat: fabric, seat2: fabric2, floor: c
    const n2 = new Node('int_dash'); n2.mesh = { prims: Object.keys(B).filter(k =>
    // steering wheel (animated), styled like the real car's
    const sw = new Node('steering'); sw.pos.set(xs, wy, wz);
    const tilt = -0.42; sw.rot.setAxisAngle(new V3(1, 0, 0), tilt);
    sw.mesh = { prims: steeringWheelPrims(o.wheelStyle || 'modern') }; sw.castShado
    root.add(sw);
    const doorCards = {};
    for (const sx of [1, -1]) { const D = DC[sx]; const n = new Node(sx > 0 ? 'door
    root.userData = { steering: sw, wheelCenter: sw.pos.clone(), wheelTilt: tilt, c
    return root;
}
// 3D steering wheels (rim radius 0.185 so the driver's hands always match)
function steeringWheelPrims(style) {
    const lea = new Material({ name: 'wheelLeather', color: [0.018, 0.018, 0.018, 1
    const pad = new Material({ name: 'wheelPad', color: [0.025, 0.025, 0.025, 1], r
    const sil = new Material({ name: 'wheelSilver', color: [0.72, 0.73, 0.75, 1], r
    const R = 0.185, rim = new MB(), sp = new MB(), sv = new MB(), wd = new MB();
    const arc = (mb, a0, a1, r = 0.02) => { const seg = 28, sides = 10, base = mb.P
    const spoke = (mb, ang, len, w, t = 0.022, r0 = 0.05) => { mb.push().rotate(0,
    const logo = () => { sv.push().translate(0, -0.005, -0.013).rotate(0, 1, 0, Mat
    if (style === 'lc') {
        arc(rim, -0.55, 0.55, 0.021); arc(rim, Math.PI - 0.55, Math.PI + 0.55, 0.021)
        spoke(sp, -0.12, 0.1, 0.045); spoke(sp, Math.PI + 0.12, 0.1, 0.045); spoke(sp
        sp.pillow(0, -0.01, 0.02, 0.15, 0.12, 0.06, 0.03);
    } else if (style === 'hilux' || style === 'avanza' || style === 'camry') {
        arc(rim, 0, Math.PI * 2, style === 'camry' ? 0.019 : 0.021);
    }
}

```

```

    spoke(sp, -0.08, 0.1, 0.05); spoke(sp, Math.PI + 0.08, 0.1, 0.05); spoke(sp,
    if (style !== 'avanza') { spoke(sv, -0.08, 0.08, 0.008, 0.024, 0.06); spoke(s
    if (style === 'hilux') for (const x of [-0.11, 0.11]) sv.push().translate(x,
    sp.pillow(0, -0.01, 0.02, style === 'avanza' ? 0.14 : 0.12, 0.11, 0.06, 0.03)
  } else if (style === 'changan') {
    arc(rim, 0, Math.PI * 2, 0.02);
    spoke(sp, -0.05, 0.1, 0.045); spoke(sp, Math.PI + 0.05, 0.1, 0.045); spoke(sp
    spoke(sv, -Math.PI / 2 - 0.35, 0.09, 0.008, 0.026, 0.06); spoke(sv, -Math.PI
    sp.pillow(0, 0.0, 0.02, 0.13, 0.11, 0.06, 0.035);
  } else if (style === 'sport' || style === 'elantra') {
    arc(rim, 0, Math.PI * 2, 0.021);
    spoke(sp, 0, 0.1, 0.05); spoke(sp, Math.PI, 0.1, 0.05);
    for (const x of [-0.05, 0.05]) { sp.box(x, -0.12, 0.012, 0.022, 0.11, 0.022);
    sp.pillow(0, 0.0, 0.02, 0.13, 0.1, 0.06, 0.03);
  } else {
    arc(rim, 0, Math.PI * 2, 0.02); spoke(sp, -0.05, 0.1, 0.045); spoke(sp, Math.
  }
  logo();
  const woodM = new Material({ name: 'wheelWood', map: woodTexture(), color: [1,
  const out = [rim.prim(lea), sp.prim(pad), sv.prim(sil)]; if (wd.I.length) out.p
  return out.filter(p => p.index.length);
}
// live instrument cluster drawn on a canvas (redrawn while you drive); one look
function clusterCanvas(style) {
  const c = document.createElement('canvas'); c.width = 512; c.height = 168; cons
  const tex = { image: c, wrapS: 33071, wrapT: 33071, srgb: true, gl: null, dirty
  const A0 = 0.75 * Math.PI, AR = 1.5 * Math.PI;
  const dial = (cx, cy, r, v, max, labels, o = {}) => {
    const face = o.face || '#0b0b0b', ink = o.ink || '#f2efe6', ring = o.ring ||
    g.fillStyle = face; g.beginPath(); g.arc(cx, cy, r + 6, 0, Math.PI * 2); g.fi
    g.strokeStyle = ring; g.lineWidth = o.ringW || 3; g.beginPath(); g.arc(cx, cy
    if (o.glow) { g.strokeStyle = o.glow; g.lineWidth = 3; g.beginPath(); g.arc(c
    const n = o.ticks || 20;
    for (let i = 0; i <= n; i++) { const a = A0 + i / n * AR; const big = i % 2 =
    g.fillStyle = ink; g.font = `bold ${o.font || 15}px Arial`; g.textAlign = 'ce
    labels.forEach((t, i) => { const a = A0 + i / (labels.length - 1) * AR; g.fil
    const a = A0 + Math.min(1, v / max) * AR;
    g.strokeStyle = needle; g.lineWidth = 4; g.beginPath(); g.moveTo(cx - Math.co
    g.fillStyle = o.cap || '#333'; g.beginPath(); g.arc(cx, cy, 8, 0, Math.PI * 2
  };
  const txt = (t, x, y, size, col, bold = true) => { g.fillStyle = col; g.font =
  const K = ['0', '40', '80', '120', '160', '200'], RPM = ['0', '2', '4', '6', '8
  const draw = (kmh, rpm, gear) => {
    g.fillStyle = '#030303'; g.fillRect(0, 0, 512, 168);
    const v = Math.round(kmh);
    if (style === 'lc') { // Land Cruiser 70: two big white-on-black d

```

```

    dial(120, 86, 72, rpm / 1000, 5, ['0', '1', '2', '3', '4', '5'], { ring: '#
    dial(392, 86, 72, kmh, 180, ['0', '40', '80', '120', '160'], { ring: '#c8c8
    g.fillStyle = '#111'; g.fillRect(222, 40, 68, 90); txt('E    F', 256, 60, 1
} else if (style === 'hilux') { // Hilux: blue-rimmed dials with a centre in
    dial(116, 86, 72, rpm / 1000, 8, RPM, { ring: '#44484e', glow: '#2f7fe0', r
    dial(396, 86, 72, kmh, 200, K, { ring: '#44484e', glow: '#2f7fe0', needle:
    g.fillStyle = '#0a1a2c'; g.fillRect(212, 34, 88, 100); g.strokeStyle = '#2f
    txt(String(gear), 256, 66, 30, '#e8f4ff'); txt(v + ' km/h', 256, 106, 15, '
} else if (style === 'changan') { // Changan: two dials with a digital centre
    dial(110, 86, 66, rpm / 1000, 8, RPM, { ring: '#707780', glow: '#27c3c9', r
    dial(402, 86, 66, kmh, 220, ['0', '60', '120', '180', '220'], { ring: '#707
    g.fillStyle = '#071114'; g.fillRect(196, 30, 120, 108); txt(String(v), 256,
} else if (style === 'elantra') { // Elantra 2024: fully digital, teal arcs a
    const gr = g.createLinearGradient(0, 0, 512, 0); gr.addColorStop(0, '#02080
    for (const [cx, val, mx] of [[110, rpm, 8000], [402, kmh, 220]]) { g.stroke
    txt(String(Math.round(rpm / 100) / 10), 110, 92, 30, '#e8ffff'); txt('x1000
    txt(String(gear), 256, 84, 34, '#2de0d0'); txt('D', 256, 120, 16, '#9ad');
} else if (style === 'camry') { // Camry 2025: blue ECO/POWER arc with a big
    g.fillStyle = '#01040a'; g.fillRect(0, 0, 512, 168);
    g.strokeStyle = '#0e2a55'; g.lineWidth = 12; g.beginPath(); g.arc(210, 120,
    const f = Math.min(1, rpm / 7000); g.strokeStyle = '#2a7cff'; g.beginPath()
    txt('ECO', 140, 60, 11, '#8fb4e6'); txt('POWER', 285, 60, 11, '#8fb4e6'); t
    g.strokeStyle = '#ddd'; g.lineWidth = 2; g.beginPath(); g.arc(470, 84, 60,
} else if (style === 'avanza') { // Avanza: big central speedometer, small ta
    dial(256, 88, 76, kmh, 180, ['0', '40', '80', '120', '160'], { face: '#1010
    dial(92, 92, 52, rpm / 1000, 8, RPM, { face: '#101010', ring: '#8a8a8a', re
    g.fillStyle = '#101010'; g.beginPath(); g.arc(420, 92, 56, 0, Math.PI * 2);
} else if (style === 'digital') {
    const gr = g.createLinearGradient(0, 0, 512, 168); gr.addColorStop(0, '#0d1
    g.strokeStyle = '#2a9df4'; g.lineWidth = 6; g.beginPath(); g.arc(130, 84, 6
    txt(String(v), 330, 78, 64, '#e8f4ff'); txt('km/h', 330, 124, 18, '#8fb8d8'
} else {
    dial(118, 84, 70, rpm / 1000, 8, RPM, { red: true }); dial(394, 84, 70, kmh
    g.fillStyle = '#0b0b0b'; g.fillRect(222, 50, 68, 68); g.strokeStyle = '#555
    txt(String(gear), 256, 76, 34, '#ffb347'); txt(String(v), 256, 104, 14, '#d
}
    tex.dirty = true;
};
draw(0, 800, 'N');
return { tex, draw };
}

// ----- number plate -----
function plateNode(plateGltf, front) {
    const src = plateGltf.root.children[0];
    const n = new Node(front ? 'plate_front' : 'plate_rear');

```

```

const m = new Node('plate_mesh'); m.mesh = src.mesh; m.pos.set(0, -0.08, 0); //
n.add(m);
if (!front) n.rot.setAxisAngle(new V3(0, 1, 0), Math.PI);
n.castShadow = false;
return n;
}

```

```

// embossed maker's lettering for tailgates (e.g. TOYOTA on the Hilux / Land Crui
function badgeNode(text, width) {

```

```

    const tex = canvasTex('badge_' + text, 1024, 160, (g, w, h) => {
        g.clearRect(0, 0, w, h); g.font = 'bold 128px Arial Black, Arial'; g.textAlign
        const gr = g.createLinearGradient(0, 20, 0, h - 20); gr.addColorStop(0, '#f4f
        g.lineWidth = 8; g.strokeStyle = 'rgba(20,20,22,0.85)'; g.strokeText(text.spl
        g.fillStyle = gr; g.fillText(text.split('').join(String.fromCharCode(8202)),
    });

```

```

    const mat = new Material({ name: 'badge', map: tex, roughness: 0.25, metalness:
    const mb = new MB(); const hw = width / 2, hh = width * 160 / 1024 / 2;
    const a = mb.v(-hw, -hh, 0, 0, 0, -1, 1, 1), b = mb.v(hw, -hh, 0, 0, 0, -1, 0,
    mb.I.push(a, b, cc, a, cc, d);
    const n = new Node('rear_badge'); n.mesh = { prims: [mb.prim(mat)] }; n.castSha
    return n;
}

```

```

// chassis seen when the car is on its roof: frame rails, axles, diff, driveshaft
function buildUnderbody(def, meta) {

```

```

    const truck = !!def.procWheels && def.mass > 1700;
    const metal = new Material({ name: 'chassis', color: truck ? [0.05, 0.045, 0.04
    const pan = new Material({ name: 'floorpan', color: [0.03, 0.03, 0.032, 1], rou
    const pipe = new Material({ name: 'exhaust', color: [0.07, 0.07, 0.075, 1], rou
    const W = meta.wheels, R = meta.wheelRadius;
    const fz = W.find(w => w.front).pos[2], rz = W.find(w => !w.front).pos[2], tx =
    const lowBody = meta.bmin ? meta.bmin[1] : R;
    const yPan = Math.max(lowBody + 0.03, R * 0.8), yRail = Math.min(yPan - 0.02, R
    const zA = (def.zMin ?? meta.bmin[2]) + 0.35, zB = (def.zMax ?? meta.bmax[2]) -
    const m = new MB(), p = new MB(), e = new MB();

```

```

    // floor pan between the axles (closes the underside), wheel arches stay open

```

```

    p.box(0, yPan + 0.01, (fz + rz) / 2, hw * 2, 0.012, (fz - rz) - R * 2.3);
    p.box(0, yPan + 0.03, fz + R * 1.2 + (zB - fz - R * 1.2) / 2, hw * 1.6, 0.012,
    p.box(0, yPan + 0.03, rz - R * 1.2 - (rz - R * 1.2 - zA) / 2, hw * 1.7, 0.012,

```

```

    // frame rails and cross members

```

```

    for (const sx of [-1, 1]) m.box(sx * (truck ? 0.44 : 0.5), yRail, (zA + zB) / 2
    for (const z of [zB - 0.1, fz - 0.45, (fz + rz) / 2, rz + 0.5, zA + 0.1]) m.box

```

```

    // axles / subframes and suspension

```

```

    m.push().translate(0, R, rz).rotate(0, 0, 1, Math.PI / 2); m.cylY(0, 0, 0, 0.04
    if (truck) { m.push().translate(0, R, fz).rotate(0, 0, 1, Math.PI / 2); m.cylY(
    else m.box(0, R + 0.02, fz, tx * 2 - 0.3, 0.07, 0.25);

```

```

m.pillow(0.08, R, rz, 0.28, 0.24, 0.26, 0.06);
for (const sx of [-1, 1]) for (const z of [fz, rz]) { m.cylY(sx * (tx - 0.25),
// gearbox + driveshaft to the rear axle
m.pillow(0, R + 0.08, fz - 0.55, 0.3, 0.26, 0.6, 0.06);
m.push().translate(0, R + 0.02, (fz - 0.85 + rz) / 2).rotate(1, 0, 0, Math.PI /
// fuel tank
m.pillow(-0.35, yPan - 0.1, rz + 0.55, 0.55, 0.2, 0.6, 0.05);
// exhaust: pipe along the right, muffler, tail pipe
e.push().translate(-0.3, yPan - 0.07, (fz - 0.4 + zA) / 2).rotate(1, 0, 0, Math
e.push().translate(-0.3, yPan - 0.08, (fz + rz) / 2 - 0.2).rotate(1, 0, 0, Math
const n = new Node('underbody'); n.mesh = { prims: [m.prim(metal), p.prim(pan),
return n;
}

// night: glowing head / tail lamp discs (unlit), shown only in night mode
// centre-stack face for cars whose own model has only a blank dash: radio, scree
function stackTexture(style) {
return canvasTex('stack_' + style, 256, 288, (g, w, h) => {
const gr = g.createLinearGradient(0, 0, 0, h); gr.addColorStop(0, '#1d1d20');
g.strokeStyle = '#8d8a84'; g.lineWidth = 4; g.strokeRect(4, 4, w - 8, h - 8);
// vents
for (const vx of [20, 146]) { g.fillStyle = '#060607'; g.fillRect(vx, 14, 90,
// radio: display + knobs + buttons
g.fillStyle = '#0a0c0c'; g.fillRect(24, 74, 208, 104);
const lc = g.createLinearGradient(0, 88, 0, 128); lc.addColorStop(0, '#6fd3ff
g.fillStyle = '#062033'; g.font = 'bold 20px Arial'; g.textAlign = 'center';
for (const kx of [44, 212]) { g.fillStyle = '#2a2a2d'; g.beginPath(); g.arc(k
for (let i = 0; i < 6; i++) { g.fillStyle = '#3a3a3e'; g.fillRect(34 + i * 32
// climate dials
for (const [kx, col] of [[56, '#3d7dff'], [128, '#bbbbbb'], [200, '#ff4b3d']])
g.fillStyle = '#222226'; g.beginPath(); g.arc(kx, 230, 30, 0, 7); g.fill();
g.fillStyle = '#e8e3da'; g.fillRect(kx - 2, 206, 4, 16);
}
});
}

function lampGlow(color, r) {
const mb = new MB(); mb.rotate(0, 1, 0, -Math.PI / 2).discX(0, r, 20, false);
const n = new Node('lamp_glow'); n.mesh = { prims: [mb.prim(new Material({ name
return n;
}

return { MB, steelWheel, buildInterior, steeringWheelPrims, clusterCanvas, plateN
})();
__M['tune'] = (() => {
// Car customisation helpers: split the body into named parts (for per-part paint

```

```

// and CPU ray-picking against the car body (for the spray can).
const { V3, Q } = __M['math'];
const { Node, Material } = __M['scene'];

const PART_NAMES = { bumperF: 'المدام الأمامي', bumperR: 'المدام الخلفي', hood: 'ت'
const PICKUPS = { hilux: 1, hilux2: 1, shas: 1 };

function cloneMat(m) { const c = Object.assign(Object.create(Object.getPrototypeOf
// classify every body triangle and rebuild the body prims as one sub-prim per (m
// original vertex buffers so dents still work
function splitParts(car, key) {
  key = String(key || '').replace(/_(net|bot)$/, '');
  if (car.parts) return car.parts;
  const d = car.def, body = car.bodyNode || car.model; const parts = {};
  const topAt = (z) => { const P = d.profile || []; let y = 1.4; for (let i = 0;
  const wz = car.wheels.map(w => w.pos.z), R = car.wheelRadius; const maxTop = Ma
  const bedZ = d.interior && d.interior.rearWall !== undefined ? d.interior.rearW
  const classify = (x, y, z, m) => {
    if (m && m.glass) return 'glass';
    if (m && (m.tail || m.head || /lamp|light|tail/i.test(m.name || ''))) return
    const top = topAt(z);
    if (z > d.zMax - 0.34 && y < (d.bumperY ?? d.lowY + 0.42)) return 'bumperF';
    if (z < d.zMin + 0.3 && y < d.lowY + 0.5) return 'bumperR';
    if (PICKUPS[key] && z < bedZ && y > d.lowY + 0.05) return 'bed';
    if (top > maxTop - 0.12 && y > top - 0.13 && Math.abs(x) < d.halfWidth - 0.12
    if (Math.abs(x) > d.halfWidth - 0.24 && wz.some(q => Math.abs(z - q) < R * 1.
    if (z > d.zMax * 0.42 && y > d.lowY + 0.2 && Math.abs(x) < d.halfWidth - 0.05
    return x > 0 ? 'sideL' : 'sideR';
  };
  body.traverse(n => {
    if (!n.mesh || n === car.doorNode || n === car.doorNodeR || /^wheel/.test(n.n
    let isBody = n === body; for (let p = n.parent; p && !isBody; p = p.parent) i
    if (!isBody && n !== body) return;
    const out = [];
    for (const p of n.mesh.prims) {
      if (!p.index || p.shadowOnly || !p.attrs.position) { out.push(p); continue;
      const P = p.attrs.position.data, I = p.index, groups = {};
      for (let i = 0; i < I.length; i += 3) {
        const a = I[i] * 3, b = I[i + 1] * 3, c = I[i + 2] * 3;
        const k = classify((P[a] + P[b] + P[c]) / 3, (P[a + 1] + P[b + 1] + P[c +
          (groups[k] || (groups[k] = [])).push(I[i], I[i + 1], I[i + 2]));
      }
      const keys = Object.keys(groups);
      if (keys.length === 1) { const k = keys[0]; p.part = k; if (p.material && p
      for (const k of keys) {

```

```

        const idx = I instanceof Uint32Array ? new Uint32Array(groups[k]) : new U
        const sp = { attrs: p.attrs, index: idx, material: p.material && p.materi
        (parts[k] || (parts[k] = [])).push(sp); out.push(sp);
    }
}
n.mesh = { ...n.mesh, prims: out };
});
for (const [k, node] of [['door_l', car.doorNode], ['door_r', car.doorNodeR]])
    node.traverse(n => { if (!n.mesh) return; n.mesh = { ...n.mesh, prims: n.mesh
    }
    car.parts = parts; return parts;
}

// paint: null = original colours; color = whole car; per-part map { part: [r,g,b
function applyPaint(car, whole, perPart) {
    const parts = car.parts || {};
    for (const [k, list] of Object.entries(parts)) for (const p of list) {
        const m = p.material; if (!m || !m.carPaint) continue;
        if (!m._orig) m._orig = { color: m.color.slice(), map: m.map };
        const c = (perPart && perPart[k]) || whole;
        if (c) { const k = m._orig.map ? 1.3 : 1; m.color = [Math.min(2, c[0] * k), M
        else { m.color = m._orig.color.slice(); m.map = m._orig.map; }
    }
}

function applyHidden(car, hide) {
    const parts = car.parts || {};
    for (const [k, list] of Object.entries(parts)) for (const p of list) p.hidden =
}

// ray (world) -> first hit on the car body/doors, in car-local space
function pickCar(car, ro, rd) {
    const o = car.worldToLocal(ro.clone(), new V3()); const e = car.worldToLocal(ro
    let best = 1e9, bn = null;
    const test = (P, I, off) => {
        for (let i = 0; i < I.length; i += 3) {
            const a = I[i] * 3, b = I[i + 1] * 3, c = I[i + 2] * 3;
            const ax = P[a] + off.x, ay = P[a + 1] + off.y, az = P[a + 2] + off.z;
            const elx = P[b] + off.x - ax, ely = P[b + 1] + off.y - ay, elz = P[b + 2]
            const px = dir.y * e2z - dir.z * e2y, py = dir.z * e2x - dir.x * e2z, pz =
            const det = elx * px + ely * py + elz * pz; if (Math.abs(det) < 1e-9) conti
            const tx = o.x - ax, ty = o.y - ay, tz = o.z - az; const u = (tx * px + ty
            const qx = ty * elz - tz * ely, qy = tz * elx - tx * elz, qz = tx * ely - t
            const t = (e2x * qx + e2y * qy + e2z * qz) * id; if (t > 0.01 && t < best)
        }
    };
    for (const list of Object.values(car.parts || {})) for (const p of list) { if (

```

```

    return bn ? { p: o.addS(dir, best), n: bn } : null;
}

// a spray dot stuck on the paint
function sprayDot(car, hit, color, size) {
    const n = hit.n, P = [], N = [], U = [], I = [];
    const t = Math.abs(n.y) < 0.9 ? new V3(0, 1, 0) : new V3(1, 0, 0); const a = ne
    const c = hit.p.clone().addS(n, 0.004); const seg = 14;
    P.push(c.x, c.y, c.z); N.push(n.x, n.y, n.z); U.push(0.5, 0.5);
    for (let i = 0; i <= seg; i++) { const g = i / seg * Math.PI * 2, r = size * (0
    if (!car.sprayNode) { car.sprayNode = new Node('spray'); car.sprayNode.castShad
    const key = color.join(','); const mat = car.sprayMats[key] || (car.sprayMats[k
    car.sprayNode.mesh.prims.push({ attrs: { position: { data: new Float32Array(P),
    }
function clearSpray(car) { if (car.sprayNode) car.sprayNode.mesh.prims.length = 0

return { PART_NAMES, splitParts, applyPaint, applyHidden, pickCar, sprayDot, clea
})();
__M['car'] = (() => {
// Vehicle: raycast suspension + slip-based tyre model + sphere body collisions +
const { V3, Q, M4, clamp, lerp } = __M['math'];
const { RigidBody } = __M['physics'];
const { Node, Material } = __M['scene'];
const { computeBounds } = __M['gltf'];
const { splitParts, applyPaint, applyHidden } = __M['tune'];
const { steelWheel, buildInterior, plateNode, badgeNode, steeringWheelPrims, buil

// quick definition for cars converted automatically from the user's model files
function autoDef(o) {
    const L = o.len / 2, H = o.h, fa = o.fa, ra = o.ra, truck = !!o.truck;
    const W = o.steer || o.wheel; const sz = W ? W[2] : fa - 1.05, sy = W ? W[1] :
    return {
        name: o.name, file: o.file, mass: o.mass, com: [0, truck ? 0.72 : 0.52, 0.1],
        profile: [[L, H * (truck ? 0.62 : 0.55)], [fa - 0.1, H * (truck ? 0.66 : 0.62
        halfWidth: o.w / 2 - 0.04, zMin: -L, zMax: L, sphereR: truck ? 0.42 : 0.37, l
        susTop: truck ? 0.32 : 0.26, travel: truck ? 0.14 : 0.11, zeta: 0.33, tyreMu:
        torque: o.torque, redline: 6000, gears: [3.5, 2.1, 1.45, 1.08, 0.86, 0.72], f
        maxSteer: 0.6, brake: 14000, dragCdA: truck ? 1.1 : 0.7, engineVoice: { cyl:
        ...(o.interior ? { interior: o.interior } : { ownInterior: true, seat: o.seat
        ...(o.steer || o.interior ? {} : { wheelPos: { x: sx, y: sy, z: sz, tilt: o.w
        wheelStyle: o.wheelStyle || 'hilux', plates: [{ front: true, x: 0, y: truck ?
        spec: o.spec, auto: true,
    };
}

const CAR_DEFS = {

```

```

shas: {
  name: 'شاص', file: 'shas.glb', mass: 2250, com: [0, 0.68, 0.1],
  profile: [[2.25, 1.28], [0.9, 1.3], [0.75, 1.93], [-0.4, 1.93], [-0.55, 1.88]
  halfWidth: 0.9, zMin: -2.84, zMax: 2.34, sphereR: 0.42, lowY: 0.74,
  susTop: 0.32, travel: 0.14, zeta: 0.3, tyreMu: 1.1, rearMu: 1.0,
  torque: 360, redline: 5800, gears: [3.9, 2.3, 1.55, 1.12, 0.86], final: 3.9,
  maxSteer: 0.62, brake: 15000, dragCdA: 1.15, engineVoice: { cyl: 6, base: 0.9
  procWheels: { rim: '#e9e6df', width: 0.265 }, driverScale: 0.93,
  interior: { floorY: 0.68, seatTop: 0.95, seatZ: 0.02, dashFront: 0.96, dashTo
  plates: [{ front: true, x: 0, y: 0.66 }, { front: false, x: 0, y: 0.6 }],
  spec: 'غمارة · ٧٩ لاندكروزر', wheelStyle: 'lc', cluster: 'lc', rearBadge: { te
},
rub: {
  name: 'ربع', file: 'rub_game.glb', mass: 1950, com: [0.02, 0.66, 0.05],
  profile: [[1.95, 1.2], [0.95, 1.25], [0.85, 1.9], [-2.0, 1.9]],
  halfWidth: 0.86, zMin: -2.05, zMax: 2.0, sphereR: 0.42, lowY: 0.72,
  susTop: 0.32, travel: 0.14, zeta: 0.28, tyreMu: 1.1, rearMu: 0.98,
  torque: 350, redline: 6000, gears: [3.7, 2.2, 1.5, 1.1, 0.85], final: 3.9, re
  maxSteer: 0.64, brake: 14000, dragCdA: 1.05, engineVoice: { cyl: 6, base: 1.0
  procWheels: { rim: '#b9bbbd', width: 0.25 }, driverScale: 0.93,
  interior: { floorY: 0.62, seatTop: 0.9, seatZ: -0.41, dashFront: 0.53, dashTo
  plates: [{ front: true, x: 0, y: 0.64 }, { front: false, x: 0.385, y: 0.96 }]
  headY: 0.97, tailY: 0.95,
  spec: 'ربع · ٧٦ لاندكروزر', wheelStyle: 'lc', cluster: 'lc', hideMats: /plate/
},
elantra: {
  name: 'النترا', file: 'elantra.glb', mass: 1420, com: [0, 0.5, 0.2],
  profile: [[2.2, 0.95], [1.0, 1.05], [0.6, 1.38], [-1.2, 1.38], [-2.35, 1.0]],
  halfWidth: 0.9, zMin: -2.4, zMax: 2.25, sphereR: 0.38, lowY: 0.55,
  susTop: 0.26, travel: 0.1, zeta: 0.35, tyreMu: 1.18, rearMu: 1.06,
  torque: 420, redline: 6800, gears: [3.4, 2.1, 1.5, 1.15, 0.92, 0.75], final:
  maxSteer: 0.58, brake: 12000, dragCdA: 0.72, engineVoice: { cyl: 4, base: 1.1
  hideMats: /Interior/, driverScale: 0.92, recline: -0.12,
  interior: { floorY: 0.18, seatTop: 0.43, seatZ: -0.12, dashFront: 0.97, dashT
  plates: [{ front: true, x: 0, y: 0.44 }, { front: false, x: 0, y: 0.53 }],
  spec: '٢٠٢٤ هيونداى النترا', wheelStyle: 'elantra',
},
hilux: {
  name: 'هايلكس', file: 'hilux.glb', mass: 1950, com: [0, 0.72, 0.15],
  profile: [[2.5, 1.2], [1.05, 1.25], [0.56, 1.74], [-0.2, 1.74], [-0.3, 1.3],
  halfWidth: 0.9, zMin: -2.7, zMax: 2.52, sphereR: 0.42, lowY: 0.74,
  susTop: 0.3, travel: 0.14, zeta: 0.3, tyreMu: 1.1, rearMu: 1.0,
  torque: 380, redline: 5200, gears: [3.6, 2.1, 1.4, 1.0, 0.8, 0.66], final: 3.
  maxSteer: 0.62, brake: 15000, dragCdA: 1.05, engineVoice: { cyl: 4, base: 0.8
  interior: { floorY: 0.62, seatTop: 0.84, seatZ: 0.1, dashFront: 0.98, dashTop
  plates: [{ front: true, x: 0, y: 0.55 }, { front: false, x: 0, y: 0.58 }],

```

```

    defaultParts: { bumperF: [0.035, 0.035, 0.04] }, bumperY: 0.72, spec: 'رة بخط'
},
changan: {
    name: 'شانجان', file: 'changan.glb', mass: 1250, com: [0, 0.5, 0.2],
    profile: [[2.1, 0.92], [1.0, 1.0], [0.3, 1.45], [-0.9, 1.5], [-1.7, 1.05], [-halfWidth: 0.87, zMin: -2.3, zMax: 2.1, sphereR: 0.36, lowY: 0.5,
    susTop: 0.26, travel: 0.11, zeta: 0.35, tyreMu: 1.12, rearMu: 1.02,
    torque: 235, redline: 6300, gears: [3.6, 2.1, 1.45, 1.05, 0.84, 0.7], final:
    maxSteer: 0.6, brake: 11500, dragCdA: 0.68, engineVoice: { cyl: 4, base: 1.1
    interior: { floorY: 0.3, seatTop: 0.56, seatZ: -0.25, dashFront: 0.8, dashTop
    plates: [{ front: true, x: 0, y: 0.5 }, { front: false, x: 0, y: 0.8 }],
    spec: 'شانجان V7 السفن', wheelStyle: 'changan', cluster: 'changan',
},
hilux2: {
    name: 'غمارتين', file: 'hilux2.glb', mass: 2080, com: [0, 0.74, 0.05],
    profile: [[2.5, 1.2], [1.05, 1.25], [0.56, 1.74], [-0.95, 1.74], [-1.0, 1.3],
    halfWidth: 0.9, zMin: -2.7, zMax: 2.52, sphereR: 0.42, lowY: 0.74,
    susTop: 0.3, travel: 0.14, zeta: 0.3, tyreMu: 1.1, rearMu: 1.0,
    torque: 380, redline: 5200, gears: [3.6, 2.1, 1.4, 1.0, 0.8, 0.66], final: 3.
    maxSteer: 0.62, brake: 15000, dragCdA: 1.08, engineVoice: { cyl: 4, base: 0.8
    interior: { floorY: 0.62, seatTop: 0.84, seatZ: 0.1, dashFront: 0.98, dashTop
    plates: [{ front: true, x: 0, y: 0.55 }, { front: false, x: 0, y: 0.58 }],
    defaultParts: { bumperF: [0.035, 0.035, 0.04] }, bumperY: 0.72, spec: 'مارتين'
},
camry: {
    name: 'كامري', file: 'camry.glb', mass: 1560, com: [0, 0.5, 0.25],
    profile: [[2.4, 0.95], [0.95, 1.05], [0.3, 1.42], [-0.9, 1.42], [-1.8, 1.08],
    halfWidth: 0.9, zMin: -2.5, zMax: 2.4, sphereR: 0.37, lowY: 0.52,
    susTop: 0.26, travel: 0.11, zeta: 0.35, tyreMu: 1.15, rearMu: 1.04,
    torque: 250, redline: 6600, gears: [3.3, 2.0, 1.4, 1.05, 0.85, 0.7], final: 3
    maxSteer: 0.58, brake: 12500, dragCdA: 0.66, engineVoice: { cyl: 4, base: 1.0
    ownInterior: true, seat: { x: 0.37, top: 0.52, z: -0.12 }, headroomY: 1.36, r
    plates: [{ front: true, x: 0, y: 0.42 }, { front: false, x: 0, y: 0.72 }],
    spec: 'تويوتا كامري ٢٠٢٥ داخلية أصلية', wheelStyle: 'camry', cluster: 'camr
},
tahoe: {
    name: 'تاهو', file: 'tahoe.glb', mass: 2550, com: [0, 0.78, 0.05],
    profile: [[2.49, 1.12], [1.45, 1.25], [0.72, 1.95], [-2.5, 1.97], [-2.64, 1.2
    halfWidth: 1.0, zMin: -2.64, zMax: 2.49, sphereR: 0.45, lowY: 0.72,
    susTop: 0.33, travel: 0.15, zeta: 0.32, tyreMu: 1.1, rearMu: 1.0,
    torque: 470, redline: 5800, gears: [3.1, 1.63, 1.0, 0.7], final: 3.42, rearBi
    maxSteer: 0.6, brake: 16000, dragCdA: 1.2, engineVoice: { cyl: 8, base: 0.8 }
    ownInterior: true, seat: { x: 0.42, top: 0.95, z: -0.22 }, headroomY: 1.7, cl
    decor: { cluster: { pos: [0.415, 1.345, 0.825], w: 0.4, h: 0.13, tilt: 0.3 },
    headY: 0.95, tailY: 1.05,
    plates: [{ front: true, x: 0, y: 0.6 }, { front: false, x: 0, y: 0.95 }],

```

```

    spec: 'شفروليه تاهو ٢٠٠٧ · أسود · داخلية أصلية',
  },
  yukon: {
    name: 'جمس ٢٠٠٠', file: 'yukon.glb', mass: 2650, com: [0, 0.76, 0.0],
    profile: [[2.56, 1.1], [1.45, 1.22], [0.8, 1.88], [-2.85, 1.9], [-2.95, 1.2]],
    halfWidth: 1.0, zMin: -2.95, zMax: 2.56, sphereR: 0.45, lowY: 0.7,
    susTop: 0.33, travel: 0.15, zeta: 0.32, tyreMu: 1.1, rearMu: 1.0,
    torque: 440, redline: 5600, gears: [3.06, 1.63, 1.0, 0.7], final: 3.73, rearB
    maxSteer: 0.58, brake: 16000, dragCdA: 1.25, engineVoice: { cyl: 8, base: 0.7
    ownInterior: true, seat: { x: 0.39, top: 0.74, z: -0.05 }, headroomY: 1.66,
    decor: { cluster: { pos: [0.386, 1.15, 0.815], w: 0.38, h: 0.12, tilt: 0.15 }
    headY: 0.95, tailY: 1.0,
    plates: [{ front: true, x: 0, y: 0.55 }, { front: false, x: 0, y: 0.95 }],
    spec: '٢٠٠٠ · داخلية أصلية XL جمس يوكن',
  },
  datsun: autoDef({ name: 'داتسن · غمارة', file: 'datsun.glb', spec: 'نيسان داتسن · غمارة',
  gmcold: autoDef({ name: 'جمس قديم', file: 'gmcold.glb', spec: 'جمس بهيهاني', le
  caprice: autoDef({ name: 'كابرس', file: 'caprice.glb', spec: '٢٠١٣ فروليه كابرس
  hilux07: autoDef({ name: 'هايلكس ٢٠٠٧', file: 'hilux07.glb', spec: '٢٠٠٧ هايلكس
  camry20: autoDef({ name: 'كامري ٢٠٢٠', file: 'camry20.glb', spec: '٢٠٢٠ كامري
  avanza: {
    name: 'أفانزا', file: 'avanza.glb', mass: 1150, com: [0, 0.62, 0.1],
    profile: [[2.0, 1.0], [1.1, 1.1], [0.6, 1.64], [-2.05, 1.64], [-2.15, 1.2]],
    halfWidth: 0.84, zMin: -2.15, zMax: 2.0, sphereR: 0.38, lowY: 0.6,
    susTop: 0.28, travel: 0.13, zeta: 0.3, tyreMu: 1.05, rearMu: 0.97,
    torque: 150, redline: 6200, gears: [3.8, 2.2, 1.45, 1.0, 0.8], final: 4.3, re
    maxSteer: 0.62, brake: 10500, dragCdA: 0.85, engineVoice: { cyl: 4, base: 1.0
    interior: { floorY: 0.42, seatTop: 0.72, seatZ: -0.05, dashFront: 0.95, dashT
    plates: [{ front: true, x: 0, y: 0.45 }, { front: false, x: 0, y: 0.62 }],
    spec: '٢٠١٥ تويوتا أفانزا', wheelStyle: 'avanza', cluster: 'avanza', rearBadg
  },
};

const UP = new V3(0, 1, 0);
const _a = new V3(), _b = new V3(), _c = new V3(), _d = new V3(), _e = new V3(),
const _hit = {};

class Car {
  constructor(def, gltf, world) {
    this.def = def; this.world = world;
    this.node = new Node('car_' + def.name);
    const src = gltf.root.children[0];
    this.model = src; this.node.add(src);
    const meta = src.userData && src.userData.wheels ? src.userData : gltf.json.e
    this.meta = meta;
    this.wheelRadius = meta.wheelRadius;

```

```

// body parts
this.bodyNode = src.find('body');
this.doorNode = src.find('door_l'); this.doorNodeR = src.find('door_r'); this
const shadow = src.find('shadow_proxy');
if (shadow) { shadow.mesh.prims.forEach(p => { p.shadowOnly = true; }); this.
// car paint materials: doubleSided dark interior for the cabin shell
this.node.traverse(n => { if (n.mesh) for (const p of n.mesh.prims) { const m
this.doorAngle = 0; this.doorTarget = 0;
this.doorHinge = this.doorNode ? this.doorNode.pos.clone() : new V3(0.9, 1.1,
if (this.doorNode) {
    // door bounds in local space (relative to hinge)
    let zmin = 0, ymin = 1e9, ymax = -1e9;
    for (const p of this.doorNode.mesh.prims) { zmin = Math.min(zmin, p.min[2])
    this.doorLen = -zmin; this.doorYmin = ymin; this.doorYmax = ymax;
}
if (def.hideMats) src.traverse(n => { if (n.mesh) n.mesh = { ...n.mesh, prims
this.addInterior();

// wheels
this.wheels = meta.wheels.map(w => {
    let node = src.find(w.name);
    const pos = new V3(...w.pos);
    if (def.procWheels && node) {
        // replace the scanned wheel with a clean high-detail steel wheel
        node.mesh = null; node.children.length = 0;
        const pw = steelWheel(meta.wheelRadius * 0.985, def.procWheels.width, def
        node.add(pw);
    }
    return {
        node, pos, front: w.front, side: w.side, steer: 0, spin: 0, spinVel: 0, s
        onGround: false, slip: 0, slipLong: 0, load: 0, contact: new V3(), normal
        skid: 0, brk: 0, sink: 0, psi: 30, dmgOff: new V3(), camber: 0, toe: 0, d
    };
});
const m = def.mass;
const L = def.zMax - def.zMin, W = def.halfWidth * 2, H = 1.6;
const inertia = new V3(m * (H * H + L * L) / 12, m * (W * W + L * L) / 12 * 1
this.body = new RigidBody(m, inertia);
this.com = new V3(...def.com);
this.k = m * 9.81 / 4 / def.travel;
this.c = 2 * def.zeta * Math.sqrt(this.k * m / 4);
this.buildSpheres();
// drivetrain
this.gear = 1; this.rpm = 900; this.reverse = false; this.shiftTimer = 0; thi
this.throttle = 0; this.brakeIn = 0; this.steerIn = 0; this.handbrake = false
this.speed = 0; this.forwardSpeed = 0; this.driftAngle = 0; this.impacts = []

```

```

this.engineOn = true;
this.scrape = 0; this.airTime = 0;
this.collectDeformables();
this.steerBias = 0; this.engineHealth = 1;
this.underbody = buildUnderbody(def, meta); this.underbody.visible = false; t
}

addPlates(plateGltf) {
  if (!plateGltf || !this.def.plates) return;
  for (const pd of this.def.plates) {
    // find the bumper surface at the plate position
    let best = pd.front ? -1e9 : 1e9;
    const scan = (n, off) => { if (!n || !n.mesh) return; for (const p of n.mesh)
    scan(this.bodyNode, new V3());
    if (!isFinite(best) || Math.abs(best) > 5) best = pd.front ? this.def.zMax
    const n = plateNode(plateGltf, pd.front);
    n.pos.set(pd.x, pd.y, best + (pd.front ? 0.012 : -0.012));
    this.model.add(n);
    (this.plates || (this.plates = [])).push({ node: n, home: n.pos.clone(), ro
  }
  // lamp positions (for night mode): scan the body surface at the usual lamp h
  const d = this.def; this.lamps = [];
  const surf = (x, y, front) => { let best = front ? -1e9 : 1e9; this.bodyNode.
  const hy = d.headY ?? d.lowY + 0.2, ty = d.tailY ?? d.lowY + 0.3;
  for (const sx of [1, -1]) {
    const hx = sx * (d.halfWidth - 0.27), hz = surf(hx, hy, true);
    const g = lampGlow([1, 0.97, 0.9, 1], 0.075); g.pos.set(hx, hy, hz + 0.01);
    const tx = sx * (d.halfWidth - 0.14), tz = surf(tx, ty, false);
    const t = lampGlow([1, 0.05, 0.03, 1], 0.06); t.rot.setAxisAngle(UP, Math.P
  }
  const rb = this.def.rearBadge;
  if (rb) {
    let best = 1e9;
    this.bodyNode.traverse(nn => { if (!nn.mesh) return; for (const p of nn.mesh)
    if (!isFinite(best) || best > 0) best = this.def.zMin;
    const n = badgeNode(rb.text, rb.w);
    n.pos.set(0, rb.y, best - 0.006);
    this.model.add(n);
    this.plates.push({ node: n, home: n.pos.clone(), rot: n.rot.clone(), off: n
  }
}

// z of the windscreen glass at height y (centre of the car), so the rear-view
windscreenZ(y) {
  let best = -1e9; const src = this.bodyNode || this.model;
  src.traverse(n => { if (!n.mesh) return; for (const p of n.mesh.prims) { if (

```

```

    return best > -1e8 ? best - 0.07 : undefined;
}

// lowest roof-skin height above the cabin at a given z (scan cars have no inne
roofY(z, minY) {
    let best = 1e9; const src = this.bodyNode || this.model;
    src.traverse(n => { if (!n.mesh) return; for (const p of n.mesh.prims) { if (
    return best < 1e8 ? best : undefined;
}

// night: the car's OWN lamp geometry lights up (head-lamp lenses white, tail l
// are only a fallback for scanned cars whose lamps aren't separate materials
lampMats() {
    if (this._lampMats) return this._lampMats;
    const head = new Set(), tail = new Set();
    this.model.traverse(n => { if (!n.mesh) return; for (const p of n.mesh.prims)
        const nm = (m.name || '').toLowerCase();
        if (m.tail || /tail|brake|red_light|rglass|eglass/.test(nm)) tail.add(m);
        else if (m.head || /head|_lamp$_|lamp_0|lights_16|glass1\.001|rvglass/.test(
        for (const m of [...head, ...tail]) m._emis0 = m.emissive.slice();
    return (this._lampMats = { head: [...head], tail: [...tail] });
}

setNight(on) {
    this.night = on; const L = this.lampMats(), own = L.head.length > 0;
    for (const l of this.lamps || []) l.node.visible = on && (!own || !l.head) &&
    for (const m of L.head) m.emissive = on ? [2.2, 2.1, 1.9] : m._emis0;
    this.updateTailGlow();
}

updateTailGlow() {
    const L = this._lampMats; if (!L) return; const b = this.brakeIn || 0;
    for (const m of L.tail) m.emissive = this.night ? [0.9 + 1.6 * b, 0.02, 0.01]
}

// world-space lights this car casts at night: two headlight spots + a red glow
pushLights(out) {
    this.updateTailGlow();
    const f = this.forward(_n).clone(); const down = this.up(new V3()).scale(-0.1
    for (const l of this.lamps || []) {
        const p = this.localToWorld(l.local, new V3());
        if (l.head) out.push({ pos: p, range: 55, dir, cos: 0.86, color: [7, 6.6, 5
        }
    }
    const rear = this.localToWorld(new V3(0, 0.35, this.def.zMin - 1.1), new V3()
    out.push({ pos: rear, range: 3.2, color: [0.35 + 0.9 * (this.brakeIn || 0), 0
}

// live gauges and a centre stack on models whose own dash is a blank shell
addDecor(D) {

```

```

const node = new Node('decor'); const prims = [];
const panel = (mat, c, w, h, tilt) => { const mb = new MB(); mb.push().transl
  const a = mb.v(w / 2, -h / 2, 0, 0, 0, -1, 0, 1), b = mb.v(-w / 2, -h / 2,
const bezel = new Material({ name: 'decor_bezel', color: [0.04, 0.04, 0.045,
if (D.cluster) { const C = D.cluster; this.cluster = clusterCanvas(C.style ||
  const bz = new MB(); bz.push().translate(C.pos[0], C.pos[1], C.pos[2] + 0.0
  panel(new Material({ name: 'cluster', map: this.cluster.tex, unlit: true, d
if (D.stack) { const C = D.stack; panel(new Material({ name: 'stack', map: st
node.mesh = { prims }; node.castShadow = false; this.model.add(node);
}

addInterior() {
  const d = this.def;
  if (d.downInterior) {
    this.seatPos = new V3(d.seat.x, d.seat.top, d.seat.z);
    const ms = this.meta.steer, sn = this.model.find('steering');
    if (ms && sn) { // the model's own steering wheel, spun about its column
      sn.rot.setAxisAngle(new V3(1, 0, 0), ms.tilt); this.steeringNode = sn; th
      this.steerCenter = new V3(...ms.center); this.steerTilt = ms.tilt; sn.cas
      { let r = 0; for (const p of sn.mesh ? sn.mesh.prims : []) { const a = p.
      if (d.decor) this.addDecor(d.decor);
      return;
    }
    this.steerCenter = new V3(d.wheelPos.x, d.wheelPos.y, d.wheelPos.z); this.s
    if (d.swapWheel) {
      // cut the model's fixed steering wheel out of the body and put a real, t
      const c = this.steerCenter, tq = new Q().setAxisAngle(new V3(1, 0, 0), th
      this.bodyNode.traverse(n => { if (!n.mesh) return; n.mesh = { ...n.mesh,
      const P = p.attrs.position.data, I = p.index, keep = [];
      for (let i = 0; i < I.length; i += 3) {
        let x = 0, y = 0, z = 0; for (let k = 0; k < 3; k++) { const j = I[i
        x = x / 3 - c.x; y = y / 3 - c.y; z = z / 3 - c.z;
        const dn = x * nrm.x + y * nrm.y + z * nrm.z, r2 = x * x + y * y + z
        if (!(Math.abs(dn) < d.swapWheel.depth && r2 < d.swapWheel.r * d.swap
      }
      if (keep.length === I.length) return p;
      const np = { ...p, index: I instanceof Uint32Array ? new Uint32Array(ke
      }) }; });
      const sw = new Node('steering'); sw.pos.copy(c); sw.rot.copy(tq); sw.mesh
      this.model.add(sw); this.steeringNode = sw; this.steerBase = sw.rot.clone
    }
    return;
  }
}
const I = d.interior; const h = this.doorHinge;
const xs = Math.min(Math.abs(h.x) - 0.42, I.halfW - 0.4);
// keep the headliner, sun visors and mirror under the real roof skin (they u

```

```

const minY = I.seatTop + 0.68, rMid = this.roofY((I.linerZ[0] + I.linerZ[1])
const linerY = rMid !== undefined ? Math.min(I.linerY, rMid - 0.04) : I.liner
const wsTop = this.windscreenZ(linerY - 0.03);
let zf = wsTop !== undefined ? Math.min(I.linerZ[1], wsTop - 0.02) : I.linerZ
for (let k = 0; k < 30; k++) { const r = this.roofY(zf, minY); if (r === unde
const linerZ = [I.linerZ[0], zf];
const mz = this.windscreenZ(linerY - 0.12), rM = mz !== undefined ? this.roof
const mirrorY = Math.min(linerY - 0.12, rM !== undefined ? rM - 0.07 : 1e9);
// the door cards must fit inside the real door skin (else they poke out of t
let doorBox = null;
if (this.doorNode) { let z0 = 1e9, z1 = -1e9, xi = 1e9; const dp = this.doorN
  this.doorNode.traverse(n => { if (!n.mesh) return; for (const p of n.mesh.p
    if (z1 > z0) doorBox = { z0: z0 + 0.04, z1: z1 - 0.04, x: xi - 0.025 }; }
const node = buildInterior({ ...I, linerY, linerZ, xs, cluster: d.cluster, wh
this.cluster = node.userData.cluster;
this.model.add(node);
// hang the door cards on the doors
const dcs = node.userData.doorCards || {};
for (const [side, door] of [['l', this.doorNode], ['r', this.doorNodeR]]) {
  const dc = dcs[side]; if (!dc || !door) continue;
  node.remove(dc); dc.pos.set(-door.pos.x, -door.pos.y, -door.pos.z); door.ad
}
this.steeringNode = node.userData.steering; this.steerBase = this.steeringNod
this.steerCenter = node.userData.wheelCenter; this.steerTilt = node.userData.
this.seatPos = new V3(xs, I.seatTop, I.seatZ);
}

buildSpheres() {
  const d = this.def, r = d.sphereR, spheres = [];
  const topAt = (z) => { const p = d.profile; for (let i = 0; i < p.length - 1;
  const n = Math.ceil((d.zMax - d.zMin - 2 * r) / 0.7);
  for (let i = 0; i <= n; i++) {
    const z = d.zMax - r - (d.zMax - d.zMin - 2 * r) * i / n;
    const top = topAt(z);
    for (const sx of [-1, 1]) {
      const x = sx * (d.halfWidth - r);
      spheres.push(new V3(x, d.lowY, z));
      const yTop = top - r; if (yTop > d.lowY + 0.3) spheres.push(new V3(x, yTo
      if (yTop > d.lowY + 0.75) spheres.push(new V3(x, (yTop + d.lowY) / 2, z))
    }
  }
}
// relative to COM
this.spheres = spheres.map(s => { const p = s.sub(this.com); p.r = r; return
// tyres are solid too (walls, kerbs, rolling onto the side)
this.meta.wheels.forEach((w, i) => { const p = new V3(...w.pos).sub(this.com)
}

```

```

collectDeformables() {
  this.deform = [];
  const add = (node) => { if (!node || !node.mesh) return; for (const p of node
  if (this.bodyNode) add(this.bodyNode);
  if (this.doorNode) add(this.doorNode);
  if (this.doorNodeR) add(this.doorNodeR);
}

place(x, y, z, yaw) {
  const b = this.body;
  b.q.setAxisAngle(UP, yaw);
  b.pos.copy(this.com).applyQ(b.q).add(_a.set(x, y, z));
  b.vel.set(0, 0, 0); b.ang.set(0, 0, 0); b.updateInertia();
  for (const w of this.wheels) { w.springLen = w.prevLen = this.def.susTop; w.s
  this.gear = 1; this.reverse = false; this.rpm = 900;
  this.syncNode();
}

// local (model) point -> world
localToWorld(l, out) { return out.copy(l).sub(this.com).applyQ(this.body.q).add
worldToLocal(wp, out) { return out.copy(wp).sub(this.body.pos).applyInvQ(this.b
forward(out) { return out.set(0, 0, 1).applyQ(this.body.q); }
up(out) { return out.set(0, 1, 0).applyQ(this.body.q); }

// tuning: engine cylinders, drive type, low range (دبل), diff locks, stance, p
applyTune(T) {
  const d = this.def; this.tune = T = T || {};
  const base = d.cyl || (d.engineVoice && d.engineVoice.cyl) || 4;
  const cyl = T.cyl || base; this.tuneTorque = (cyl / base) ** 0.85 * (T.turbo
  this.tuneRear = T.drive === 'fwd' ? 0.02 : T.drive === 'rwd' ? 0.98 : T.drive
  if (T.dbl === 'low' || T.dbl === 'high') { if (this.tuneRear === undefined ||
  this.tuneDiff = T.diff === 'rear' ? [false, true] : T.diff === 'both' ? [true
  for (const w of this.wheels) { const h = (w.front ? T.hf : T.hr) || 0; w.ride
  this.voice = { ...(d.engineVoice || {}), cyl, base: (d.engineVoice ? d.engine
  const dp = d.defaultParts && !T.color ? { ...d.defaultParts, ...(T.parts || {
  if (T.color || dp || T.hide || this.parts) { splitParts(this, this.key); appl
}
torqueAt(rpm) {
  const x = rpm / this.def.redline;
  const shape = x < 0.55 ? 0.62 + 0.38 * (x / 0.55) : 1 - Math.max(0, (x - 0.55
  return this.def.torque * clamp(shape, 0.3, 1);
}

step(dt, input) {
  const d = this.def, b = this.body;

```

```

this.throttle = input.throttle; this.brakeIn = input.brake; this.handbrake =
// steering smoothing & speed sensitivity
const fwd = this.forward(_a); const vF = b.vel.dot(fwd);
this.forwardSpeed = vF; this.speed = b.vel.len();
const maxSteer = d.maxSteer * lerp(1, 0.55, clamp((Math.abs(vF) - 10) / 35, 0
// allow counter-steer freedom while drifting
const target = clamp(input.steer * (Math.abs(this.driftAngle) > 0.25 ? d.maxS
this.steer += (target - this.steer) * (1 - Math.exp(-dt * 8)));

// gear logic (automatic with reverse)
let driveCmd = 0, brakeCmd = 0;
if (!this.reverse) {
    driveCmd = this.throttle; brakeCmd = this.brakeIn;
    if (this.brakeIn > 0.3 && vF < 0.8 && this.throttle < 0.1) { this.revTimer
    else this.revTimer = 0;
} else {
    driveCmd = -this.brakeIn; brakeCmd = this.throttle;
    if (this.throttle > 0.3 && vF > -0.8) { this.reverse = false; brakeCmd = 0;
}
if (!this.engineOn || this.noFuel) driveCmd = 0;

// engine rpm: road speed through the gearbox, clutch slip at launch, flare w
const gr = this.reverse ? 3.6 : d.gears[this.gear - 1];
const T = this.tune || {}; const low = T.dbl === 'low' ? 2.6 : 1;
const ratio = gr * d.final * low;
const toRpm = ratio * 60 / (2 * Math.PI);
const rpmRoad = Math.abs(vF) / this.wheelRadius * toRpm;
const rearW = this.wheels.filter(w => !w.front);
let wheelOmega = 0; for (const w of rearW) wheelOmega += Math.abs(w.spinVel);
const rpmWheel = wheelOmega * toRpm;
const launch = 1000 + Math.abs(driveCmd) * 2600;
let rpmT = Math.max(rpmRoad, rpmWheel, this.gear === 1 || this.reverse ? laun
if (driveCmd === 0 && rpmRoad < 1200) rpmT = Math.max(850, rpmRoad);
this.rpm += (rpmT - this.rpm) * (1 - Math.exp(-dt * 10));
if (this.rpm > d.redline) this.rpm = d.redline;
this.shiftTimer -= dt; this.sinceUp = (this.sinceUp || 0) + dt;
if (!this.reverse && this.shiftTimer <= 0) {
    if (rpmRoad > d.redline * 0.88 && this.gear < d.gears.length) { this.gear++
    else if (rpmRoad < d.redline * 0.33 && this.gear > 1 && this.sinceUp > 1.0)
}
const shifting = this.shiftTimer > 0.15;
const engTorque = this.torqueAt(this.rpm) * (this.tuneTorque || 1) * this.eng
const driveForceTotal = Math.sign(driveCmd) * engTorque * ratio * 0.88 / this

// ---- wheels ----
const upW = this.up(_b);

```

```

const R = this.wheelRadius;
const restLen0 = d.susTop + d.travel;
let grounded = 0; this.anySpin = false;
const muScale = input.surfaceGrip || 1;
// drift detection (body slip angle)
const velFlat = _c.copy(b.vel).addS(upW, -b.vel.dot(upW));
const sp = velFlat.len();
if (sp > 3) { const side = _d.set(1, 0, 0).applyQ(b.q); this.driftAngle = Mat
else this.driftAngle *= 0.9;
const drifting = Math.abs(this.driftAngle) > 0.2 && sp > 6;

// pass 1: suspension raycasts
for (const w of this.wheels) {
    const restLen = restLen0 + (w.rideAdj || 0) - w.brk * 0.16; // a broken cor
    const origin = this.localToWorld(_p.copy(w.pos).addS(UP, d.susTop), _e);
    const dir = _d.copy(upW).scale(-1);
    if (w.detached) { w.hitOk = false; w.comp = 0; continue; }
    let hit = this.world.raycast(origin, dir, restLen + R, _hit), bestT = hit ?
    // extra rays ahead/behind: the tyre is round, so a step edge lifts it befo
    for (const f of TYRE_RAYS) {
        const dz = f * R, o2 = _o2.copy(origin).addS(fwd, dz);
        const h2 = this.world.raycast(o2, dir, restLen + R, _hit2);
        if (!h2) continue;
        const te = h2.t - Math.sqrt(R * R - dz * dz) + R;
        if (te < bestT - 0.005) { bestT = te; bn = [h2.nx, h2.ny, h2.nz, h2.tag];
    }
    if (bn && bn[3] === 3 && w.sink > 0) bestT += w.sink; // bogged down: the t
    if (w.psi < 30) bestT += R * 0.16 * (1 - w.psi / 30); // soft / flat tyre:
    w.hitOk = bestT < 1e8;
    if (w.hitOk) { w.hitT = bestT; w.hitN = (w.hitN || new V3()).set(bn[0], bn[
    else w.comp = 0;
}
const kArb = this.k * 0.45;
const upright = clamp((upW.y - 0.25) / 0.35, 0, 1); // suspension fades out w
const rollH = this.com.y * 0.55;
for (const w of this.wheels) {
    const restLen = restLen0 + (w.rideAdj || 0) - w.brk * 0.16;
    const partner = this.wheels.find(o => o !== w && o.front === w.front);
    w.steer = w.front ? -this.steer : 0;
    if (w.hitOk) {
        const hit = { t: w.hitT, nx: w.hitN.x, ny: w.hitN.y, nz: w.hitN.z, tag: w
        const origin = _e.copy(w.origin); const dir = _d.copy(upW).scale(-1);
        w.onGround = true; grounded++;
        const len = Math.max(hit.t - R, 0.0);
        w.springLen = Math.min(len, restLen);
        const compression = restLen - w.springLen;

```

```

const cv = clamp((w.prevLen - w.springLen) / dt, -2.2, 2.2); w.prevLen =
let F = this.k * compression + this.c * cv + kArb * (compression - (partn
if (w.springLen < 0.04) F += (0.04 - w.springLen) * this.k * 3; // bump s
F = clamp(F, 0, d.mass * 9.81 * 0.95) * upright;
w.load = F;
const n = _j.set(hit.nx, hit.ny, hit.nz);
w.normal.copy(n); w.tag = hit.tag;
const cp = w.contact.copy(origin).addS(dir, hit.t);
// wheel heading
const hq = _q1.setAxisAngle(UP, w.steer + w.toe);
const wf = _f.set(0, 0, 1).applyQ(hq).applyQ(b.q);
wf.addS(n, -wf.dot(n)).norm();
const ws = _s.crossVecs(n, wf); // left
const v = b.velAt(cp, _v);
const vf = v.dot(wf), vs = v.dot(ws);
// suspension force along body up (approx ground normal)
_imp.copy(n).scale(F * dt);
b.applyImpulse(_imp, cp);

const soft = w.tag === 3 ? 1 + (30 - w.psi) / 55 : 1, flat = w.psi <= 0.5
const mu = (w.front ? d.tyreMu : d.rearMu) * muScale * (w.tag === 3 ? 0.7
const Fz0 = d.mass * 9.81 / 4; const loadSens = clamp(1 - 0.14 * (F / Fz0
const Fmax = mu * F * loadSens;
// ---- combined-slip tyre model ----
let Fx = 0, Fy = 0; let spinning = false; let slipSpeed = 0;
const rb = this.tuneRear ?? d.rearBias; let driveShare = w.front ? (1 - r
if (this.tuneDiff) { const opp = this.wheels.find(o => o !== w && o.front
const Fd = driveForceTotal * driveShare;
const mEff = d.mass / 4;
const brakeF = brakeCmd * d.brake / 4 * (w.front ? 1.15 : 0.85);
const hb = this.handbrake && !w.front;
const muK = 0.82; // sliding / static friction ratio
const vpl = Math.hypot(vf, vs);
// lateral demand from slip angle
const alpha = Math.atan2(vs, Math.abs(vf) + 0.6);
let latMu = 1;
if (!w.front && drifting && this.throttle > 0.3) latMu = 0.9; // drift as
let Fy0 = -Fmax * latMu * Math.sin(1.62 * Math.atan(8 * alpha));
const lowSpeed = vpl < 2.5;
if (lowSpeed) Fy0 = clamp(-vs * mEff / dt * 0.8, -Fmax, Fmax);
if (hb && vpl > 0.8) {
    // locked wheel: friction opposes the sliding direction
    Fx = -muK * Fmax * vf / vpl; Fy = -muK * Fmax * vs / vpl;
    slipSpeed = vpl;
} else {
    let Fxd = Fd - vf * 4;

```

```

    if (w.tag === 3 && w.sink > 0.005) { // digging through sand: a big rol
        const sd = F * (0.06 + 2.6 * w.sink / R) * clamp((w.psi - 8) / 22, 0, 1);
        Fxd -= Math.sign(vf) * Math.min(sd, Math.abs(vf) * mEff / dt + (Math.
    }
    if (hb) Fxd = clamp(-vf * mEff / dt, -Fmax, Fmax);
    if (brakeF > 0) Fxd += -Math.sign(vf) * Math.min(brakeF, Math.abs(vf) *
    const demand = Math.hypot(Fxd, Fy0);
    if (demand <= Fmax) { Fx = Fxd; Fy = Fy0; }
    else if (Math.abs(Fd) > 0.45 * Fmax && Math.sign(Fd) === Math.sign(Fxd)
        // wheelspin: the tyre surface runs faster than the ground
        spinning = true;
        const excess = Math.abs(Fxd) / Fmax;
        slipSpeed = clamp((excess - 0.45) * 22 + this.throttle * 4, 1.5, 26);
        const sx = Math.sign(Fd) * slipSpeed;
        const sy = lowSpeed ? 0 : vs;
        const sl = Math.hypot(sx, sy);
        Fx = muK * Fmax * sx / sl;
        Fy = lowSpeed ? Fy0 * 0.5 : -muK * Fmax * sy / sl;
    } else {
        // saturated (hard cornering / braking): keep direction, cap at slidi
        const k = Fmax * (demand > Fmax * 1.25 ? muK : 0.97) / demand;
        Fx = Fxd * k; Fy = Fy0 * k;
    }
}

_imp.copy(wf).scale(Fx * dt); b.applyImpulse(_imp, cp);
_imp.copy(ws).scale(Fy * dt); b.applyImpulse(_imp, _cp2.copy(cp).addS(upW
// wheel visuals & slip
const rollOmega = vf / R;
if (hb) { w.spinVel = 0; }
else if (spinning) { w.spinVel += ((vf + Math.sign(Fd) * slipSpeed) / R -
else w.spinVel = rollOmega;
w.slipLong = hb ? Math.min(1, slipSpeed / 6) : spinning ? clamp(slipSpeed
w.slip = clamp(Math.max(Math.abs(vs) - 1.6, 0) / 6, 0, 1);
w.skid = Math.max(w.slip, w.slipLong);
// ---- bogging in soft sand: spinning (or crawling) digs the tyre in, ro
// Heavy low cars (sedans) dig in at once, trucks only when you spin them
if (w.tag === 3 && this.bogSand) {
    const pf = clamp(w.psi / 30, 0.25, 1.15), bog = (d.bog ?? (d.lowY < 0.6
    const dig = ((hb && this.throttle > 0.4 && !w.front ? 0.55 * this.throt
    // rolling on top climbs out; with the tyres let down (<=20) a gentle t
    const low = clamp((22 - w.psi) / 12, 0, 1);
    const climb = (vpl > 4 && w.slipLong < 0.15 ? 0.1 : 0) + (this.throttle
    w.sink = clamp(w.sink + dt * (bog * dig * pf * pf - climb / pf), 0, R *
    if (hb && this.throttle > 0.4 && !w.front) { w.spinVel = this.throttle
} else if (w.sink > 0) w.sink = Math.max(0, w.sink - dt * 0.5);
} else {

```

```

        w.onGround = false; w.springLen += (restLen0 - w.springLen) * Math.min(1,
        if (!w.front) { w.spinVel += (driveCmd * 60 - w.spinVel) * dt * 2; }
        w.spinVel *= 0.995;
    }
    w.spin += w.spinVel * dt;
}
this.grounded = grounded;
this.airTime = grounded === 0 ? this.airTime + dt : 0;
// aerodynamic drag + downforce-ish stability
const v2 = b.vel.len();
_imp.copy(b.vel).scale(-0.5 * 1.225 * d.dragCdA * v2 * dt); b.vel.addS(_imp,
// gravity
b.vel.y -= 9.81 * dt;
// angular damping (tyre scrub / air), stronger yaw damping when not drifting
const ad = Math.exp(-dt * (grounded ? 0.25 : 0.05)); b.ang.scale(ad);
// drift control: past ~35 degrees of slide, any yaw that makes the slide big
// so a handbrake/power drift holds its angle instead of snapping round (a 18
const dA = this.driftAngle - (this.prevDrift ?? this.driftAngle); this.prevDr
if (grounded >= 2 && sp > 4) {
    const over = clamp((Math.abs(this.driftAngle) - 0.45) / 0.45, 0, 1);
    const growing = Math.sign(dA) === Math.sign(this.driftAngle) && Math.abs(dA
    let yawRate = b.ang.dot(upW);
    const k = over * (growing ? 12 : 2) * (this.handbrake ? 0.7 : 1);
    if (k > 0) { b.ang.addS(upW, -yawRate * Math.min(1, dt * k)); yawRate = b.a
    const cap = lerp(3.0, 1.1, over);
    if (growing && Math.abs(yawRate) > cap) b.ang.addS(upW, (Math.sign(yawRate)
}
// hajwala weight: at speed the slide angle follows the steering like a heavy
// progressively, centre the wheel (or lift) and it swings back straight, cou
// No snap spin that leaves you stuck sideways.
{
    const vf = _e.copy(b.vel); vf.y = 0; const vs = vf.len();
    const head = vs > 1 ? Math.atan2(vf.x, vf.z) : (this.prevHead ?? 0);
    let hr = 0; if (this.prevHead !== undefined) { let dh = head - this.prevHea
    const w = clamp((vs - 12) / 10, 0, 1) * (grounded >= 2 ? 1 : 0) * (this.han
    if (w > 0 && (Math.abs(this.driftAngle) > 0.08 || Math.abs(input.steer) > 0
        // counter-steer HOLDS the slide (the harder you counter, the shallower),
        // lifting off lets it straighten; it never flips to the other side by it
        let tgt;
        if (Math.abs(this.driftAngle) > 0.15) {
            const sd = Math.sign(this.driftAngle), c = input.steer * sd; // c > 0:
            const th = clamp(this.throttle, 0, 1);
            let mag = c >= 0 ? 0.62 - 0.28 * c : 0.62 + 0.38 * Math.min(1, -c);
            mag *= clamp((th - 0.08) / 0.5, 0, 1);
            tgt = sd * mag;
        } else tgt = clamp(-input.steer * 0.35 * clamp(this.throttle, 0, 1), -0.3

```

```

    const err = tgt - this.driftAngle, yawNow = b.ang.dot(upW);
    const want = hr * Math.sign(upW.y || 1) + clamp(err * 2.2, -1.6, 1.6); //
    b.ang.addS(upW, (want - yawNow) * Math.min(1, dt * 2.6 * w));
  }
}
// drift momentum ("وزنية"): on the gas a slide carries its speed instead of
// and the heavy body keeps travelling along its line (the tyres sweep it rou
{
  const vf = _e.copy(b.vel); vf.y = 0; const vs = vf.len();
  if (grounded >= 2 && Math.abs(this.driftAngle) > 0.15 && vs > 4 && this.thr
    if (!(this.driftV > 0)) this.driftV = vs;
    this.driftV = Math.max(vs, this.driftV - dt * 0.35); // barel
    const want = Math.min(this.driftV, 60) - vs;
    if (want > 0) b.vel.addS(vf.scale(1 / vs), Math.min(want, dt * 16 * this.
  } else this.driftV = 0;
}
// two wheels ("تطعيس" on the side wheels): when only one side touches the gr
{
  const lg = this.wheels.filter(w => w.onGround && w.side > 0).length, rg = t
  const sideW = _d.set(1, 0, 0).applyQ(b.q); const roll = Math.asin(clamp(sid
  this.twoWheel = ((lg === 0 && rg >= 1) || (rg === 0 && lg >= 1)) && Math.ab
  this.twoWheelT = this.twoWheel ? (this.twoWheelT || 0) + dt : 0;
  if (this.twoWheel && this.twoWheelT < 4.5 && this.throttle > 0.05) {
    // the assist fades out: a few seconds on two wheels, then gravity wins a
    const fade = Math.exp(-this.twoWheelT / 2.5) * Math.min(1, this.throttle
    const fw = this.forward(_e); const rr = b.ang.dot(fw);
    b.ang.addS(fw, -rr * Math.min(1, dt * 3.5 * fade));
    const tw = Math.abs(this.wheels[0].pos.x), bal = Math.atan2(tw, this.com.
    const hold = Math.sign(roll) * (bal - 0.04) - this.steer / d.maxSteer * 0
    b.ang.addS(fw, (hold - roll) * dt * 28 * fade);
  }
}
// ---- water: buoyancy, heavy drag, and a drowned engine if the water reache
this.submerge = 0;
if (input.water !== null && input.water !== undefined) {
  const level = input.water; let sub = 0;
  for (const s of this.spheres) {
    const cy = _a.copy(s).applyQ(b.q).add(b.pos).y; const f = clamp((level -
    if (f <= 0) continue; sub += f;
    const cp = _a.copy(s).applyQ(b.q).add(b.pos);
    const v = b.velAt(cp, _v);
    _imp.set(0, d.mass * 9.81 * 0.95 / this.spheres.length * 1.25 * f * dt, 0
    _imp.copy(v).scale(-Math.min(1, dt * 2.2 * f) * d.mass / this.spheres.len
  }
  this.submerge = sub / this.spheres.length;
  const intake = this.localToWorld(_p.set(0, d.lowY + 0.45, d.zMax - 0.8), _e

```

```

    if (level > intake && this.engineOn && Math.abs(this.forwardSpeed) < 25) {
    }
    this.collide(dt);
    if (b.ang.lenSq() > 400) b.ang.scale(20 / b.ang.len());
    if (b.vel.lenSq() > 10000) b.vel.scale(100 / b.vel.len());
    b.integrate(dt);
    if (!Number.isFinite(b.pos.x + b.pos.y + b.pos.z + b.q.x + b.q.w + b.vel.x +
        if (this._good) { b.pos.copy(this._good.p); b.q.copy(this._good.q); } b.vel
    ) else { if (!this._good) this._good = { p: new V3(), q: new Q() }; this._goo
    this.syncNode();
}

collide(dt) {
    const b = this.body; const d = this.def;
    let scrape = 0;
    for (const s of this.spheres) {
        const r = s.r;
        const c = _a.copy(s).applyQ(b.q).add(b.pos);
        this.world.sphere(c, r, (nx, ny, nz, depth, px, py, pz, tag) => {
            const n = _n.set(nx, ny, nz);
            const cp = _cp.copy(c).addS(n, -r);
            const v = b.velAt(cp, _v);
            const vn = v.dot(n);
            // positional correction
            const corr = Math.min(Math.max(depth - 0.01, 0) * 0.2, 0.03);
            b.pos.addS(n, corr);
            if (vn < 0) {
                const k = b.invMassAt(cp, n);
                const e = -vn > 6 ? 0.08 : 0.0;
                let jn = -(1 + e) * vn / k;
                if (s.tyre && ny > 0.2) jn = Math.min(jn, d.mass * 9.81 * 1.1 * dt); //
                _imp.copy(n).scale(jn); b.applyImpulse(_imp, cp);
                // friction
                const vt = _vt.copy(v).addS(n, -vn); const vtl = vt.len();
                if (vtl > 1e-4) {
                    vt.scale(1 / vtl);
                    const kt = b.invMassAt(cp, vt);
                    const jt = Math.min(vtl / kt, (s.tyre ? 0.08 : 0.55) * jn); // tyres
                    _imp.copy(vt).scale(-jt); b.applyImpulse(_imp, cp);
                    if (tag !== 9) scrape = Math.max(scrape, vtl);
                }
                if (-vn > 2.5 && !(s.tyre && ny > 0.2 && -vn < 7)) this.impactts.push({
            }
        });
    }
    this.scrape = scrape;
}

```

```

}

syncNode() {
  if (!this._lampMats) this.lampMats(); this.updateTailGlow();
  const b = this.body; const n = this.node;
  if (this.underbody) this.underbody.visible = (1 - 2 * (b.q.x * b.q.x + b.q.z
  n.rot.copy(b.q);
  n.pos.copy(this.com).scale(-1).applyQ(b.q).add(b.pos);
  for (const w of this.wheels) {
    if (!w.node || w.detached) continue;
    const y = w.pos.y + this.def.susTop - w.springLen;
    w.node.pos.set(w.pos.x + w.dmgOff.x, Math.min(y, w.pos.y + 0.2) + w.dmgOff.
    _q1.setAxisAngle(UP, w.steer + w.toe);
    if (w.camber) { _q3.setAxisAngle(_x.set(0, 0, 1), w.camber); _q1.mul(_q3);
    _q2.setAxisAngle(_x.set(1, 0, 0), w.spin);
    w.node.rot.copy(_q1).mul(_q2);
  }
  if (this.doorNode) {
    this.doorNode.rot.setAxisAngle(UP, -this.doorAngle);
  }
  if (this.doorNodeR) {
    this.doorNodeR.rot.setAxisAngle(UP, this.doorAngleR);
  }
  if (this.steeringNode) {
    _q1.setAxisAngle(_x.set(0, 0, 1), this.wheelSpin());
    this.steeringNode.rot.copy(this.steerBase);
    if (this.wheelBend) { _q2.setAxisAngle(_x.set(1, 0, 0), this.wheelBend); t
    this.steeringNode.rot.mul(_q1);
  }
}

// steering-wheel rotation (rad) shown in the cabin: full lock = 1.5 turns, tur
wheelSpin() { return clamp(this.steer / this.def.maxSteer, -1, 1) * 3 * Math.PI

// ---- visual deformation ----
dent(worldPoint, worldDir, amount, radius, renderer) {
  const lp = this.worldToLocal(worldPoint, _a);
  const ld = _b.copy(worldDir).applyInvQ(this.body.q).norm();
  if (this.onDent) this.onDent(lp.clone(), ld.clone(), amount, radius);
  if (lp.z > this.def.zMax - 1.3 && amount > 0.035) this.wheelBend = Math.min(0
  let changed = 0;
  for (const D of this.deform) {
    const p = D.prim;
    // point relative to node space (body at origin; door has hinge offset + ro
    let px = lp.x, py = lp.y, pz = lp.z, dx = ld.x, dy = ld.y, dz = ld.z;
    if (D.node === this.doorNode || D.node === this.doorNodeR) {

```

```

    _c.set(px, py, pz).sub(D.node.pos).applyInvQ(D.node.rot); px = _c.x; py =
    _c.set(dx, dy, dz).applyInvQ(D.node.rot); dx = _c.x; dy = _c.y; dz = _c.z
}
if (px < p.min[0] - radius || px > p.max[0] + radius || py < p.min[1] - rad
const pos = p.attrs.position.data, nrm = p.attrs.normal.data;
if (!D.orig) D.orig = new Float32Array(pos);
const r2 = radius * radius; let moved = 0;
for (let i = 0; i < pos.length; i += 3) {
    const ex = pos[i] - px, ey = pos[i + 1] - py, ez = pos[i + 2] - pz;
    const dd = ex * ex + ey * ey + ez * ez; if (dd > r2) continue;
    const r = Math.sqrt(dd), q = 1 - r / radius; const s = amount * q * q;
    pos[i] += dx * s; pos[i + 1] += dy * s; pos[i + 2] += dz * s;
    // limit total displacement
    const ox = pos[i] - D.orig[i], oy = pos[i + 1] - D.orig[i + 1], oz = pos[
    const ol = Math.hypot(ox, oy, oz); if (ol > 1.1) { const k = 1.1 / ol; po
    // analytic normal tilt: the dent bowl slopes toward its centre
    if (r > 1e-4) {
        let nx = nrm[i], ny = nrm[i + 1], nz = nrm[i + 2];
        let tx = ex / r, ty = ey / r, tz = ez / r; const tn = tx * nx + ty * ny
        const inward = -(dx * nx + dy * ny + dz * nz); // >0 when pushing into
        const g = 2 * amount * q / radius * Math.sign(inward) * Math.min(1, Mat
        nx -= tx * g; ny -= ty * g; nz -= tz * g; const l = Math.hypot(nx, ny,
        nrm[i] = nx / l; nrm[i + 1] = ny / l; nrm[i + 2] = nz / l;
    }
    moved++;
}
if (moved) { D.dirty = true; changed += moved; }
}
if (changed && renderer) for (const D of this.deform) if (D.dirty) { renderer
// parts bolted to the body move with the crushed metal: plates and wheels
const push = (pt, off, lim) => {
    const ex = pt.x - lp.x, ey = pt.y - lp.y, ez = pt.z - lp.z; const r = Math.
    const q = 1 - r / radius, s = amount * q * q; off.x += ld.x * s; off.y += l
    const l = off.len(); if (l > lim) off.scale(lim / l); return s;
};
for (const P of this.plates || []) {
    if (push(P.home, P.off, 1.0) > 0) {
        P.node.pos.copy(P.home).add(P.off);
        const bend = clamp(P.off.len() * 1.4, 0, 0.9); P.node.rot.copy(P.rot).mul
    }
}
for (const w of this.wheels) {
    const hub = _p.copy(w.pos);
    const s = push(hub, w.dmgOff, 0.35);
    if (s > 0) {
        // a hard hit near a wheel bends the suspension: camber, toe and a saggin

```

```

        w.brk = clamp(w.brk + s * 2.2, 0, 1);
        if (!w.detached && w.brk >= 1 && s > 0.1 && amount > 0.35) { w.detached =
        w.camber = clamp(w.camber + s * 1.6 * -w.side * Math.sign(ld.x || 1) * (1
        w.toe = clamp(w.toe + s * 0.9 * (ld.z > 0 ? 1 : -1) * w.side, -0.35, 0.35
        if (w.front) this.steerBias = clamp(this.steerBias + w.toe * 0.08, -0.12,
        }
    }
    return changed;
}

recomputeNormals(D, moved) {
    const p = D.prim; const idx = p.index; const pos = p.attrs.position.data; con
    const nv = pos.length / 3;
    if (!D.adj) { // vertex -> triangles CSR
        const cnt = new Uint32Array(nv + 1);
        for (let i = 0; i < idx.length; i++) cnt[idx[i] + 1]++;
        for (let i = 0; i < nv; i++) cnt[i + 1] += cnt[i];
        const list = new Uint32Array(idx.length); const fill = cnt.slice(0, nv);
        for (let i = 0; i < idx.length; i++) list[fill[idx[i]]++] = (i / 3) | 0;
        D.adj = { start: cnt, list };
    }
    const { start, list } = D.adj;
    for (const v of moved) {
        let nx = 0, ny = 0, nz = 0;
        for (let k = start[v]; k < start[v + 1]; k++) {
            const t = list[k] * 3; const a = idx[t] * 3, bb = idx[t + 1] * 3, c = idx
            const e1x = pos[bb] - pos[a], e1y = pos[bb + 1] - pos[a + 1], e1z = pos[b
            const e2x = pos[c] - pos[a], e2y = pos[c + 1] - pos[a + 1], e2z = pos[c +
            nx += e1y * e2z - e1z * e2y; ny += e1z * e2x - e1x * e2z; nz += e1x * e2y
        }
        const l = Math.hypot(nx, ny, nz);
        if (l > 1e-12) {
            // blend with original normal direction sign
            const ox = nrm[v * 3], oy = nrm[v * 3 + 1], oz = nrm[v * 3 + 2];
            const sgn = (nx * ox + ny * oy + nz * oz) < 0 ? -1 : 1;
            nrm[v * 3] = sgn * nx / l; nrm[v * 3 + 1] = sgn * ny / l; nrm[v * 3 + 2]
        }
    }
}

repair(renderer) {
    this.wheelBend = 0; for (const w of this.wheels || []) { w.psi = 30; w.sink =
    for (const D of this.deform) if (D.orig) { D.prim.attrs.position.data.set(D.o
    // normals: reload from cache
    for (const D of this.deform) if (D.nrmOrig) D.prim.attrs.normal.data.set(D.nr
    if (renderer) for (const D of this.deform) renderer.updatePositions(D.prim);

```

```

    this.damage = 0; this.steerBias = 0; this.engineHealth = 1;
    for (const w of this.wheels) { w.brk = 0; w.camber = 0; w.toe = 0; w.dmgOff.s
    this.detachEvents = [];
    this.flooded = false; this.floodMsg = false; this.engineOn = true;
    for (const P of this.plates || []) { P.off.set(0, 0, 0); P.node.pos.copy(P.ho
    }
    snapshotNormals() { for (const D of this.deform) D.nrmOrig = new Float32Array(D
    }

const TYRE_RAYS = [-0.96, -0.85, -0.65, -0.4, 0.4, 0.65, 0.85, 0.96];
const _o2 = new V3(), _hit2 = {};
const _q3 = new Q(), _cp2 = new V3(), _q1 = new Q(), _q2 = new Q(), _x = new V3()
const _x2 = new V3();

// ---- small procedural meshes ----
function boxPrim(sx, sy, sz, material) {
    const hx = sx / 2, hy = sy / 2, hz = sz / 2;
    const P = [], N = [], U = [], I = [];
    const face = (nx, ny, nz, ux, uy, uz, vx, vy, vz) => {
        const b = P.length / 3;
        for (const [a, c] of [[-1, -1], [1, -1], [1, 1], [-1, 1]]) {
            P.push(nx * hx + ux * a * hx + vx * c * hx, ny * hy + uy * a * hy + vy * c
            N.push(nx, ny, nz); U.push((a + 1) / 2, (c + 1) / 2);
        }
        I.push(b, b + 1, b + 2, b, b + 2, b + 3);
    };
    face(1, 0, 0, 0, 0, -1, 0, 1, 0); face(-1, 0, 0, 0, 0, 1, 0, 1, 0);
    face(0, 1, 0, 1, 0, 0, 0, 0, -1); face(0, -1, 0, 1, 0, 0, 0, 0, 1);
    face(0, 0, 1, 1, 0, 0, 0, 1, 0); face(0, 0, -1, -1, 0, 0, 0, 1, 0);
    return makePrim(P, N, U, I, material);
}

function torusPrim(R, r, material, seg = 32, sides = 8) {
    const P = [], N = [], U = [], I = [];
    for (let i = 0; i <= seg; i++) {
        const a = i / seg * Math.PI * 2;
        for (let j = 0; j <= sides; j++) {
            const b = j / sides * Math.PI * 2;
            const cx = Math.cos(a), cy = Math.sin(a);
            P.push((R + r * Math.cos(b)) * cx, (R + r * Math.cos(b)) * cy, r * Math.sin
            N.push(Math.cos(b) * cx, Math.cos(b) * cy, Math.sin(b)); U.push(i / seg, j
        }
    }
    for (let i = 0; i < seg; i++) for (let j = 0; j < sides; j++) {
        const a = i * (sides + 1) + j, b = a + sides + 1;
        I.push(a, b, a + 1, b, b + 1, a + 1);
    }
}

```

```

    return makePrim(P, N, U, I, material);
}
function makePrim(P, N, U, I, material) {
    const prim = {
        attrs: { position: { data: new Float32Array(P), size: 3, count: P.length / 3
        index: P.length / 3 > 65535 ? new Uint32Array(I) : new Uint16Array(I), materi
    };
    computeBounds(prim);
    return prim;
}

return { CAR_DEFS, Car, boxPrim, torusPrim, makePrim };
})();
__M['maps'] = (() => {
// Maps: city (from the provided model), a long Saudi highway through the desert,
const { V3, Q, M4, clamp, lerp } = __M['math'];
const { Node, Material } = __M['scene'];
const { CollisionWorld } = __M['physics'];
const { MB } = __M['carparts'];
const { makePrim } = __M['car'];

// ----- noise -----
function hash2(x, y, s) { let h = (x * 374761393 + y * 668265263 + s * 982451653)
function vnoise(x, y, s = 0) {
    const xi = Math.floor(x), yi = Math.floor(y), xf = x - xi, yf = y - yi;
    const u = xf * xf * (3 - 2 * xf), v = yf * yf * (3 - 2 * yf);
    const a = hash2(xi, yi, s), b = hash2(xi + 1, yi, s), c = hash2(xi, yi + 1, s),
    return a + (b - a) * u + (c - a) * v + (a - b - c + d) * u * v;
}
function fbm(x, y, oct = 4, s = 0) { let t = 0, a = 0.5, f = 1; for (let i = 0; i
const sstep = (a, b, x) => { const t = clamp((x - a) / (b - a), 0, 1); return t *
function rng(seed) { let s = seed >>> 0; return () => { s = (s * 1664525 + 101390

// ----- heightfield terrain -----
class Terrain {
    constructor(x0, z0, x1, z1, step, fn, tagFn) {
        this.x0 = x0; this.z0 = z0; this.step = step;
        this.nx = Math.ceil((x1 - x0) / step) + 1; this.nz = Math.ceil((z1 - z0) / st
        this.H = new Float32Array(this.nx * this.nz);
        for (let j = 0; j < this.nz; j++) for (let i = 0; i < this.nx; i++) this.H[j
        this.tagFn = tagFn;
        this.h = (x, z) => {
            let fx = (x - this.x0) / this.step, fz = (z - this.z0) / this.step;
            fx = clamp(fx, 0, this.nx - 1.001); fz = clamp(fz, 0, this.nz - 1.001);
            const i = Math.floor(fx), j = Math.floor(fz), u = fx - i, v = fz - j;
            const H = this.H, n = this.nx;

```

```

    // same triangle split as the render mesh
    const a = H[j * n + i], b = H[j * n + i + 1], c = H[(j + 1) * n + i], d = H
    return u + v < 1 ? a + (b - a) * u + (c - a) * v : d + (c - d) * (1 - u) +
};
this.n = (x, z) => {
    const e = this.step * 0.5;
    const dx = this.h(x + e, z) - this.h(x - e, z), dz = this.h(x, z + e) - thi
    const l = Math.hypot(dx, 2 * e, dz); return [-dx / l, 2 * e / l, -dz / l];
};
this.tag = (x, z) => this.tagFn ? this.tagFn(x, z) : 3;
}
mesh(material, uvScale, chunk = 64) {
    // split into chunks for frustum culling
    const nodes = [];
    for (let cj = 0; cj < this.nz - 1; cj += chunk) for (let ci = 0; ci < this.nx
    const P = [], N = [], U = [], I = [];
    const w = Math.min(chunk, this.nx - 1 - ci) + 1, hh = Math.min(chunk, this.
    for (let j = 0; j < hh; j++) for (let i = 0; i < w; i++) {
        const x = this.x0 + (ci + i) * this.step, z = this.z0 + (cj + j) * this.s
        const y = this.H[(cj + j) * this.nx + ci + i];
        const n = this.n(x, z);
        P.push(x, y, z); N.push(n[0], n[1], n[2]); U.push(x * uvScale, z * uvScal
    }
    for (let j = 0; j < hh - 1; j++) for (let i = 0; i < w - 1; i++) {
        const a = j * w + i, b = a + 1, c = a + w, d = c + 1;
        I.push(a, c, b, b, c, d);
    }
    const n = new Node('terrain'); n.mesh = { prims: [makePrim(P, N, U, I, mate
    }
    return nodes;
}
}

// ----- collision helpers -----
function colBox(world, cx, cy, cz, sx, sy, sz, yaw = 0, tag = 5) {
    const c = Math.cos(yaw), s = Math.sin(yaw); const P = [];
    for (const [a, b, d] of [[-1, -1, -1], [1, -1, -1], [1, 1, -1], [-1, 1, -1], [-
    const x = a * sx / 2, z = d * sz / 2; P.push(cx + x * c + z * s, cy + b * sy
    }
    const I = [0, 2, 1, 0, 3, 2, 4, 5, 6, 4, 6, 7, 0, 1, 5, 0, 5, 4, 3, 7, 6, 3, 6,
    world.addTriangles(P, I, null, tag);
}

function colCyl(world, cx, cy0, cz, r, h, tag = 5, seg = 8) {
    const P = [], I = [];
    for (let i = 0; i < seg; i++) { const a = i / seg * Math.PI * 2; P.push(cx + Ma
    for (let i = 0; i < seg; i++) { const a = i * 2, b = ((i + 1) % seg) * 2; I.pus

```

```

    world.addTriangles(P, I, null, tag);
}

// ----- canvas textures -----
const texCache = {};
function ctex(key, w, h, draw, repeat = true) {
    if (texCache[key]) return texCache[key];
    const c = document.createElement('canvas'); c.width = w; c.height = h; draw(c.g
    return (texCache[key] = { image: c, wrapS: repeat ? 10497 : 33071, wrapT: repea
}
function noiseFill(g, w, h, base, amp) {
    const d = g.createImageData(w, h); const r = rng(w * 7 + h);
    for (let i = 0; i < w * h; i++) { const n = (r() - 0.5) * amp; d.data[i * 4] =
    g.putImageData(d, 0, 0);
}
function roadTexture() {
    // across the road (u): 0..1 = -15..+15 m ; along (v): one texture = 12 m
    return ctex('road', 512, 256, (g, w, h) => {
        noiseFill(g, w, h, [58, 58, 60], 26);
        const X = (m) => (m + 15) / 30 * w;
        // shoulders slightly lighter
        g.fillStyle = 'rgba(120,115,105,0.25)'; g.fillRect(X(-15), 0, X(-12.3) - X(-1
        // edge lines
        g.fillStyle = '#e8e6df'; g.fillRect(X(-12.4), 0, X(-12.2) - X(-12.4), h); g.f
        g.fillStyle = '#e2b23a'; g.fillRect(X(-1.9), 0, X(-1.75) - X(-1.9), h); g.fil
        // dashed lane lines
        g.fillStyle = '#ecebe6';
        for (const m of [-8.6, -5.0, 5.0, 8.6]) g.fillRect(X(m) - 1.5, 0, 3, h * 0.33
        // tyre polish in the lanes
        g.fillStyle = 'rgba(20,20,22,0.18)';
        for (const m of [-10.8, -9.4, -7.2, -5.8, -3.6, -2.4, 2.4, 3.6, 5.8, 7.2, 9.4
        });
    }
}
function signTexture(lines, bg = '#0f6b3a', key) {
    return ctex('sign_' + key, 512, 256, (g, w, h) => {
        g.fillStyle = bg; g.fillRect(0, 0, w, h);
        g.strokeStyle = '#f2f2f2'; g.lineWidth = 8; g.strokeRect(10, 10, w - 20, h -
        g.fillStyle = 'ffffff'; g.textAlign = 'center'; g.direction = 'rtl';
        const n = lines.length;
        lines.forEach((t, i) => { g.font = (i === 0 ? 'bold 64px' : 'bold 46px') + '
        }, false);
    }
}
function stuccoTexture(base) {
    return ctex('stucco' + base.join(), 256, 256, (g, w, h) => {
        noiseFill(g, w, h, base, 18);
        g.fillStyle = 'rgba(0,0,0,0.06)'; for (let i = 0; i < 40; i++) g.fillRect(Mat

```

```

    });
}
function mountainTexture() { return ctex('mount', 256, 256, (g, w, h) => noiseFil

// ----- city -----
function buildCity(assets) {
    const city = assets.city.root;
    const w = new CollisionWorld(4);
    city.updateWorld();
    const tagOf = (name, mat) => {
        if (/Roads/.test(name)) return 1; if (/Sidewalks/.test(name)) return 2;
        if (/Terrain/.test(name)) return /sand|grass/.test(mat) ? 3 : 2;
        if (/Buildings/.test(name)) return 4; if (/Props/.test(name)) return 5; if (/
    };
    let sandTex = null;
    city.traverse(n => {
        if (!n.mesh) return;
        for (const p of n.mesh.prims) {
            const m = p.material;
            if (/glass_lit/.test(m.name)) m.emissive = [0.9, 0.75, 0.45];
            if (/shop_/.test(m.name)) { m.roughness = 0.15; m.envBoost = 1.4; }
            if (/asphalt/.test(m.name)) m.roughness = 0.85;
            if (/sand/.test(m.name)) sandTex = m.map;
            if (/col$/.test(n.name) || /trunk/.test(m.name)) w.addTriangles(p.attrs.po
        }
    });
    w.build();
    const root = new Node('map_city'); root.add(city);
    const S = 3000; const sandMat = new Material({ name: 'desert', map: sandTex, ro
    const g = new Node('desert'); g.mesh = { prims: [makePrim([-S, -0.32, -S, S, -0
    g.castShadow = false; root.add(g);
    root.add(mountains(sandTex, 900, 1800, 0.55, 1));
    const cones = [];
    for (let i = 0; i < 10; i++) { const a = i / 10 * Math.PI * 2; cones.push([Math
    for (let i = 0; i < 7; i++) cones.push([-7 + (i % 2 ? 1.5 : -1.5), 30 + i * 9])
    return {
        name: 'city', root, world: w, sandTex,
        spawn: { x: -7, z: -40, yaw: 0 }, showroom: { x: 0, z: 0, yaw: 0.6 },
        cones, npcs: [[18, 20, -2.4], [19.8, 21.6, -2.2], [-18, 19, 2.4], [-19, -18,
        env: { sunDir: new V3(-0.22, 0.8, -0.58).norm(), sunColor: [3.1, 2.85, 2.5],
    };
}

// distant rocky mountains (visual only)
function mountains(tex, r0, r1, heightScale, seed, gapX = 0) {
    const mat = new Material({ name: 'mountain', map: mountainTexture(), color: [0.

```

```

const P = [], N = [], U = [], I = []; const seg = 160, rings = 8;
for (let k = 0; k <= rings; k++) {
  const t = k / rings; const r = lerp(r0, r1, t);
  for (let i = 0; i <= seg; i++) {
    const a = i / seg * Math.PI * 2;
    const prof = Math.sin(t * Math.PI) ** 0.8;
    const hgt = (fbm(Math.cos(a) * 3 + seed, Math.sin(a) * 3 + t * 2, 5, seed)
    const gx = gapX ? sstep(gapX, gapX * 2.2, Math.abs(Math.cos(a) * r)) : 1;
    P.push(Math.cos(a) * r, Math.max(-2, hgt * gx - 2 * (1 - gx)), Math.sin(a)
  }
}
for (let k = 0; k < rings; k++) for (let i = 0; i < seg; i++) { const a = k * (
// normals
const nv = P.length / 3; const acc = new Float32Array(nv * 3);
for (let i = 0; i < I.length; i += 3) {
  const [a, b, c] = [I[i], I[i + 1], I[i + 2]];
  const e1 = [P[b * 3] - P[a * 3], P[b * 3 + 1] - P[a * 3 + 1], P[b * 3 + 2] -
  const n = [e1[1] * e2[2] - e1[2] * e2[1], e1[2] * e2[0] - e1[0] * e2[2], e1[0]
  for (const v of [a, b, c]) { acc[v * 3] += n[0]; acc[v * 3 + 1] += n[1]; acc[
}
for (let v = 0; v < nv; v++) { const l = Math.hypot(acc[v * 3], acc[v * 3 + 1],
const n = new Node('mountains'); n.mesh = { prims: [makePrim(P, N, U, I, mat)]
mat.doubleSided = true;
return n;
}

// ----- vegetation / props builders -----
function shrubs(mb, x, y, z, s, r) {
  const n = 3 + (r() * 3 | 0);
  for (let i = 0; i < n; i++) {
    const a = r() * 6.28, d = r() * 0.5 * s;
    const sx = (0.35 + r() * 0.4) * s;
    mb.push().translate(x + Math.cos(a) * d, y + sx * 0.3, z + Math.sin(a) * d).r
    mb.box(0, 0, 0, sx, sx * 0.65, sx); mb.pop();
  }
}
function palm(trunk, fronds, x, y, z, h, r) {
  const seg = 6; const lean = (r() - 0.5) * 0.25, lz = (r() - 0.5) * 0.25;
  for (let i = 0; i < seg; i++) {
    const t0 = i / seg, t1 = (i + 1) / seg;
    const rad = lerp(0.28, 0.2, t0);
    trunk.push().translate(x + lean * h * (t0 + t1) / 2, y + h * (t0 + t1) / 2, z
    trunk.box(0, 0, 0, rad * 2, h / seg + 0.02, rad * 2);
    trunk.pop();
  }
  const tx = x + lean * h, tz = z + lz * h, ty = y + h;

```

```

const nf = 8;
for (let i = 0; i < nf; i++) {
  const a = i / nf * Math.PI * 2 + r();
  fronds.push().translate(tx, ty, tz).rotate(0, 1, 0, a).rotate(1, 0, 0, 0.5 +
  // frond: a long drooping leaf made of 2 segments
  fronds.box(0, 0, 1.0, 0.9, 0.04, 2.0);
  fronds.push().translate(0, -0.25, 2.6).rotate(1, 0, 0, 0.5); fronds.box(0, 0,
  fronds.pop());
}
}

function streetLight(pole, lamp, x, y, z, yaw) {
  pole.push().translate(x, y, z).rotate(0, 1, 0, yaw);
  pole.box(0, 5, 0, 0.18, 10, 0.18);
  pole.box(1.2, 9.8, 0, 2.4, 0.12, 0.12); pole.box(-1.2, 9.8, 0, 2.4, 0.12, 0.12)
  pole.pop();
  lamp.push().translate(x, y, z).rotate(0, 1, 0, yaw);
  lamp.box(2.3, 9.7, 0, 0.7, 0.14, 0.3); lamp.box(-2.3, 9.7, 0, 0.7, 0.14, 0.3);
  lamp.pop();
}

function najdiHouse(wall, trim, x, y, z, yaw, r) {
  const W = 10 + r() * 8, D = 9 + r() * 6, H = 4 + (r() < 0.5 ? 3.5 : 0);
  wall.push().translate(x, y, z).rotate(0, 1, 0, yaw);
  wall.box(0, H / 2, 0, W, H, D);
  // crenellations (triangular merlons approximated by small boxes)
  for (let i = -W / 2 + 0.6; i < W / 2; i += 1.2) { wall.box(i, H + 0.35, D / 2 -
  for (let i = -D / 2 + 0.6; i < D / 2; i += 1.2) { wall.box(W / 2 - 0.15, H + 0.
  wall.pop();
  trim.push().translate(x, y, z).rotate(0, 1, 0, yaw);
  trim.box(0, 1.2, D / 2 + 0.03, 1.4, 2.4, 0.1); // wooden door
  for (let i = -W / 2 + 2; i < W / 2 - 1; i += 3) trim.box(i, H - 1.2, D / 2 + 0.
  trim.pop();
  return { W, D, H };
}

function mosque(wall, dome, x, y, z, yaw) {
  wall.push().translate(x, y, z).rotate(0, 1, 0, yaw);
  wall.box(0, 4, 0, 16, 8, 16);
  wall.box(10, 11, 6, 2.6, 22, 2.6); wall.box(10, 22.5, 6, 3.2, 1, 3.2); wall.box
  wall.pop();
  dome.push().translate(x, y + 8, z).rotate(0, 1, 0, yaw);
  // dome as stacked shrinking boxes (stylised)
  for (let k = 0; k < 7; k++) { const t = k / 7; const r = 5.5 * Math.cos(t * Mat
  dome.box(0, 6.4, 0, 0.2, 1.4, 0.2);
  dome.pop();
}

// ----- highway -----

```

```

function buildHighway(assets, sandTex) {
    const L = 4200; // half length (m)
    const cx = (z) => 45 * Math.sin(z / 650) + 22 * Math.sin(z / 240 + 1.3); // roa
    const dcx = (z) => 45 / 650 * Math.cos(z / 650) + 22 / 240 * Math.cos(z / 240 +
    const HW = 15;
    const dunes = (x, z) => {
        const d = Math.abs(x - cx(z));
        const k = sstep(24, 110, d);
        const base = (fbm(x / 180, z / 180, 4, 3) - 0.35) * 26 + Math.max(0, fbm(x /
        return Math.max(0, base) * k + (d > HW + 2 ? (fbm(x / 9, z / 9, 2, 9) - 0.5)
    };
    const terrain = new Terrain(-420, -L - 50, 420, L + 50, 8, dunes, (x, z) => Mat
    const world = new CollisionWorld(8);
    world.terrain = terrain; world.bounds = L + 40; world.boundsX = 400;
    const root = new Node('map_highway');
    const sandMat = new Material({ name: 'sand', map: sandTex, color: [1.05, 0.92,
    for (const n of terrain.mesh(sandMat, 1 / 7)) root.add(n);
    // far flat desert + mountains
    const S = 6000; const far = new Node('far_sand'); far.mesh = { prims: [makePrim
    { const mt = mountains(sandTex, 1500, 3200, 1.1, 4, 260); mt.userData = { follo
    // road surface
    const roadMat = new Material({ name: 'road', map: roadTexture(), roughness: 0.8
    const step = 5;
    const chunkLen = 400;
    for (let z0 = -L; z0 < L; z0 += chunkLen) {
        const P = [], N = [], U = [], I = []; const across = [-15, -12.3, 12.3, 15];
        let row = 0;
        for (let z = z0; z <= Math.min(L, z0 + chunkLen); z += step, row++) {
            const c = cx(z), t = dcx(z); const nl = Math.hypot(1, t); const px = 1 / nl
            for (const a of across) { P.push(c + px * a, 0.03, z + pz * a); N.push(0, 1
        }
        for (let r = 0; r < row - 1; r++) for (let k = 0; k < across.length - 1; k++)
            const n = new Node('road'); n.mesh = { prims: [makePrim(P, N, U, I, roadMat)]
    }
    // props, merged per chunk and material
    const concrete = new Material({ name: 'jersey', map: stuccoTexture([196, 192, 1
    const metal = new Material({ name: 'pole', color: [0.55, 0.57, 0.6, 1], roughne
    const lampMat = new Material({ name: 'lamp', color: [0.95, 0.9, 0.75, 1], emiss
    const trunkMat = new Material({ name: 'palmTrunk', color: [0.42, 0.32, 0.22, 1]
    const frondMat = new Material({ name: 'palmFrond', color: [0.28, 0.42, 0.16, 1]
    const shrubMat = new Material({ name: 'shrub', color: [0.42, 0.45, 0.26, 1], ro
    const wallMat = new Material({ name: 'najdi', map: stuccoTexture([196, 160, 118
    const trimMat = new Material({ name: 'wood', color: [0.33, 0.2, 0.1, 1], roughn
    const whiteMat = new Material({ name: 'white', map: stuccoTexture([236, 232, 22
    const domeMat = new Material({ name: 'dome', color: [0.2, 0.55, 0.45, 1], rough
    const chunks = new Map();

```

```

const chunkOf = (z) => { const k = Math.floor(z / 500); if (!chunks.has(k)) chu
const r = rng(1234);
// median barrier + street lights
for (let z = -L; z < L; z += 10) {
  const c = cx(z + 5), yaw = Math.atan2(dcx(z + 5), 1);
  const ch = chunkOf(z);
  ch.jersey.push().translate(c, 0, z + 5).rotate(0, 1, 0, yaw); ch.jersey.box(0
colBox(world, c, 0.45, z + 5, 0.7, 0.9, 10, yaw, 6);
  if (Math.round(z) % 50 === 0) { streetLight(ch.pole, ch.lamp, c, 0.9, z + 5,
}
// palms along both shoulders and shrubs in the sand
for (let z = -L; z < L; z += 56) {
  for (const side of [-1, 1]) {
    const zz = z + (side > 0 ? 28 : 0); const c = cx(zz); const x = c + side *
    const ch = chunkOf(zz);
    palm(ch.trunk, ch.frond, x, terrain.h(x, zz), zz, 7 + r() * 3, r);
    colCyl(world, x, terrain.h(x, zz), zz, 0.3, 8, 7);
  }
}
for (let i = 0; i < 1400; i++) {
  const z = (r() * 2 - 1) * L; const side = r() < 0.5 ? -1 : 1; const d = 24 +
  if (Math.abs(x) > 415) continue;
  shrubs(chunkOf(z).shrub, x, terrain.h(x, z) - 0.05, z, 0.8 + r() * 1.4, r);
}
// landmarks: overpasses, gantry signs, houses, a mosque, a petrol station, sah
const signs = [];
const landmark = (z, fn) => fn(chunkOf(z), cx(z), z, Math.atan2(dcx(z), 1));
for (const zb of [-2600, -300, 2300]) landmark(zb, (ch, c, z, yaw) => {
  ch.white.push().translate(c, 0, z).rotate(0, 1, 0, yaw);
  ch.white.box(0, 7.2, 0, 70, 1.2, 14); ch.white.box(0, 8.2, 6.8, 70, 1.0, 0.4)
  for (const px of [-20, -1.4, 1.4, 20]) ch.white.box(px, 3.3, 0, 1.6, 6.6, 8);
  ch.white.pop();
  for (const px of [-20, -1.4, 1.4, 20]) colBox(world, c + px * Math.cos(yaw),
  colBox(world, c, 7.2, z, 70, 1.2, 14, yaw, 4);
});
const signDefs = [[-3600, ['\u0627\u0644\u0631\u064a\u0627\u0636', '\u0627\u0644\u0631\u064a\u0627\u0636'], [-['\u0627\u0644\u0646\u062e\u064a\u0644', '\u0627\u0644\u0646\u062e\u064a\u0644'], 20
for (const [zs, lines] of signDefs) landmark(zs, (ch, c, z, yaw) => {
  ch.pole.push().translate(c, 0, z).rotate(0, 1, 0, yaw);
  ch.pole.box(-13.5, 4, 0, 0.35, 8, 0.35); ch.pole.box(13.5, 4, 0, 0.35, 8, 0.3
  ch.pole.pop();
  colCyl(world, c - 13.5 * Math.cos(yaw), 0, z + 13.5 * Math.sin(yaw), 0.25, 8,
  signs.push({ c, z, yaw, lines });
});
// houses & mosque & petrol station
for (let k = 0; k < 26; k++) {
  const z = -L + 200 + k * 310 + r() * 80; const side = r() < 0.5 ? -1 : 1; con

```

```

    const ch = chunkOf(z); const y = terrain.h(x, z) - 0.2; const yaw = (r() - 0.
    const d = najdiHouse(ch.najdi, ch.wood, x, y, z, yaw, r);
    colBox(world, x, y + d.H / 2, z, d.W, d.H, d.D, yaw, 4);
}
landmark(-1500, (ch, c, z) => { const x = c + 95; mosque(ch.white, ch.dome, x,
landmark(1250, (ch, c, z, yaw) => {
    const x = c - 48; const y = 0;
    ch.white.box(x, 5.4, z, 22, 0.8, 16); // ca
    for (const [px, pz] of [[-8, -5], [8, -5], [-8, 5], [8, 5]]) { ch.pole.box(x
    for (const pz of [-4, 4]) { ch.najdi.box(x, 0.8, z + pz, 1.2, 1.6, 0.8); colB
    ch.white.box(x - 20, 2.5, z, 10, 5, 14); colBox(world, x - 20, 2.5, z, 10, 5,
    signs.push({ c: x, z: z - 9, yaw: 0, lines: ['بنزين', 'محطة'], y: 7.2
});
// saher speed camera
const saher = [];
for (const zs of [-3000, 400, 2900]) landmark(zs, (ch, c, z) => { const x = c +
// emit chunk meshes
const mats = { jersey: concrete, pole: metal, lamp: lampMat, trunk: trunkMat, f
for (const [k, ch] of chunks) {
    const n = new Node('props_' + k); const prims = [];
    for (const key in ch) if (ch[key].I.length) prims.push(ch[key].prim(mats[key]
    n.mesh = { prims }; root.add(n);
}
// sign faces
let si = 0;
for (const s of signs) {
    const mat = new Material({ name: 'signFace', map: signTexture(s.lines, s.col
    const mb = new MB(); const w = s.small ? 3.2 : 8, h = s.small ? 1.6 : 3.6, y
    mb.push().translate(s.c, y, s.z).rotate(0, 1, 0, s.yaw + Math.PI);
    const a = mb.v(-w / 2, -h / 2, 0, 0, 0, 1, 0, 1), b = mb.v(w / 2, -h / 2, 0,
    mb.quad(a, b, c, d); mb.pop();
    const n = new Node('sign'); n.mesh = { prims: [mb.prim(mat)] }; root.add(n);
}
world.build();
const z0 = -L + 150;
const cones = []; for (let i = 0; i < 8; i++) cones.push([cx(z0 + 60 + i * 12)
return {
    name: 'highway', root, world, terrain, saher,
    spawn: { x: cx(z0) - 7, z: z0, yaw: Math.atan2(dcx(z0), 1) }, showroom: { x:
    cones, npcs: [[cx(z0 + 40) + 17, z0 + 40, -1.6], [cx(z0 + 43) + 17.5, z0 + 43
    env: { sunDir: new V3(-0.35, 0.62, -0.7).norm(), sunColor: [3.3, 2.8, 2.2], s
    roadX: cx,
    // endless: drive off one end and you continue from the other, on the same sp
    wrap: (x, z) => { const W = L - 120; if (Math.abs(z) < W) return null; const
};
}

```

```
// ----- desert dunes -----
function buildDesert(assets, sandTex) {
  const R = 1400;
  const dunes = (x, z) => {
    const d = Math.hypot(x, z);
    // flat hard-packed arena in the middle for hajwala, rolling dunes around it
    const arena = sstep(160, 320, d);
    // crescent dune ridges (sharp crest, soft windward side)
    const ridge = (u) => { const f = u - Math.floor(u); return f < 0.7 ? (f / 0.7
    const warp = fbm(x / 220, z / 220, 3, 2) * 3;
    const r1 = ridge(z / 95 + x / 380 + warp) * (0.6 + fbm(x / 300, z / 300, 2, 8
    const r2 = ridge(x / 70 - z / 260 + warp * 0.7) * 7;
    const rolling = (fbm(x / 120, z / 120, 4, 5) - 0.4) * 10;
    const near = (fbm(x / 40, z / 40, 3, 11) - 0.5) * 1.6;
    return (r1 + r2 + rolling) * arena + near * (0.4 + arena) + Math.max(0, d - R
  };
  const terrain = new Terrain(-R, -R, R, R, 8, dunes, () => 3);
  const world = new CollisionWorld(8); world.terrain = terrain; world.bounds = R
  const root = new Node('map_desert');
  const sandMat = new Material({ name: 'duneSand', map: sandTex, color: [1.12, 0.
  for (const n of terrain.mesh(sandMat, 1 / 6)) root.add(n);
  const S = 7000; const far = new Node('far_sand'); far.mesh = { prims: [makePrim
  root.add(mountains(sandTex, 1900, 3600, 1.4, 7));
  // shrubs, acacia trees and rocks
  const r = rng(99);
  const shrubMat = new Material({ name: 'shrub', color: [0.4, 0.42, 0.24, 1], rou
  const rockMat = new Material({ name: 'rock', map: mountainTexture(), color: [0.
  const trunkMat = new Material({ name: 'acaciaTrunk', color: [0.3, 0.22, 0.15, 1
  const canopyMat = new Material({ name: 'acacia', color: [0.33, 0.4, 0.18, 1], r
  const tentMat = new Material({ name: 'tent', color: [0.12, 0.1, 0.09, 1], rough
  const rugMat = new Material({ name: 'rug', color: [0.55, 0.12, 0.1, 1], roughne
  const chunks = new Map();
  const chunkOf = (x, z) => { const k = Math.floor(x / 500) + ',' + Math.floor(z
  for (let i = 0; i < 2000; i++) {
    const x = (r() * 2 - 1) * R * 0.95, z = (r() * 2 - 1) * R * 0.95; if (Math.hy
    const y = terrain.h(x, z); const n = terrain.n(x, z); if (n[1] < 0.9 && r() <
    const ch = chunkOf(x, z); const t = r();
    if (t < 0.8) shrubs(ch.shrub, x, y - 0.05, z, 0.6 + r() * 1.2, r);
    else if (t < 0.93) { const s = 0.5 + r() * 1.6; ch.rock.pillow(x, y + s * 0.2
    else {
      const h = 2.2 + r() * 1.5; ch.trunk.box(x, y + h / 2, z, 0.25, h, 0.25);
      for (let k = 0; k < 4; k++) ch.canopy.pillow(x + (r() - 0.5) * 2, y + h + r
      colCyl(world, x, y, z, 0.25, h, 7);
    }
  }
}
```

```

for (const [k, ch] of chunks) { const n = new Node('dprops_' + k); n.mesh = { p
// a desert camp (بيت شعر) near the arena
{ const mb = new MB(), rug = new MB(); const x = 60, z = 70, y = terrain.h(x, z
mb.push().translate(x, y, z); mb.box(0, 2.2, 0, 12, 0.08, 7); for (const px o
rug.box(x, y + 0.03, z, 10, 0.05, 5.5);
const n = new Node('camp'); n.mesh = { prims: [mb.prim(tentMat), rug.prim(rug
colBox(world, x, y + 1.1, z - 3.5, 12, 2.2, 0.2, 0, 4); }
world.build();
const cones = []; for (let i = 0; i < 8; i++) { const a = i / 8 * Math.PI * 2;
return {
name: 'desert', root, world, terrain,
spawn: { x: -20, z: -30, yaw: 0.3 }, showroom: { x: 0, z: 0, yaw: 0.8 },
cones, npcs: [[50, 64, 2.6], [53, 66, 2.8], [66, 64, -2.4]],
env: { sunDir: new V3(-0.55, 0.42, -0.55).norm(), sunColor: [3.6, 2.7, 1.9],
};
}

// ----- Riyadh: photogrammetry block of the real city + King Fahd Road with
function glassTexture(tint, key) {
// 4 floors (4 m each) x 4 mullion bays (4 m) of curtain wall
return ctex('glass_' + key, 256, 256, (g, w, h) => {
const [r, gg, b] = tint;
const grd = g.createLinearGradient(0, 0, w, h); grd.addColorStop(0, `rgb(${r
g.fillStyle = grd; g.fillRect(0, 0, w, h);
const rr = rng(r * 3 + b);
for (let fy = 0; fy < 4; fy++) for (let bx = 0; bx < 4; bx++) { const k = rr(
g.fillStyle = 'rgba(210,215,220,0.55)'; for (let fy = 0; fy < 4; fy++) g.fill
g.fillStyle = 'rgba(190,196,204,0.6)'; for (let bx = 0; bx < 4; bx++) g.fillR
});
}
function concreteTexture() {
return ctex('urbanGround', 256, 256, (g, w, h) => {
noiseFill(g, w, h, [168, 156, 136], 22);
g.fillStyle = 'rgba(90,80,64,0.08)'; const r = rng(77); for (let i = 0; i < 6
});
}
// loft closed cross-sections into a faceted wall (flat normals per facet, UVs in
function loft(mb, rings, cap = true) {
for (let k = 0; k < rings.length - 1; k++) {
const A = rings[k], B = rings[k + 1], n = A.pts.length; let s = 0;
let cx = 0, cz = 0; for (const p of A.pts) { cx += p[0]; cz += p[1]; } cx /=
for (let i = 0; i < n; i++) {
const j = (i + 1) % n;
const a = [A.pts[i][0], A.y, A.pts[i][1]], b = [A.pts[j][0], A.y, A.pts[j][
const e1 = [b[0] - a[0], b[1] - a[1], b[2] - a[2]], e2 = [d[0] - a[0], d[1]
let nx = e1[1] * e2[2] - e1[2] * e2[1], ny = e1[2] * e2[0] - e1[0] * e2[2],

```

```

    const l = Math.hypot(nx, ny, nz) || 1; nx /= l; ny /= l; nz /= l;
    // orient outward (away from the ring centre)
    const out = nx * ((a[0] + b[0]) / 2 - cx) + nz * ((a[2] + b[2]) / 2 - cz) >
    if (!out) { nx = -nx; ny = -ny; nz = -nz; }
    const len = Math.hypot(e1[0], e1[2]);
    const va = mb.v(a[0], a[1], a[2], nx, ny, nz, s / 16, -A.y / 16), vb = mb.v
    const vc = mb.v(c[0], c[1], c[2], nx, ny, nz, (s + len) / 16, -B.y / 16), v
    // (b-a)x(d-a) == +n when out, so a,b,d / b,c,d is front-facing; otherwise
    if (out) mb.I.push(va, vb, vd, vb, vc, vd); else mb.I.push(va, vd, vb, vb,
    s += len;
  }
}
if (cap) {
  const T = rings[rings.length - 1]; const n = T.pts.length; let cx = 0, cz = 0
  const c = mb.v(cx, T.y, cz, 0, 1, 0, 0, 0);
  const vs = T.pts.map(p => mb.v(p[0], T.y, p[1], 0, 1, 0, 0.1, 0.1));
  for (let i = 0; i < n; i++) {
    const p = T.pts[i], q = T.pts[(i + 1) % n]; const up = (q[0] - cx) * (p[1]
    if (up < 0) mb.I.push(c, vs[(i + 1) % n], vs[i]); else mb.I.push(c, vs[i],
  }
}
}
function sphere(mb, cx, cy, cz, r, seg = 24) {
  const base = mb.P.length / 3;
  for (let j = 0; j <= seg; j++) { const th = j / seg * Math.PI; for (let i = 0;
  for (let j = 0; j < seg; j++) for (let i = 0; i < seg; i++) { const a = base +
}
const lensPts = (W, D, n = 14, x0 = -1) => {
  // pointed-ellipse (lens) plan, optional clip to x >= x0 (for the two arms of K
  const top = [], bot = []; const a = x0 < 0 ? -W / 2 : x0;
  for (let i = 0; i <= n; i++) { const x = lerp(a, W / 2, i / n); const z = D / 2
  return x0 < 0 ? [...top.slice(0, n), ...bot.reverse().slice(0, n)] : [...top, .
};
const tf = (pts, x, z, yaw) => { const c = Math.cos(yaw), s = Math.sin(yaw); retu

function kingdomCentre(glass, steel, world, X, Z, yaw) {
  // برج المملكة: lens-shaped tower, parabolic opening in the upper third with th
  const rings = [];
  for (let y = 0; y <= 186; y += 6) { const t = y / 186; rings.push({ y, pts: tf(
  loft(glass, rings, false);
  for (const side of [1, -1]) {
    const arm = [];
    for (let y = 186; y <= 300; y += 4) {
      const t = (y - 186) / 114; const W = lerp(66, 56, t), D = lerp(33, 26, t *
      arm.push({ y, pts: tf(lensPts(W, D, 10, a).map(([u, v]) => [u * side, v * s
    }
  }

```

```

    loft(glass, arm, true);
}
// sky bridge + the steel arch under it
steel.push().translate(X, 0, Z).rotate(0, 1, 0, yaw); steel.box(0, 292, 0, 50,
colBox(world, X, 90, Z, 64, 180, 26, yaw, 4);
}
function faisaliah(glass, steel, gold, world, X, Z, yaw) {
    // برج الفيصلية: tapering square tower, open lattice top with the golden globe
    const half = (y) => 27 * Math.pow(1 - y / 250, 0.72);
    const sq = (s) => [[-s, -s], [s, -s], [s, s], [-s, s]];
    const rings = []; for (let y = 0; y <= 178; y += 6) rings.push({ y, pts: tf(sq(
    loft(glass, rings, true);
    // four corner legs continuing to the top
    for (const [sx, sz] of [[-1, -1], [1, -1], [1, 1], [-1, 1]]) {
        const leg = []; for (let y = 176; y <= 238; y += 4) { const h = Math.max(1.2,
        loft(steel, leg, true);
    }
    sphere(gold, X, 206, Z, 11.5);
    steel.push().translate(X, 0, Z); steel.box(0, 252, 0, 0.9, 30, 0.9); steel.box(
    colBox(world, X, 60, Z, 50, 120, 50, yaw, 4);
}
function rajhiTower(glass, fins, world, X, Z, yaw) {
    // برج الراجحي (مقر المصرف): slender glass slab with rounded ends, vertical fin
    const plan = (W, D) => { const pts = []; const r = D / 2; for (let i = 0; i <=
    const rings = []; for (let y = 0; y <= 176; y += 8) rings.push({ y, pts: tf(pla
    rings.push({ y: 184, pts: tf(plan(40, 22), X, Z, yaw) }); rings.push({ y: 196,
    loft(glass, rings, true);
    fins.push().translate(X, 0, Z).rotate(0, 1, 0, yaw);
    for (let i = -3; i <= 3; i++) { fins.box(i * 5.2, 90, 13.3, 0.6, 180, 1.2); fin
    fins.box(0, 205, 0, 1.2, 18, 1.2);
    fins.pop();
    colBox(world, X, 90, Z, 46, 180, 26, yaw, 4);
}

function buildRiyadh(assets, sandTex) {
    const src = assets.riyadh.root; src.updateWorld();
    const world = new CollisionWorld(6);
    let meta = null; src.traverse(n => { if (!meta && n.userData && n.userData.grid
    // photogrammetry material: baked lighting, keep it matte
    src.traverse(n => {
        if (!n.mesh) return;
        if (/walls-col/.test(n.name)) { for (const p of n.mesh.prims) world.addTriang
        for (const p of n.mesh.prims) { const m = p.material; m.roughness = 1; m.meta
    });
    // ground heightfield of the block (cm, int16) blended to 0 outside
    const gd = meta.grid; const raw = atob(gd.h); const buf = new Uint8Array(raw.le

```

```

const H16 = new Int16Array(buf.buffer); const GH = new Float32Array(H16.length)
const gx1 = gd.x0 + (gd.nx - 1) * gd.step, gz1 = gd.z0 + (gd.nz - 1) * gd.step;
const gridH = (x, z) => {
  if (x <= gd.x0 || z <= gd.z0 || x >= gx1 || z >= gz1) return 0;
  const fx = (x - gd.x0) / gd.step, fz = (z - gd.z0) / gd.step; const i = Math.
  const a = GH[j * gd.nx + i], b = GH[j * gd.nx + i + 1], c = GH[(j + 1) * gd.n
  const h = (a * (1 - u) + b * u) * (1 - v) + (c * (1 - u) + d * u) * v;
  const edge = Math.min(x - gd.x0, gx1 - x, z - gd.z0, gz1 - z);
  return h * sstep(2, 14, edge);
};
// roads: King Fahd Road (north-south, east of the block) + a ring around the b
const BX = 260, L = 2600;
const roads = [[BX, -L, BX, L], [-236, -470, BX, -470], [-236, 470, BX, 470], [
const onRoad = (x, z) => { for (const [x0, z0, x1, z1] of roads) { if (x0 === x
const inBlock = (x, z) => x > gd.x0 + 6 && x < gx1 - 6 && z > gd.z0 + 6 && z <
const terrain = {
  h: gridH,
  n: (x, z) => { const e = 1; const dx = gridH(x + e, z) - gridH(x - e, z), dz
  tag: (x, z) => inBlock(x, z) || onRoad(x, z) ? 1 : 3,
};
world.terrain = terrain; world.bounds = L + 60; world.boundsX = 900;
const root = new Node('map_riyadh'); root.add(src);
// outer ground: concrete/sand plane around the block (4 quads leaving the bloc
const gMat = new Material({ name: 'urban', map: concreteTexture(), roughness: 1
const bx0 = gd.x0 + 4, bx1 = gx1 - 4, bz0 = gd.z0 + 4, bz1 = gz1 - 4, S = 4000;
const P = [], N = [], U = [], I = [];
const quad = (x0, z0, x1, z1, y) => { const b = P.length / 3; for (const [x, z]
quad(-S, -S, S, bz0, -0.03); quad(-S, bz1, S, S, -0.03); quad(-S, bz0, bx0, bz1
{ const n = new Node('ground'); n.mesh = { prims: [makePrim(P, N, U, I, gMat)]
root.add(mountains(sandTex, 2200, 3600, 0.35, 9));
// road meshes
const roadMat = new Material({ name: 'road', map: roadTexture(), roughness: 0.8
{ const P = [], N = [], U = [], I = [];
  for (const [x0, z0, x1, z1] of roads) {
    const len = Math.hypot(x1 - x0, z1 - z0), dx = (x1 - x0) / len, dz = (z1 -
    const b = P.length / 3;
    for (const [s, a] of [[-ext, -15], [-ext, 15], [len + ext, 15], [len + ext,
    I.push(b, b + 1, b + 2, b, b + 2, b + 3);
  }
  // make both windings visible from above
  for (let i = 0; i < I.length; i += 3) { const a = I[i], b = I[i + 1], c = I[i
  const n = new Node('roads'); n.mesh = { prims: [makePrim(P, N, U, I, roadMat)
// materials
const concrete = new Material({ name: 'jersey', map: stuccoTexture([196, 192, 1
const metal = new Material({ name: 'pole', color: [0.55, 0.57, 0.6, 1], roughne
const lampMat = new Material({ name: 'lamp', color: [0.95, 0.9, 0.75, 1], emiss

```

```

const trunkMat = new Material({ name: 'palmTrunk', color: [0.42, 0.32, 0.22, 1]
const frondMat = new Material({ name: 'palmFrond', color: [0.28, 0.42, 0.16, 1]
const kcGlass = new Material({ name: 'kcGlass', map: glassTexture([70, 96, 118]
const fzGlass = new Material({ name: 'fzGlass', map: glassTexture([92, 104, 96]
const rjGlass = new Material({ name: 'rjGlass', map: glassTexture([54, 92, 96],
const steel = new Material({ name: 'steel', color: [0.72, 0.74, 0.77, 1], rough
const gold = new Material({ name: 'gold', color: [0.95, 0.72, 0.3, 1], roughnes
const bldGlass = [glassTexture([80, 92, 104], 'b1'), glassTexture([110, 104, 92
const stone = new Material({ name: 'stone', map: stuccoTexture([214, 196, 164])
const mb = { jersey: new MB(), pole: new MB(), lamp: new MB(), trunk: new MB(),
const r = rng(2030);
// King Fahd Road median: jersey barrier, palms, double street lights
for (let z = -L; z < L; z += 10) {
  if (Math.abs(z + 5 - 470) < 18 || Math.abs(z + 5 + 470) < 18 || Math.abs(z +
  mb.jersey.box(BX, 0.45, z + 5, 0.6, 0.9, 10.05); mb.jersey.box(BX, 0.08, z +
  colBox(world, BX, 0.45, z + 5, 0.7, 0.9, 10, 0, 6);
  if (Math.round(z) % 40 === 0) { streetLight(mb.pole, mb.lamp, BX, 0.9, z + 5,
}
for (let z = -L + 25; z < L; z += 34) for (const side of [-1, 1]) {
  const x = BX + side * 19; if (onRoad(x, z) && Math.abs(x - BX) < 15) continue
  if ([470, -470, 1400, -1400].some(j => Math.abs(z - j) < 20)) continue;
  palm(mb.trunk, mb.frond, x, 0, z, 8 + r() * 4, r); colCyl(world, x, 0, z, 0.3
}
// landmark towers along King Fahd Road
kingdomCentre(mb.kc, mb.steel, world, 345, 900, 0);
faisaliah(mb.fz, mb.steel, mb.gold, world, 350, -900, 0.785);
rajhiTower(mb.rj, mb.steel, world, 335, 60, Math.PI / 2);
// filler towers & low-rise along both sides (outside the photogrammetry block)
const keepOut = [[345, 900, 70], [350, -900, 60], [335, 60, 45]];
const place = (x, z, w, d, h, yaw) => {
  if (keepOut.some(([kx, kz, kr]) => Math.hypot(x - kx, z - kz) < kr + Math.max
  if (inBlock(x, z) || onRoad(x, z)) return;
  const tall = h > 30; const m = tall ? mb['b' + ((r() * 3) | 0)] : mb.stone;
  const pts = [[-w / 2, -d / 2], [w / 2, -d / 2], [w / 2, d / 2], [-w / 2, d /
  const rings = [{ y: 0, pts: tf(pts, x, z, yaw) }, { y: h, pts: tf(pts, x, z,
  if (tall && r() < 0.5) rings.push({ y: h + 6, pts: tf(pts.map(([a, b]) => [a
  loft(m, rings, true);
  if (!tall) { mb.stone.push().translate(x, 0, z).rotate(0, 1, 0, yaw); for (le
  colBox(world, x, h / 2, z, w, h, d, yaw, 4);
};
for (let z = -L + 60; z < L - 60; z += 48 + r() * 30) {
  place(BX + 62 + r() * 30, z, 24 + r() * 18, 22 + r() * 16, r() < 0.45 ? 40 +
  if (Math.abs(z) > 520) place(BX - 62 - r() * 30, z, 24 + r() * 18, 22 + r() *
  if (Math.abs(z) > 520 && Math.abs(z) < 1380) place(-300 - r() * 40, z, 26 + r
}
// residential districts (villas with Najdiparapets) filling the city around t

```

```

for (let gx = -820; gx <= 820; gx += 36) for (let gz = -L + 40; gz <= L - 40; g
  if (r() < 0.4) continue;
  const x = gx + (r() - 0.5) * 6, z = gz + (r() - 0.5) * 6;
  if (Math.abs(x - BX) < 60 || (x > -290 && x < BX && Math.abs(z) < 520) || Mat
  if (Math.hypot(x - BX, z - (-1560)) < 120) continue;
  const far = Math.abs(x - BX) > 500;
  const w = 13 + r() * 8, d = 13 + r() * 8, h = 6.5 + (r() < 0.3 ? 3.5 : 0) + (
  const pts = [[-w / 2, -d / 2], [w / 2, -d / 2], [w / 2, d / 2], [-w / 2, d /
  loft(mb.stone, [{ y: 0, pts: tf(pts, x, z, 0) }, { y: h, pts: tf(pts, x, z, 0
  if (!far) colBox(world, x, h / 2, z, w, h, d, 0, 4);
}
// road signs (Arabic) on gantries over King Fahd Road
const signs = [];
for (const [zs, lines] of [[-2200, ['العلياء', '↑ طريق الملك فهد']], [-['م ٥٠٠ م
  mb.pole.box(BX - 13.5, 4, zs, 0.35, 8, 0.35); mb.pole.box(BX + 13.5, 4, zs, 0
  colCyl(world, BX - 13.5, 0, zs, 0.25, 8, 5); colCyl(world, BX + 13.5, 0, zs,
  signs.push({ c: BX + 7, z: zs, yaw: 0, lines }); signs.push({ c: BX - 7, z: z
}
const saher = [];
for (const zs of [-1800, 1100]) { const x = BX + 17; mb.pole.box(x, 2.5, zs, 0.
const mats = { jersey: concrete, pole: metal, lamp: lampMat, trunk: trunkMat, f
{ const n = new Node('riyadh_props'); const prims = []; for (const k in mb) if
let si = 0;
// ---- park (حديقة) next to the start: grass, trees, a swing and benches
{ const sz0 = -1560, px = BX - 58, pz = sz0 + 30, gy = terrain.h(px, pz);
  const grass = new Material({ name: 'parkGrass', color: [0.24, 0.5, 0.18, 1],
  const g = new MB(), w = new MB(), l = new MB(), ir = new MB(), pa = new MB();
  g.box(px, gy + 0.03, pz, 44, 0.06, 44); pa.box(px, gy + 0.07, pz, 3, 0.04, 44
  const r = rng(77);
  for (let i = 0; i < 18; i++) { const a = r() * Math.PI * 2, d = 8 + r() * 12,
    w.box(x, gy + h / 2, z, 0.35, h, 0.35); sphere(l, x, gy + h + 1.2, z, 1.8 +
  // swing: A-frames + top bar + two seats on chains
  const sx = px + 10, sz2 = pz - 10;
  for (const dz of [-1.6, 1.6]) { ir.push().translate(sx - 0.7, gy + 1.2, sz2 +
  ir.box(sx, gy + 2.45, sz2, 0.1, 0.1, 3.4);
  for (const dz of [-0.8, 0.8]) { ir.box(sx - 0.2, gy + 1.5, sz2 + dz, 0.03, 1.
  colBox(world, sx, gy + 1.2, sz2, 1.6, 2.5, 3.4, 0, 7);
  for (const [bx, bz] of [[px - 6, pz + 3], [px + 6, pz - 3], [px - 3, pz - 7]]
  const n = new Node('park'); n.mesh = { prims: [g.prim(grass), w.prim(wood), l
  signs.push({ c: px + 23, z: pz, yaw: -Math.PI / 2, lines: ['مراجيح', 'حديقة']
}
// ---- mall (مول) east of the road
if (assets.mall) {
  const src = assets.mall.root; const mx = BX + 250, mz = -1150;
  let y0 = 1e9, x0 = 1e9, x1 = -1e9, z0 = 1e9, z1 = -1e9; src.updateWorld();
  src.traverse(nn => { if (!nn.mesh) return; for (const p of nn.mesh.prims) { c

```

```

    const holder = new Node('mall'); holder.add(src); const gy = terrain.h(mx, mz);
    holder.pos.set(mx - (x0 + x1) / 2, gy - y0, mz - (z0 + z1) / 2); holder.update();
    src.traverse(nn => { if (!nn.mesh) return; for (const p of nn.mesh.prims) world.signs.push({ c: BX + 14, z: mz - 60, yaw: -Math.PI / 2, lines: ['← المول', 'M'] });
  });
  for (const s of signs) {
    const mat = new Material({ name: 'signFace', map: signTexture(s.lines, s.colors) });
    const m2 = new MeshBuilder(); const w = s.small ? 3.2 : 8, h = s.small ? 1.6 : 3.6, y = s.y;
    m2.push().translate(s.c, y, s.z).rotate(0, 1, 0, s.yaw + Math.PI);
    const a = m2.v(-w / 2, -h / 2, 0, 0, 0, 1, 0, 1), b = m2.v(w / 2, -h / 2, 0, 0, 0, 1, 0, 1), c = m2.v(-w / 2, h / 2, 0, 0, 0, 1, 0, 1), d = m2.v(w / 2, h / 2, 0, 0, 0, 1, 0, 1);
    m2.quad(a, b, c, d); m2.pop();
    const n = new Node('sign'); n.mesh = { prims: [m2.prim(mat)] }; root.add(n);
  }
  world.build();
  const sz = -1560;
  const cones = []; for (let i = 0; i < 8; i++) cones.push([BX - 8 + (i % 2 ? 2 : -2), sz, 0]);
  return {
    name: 'riyadh', root, world, terrain, saher,
    spawn: { x: BX - 8, z: sz, yaw: 0 }, showroom: { x: BX - 7, z: sz - 30, yaw: 0 },
    cones, npcs: [[BX - 24, sz + 30, 1.5], [BX - 24.6, sz + 32, 1.7], [BX + 24, sz + 30, 1.5], [BX + 24.6, sz + 32, 1.7]],
    env: { sunDir: new V3(-0.4, 0.66, -0.62).norm(), sunColor: [3.2, 2.85, 2.35], fog: 0.5 };
  };
}

// ----- physics test ground (ماب التجارب): bumps, stairs, two-wheel ramp, w
function skidTexture() {
  const c = document.createElement('canvas'); c.width = c.height = 512; const g = c.getContext('2d');
  g.fillStyle = '#23303a'; g.fillRect(0, 0, 512, 512);
  g.strokeStyle = 'rgba(255,255,255,0.8)'; g.lineWidth = 6; for (const r of [120, 240, 360]) g.strokeRect(8, 8, 496, 504);
  return { image: c, wrapS: 33071, wrapT: 33071, srgb: true, gl: null };
}

// terrain patch from a height function (visual + collision); uv = world units *
function heightPatch(world, mb, x0, z0, x1, z1, fn, n, tag, uvs) {
  const nx = n, nz = Math.max(2, Math.round(n * (z1 - z0) / (x1 - x0))), P = [], H = (x, z) => fn(x, z);
  for (let j = 0; j <= nz; j++) for (let i = 0; i <= nx; i++) {
    const x = x0 + (x1 - x0) * i / nx, z = z0 + (z1 - z0) * j / nz, y = H(x, z),
    const gx = (H(x + 1, z) - H(x - 1, z)) / 2, gz = (H(x, z + 1) - H(x, z - 1)) / 2;
    P.push(x, y, z);
    mb.v(x, y, z, -gx / 1, 1 / 1, -gz / 1, uvs >= 40 ? i / nx : x * uvs, uvs >= 40 ? j / nz : z * uvs);
  }
  for (let j = 0; j < nz; j++) for (let i = 0; i < nx; i++) { const a = j * (nx + 1) + i; world.addTriangles(P, a, null, tag); }
}

function prism(world, mb, x, z0, width, profile, tag = 2) {

```

```

// profile: [[z, y], ...] along the lane (closed at y=0), extruded across x (wi
const hx = width / 2, P = [], I = [];
const pts = [[profile[0][0], 0], ...profile, [profile[profile.length - 1][0], 0]
for (const [z, y] of pts) P.push(x - hx, y, z0 + z, x + hx, y, z0 + z);
const n = pts.length;
for (let i = 0; i < n - 1; i++) { const a = i * 2; I.push(a, a + 2, a + 1, a +
// side caps (fan)
for (let i = 1; i < n - 1; i++) { I.push(0, (i + 1) * 2, i * 2); I.push(1, i *
world.addTriangles(P, I, null, tag);
// visual: flat-shaded quads with proper normals
for (let i = 0; i < n - 1; i++) {
    const [za, ya] = pts[i], [zb, yb] = pts[i + 1]; const dz = zb - za, dy = yb -
    const a = mb.v(x - hx, ya, z0 + za, 0, ny, nz, 0, za / 2), b = mb.v(x + hx, y
    mb.I.push(a, c, b, a, d, c);
}
for (const s of [-1, 1]) { const base = mb.v(x + s * hx, pts[0][1], z0 + pts[0]
}
function buildTest(assets, sandTex) {
    const world = new CollisionWorld(4); world.bounds = 150; world.boundsX = 150; w
    const root = new Node('map_test');
    const asphalt = new Material({ name: 'testAsphalt', map: roadTexture(), roughne
    const concrete = new Material({ name: 'testConcrete', map: stuccoTexture([190,
    const yellow = new Material({ name: 'testYellow', map: stuccoTexture([226, 180,
    const red = new Material({ name: 'testRed', map: stuccoTexture([190, 60, 50]),
    const water = new Material({ name: 'water', color: [0.05, 0.3, 0.42, 0.78], rou
    // ground with a hole for the pool
    const pool = { x0: -18, x1: 18, z0: 40, z1: 64, depth: 1.6, level: -0.12 };
    const ramp = 7; // pool entry/exit slopes along z
    const gP = [], gN = [], gU = [], gI = [], col = [];
    const quad = (x0, z0, x1, z1, y0 = 0, y1 = 0) => {
        const b = gP.length / 3;
        for (const [x, z, y] of [[x0, z0, y0], [x1, z0, y0], [x1, z1, y1], [x0, z1, y
        gI.push(b, b + 2, b + 1, b, b + 3, b + 2);
    };
    const S = 150;
    quad(-S, -S, S, pool.z0 - ramp); quad(-S, pool.z1 + ramp, S, S); quad(-S, pool.
    const d = -pool.depth;
    quad(pool.x0, pool.z0 - ramp, pool.x1, pool.z0, 0, d); quad(pool.x0, pool.z0, p
    world.addTriangles(gP, gI, null, 1);
    // pool side walls
    const walls = new MB();
    for (const x of [pool.x0, pool.x1]) { walls.box(x, d / 2, (pool.z0 + pool.z1) /
    { const n = new Node('ground'); n.mesh = { prims: [makePrim(gP, gN, gU, gI, asp
    // water surface
    { const P = [pool.x0, pool.level, pool.z0 - ramp * (pool.level / d), pool.x1, p
    const n = new Node('water'); n.mesh = { prims: [makePrim(P, [0, 1, 0, 0, 1, 0

```

```

const mbY = new MB(), mbR = new MB(), mbC = new MB();
// 1) alternating bumps (x = -40): left wheels, right wheels, left ... the car
for (let i = 0; i < 14; i++) {
    const side = i % 2 ? -1 : 1; const z = -90 + i * 5.2;
    prism(world, i % 2 ? mbY : mbR, -40 + side * 0.85, z, 1.3, [[0, 0.02], [0.35,
}
// full-width speed bumps
for (let i = 0; i < 4; i++) prism(world, mbY, -40, -10 + i * 9, 6, [[0, 0.01],
// 2) stairs up (x = 0) then a ramp down
const step = 0.15, depth = 0.8, nSteps = 7;
const prof = [[0, 0]]; for (let i = 0; i < nSteps; i++) { prof.push([i * depth,
const top = nSteps * step, zTop = nSteps * depth; prof.push([zTop + 6, top]); p
prism(world, mbC, 0, -60, 6, prof, 2);
// 3) two-wheel side ramp (x = 40): only the left wheels climb, the car rolls o
prism(world, mbR, 40 + 0.95, -70, 1.3, [[0, 0.02], [9, 1.05], [9.4, 1.05]], 2);
prism(world, mbR, 48 + 0.95, -20, 1.3, [[0, 0.02], [7, 0.75], [7.3, 0.75]], 2);
// 4) jump ramp (x = -40, far end)
prism(world, mbY, -40, 55, 5, [[0, 0.02], [10, 1.6], [10.5, 1.6]], 2);
// 5) rough rock garden (x = 40, far end): random small blocks
{ const r = rng(42); for (let i = 0; i < 70; i++) { const x = 40 + (r() - 0.5)
{ const n = new Node('test_props'); n.mesh = { prims: [mbY.prim(yellow), mbR.pr
// signs
const signs = [[-40, -97, ['يمين ويسار', 'مطبات']], [['اطلع فيه', 'درج'], 67- ,
let si = 0;
for (const [x, z, lines] of signs) {
    const mat = new Material({ name: 'signFace', map: signTexture(lines, '#0f6b3a
    const m2 = new MB(); m2.push().translate(x + 5, 2.6, z).rotate(0, 1, 0, Math.
    const a = m2.v(-1.6, -0.8, 0, 0, 0, 1, 0, 1), b = m2.v(1.6, -0.8, 0, 0, 0, 1,
    const pole = new MB(); pole.box(x + 5, 0.9, z + 0.05, 0.12, 1.8, 0.12);
    const n = new Node('sign'); n.mesh = { prims: [m2.prim(mat), pole.prim(concre
}
root.add(mountains(sandTex, 900, 1800, 0.45, 5));
{ const sandMat = new Material({ name: 'sand', map: sandTex, color: [1.05, 0.92
    const fP = [], fN = [], fU = [], fI = [];
    for (const [x0, z0, x1, z1] of [[-F, -F, F, -S], [-F, S, F, F], [-F, -S, -S,
    const far = new Node('far_sand'); far.mesh = { prims: [makePrim(fP, fN, fU, f
// ---- more tests on the east side (x 70..140) and west side (x -100)
const sandM = new Material({ name: 'sandPatch', map: sandTex, color: [1.05, 0.9
const skidM = new Material({ name: 'skidPad', map: skidTexture(), roughness: 0.
const mbS = new MB(), mbK = new MB(), mbW = new MB(), mbC2 = new MB(), mbR2 = n
// 6) wet skid pad (low grip) for drifting circles
heightPatch(world, mbK, 72, -118, 118, -72, () => 0.015, 4, 8, 46);
// 7) loose sand patch with small ripples
heightPatch(world, mbS, 75, -55, 115, -25, (x, z) => 0.02 + 0.06 * (1 + Math.si
// 8) side slope (rollover test): climb across 9 m to 4.6 m, flat top, back dow
heightPatch(world, mbC2, 80, -5, 104, 25, (x) => { const u = x - 80; return u <

```

```

// 9) big dune to jump over
heightPatch(world, mbS, 78, 45, 112, 95, (x, z) => 0.02 + 4.2 * Math.exp(-(x -
// 10) crash wall + concrete blocks
mbW.box(132, 1.25, 0, 1.2, 2.5, 60); colBox(world, 132, 1.25, 0, 1.2, 2.5, 60,
for (let i = 0; i < 5; i++) { const z = -22 + i * 11; mbW.box(126, 0.45, z, 1.0
// 11) wave road (car pitches and floats over the crests)
heightPatch(world, mbC2, -100, -80, -90, 10, (x, z) => 0.02 + 0.22 * (1 + Math.
// 12) two narrow planks across the pool (one per wheel track)
for (const sx of [-0.8, 0.8]) prism(world, mbR2, 10 + sx, 28, 0.42, [[0, 0.02],
// 13) quarter pipe wall-ride (west edge)
heightPatch(world, mbC2, -134, 90, -112, 140, (x) => { const u = (-112 - x); re
// 14) steep hill climb (30%) with a plateau
heightPatch(world, mbS, -134, -135, -112, -60, (x, z) => { const u = z + 135; r
// 15) row of kickers (small jumps)
for (let i = 0; i < 4; i++) prism(world, mbR2, -60 + i * 30, 104, 5, [[0, 0.02]
// 16) log bumps across a lane
for (let i = 0; i < 12; i++) { const z = -130 + i * 4; const prof = []; for (le
// 17) mud bowl (slippery dip)
heightPatch(world, mbK, 50, 112, 74, 138, (x, z) => 0.02 - 1.2 * Math.max(0, 1
// 18) narrow gate (low bar) and a zig-zag of concrete blocks
for (const sx of [-1.35, 1.35]) { mbW.box(25 + sx, 1.3, 10, 0.4, 2.6, 0.4); col
mbW.box(25, 2.3, 10, 3.1, 0.25, 0.4); colBox(world, 25, 2.3, 10, 3.1, 0.25, 0.4
for (let i = 0; i < 6; i++) { const x = 25 + (i % 2 ? 1.6 : -1.6), z = 20 + i *
// 19) banked oval turn (east-north)
heightPatch(world, mbC2, 80, 100, 132, 140, (x, z) => { const d = Math.hypot((x
// 20) side-by-side see-saw style tilt board (fixed): left and right wheels on
prism(world, mbR2, -20 + 0.95, 100, 1.3, [[0, 0.02], [6, 0.55], [14, 0.55], [20
prism(world, mbR2, -20 - 0.95, 108, 1.3, [[0, 0.02], [6, 0.55], [14, 0.55], [20
{ const n = new Node('test_more'); n.mesh = { prims: [mbS.prim(sandM), mbK.prim
for (const [x, z, lines] of [[95, -122, ['دُور وفخط', 'حلبة تزلج', 'ثقيل'],
const mat = new Material({ name: 'signFace', map: signTexture(lines, '#0f6b3a
const m2 = new MB(); m2.push().translate(x + 5, 2.6, z).rotate(0, 1, 0, Math.
const a = m2.v(-1.6, -0.8, 0, 0, 0, 1, 0, 1), b = m2.v(1.6, -0.8, 0, 0, 0, 1,
const pole = new MB(); pole.box(x + 5, 0.9, z + 0.05, 0.12, 1.8, 0.12);
const n = new Node('sign'); n.mesh = { prims: [m2.prim(mat), pole.prim(concre
}
world.build();
const slalom = []; for (let i = 0; i < 10; i++) slalom.push([-80, -110 + i * 11
return {
name: 'test', root, world, water: [pool], deepSand: [{ x0: 75, x1: 115, z0: -
spawn: { x: -40, z: -110, yaw: 0 }, showroom: { x: -10, z: -110, yaw: 0.6 },
cones: [[8, -110], [10, -106], [12, -110], [10, -114], ...slalom], npcs: [[-3
env: { sunDir: new V3(-0.3, 0.75, -0.55).norm(), sunColor: [3.1, 2.85, 2.5],
};
}
}

```

```

// ----- the caravan / room you walk into from the Shas bed -----
function buildRoom(assets) {
  const src = assets.caravan.root; src.updateWorld();
  const world = new CollisionWorld(2); world.bounds = 30; world.boundsX = 30; wor
  let x0 = 1e9, x1 = -1e9, z0 = 1e9, z1 = -1e9; const v = new V3();
  src.traverse(n => { if (!n.mesh) return; for (const p of n.mesh.prims) { if (!p
    const a = p.attrs.position.data; for (let i = 0; i < a.length; i += 60) { v.s
  world.build();
  const root = new Node('map_room'); root.add(src);
  let cx = (x0 + x1) / 2, cz = (z0 + z1) / 2;
  // stand on open floor: pick the free floor spot (with free floor around it) cl
  const floor = (x, z) => world.heightAt(x, z, 2.2) < 0.2;
  const free = []; for (let x = x0 + 0.5; x < x1 - 0.5; x += 0.35) for (let z = z
  free.sort((a, b) => Math.hypot(a[0] - cx, a[1] - cz) - Math.hypot(b[0] - cx, b[
  const sp = free[0] || [cx, cz]; const ex = free.find(f => Math.hypot(f[0] - sp[
  cx = sp[0]; cz = sp[1];
  return {
    name: 'room', root, world, room: true,
    spawn: { x: cx, z: cz, yaw: 0 }, showroom: { x: cx, z: cz, yaw: 0 }, exit: {
    cones: [], npcs: [],
    env: { sunDir: new V3(-0.3, 0.8, -0.4).norm(), sunColor: [1.6, 1.5, 1.35], sk
  };
}

return { Terrain, signTexture, buildCity, buildHighway, buildDesert, buildRiyadh,
})();
__M['character'] = (() => {
// Skinned character with procedural animation (model-space bone rotations)
const { V3, Q, M4, clamp, lerp } = __M['math'];
const { Node, Skin } = __M['scene'];

const BONES = ['pelvis', 'spine_01', 'spine_02', 'spine_03', 'neck_01', 'head', '
  'clavicle_r', 'upperarm_r', 'lowerarm_r', 'hand_r', 'thigh_l', 'calf_l', 'foot_

const _q = new Q(), _q2 = new Q(), _v = new V3(), _t1 = new V3(), _t2 = new V3();
let _m1 = null, _m2 = null;
function qToVec(q) { let { x, y, z, w } = q; if (w < 0) { x = -x; y = -y; z = -z;

// deep clone a gltf character subtree with remapped skin joints
function cloneCharacter(srcRoot) {
  const map = new Map();
  const cl = (n) => { const c = new Node(n.name); c.pos.copy(n.pos); c.rot.copy(n
  const root = cl(srcRoot);
  srcRoot.traverse(n => { if (n.skin) map.get(n).skin = new Skin(n.skin.joints.ma
  return root;
}

```

```

class Character {
  constructor(modelRoot) {
    this.root = new Node('character');
    this.model = modelRoot; this.root.add(modelRoot);
    this.bones = {};
    modelRoot.traverse(n => {
      const m = /^(.*)_\d+$/ .exec(n.name);
      if (m && BONES.includes(m[1]) && !this.bones[m[1]]) this.bones[m[1]] = n;
    });
    // rest data in model space
    this.root.updateWorld();
    const inv = M4.invert(M4.create(), this.root.world);
    const tmp = M4.create(), p = new V3(), s = new V3();
    this.rest = {};
    for (const [k, b] of Object.entries(this.bones)) {
      M4.mul(tmp, inv, b.world);
      const W = new Q(); M4.decompose(tmp, p, W, s);
      this.rest[k] = { local: b.rot.clone(), W, Winv: W.clone().conj(), pos: b.po
    }
    this.pelvisRestY = this.rest.pelvis.wpos.y;
    this.pose = {}; // bone -> V3 rotation vector (model space)
    this.pelvisOffset = new V3();
    // gameplay state
    this.pos = new V3(); this.yaw = 0; this.vel = new V3();
    this.state = 'idle'; this.t = 0; this.phase = 0; this.walkBlend = 0; this.spe
    this.anim = null; // scripted action {name, t, dur, fn}
    this.headLook = null; // world point
    this.visible = true;
  }

  // apply pose rotation vectors to bones
  applyPose(pose) {
    for (const k in this.bones) {
      const b = this.bones[k], r = this.rest[k];
      const v = pose[k];
      if (!v || (v[0] === 0 && v[1] === 0 && v[2] === 0)) { b.rot.copy(r.local);
      const ang = Math.hypot(v[0], v[1], v[2]);
      _v.set(v[0] / ang, v[1] / ang, v[2] / ang);
      _q.setAxisAngle(_v, ang);
      // local = restLocal * Winv * dq * W
      _q2.copy(r.local).mul(r.Winv).mul(_q).mul(r.W);
      b.rot.copy(_q2);
    }
    // pelvis translation (model space -> pelvis parent space approx: parent chai
    const pb = this.bones.pelvis; if (pb) {

```

```

    const r = this.rest.pelvis;
    if (!this._pelvisParentInv) {
        const par = pb.parent; this.root.updateWorld();
        const inv = M4.invert(M4.create(), this.root.world); const tmp = M4.mul(M
        this._pelvisParentInv = M4.invert(M4.create(), tmp);
    }
    const o = this.pelvisOffset;
    _v.set(r.wpos.x + o.x, r.wpos.y + o.y, r.wpos.z + o.z).applyM4(this._pelvis
    pb.pos.copy(_v);
}

// ----- procedural poses -----
static blend(a, b, t) {
    const out = {};
    const keys = new Set([...Object.keys(a), ...Object.keys(b)]);
    for (const k of keys) {
        const x = a[k] || [0, 0, 0], y = b[k] || [0, 0, 0];
        out[k] = [x[0] + (y[0] - x[0]) * t, x[1] + (y[1] - x[1]) * t, x[2] + (y[2]
    }
    return out;
}

static add(a, b, w = 1) {
    const out = { ...a };
    for (const k in b) { const x = out[k] || [0, 0, 0], y = b[k]; out[k] = [x[0]
    return out;
}

poseIdle(t) {
    // relaxed stance: slow breathing, subtle weight shift between the legs, arms
    const br = Math.sin(t * 1.6) * 0.022, sh = Math.sin(t * 0.37) * 0.035, sw = M
    return {
        pelvis: [0, sh * 0.6, sh],
        thigh_l: [0.02, 0, 0.04 - sh], thigh_r: [0.02, 0, -0.04 - sh], calf_l: [0.0
        spine_01: [br * 0.5, -sh * 0.4, -sh * 0.8], spine_02: [br, 0, 0], spine_03:
        clavicle_l: [0, 0, 0.05], clavicle_r: [0, 0, -0.05],
        upperarm_l: [0.03 + sw * 0.3, 0.12, -1.07 - br * 0.4], upperarm_r: [0.03 -
        lowerarm_l: [0, -0.3, 0], lowerarm_r: [0, 0.3, 0],
        hand_l: [0, 0, 0.12], hand_r: [0, 0, -0.12],
    };
}

// gait: ph = phase (rad), run = 0..1 blend between walk and run
poseWalk(ph, run = 0) {
    if (run === true) run = 1; if (run === false) run = 0;
    const L = (a, b) => a + (b - a) * run;
    const s = Math.sin(ph), c = Math.cos(ph), s2 = Math.sin(ph * 2);
    const A = L(0.46, 0.8), K = L(1.05, 1.75), B = L(0.42, 0.85);

```

```

// thigh swing with a quicker forward swing than stance (asymmetric)
const swing = (x) => -A * Math.sin(x) - 0.08 * Math.sin(2 * x) * L(1, 2);
const tl = swing(ph), tr = swing(ph + Math.PI);
// knee: flexes during swing, small flex at heel strike (loading response)
const knee = (x) => { const cx = Math.cos(x); return 0.1 + K * Math.max(0, cx
const cl = knee(ph), cr = knee(ph + Math.PI);
// foot: toe-off plantar flexion and heel strike dorsiflexion
const foot = (x, t, k) => -(t + k) * 0.45 + 0.25 * Math.max(0, -Math.sin(x +
const lean = L(-0.05, -0.28);
const hipRoll = 0.045 * s * L(1, 0.5), hipYaw = 0.1 * s * L(1, 0.7); // firm,
return {
  pelvis: [L(0.02, 0.1), hipYaw, hipRoll],
  thigh_l: [tl - L(0.02, 0.1), 0, 0.03 - hipRoll], thigh_r: [tr - L(0.02, 0.1
  calf_l: [cl, 0, 0], calf_r: [cr, 0, 0],
  foot_l: [foot(ph, tl, cl), 0, 0], foot_r: [foot(ph + Math.PI, tr, cr), 0, 0
  spine_01: [lean, -hipYaw * 1.2, -hipRoll * 0.8], spine_02: [L(0.02, -0.05),
  neck_01: [-lean * 0.55, hipYaw * 0.9, 0], head: [-lean * 0.25 + 0.02 * s2,
  clavicle_l: [0.05 * s, 0, 0.05], clavicle_r: [-0.05 * s, 0, -0.05],
  upperarm_l: [B * s, L(0.1, 0.15), -L(1.1, 0.98)], upperarm_r: [-B * s, -L(0
  // elbow flexion is a rotation about the vertical axis in the T-pose frame
  lowerarm_l: [0, -(L(0.3, 1.5) + Math.max(0, -s) * L(0.2, 0.3)), 0], lowerar
  hand_l: [L(0, 0.25), 0, L(0.12, 0.3)], hand_r: [L(0, 0.25), 0, -L(0.12, 0.3
  }];
}
// vertical pelvis bob for the gait (added to the pelvis translation)
gaitBob(ph, run = 0) { const c2 = Math.cos(ph * 2); return (run ? 0.05 : 0.028)
poseSit() {
  return {
    pelvis: [-0.16, 0, 0],
    thigh_l: [-1.28, 0.06, 0.1], thigh_r: [-1.28, -0.06, -0.1],
    calf_l: [0.55, 0, 0], calf_r: [0.62, 0, 0],
    foot_l: [0.45, 0, 0], foot_r: [0.4, 0, 0],
    spine_01: [0.04, 0, 0], spine_02: [0.02, 0, 0], spine_03: [0.02, 0, 0], nec
    upperarm_l: [-0.7, 0.3, -0.85], upperarm_r: [-0.7, -0.3, 0.85],
    lowerarm_l: [-0.9, 0.3, 0.2], lowerarm_r: [-0.9, -0.3, -0.2],
  };
}
poseDrive(steer) {
  const p = this.poseSit();
  p.spine_03 = [-0.06, -steer * 0.06, 0];
  p.head = [-0.05, -steer * 0.22, 0];
  return p;
}
// ---- two-bone arm IK: target in character-model space ----
armIK(pose, side, target, pole) {
  const up = 'upperarm_' + side, lo = 'lowerarm_' + side, ha = 'hand_' + side;

```

```

    if (!this.bones[up] || !this.bones[lo] || !this.bones[ha]) return pose;
    // current (posed) shoulder position & parent rotation, with arm deltas clear
    const p2 = { ...pose }; p2[up] = [0, 0, 0]; p2[lo] = [0, 0, 0];
    this.applyPose(p2); this.root.updateWorld(this.root.parent ? this.root.parent
    const inv = _m1 || (_m1 = M4.create()); M4.invert(inv, this.root.world);
    const rel = _m2 || (_m2 = M4.create());
    const posOf = (b, out) => { M4.mul(rel, inv, b.world); return out.set(rel[12]
    const rotOf = (b, out) => { M4.mul(rel, inv, b.world); M4.decompose(rel, _t1,
    const S = posOf(this.bones[up], new V3());
    const rU = this.rest[up], rL = this.rest[lo], rH = this.rest[ha];
    const a = rU.wpos.dist(rL.wpos), b = rL.wpos.dist(rH.wpos);
    const restDirU = rL.wpos.clone().sub(rU.wpos).norm(), restDirL = rH.wpos.clon
    const toT = target.clone().sub(S); let d = toT.len(); const dir = toT.clone()
    d = clamp(d, Math.abs(a - b) + 0.01, a + b - 0.005);
    const x = (a * a - b * b + d * d) / (2 * d), h = Math.sqrt(Math.max(0, a * a
    const pl = pole.clone().addS(dir, -pole.dot(dir)).norm();
    const E = S.clone().addS(dir, x).addS(pl, h);
    const dU = E.clone().sub(S).norm();
    const T2 = S.clone().addS(dir, d);
    const dL = T2.clone().sub(E).norm();
    // parent (clavicle) delta rotation relative to rest
    const parent = this.bones[up].parent; let Dp = new Q();
    { let pb = parent; while (pb && !Object.values(this.bones).includes(pb)) pb =
      if (pb) { const k = Object.keys(this.bones).find(k2 => this.bones[k2] === p
    const DpInv = Dp.clone().conj();
    const qU = new Q().setFromUnitVectors(restDirU, dU.clone().applyQ(DpInv));
    const DU = Dp.clone().mul(qU); const DUinv = DU.clone().conj();
    const qL = new Q().setFromUnitVectors(restDirL, dL.clone().applyQ(DUinv));
    const out = { ...pose }; out[up] = qToVec(qU); out[lo] = qToVec(qL);
    return out;
  }
  poseKick(u) { // u 0..1
    const wind = clamp(u / 0.3, 0, 1), strike = clamp((u - 0.28) / 0.2, 0, 1), re
    const e = (x) => x * x * (3 - 2 * x);
    let th = 0.35 * e(wind) - 1.85 * e(strike); th *= 1 - e(rec);
    let ca = 1.2 * e(wind) * (1 - e(strike)) + 0.1 * e(strike); ca *= 1 - e(rec);
    const lean = (0.3 * e(strike)) * (1 - e(rec));
    return {
      thigh_r: [th, 0, -0.05], calf_r: [ca, 0, 0], foot_r: [-0.3 * e(strike) * (1
      thigh_l: [-0.1 * e(strike) * (1 - e(rec)), 0, 0.03], calf_l: [0.2 * e(strik
      spine_01: [lean, 0.15 * e(wind) * (1 - e(rec)), 0], spine_03: [0.1 * e(stri
      upperarm_l: [-0.6 * e(strike) * (1 - e(rec)), 0, -1.0 + 0.5 * e(strike) * (
      upperarm_r: [0.5 * e(strike) * (1 - e(rec)), 0, 1.0 - 0.4 * e(strike) * (1
      lowerarm_l: [-0.6, 0, 0], lowerarm_r: [-0.6, 0, 0],
    };
  }
}

```

```

poseReachDoor(u, side = 1) { // left arm reaches out forward-left to door handl
  const arm = side > 0 ? 'l' : 'r';
  const p = this.poseIdle(this.t);
  const reach = Math.sin(clamp(u, 0, 1) * Math.PI);
  p['upperarm_' + arm] = [-1.1 * reach, 0.3 * reach * side, (side > 0 ? -1.2 :
  p['lowerarm_' + arm] = [-0.3 - 0.5 * reach, 0, 0];
  p.spine_01 = [-0.12 * reach, 0.25 * reach * side, 0];
  return p;
}

poseFall(u) { // limp / flailing
  return {
    upperarm_l: [-0.8 - Math.sin(u * 20) * 0.3, 0, -0.3], upperarm_r: [-0.9 + M
    lowerarm_l: [-0.4, 0, 0], lowerarm_r: [-0.5, 0, 0],
    thigh_l: [-0.4, 0, 0.15], thigh_r: [-0.2, 0, -0.1], calf_l: [0.5, 0, 0], ca
    spine_01: [0.2, 0, 0], neck_01: [0.3, 0, 0],
  };
}

poseLying(t) {
  return {
    upperarm_l: [-0.2, 0, -0.5], upperarm_r: [-0.3, 0, 0.6], lowerarm_l: [-0.3,
    thigh_l: [-0.1, 0, 0.1], thigh_r: [0.05, 0, -0.12], calf_l: [0.3, 0, 0], ca
    head: [0.1 + Math.sin(t * 2) * 0.05, 0.3, 0],
  };
}

poseGetUp(u) { // from lying on back to crouch to stand (root pitch handled out
  const k = clamp(u, 0, 1);
  const crouch = Math.sin(k * Math.PI);
  return {
    thigh_l: [-1.6 * crouch, 0, 0.1], thigh_r: [-1.2 * crouch, 0, -0.1], calf_l
    foot_l: [-0.4 * crouch, 0, 0], foot_r: [-0.4 * crouch, 0, 0],
    spine_01: [-0.5 * crouch, 0, 0], upperarm_l: [-0.6 * crouch, 0, -1.1], uppe
    lowerarm_l: [-0.5, 0, 0], lowerarm_r: [-0.5, 0, 0],
  };
}

poseCheer(t) { // spectators celebrating hajwala
  const w = Math.sin(t * 9);
  return {
    upperarm_l: [-0.3, 0, 0.9 + w * 0.3], upperarm_r: [-0.3, 0, -0.9 - w * 0.3]
    spine_01: [0.05, 0, 0], head: [0.15, 0, 0],
  };
}

poseStep(u, dir) { // quick sidestep/dodge
  const s = Math.sin(clamp(u, 0, 1) * Math.PI);
  return Character.add(this.poseIdle(this.t), { thigh_l: [-0.5 * s, 0, 0.3 * s
}

```

```

headLookPose(pose, worldTarget) {
  if (!worldTarget) return pose;
  const dx = worldTarget.x - this.root.pos.x, dz = worldTarget.z - this.root.po
  let a = Math.atan2(dx, dz) - this.yaw; while (a > Math.PI) a -= 2 * Math.PI;
  a = clamp(a, -1.2, 1.2);
  const p = { ...pose };
  const h = p.head || [0, 0, 0], n = p.neck_01 || [0, 0, 0];
  p.head = [h[0], h[1] + a * 0.6, h[2]]; p.neck_01 = [n[0], n[1] + a * 0.4, n[2]
  return p;
}

setTransform(pos, yaw, pitch = 0, roll = 0) {
  this.root.pos.copy(pos);
  _q.setAxisAngle(_v.set(0, 1, 0), yaw);
  if (pitch || roll) { _q2.setAxisAngle(_v.set(1, 0, 0), pitch); _q.mul(_q2); _
  this.root.rot.copy(_q);
}
}

return { cloneCharacter, Character };
})();

__M['fx'] = (() => {
// Particles (tyre smoke, dust, sparks, debris) and skid marks
const { V3 } = __M['math'];
const S = __M['shaders'];

function makeSmokeTexture(gl) {
  const c = document.createElement('canvas'); c.width = c.height = 128; const x =
  const img = x.createImageData(128, 128);
  // soft puff with some noise
  const rnd = (i) => { const s = Math.sin(i * 127.1) * 43758.5453; return s - Mat
  for (let yy = 0; yy < 128; yy++) for (let xx = 0; xx < 128; xx++) {
    const dx = (xx - 63.5) / 64, dy = (yy - 63.5) / 64; const r = Math.hypot(dx,
    let n = 0; for (let k = 0; k < 6; k++) { const a = rnd(k) * 6.28; const cx =
    const a = Math.max(0, 1 - r) ** 1.6 * Math.min(1, 0.5 + n * 0.4);
    const o = (yy * 128 + xx) * 4; img.data[o] = img.data[o + 1] = img.data[o + 2]
  }
  x.putImageData(img, 0, 0);
  return upload(gl, c);
}

function makeSparkTexture(gl) {
  const c = document.createElement('canvas'); c.width = c.height = 64; const x =
  const g = x.createRadialGradient(32, 32, 0, 32, 32, 32);
  g.addColorStop(0, 'rgba(255,255,255,1)'); g.addColorStop(0.25, 'rgba(255,220,15
  x.fillStyle = g; x.fillRect(0, 0, 64, 64);
  return upload(gl, c);
}

```

```

}

function upload(gl, canvas) {
    const t = gl.createTexture(); gl.bindTexture(gl.TEXTURE_2D, t);
    gl.pixelStorei(gl.UNPACK_PREMULTIPLY_ALPHA_WEBGL, false);
    gl.texImage2D(gl.TEXTURE_2D, 0, gl.RGBA, gl.RGBA, gl.UNSIGNED_BYTE, canvas);
    gl.generateMipmap(gl.TEXTURE_2D);
    gl.texParameteri(gl.TEXTURE_2D, gl.TEXTURE_MIN_FILTER, gl.LINEAR_MIPMAP_LINEAR);
    gl.texParameteri(gl.TEXTURE_2D, gl.TEXTURE_WRAP_S, gl.CLAMP_TO_EDGE); gl.texPar
    return t;
}

class ParticlePool {
    constructor(r, max, additive, tex) {
        this.r = r; this.max = max; this.additive = additive; this.tex = tex;
        this.n = 0;
        this.p = new Float32Array(max * 3); this.v = new Float32Array(max * 3);
        this.life = new Float32Array(max); this.maxLife = new Float32Array(max);
        this.size = new Float32Array(max); this.grow = new Float32Array(max);
        this.col = new Float32Array(max * 4); this.rot = new Float32Array(max); this.
        this.drag = new Float32Array(max); this.grav = new Float32Array(max);
        this.inst = new Float32Array(max * 9);
        const gl = r.gl;
        this.vao = gl.createVertexArray(); gl.bindVertexArray(this.vao);
        const q = gl.createBuffer(); gl.bindBuffer(gl.ARRAY_BUFFER, q);
        gl.bufferData(gl.ARRAY_BUFFER, new Float32Array([-1, -1, 1, -1, 1, 1, -1, -1,
        gl.enableVertexAttribArray(0); gl.vertexAttribPointer(0, 2, gl.FLOAT, false,
        this.ib = gl.createBuffer(); gl.bindBuffer(gl.ARRAY_BUFFER, this.ib);
        gl.bufferData(gl.ARRAY_BUFFER, this.inst.byteLength, gl.DYNAMIC_DRAW);
        gl.enableVertexAttribArray(1); gl.vertexAttribPointer(1, 4, gl.FLOAT, false,
        gl.enableVertexAttribArray(2); gl.vertexAttribPointer(2, 4, gl.FLOAT, false,
        gl.enableVertexAttribArray(3); gl.vertexAttribPointer(3, 1, gl.FLOAT, false,
        gl.bindVertexArray(null);
    }

    emit(pos, vel, o) {
        let i;
        if (this.n < this.max) i = this.n++; else { // replace oldest-ish
            i = (this._ri = ((this._ri || 0) + 1) % this.max);
        }
        this.p[i * 3] = pos.x; this.p[i * 3 + 1] = pos.y; this.p[i * 3 + 2] = pos.z;
        this.v[i * 3] = vel.x; this.v[i * 3 + 1] = vel.y; this.v[i * 3 + 2] = vel.z;
        this.life[i] = 0; this.maxLife[i] = o.life || 2;
        this.size[i] = o.size || 0.5; this.grow[i] = o.grow || 0;
        const c = o.color || [1, 1, 1, 1]; this.col.set(c, i * 4);
        this.rot[i] = Math.random() * 6.28; this.rotV[i] = (Math.random() - 0.5) * (o
        this.drag[i] = o.drag ?? 1.5; this.grav[i] = o.gravity ?? -0.3;
    }
}

```

```

update(dt) {
  let j = 0;
  for (let i = 0; i < this.n; i++) {
    this.life[i] += dt;
    if (this.life[i] >= this.maxLife[i]) continue;
    if (j !== i) {
      this.p.copyWithIn(j * 3, i * 3, i * 3 + 3); this.v.copyWithIn(j * 3, i *
      this.life[j] = this.life[i]; this.maxLife[j] = this.maxLife[i]; this.size
      this.col.copyWithIn(j * 4, i * 4, i * 4 + 4); this.rot[j] = this.rot[i];
    }
    const dr = Math.exp(-this.drag[j] * dt);
    this.v[j * 3] *= dr; this.v[j * 3 + 1] = this.v[j * 3 + 1] * dr + this.grav
    this.p[j * 3] += this.v[j * 3] * dt; this.p[j * 3 + 1] += this.v[j * 3 + 1]
    if (this.p[j * 3 + 1] < 0.05 && this.v[j * 3 + 1] < 0) { this.p[j * 3 + 1]
    this.size[j] += this.grow[j] * dt;
    this.rot[j] += this.rotV[j] * dt;
    j++;
  }
  this.n = j;
}

draw(cam, camRight, camUp) {
  if (!this.n) return;
  const r = this.r, gl = r.gl;
  // sort back-to-front for smoke
  let order = null;
  if (!this.additive) {
    order = new Array(this.n); const dz = new Float32Array(this.n);
    for (let i = 0; i < this.n; i++) { order[i] = i; dz[i] = (this.p[i * 3] - c
    order.sort((a, b) => dz[b] - dz[a]);
  }
  const I = this.inst;
  for (let k = 0; k < this.n; k++) {
    const i = order ? order[k] : k; const t = this.life[i] / this.maxLife[i];
    const fade = this.additive ? (1 - t) : Math.min(1, t * 6) * (1 - t) * (1 -
    I[k * 9] = this.p[i * 3]; I[k * 9 + 1] = this.p[i * 3 + 1]; I[k * 9 + 2] =
    I[k * 9 + 4] = this.col[i * 4]; I[k * 9 + 5] = this.col[i * 4 + 1]; I[k * 9
  }
  const p = r.program(S.PARTICLE_VS, S.PARTICLE_FS, []);
  gl.useProgram(p.prog); r.setEnvUniforms(p.u, cam);
  gl.uniformMatrix4fv(p.u.uVP, false, cam.vp);
  gl.uniform3f(p.u.uCamRight, camRight.x, camRight.y, camRight.z); gl.uniform3f
  gl.uniform1f(p.u.uAdditive, this.additive ? 1 : 0);
  gl.activeTexture(gl.TEXTURE0); gl.bindTexture(gl.TEXTURE_2D, this.tex); gl.un
  gl.bindVertexArray(this.vao);
  gl.bindBuffer(gl.ARRAY_BUFFER, this.ib); gl.bufferSubData(gl.ARRAY_BUFFER, 0,
  gl.enable(gl.BLEND); gl.depthMask(false); gl.disable(gl.CULL_FACE);

```

```

        if (this.additive) gl.blendFunc(gl.ONE, gl.ONE); else gl.blendFuncSeparate(gl
gl.drawArraysInstanced(gl.TRIANGLES, 0, 6, this.n);
gl.depthMask(true); gl.bindVertexArray(null);
gl.blendFuncSeparate(gl.SRC_ALPHA, gl.ONE_MINUS_SRC_ALPHA, gl.ONE, gl.ONE_MIN
    }
}

```

```

class SkidMarks {
    constructor(r, maxSeg = 3000, color = [0.05, 0.05, 0.05, 0.8]) {
        this.color = color; this.r = r; this.max = maxSeg; this.head = 0; this.count
        this.data = new Float32Array(maxSeg * 6 * 4); // 6 verts per segment, xyz + a
        const gl = r.gl;
        this.vao = gl.createVertexArray(); gl.bindVertexArray(this.vao);
        this.buf = gl.createBuffer(); gl.bindBuffer(gl.ARRAY_BUFFER, this.buf);
        gl.bufferData(gl.ARRAY_BUFFER, this.data.byteLength, gl.DYNAMIC_DRAW);
        gl.enableVertexAttribArray(0); gl.vertexAttribPointer(0, 3, gl.FLOAT, false,
        gl.enableVertexAttribArray(1); gl.vertexAttribPointer(1, 1, gl.FLOAT, false,
        gl.bindVertexArray(null);
        this.dirtyFrom = -1; this.dirtyTo = -1;
    }
    add(a0, a1, b0, b1, alphaA, alphaB) {
        const i = this.head; const D = this.data; let o = i * 24;
        const put = (p, al) => { D[o++] = p.x; D[o++] = p.y; D[o++] = p.z; D[o++] = a
        put(a0, alphaA); put(b0, alphaB); put(b1, alphaB); put(a0, alphaA); put(b1, a
        const gl = this.r.gl; gl.bindBuffer(gl.ARRAY_BUFFER, this.buf); gl.bufferSubD
        this.head = (this.head + 1) % this.max; this.count = Math.min(this.count + 1,
    }
    clear() { this.count = 0; this.head = 0; }
    draw(cam) {
        if (!this.count) return;
        const r = this.r, gl = r.gl; const p = r.program(S.DECAL_VS, S.DECAL_FS, []);
        gl.useProgram(p.prog); r.setEnvUniforms(p.u, cam);
        gl.uniformMatrix4fv(p.u.uVP, false, cam.vp); gl.uniform4f(p.u.uColor, ...this
        gl.enable(gl.BLEND); gl.blendFuncSeparate(gl.SRC_ALPHA, gl.ONE_MINUS_SRC_ALPH
        gl.depthMask(false); gl.enable(gl.POLYGON_OFFSET_FILL); gl.polygonOffset(-2,
        gl.bindVertexArray(this.vao); gl.drawArrays(gl.TRIANGLES, 0, this.count * 6);
        gl.disable(gl.POLYGON_OFFSET_FILL); gl.depthMask(true); gl.bindVertexArray(nu
    }
}

```

```

class FX {
    constructor(r) {
        this.r = r;
        const gl = r.gl;
        this.smoke = new ParticlePool(r, 360, false, makeSmokeTexture(gl));
        this.sparks = new ParticlePool(r, 300, true, makeSparkTexture(gl));
    }
}

```

```

this.skids = new SkidMarks(r);
this.ruts = new SkidMarks(r, 4000, [0.62, 0.42, 0.26, 0.55]);
this.rutTrails = new Map();
this.camRight = new V3(); this.camUp = new V3();
this.trails = new Map();
r.custom.push((rr, cam, pass) => {
  const v = cam.view; this.camRight.set(v[0], v[4], v[8]); this.camUp.set(v[1]
  if (pass === 'opaque') { this.skids.draw(cam); this.ruts.draw(cam); }
  else { this.smoke.draw(cam, this.camRight, this.camUp); this.sparks.draw(ca
  });
}
update(dt) { this.smoke.update(dt); this.sparks.update(dt); }

rutTrail(key, contact, sideDir, intensity, width = 0.3) { return this.wheelTrai
// per wheel skid trail
wheelTrail(key, contact, sideDir, intensity, width = 0.26, sys = this.skids, tr
  let t = trails.get(key);
  const on = intensity > 0.12;
  if (!on) { if (t) t.active = false; return; }
  const a = contact.clone(); a.y += 0.025;
  const l = new V3().copy(a).addS(sideDir, width / 2), rr = new V3().copy(a).ad
  if (t && t.active && t.last.dist(a) < 3) {
    if (t.last.dist(a) > 0.25) {
      sys.add(t.l, t.r, l, rr, t.alpha, Math.min(1, intensity * 1.2));
      t.last = a; t.l = l; t.r = rr; t.alpha = Math.min(1, intensity * 1.2);
    }
  } else { trails.set(key, { last: a, l, r: rr, alpha: Math.min(1, intensity),
}
tyreSmoke(pos, carVel, intensity) {
  if (intensity < 0.1) return;
  const n = Math.random() < intensity * 1.2 ? (intensity > 0.6 ? 2 : 1) : 0;
  for (let i = 0; i < n; i++) {
    const v = new V3(carVel.x * 0.25 + (Math.random() - 0.5) * 1.5, 0.6 + Math.
    const p = pos.clone(); p.y += 0.25;
    const g = 0.72 + Math.random() * 0.12;
    this.smoke.emit(p, v, { life: 1.8 + Math.random() * 1.2, size: 0.55 + Math.
  }
}
dust(pos, carVel, intensity) {
  if (Math.random() > intensity) return;
  const v = new V3(carVel.x * 0.2 + (Math.random() - 0.5), 0.5 + Math.random()
  this.smoke.emit(pos.clone(), v, { life: 2.0, size: 0.6, grow: 1.2, color: [0.
}
splash(pos, carVel, intensity) {
  if (Math.random() > intensity) return;
  const v = new V3(carVel.x * 0.3 + (Math.random() - 0.5) * 2.5, 2 + Math.rando

```

```

    this.smoke.emit(pos, v, { life: 0.9 + Math.random() * 0.5, size: 0.35, grow:
}
sparkBurst(pos, normal, speed) {
    const n = Math.min(40, 6 + speed * 2) | 0;
    for (let i = 0; i < n; i++) {
        const v = new V3((Math.random() - 0.5) * 6 + normal.x * 3, Math.random() *
        this.sparks.emit(pos, v, { life: 0.3 + Math.random() * 0.4, size: 0.05 + Ma
    }
}
impactDust(pos, speed) {
    const n = Math.min(14, 3 + speed) | 0;
    for (let i = 0; i < n; i++) {
        const v = new V3((Math.random() - 0.5) * 4, Math.random() * 2, (Math.random
        this.smoke.emit(pos.clone(), v, { life: 1.5 + Math.random(), size: 0.4, gro
    }
}
}

return { FX };
})();
__M['audio'] = (() => {
// Procedural sound: engine, tyre squeal, impacts, doors, kicks, horn, gear shift
class Audio {
    constructor() { this.ctx = null; this.enabled = true; this.engines = new Map();
    init() {
        if (this.ctx) { if (this.ctx.state === 'suspended') this.ctx.resume(); return
        const AC = window.AudioContext || window.webkitAudioContext; if (!AC) return;
        const ctx = this.ctx = new AC();
        this.master = ctx.createGain(); this.master.gain.value = 0.8;
        const comp = ctx.createDynamicsCompressor(); comp.threshold.value = -14; comp
        this.master.connect(comp); comp.connect(ctx.destination);
        // noise buffer
        const len = ctx.sampleRate * 2; const nb = ctx.createBuffer(1, len, ctx.sampl
        let b0 = 0; for (let i = 0; i < len; i++) { const w = Math.random() * 2 - 1;
        this.noise = nb;
        // tyre squeal: band-passed noise + wobbling tone
        const sq = ctx.createBufferSource(); sq.buffer = nb; sq.loop = true;
        const bp = ctx.createBiquadFilter(); bp.type = 'bandpass'; bp.frequency.value
        const sqTone = ctx.createOscillator(); sqTone.type = 'triangle'; sqTone.frequ
        const sqToneG = ctx.createGain(); sqToneG.gain.value = 0.18;
        const lfo = ctx.createOscillator(); lfo.frequency.value = 9; const lfoG = ctx
        this.squealGain = ctx.createGain(); this.squealGain.gain.value = 0;
        sq.connect(bp); bp.connect(this.squealGain); sqTone.connect(sqToneG); sqToneG
        sq.start(); sqTone.start(); lfo.start();
        this.squealTone = sqTone; this.squealBP = bp;
        // wind / road noise

```

```

const rn = ctx.createBufferSource(); rn.buffer = nb; rn.loop = true;
const lp = ctx.createBiquadFilter(); lp.type = 'lowpass'; lp.frequency.value
this.roadGain = ctx.createGain(); this.roadGain.gain.value = 0;
rn.connect(lp); lp.connect(this.roadGain); this.roadGain.connect(this.master)
this.roadLP = lp;
// scrape
const sc = ctx.createBufferSource(); sc.buffer = nb; sc.loop = true;
const hp = ctx.createBiquadFilter(); hp.type = 'bandpass'; hp.frequency.value
this.scrapeGain = ctx.createGain(); this.scrapeGain.gain.value = 0;
sc.connect(hp); hp.connect(this.scrapeGain); this.scrapeGain.connect(this.mas
}
engine(key, voice) {
  if (!this.ctx) return null;
  let e = this.engines.get(key); if (e) return e;
  const ctx = this.ctx;
  const out = ctx.createGain(); out.gain.value = 0; out.connect(this.master);
  const shaper = ctx.createWaveShaper(); const curve = new Float32Array(1024);
  for (let i = 0; i < 1024; i++) { const x = i / 512 - 1; curve[i] = Math.tanh(
  const lp = ctx.createBiquadFilter(); lp.type = 'lowpass'; lp.frequency.value
  shaper.connect(lp); lp.connect(out);
  const o1 = ctx.createOscillator(); o1.type = 'sawtooth';
  const o2 = ctx.createOscillator(); o2.type = 'square';
  const o3 = ctx.createOscillator(); o3.type = 'sawtooth';
  const g1 = ctx.createGain(); g1.gain.value = 0.5; const g2 = ctx.createGain()
  o1.connect(g1); o2.connect(g2); o3.connect(g3); g1.connect(shaper); g2.conne
  // intake noise
  const nz = ctx.createBufferSource(); nz.buffer = this.noise; nz.loop = true;
  const nbp = ctx.createBiquadFilter(); nbp.type = 'bandpass'; nbp.frequency.va
  const ng = ctx.createGain(); ng.gain.value = 0.05; nz.connect(nbp); nbp.conne
  o1.start(); o2.start(); o3.start(); nz.start();
  e = { out, o1, o2, o3, lp, ng, nbp, voice };
  this.engines.set(key, e);
  return e;
}
updateEngine(key, voice, rpm, throttle, volume) {
  const e = this.engine(key, voice); if (!e) return;
  const t = this.ctx.currentTime;
  const f = rpm / 60 * (voice.cyl / 2) * voice.base; // firing frequency
  e.o1.frequency.setTargetAtTime(f, t, 0.03);
  e.o2.frequency.setTargetAtTime(f * 0.5, t, 0.03);
  e.o3.frequency.setTargetAtTime(f * 1.01 * 2, t, 0.03);
  e.lp.frequency.setTargetAtTime(350 + throttle * 1700 + rpm * 0.12, t, 0.05);
  e.ng.gain.setTargetAtTime(0.03 + throttle * 0.12, t, 0.05);
  e.nbp.frequency.setTargetAtTime(300 + rpm * 0.25, t, 0.05);
  e.out.gain.setTargetAtTime(volume * (0.16 + throttle * 0.16), t, 0.05);
}

```

```

stopEngine(key) { const e = this.engines.get(key); if (e && this.ctx) e.out.gai
setSqueal(v, speed) {
  if (!this.ctx) return; const t = this.ctx.currentTime;
  this.squealGain.gain.setTargetAtTime(Math.min(0.5, v * 0.55), t, 0.05);
  this.squealTone.frequency.setTargetAtTime(700 + Math.min(speed, 40) * 8, t, 0
}
setRoad(speed) { if (!this.ctx) return; const t = this.ctx.currentTime; this.ro
setScrape(v) { if (!this.ctx) return; this.scrapeGain.gain.setTargetAtTime(Math
burst(opts) {
  if (!this.ctx) return; const ctx = this.ctx, t = ctx.currentTime;
  const src = ctx.createBufferSource(); src.buffer = this.noise; src.playbackRa
  const f = ctx.createBiquadFilter(); f.type = opts.type || 'lowpass'; f.freq
  const g = ctx.createGain(); g.gain.setValueAtTime(opts.gain || 0.5, t); g.gai
  src.connect(f); f.connect(g); g.connect(this.master); src.start(t, Math.rando
}
thump(freq = 70, dur = 0.25, gain = 0.8) {
  if (!this.ctx) return; const ctx = this.ctx, t = ctx.currentTime;
  const o = ctx.createOscillator(); o.type = 'sine'; o.frequency.setValueAtTime
  const g = ctx.createGain(); g.gain.setValueAtTime(gain, t); g.gain.exponentia
  o.connect(g); g.connect(this.master); o.start(t); o.stop(t + dur + 0.02);
}
crash(speed) {
  const k = Math.min(1, speed / 20);
  this.thump(55, 0.35, 0.5 + k * 0.5);
  this.burst({ freq: 1800, dur: 0.25 + k * 0.4, gain: 0.3 + k * 0.6, type: 'low
  this.burst({ freq: 3500, dur: 0.15 + k * 0.3, gain: 0.15 + k * 0.35, type: 'b
  if (speed > 12) setTimeout(() => this.burst({ freq: 5200, dur: 0.5, gain: 0.2
}
door(open) {
  if (open) { this.burst({ freq: 2200, dur: 0.05, gain: 0.12, type: 'bandpass',
  else { this.thump(90, 0.15, 0.22); this.burst({ freq: 900, dur: 0.1, gain: 0.
}
kick() { this.thump(120, 0.12, 0.6); this.burst({ freq: 1400, dur: 0.08, gain:
whoosh() { this.burst({ freq: 700, dur: 0.25, gain: 0.12, type: 'bandpass', q:
step() { this.burst({ freq: 500 + Math.random() * 300, dur: 0.07, gain: 0.07, t
shift() { this.burst({ freq: 400, dur: 0.12, gain: 0.12 }); }
horn(on) {
  if (!this.ctx) return; const ctx = this.ctx;
  if (on && !this.hornNodes) {
    const g = ctx.createGain(); g.gain.value = 0.18; g.connect(this.master);
    const a = ctx.createOscillator(); a.type = 'square'; a.frequency.value = 41
    const b = ctx.createOscillator(); b.type = 'square'; b.frequency.value = 52
    const f = ctx.createBiquadFilter(); f.type = 'lowpass'; f.frequency.value =
    a.connect(f); b.connect(f); f.connect(g); a.start(); b.start();
    this.hornNodes = { g, a, b };
  } else if (!on && this.hornNodes) {

```

```

const { g, a, b } = this.hornNodes; g.gain.setTargetAtTime(0, ctx.currentTime, 0.1);
    }
}

setMuted(m) { if (this.master) this.master.gain.value = m ? 0 : 0.8; }

return { Audio };
})();

__M['ui'] = (() => {
// Touch + keyboard controls and HUD helpers
const { clamp } = __M['math'];

class Input {
  constructor() {
    this.steer = 0; this.throttle = 0; this.brake = 0; this.handbrake = false;
    this.moveX = 0; this.moveY = 0; this.run = false;
    this.keys = new Set();
    this.touchSteer = 0; this.touchGas = 0; this.touchBrake = 0; this.touchHB = false;
    this.pressed = new Set(); // one-shot actions
    this.camDX = 0; this.camDY = 0; this.camDragging = false;
    window.addEventListener('keydown', e => { const tg = e.target; if (tg && (tg.tagName === 'INPUT' || tg.tagName === 'TEXTAREA')) return;
      this.keys.add(e.code);
      const map = { KeyF: 'door', KeyE: 'door', KeyK: 'kick', KeyQ: 'kick', KeyC: 'kick',
        if (map[e.code]) this.pressed.add(map[e.code]);
        if (['Space', 'ArrowUp', 'ArrowDown', 'ArrowLeft', 'ArrowRight'].includes(e.code)) this.pressed.add(e.code);
      });
    window.addEventListener('keyup', e => { this.keys.delete(e.code); if (e.code === 'Space') this.pressed.delete(e.code); });
    window.addEventListener('blur', () => this.keys.clear());
  }

  take(a) { const h = this.pressed.has(a); this.pressed.delete(a); return h; }

  update(dt) {
    const k = this.keys;
    const kl = k.has('ArrowLeft') || k.has('KeyA'), kr = k.has('ArrowRight') || k.has('KeyD');
    const ku = k.has('ArrowUp') || k.has('KeyW'), kd = k.has('ArrowDown') || k.has('KeyS');
    let ks = (kr ? 1 : 0) - (kl ? 1 : 0);
    if (ks !== 0) this.kbSteer = clamp((this.kbSteer || 0) + ks * dt * 3.2, -1, 1);
    else this.kbSteer = (this.kbSteer || 0) * Math.exp(-dt * 8);
    const keyActive = kl || kr || ku || kd || k.has('Space');
    const ts = this.useArrows ? (this.arrowSteer || 0) : this.touchSteer;
    this.steer = this.steerHeld ? this.touchSteer : (Math.abs(this.kbSteer) > 0.05 ? this.kbSteer : 0);
    this.throttle = Math.max(ku ? 1 : 0, this.touchGas);
    this.brake = Math.max(kd ? 1 : 0, this.touchBrake);
    this.handbrake = k.has('Space') || this.touchHB;

    // on-foot move
    let mx = this.joyX || 0, my = this.joyY || 0;
  }
}

```

```

    if (kl || kr || ku || kd) { mx = (kr ? 1 : 0) - (kl ? 1 : 0); my = (ku ? 1 :
    this.moveX = mx; this.moveY = my;
    this.keyActive = keyActive;
  }
}

```

// Steering wheel widget: drag to rotate, springs back

```
const STEER_OPTS = { max: 3 * Math.PI, ret: 7, curve: 1 }; // max = wheel rotatio
```

```
function bindSteering(el, input) {
  let id = null, a0 = 0, angle = 0;
  const O = STEER_OPTS;
  const center = () => { const r = el.getBoundingClientRect(); return [r.left + r
  const ang = (x, y) => { const [cx, cy] = center(); return Math.atan2(y - cy, x
  const set = (a) => { angle = clamp(a, -O.max, O.max); const x = angle / O.max;
  el.addEventListener('pointerdown', e => { if (input.layoutMode) return; id = e.
  el.addEventListener('pointermove', e => {
    if (e.pointerId !== id) return;
    // unwrapped: the wheel keeps turning past half a turn (up to 1.5 turns each
    const a = ang(e.clientX, e.clientY); let d = a - a0; while (d > Math.PI) d -=
    a0 = a; set(angle + d);
  });
  const up = e => { if (e.pointerId !== id) return; id = null; input.steerHeld =
  el.addEventListener('pointerup', up); el.addEventListener('pointercancel', up);
  window.addEventListener('pointerup', up, true); window.addEventListener('pointe
  return {
    tick(dt) { if (input.steerHeld) return; if (Math.abs(angle) > 0.001) set(angl
    showKb(s) { if (id === null) { angle = s * O.max; el.style.transform = `rotat
    reset() { id = null; input.steerHeld = false; set(0); },
  };
}

```

// analog pedal: the higher your finger on the pedal, the harder it presses (bott

```
function bindPedal(el, input, key) {
  let id = null;
  const val = (e) => { const r = el.getBoundingClientRect(); const t = clamp((r.b
  const set = (v) => { input[key] = v; el.style.transform = v > 0 ? `perspective(
  el.addEventListener('pointerdown', e => { if (input.layoutMode) return; id = e.
  el.addEventListener('pointermove', e => { if (e.pointerId === id) set(val(e));
  const up = e => { if (e.pointerId !== id) return; id = null; set(0); };
  for (const t of ['pointerup', 'pointercancel', 'lostpointercapture']) el.addEve
  window.addEventListener('pointerup', up, true); window.addEventListener('pointe
  return { release() { id = null; set(0); } };
}

```

// arrow steering buttons: steer ramps while held, springs back when released

```
function bindArrows(left, right, input) {
  const held = { l: new Set(), r: new Set() };

```

```

const hook = (el, k) => {
  el.addEventListener('pointerdown', e => { if (input.layoutMode) return; held[
    const up = e => { if (held[k].delete(e.pointerId) && !held[k].size) el.classL
    for (const t of ['pointerup', 'pointercancel', 'lostpointercapture']) el.addE
    window.addEventListener('pointerup', up, true); window.addEventListener('poin
  });
  hook(left, 'l'); hook(right, 'r');
  let v = 0;
  return {
    active: () => held.l.size || held.r.size,
    tick(dt) {
      const t = (held.r.size ? 1 : 0) - (held.l.size ? 1 : 0);
      if (t) v = clamp(v + t * dt * 2.6, -1, 1); else v *= Math.exp(-dt * 9);
      input.arrowSteer = v;
    },
    release() { held.l.clear(); held.r.clear(); left.classList.remove('down'); ri
  };
}

function bindHold(el, onDown, onUp) {
  const ids = new Set();
  el.addEventListener('pointerdown', e => { if (el.closest('#hud')
  const up = e => { if (!ids.has(e.pointerId)) return; ids.delete(e.pointerId); i
  el.addEventListener('pointerup', up); el.addEventListener('pointercancel', up);
  window.addEventListener('pointerup', up, true);
}

function bindTap(el, fn) {
  el.addEventListener('pointerdown', e => { e.preventDefault(); e.stopPropagation
  const up = () => el.classList.remove('down');
  el.addEventListener('pointerup', up); el.addEventListener('pointercancel', up);
}

// virtual joystick
function bindJoystick(el, knob, input) {
  let id = null, cx = 0, cy = 0;
  const R = () => el.getBoundingClientRect().width / 2;
  el.addEventListener('pointerdown', e => { id = e.pointerId; el.setPointerCaptur
  const move = (e) => {
    if (e.pointerId !== id) return;
    let dx = e.clientX - cx, dy = e.clientY - cy; const l = Math.hypot(dx, dy), m
    if (l > m) { dx *= m / l; dy *= m / l; }
    knob.style.transform = `translate(${dx}px, ${dy}px)`;
    input.joyX = dx / m; input.joyY = -dy / m;
  };
  el.addEventListener('pointermove', move);
  const up = e => { if (e.pointerId !== id) return; id = null; knob.style.transfo

```

```

    el.addEventListener('pointerup', up); el.addEventListener('pointercancel', up);
}

// camera drag on the 3D view
function bindCameraDrag(el, input) {
    const pts = new Map();
    el.addEventListener('pointerdown', e => { pts.set(e.pointerId, [e.clientX, e.clientY]);
    el.addEventListener('pointermove', e => {
        const p = pts.get(e.pointerId); if (!p) return;
        input.camDX += e.clientX - p[0]; input.camDY += e.clientY - p[1]; p[0] = e.clientX; p[1] = e.clientY;
    });
    const up = e => { pts.delete(e.pointerId); if (!pts.size) input.camDragging = false;
    el.addEventListener('pointerup', up); el.addEventListener('pointercancel', up);
}

return { Input, STEER_OPTS, bindSteering, bindPedal, bindArrows, bindHold, bindTa
    })();
__M['game'] = (() => {
// Gameplay: world, cars, player (on foot / driving / door sequences), spectators
const { V3, Q, M4, clamp, lerp, smooth, angleWrap } = __M['math'];
const { Camera } = __M['renderer'];
const { CollisionWorld } = __M['physics'];
const { Car, CAR_DEFS, boxPrim, makePrim } = __M['car'];
const { Character, cloneCharacter } = __M['character'];
const { Node, Material } = __M['scene'];
const { computeBounds, playAnimation } = __M['gltf'];
const { buildCity, buildHighway, buildDesert, buildRiyadh, buildTest, buildRoom,

const UP = new V3(0, 1, 0);
const ease = (x) => { x = clamp(x, 0, 1); return x * x * (3 - 2 * x); };

class Game {
    constructor(renderer, audio, fx, assets, input, hud) {
        this.r = renderer; this.audio = audio; this.fx = fx; this.assets = assets; this.scene = new Node('scene');
        this.cam = new Camera(); this.cam.far = 2600; this.cam.near = 0.15;
        this.camMode = 0; this.camYawOff = 0; this.camPitchOff = 0; this.camIdle = 0;
        this.camPos = new V3(0, 5, -10); this.camLook = new V3();
        this.time = 0; this.mode = 'menu';
        this.maps = {}; this.mapKey = null;
        this.mapRoot = new Node('map'); this.scene.add(this.mapRoot);
        this.loadMap('city', true);
        this.cars = {}; this.pristine = {};
        for (const key of Object.keys(CAR_DEFS)) {
            // untouched copy of the model for other players' cars (online), taken before
            const src0 = assets[key].root.children[0];

```

```

        if (src0) { const c = src0.clone(true); c.traverse(n => { if (n.mesh && /^(
        const car = new Car(CAR_DEFS[key], assets[key], this.world);
        car.key = key; car.addPlates(assets.plate); car.snapshotNormals(); if (car.
        this.cars[key] = car;
    }
    this.carKey = 'shas';
    // player character
    this.manSrc = assets.man.root;
    this.player = new Character(cloneCharacter(this.manSrc));
    this.playerMode = 'car'; // car | foot | seq
    this.seq = null;
    this.buildCones();
    this.buildSpectators();
    this.placeMapActors();
    this.score = 0; this.driftPts = 0; this.driftTime = 0; this.driftCombo = 1; t
    this.lastDent = 0; this.lastCrashSound = 0;
    this.kick = null;
}

// ----- world -----
loadMap(key, first) {
    if (!this.maps[key]) {
        const sandTex = this.maps.city ? this.maps.city.sandTex : null;
        this.maps[key] = key === 'room' ? buildRoom(this.assets) : key === 'city' ?
    }
    const m = this.map = this.maps[key]; this.mapKey = key;
    this.mapRoot.children.length = 0; this.mapRoot.add(m.root);
    this.world = m.world;
    if (this.cars) for (const k in this.cars) this.cars[k].world = m.world;
    this.applyEnv();
    if (!first) { this.placeMapActors(); this.fx.skids.clear(); if (this.fx.ruts)
}
// day / night lighting for the current map
applyEnv() {
    const m = this.map, e = this.r.env, w = this.weather || (this.night ? 'night'
    this.night = w === 'night';
    Object.assign(e, { sunDir: m.env.sunDir.clone(), sunColor: m.env.sunColor, sk
    if (w === 'night') Object.assign(e, { sunDir: new V3(0.35, 0.62, -0.25).norm(
    else if (w === 'cloudy') Object.assign(e, { sunDir: new V3(-0.2, 0.9, -0.4).n
    else if (w === 'sunset') Object.assign(e, { sunDir: new V3(-0.8, 0.14, -0.58)
    else if (w === 'dusk') Object.assign(e, { sunDir: new V3(-0.85, 0.05, -0.52).
    const lamps = w === 'night' || w === 'dusk';
    if (this.cars) for (const k in this.cars) this.cars[k].setNight(lamps);
    if (this.net) this.net.setNight(lamps);
}
setWeather(w) { this.weather = w; this.applyEnv(); }

```

```

setNight(on) { this.weather = on ? 'night' : 'day'; this.applyEnv(); }
placeMapActors() {
  const m = this.map;
  this.cones.forEach((c, i) => {
    const sp = m.cones[i];
    c.node.visible = !!sp;
    if (sp) { const y = this.world.heightAt(sp[0], sp[1], 5); c.home.set(sp[0],
  ));
  this.syncCones();
  // workshop: a yellow pad near the start of every map (park on it 10 s: tyres
  if (m.room) { this.workshop = null; this.fuelPads = []; }
  else { const sp = m.spawn, c = Math.cos(sp.yaw), sn = Math.sin(sp.yaw);
    const x = sp.x - c * 9 + sn * 6, z = sp.z + sn * 9 + c * 6, y = this.world.
    this.workshop = { x, z, r: 3.2 };
    if (!this.wsNode) {
      const pad = new Material({ name: 'wsPad', color: [0.62, 0.25, 0.95, 0.55]
      const face = new Material({ name: 'wsSign', map: signTexture(['\u062b\u0648\u0627\u0646\u064a'])
      const grey = new Material({ name: 'wsPole', color: [0.5, 0.5, 0.52, 1], r
      const n = new Node('workshop'); const P = [], N = [], U = [], I = [];
      for (let i = 0; i <= 24; i++) { const a = i / 24 * Math.PI * 2; P.push(Ma
      P.push(0, 0.06, 0); N.push(0, 1, 0); U.push(0, 0); for (let i = 0; i < 24
      const sign = new Node('wsSignN'); sign.mesh = { prims: [makePrim([-1.4, 2
      sign.pos.set(0, 0, -3.4); const pole = new Node('wsPoleN'); pole.mesh = {
      n.mesh = { prims: [makePrim(P, N, U, I, pad)] }; n.add(sign); n.castShado
    }
    if (this.wsNode.parent) this.wsNode.parent.remove(this.wsNode);
    this.wsNode.pos.set(x, y, z); m.root.add(this.wsNode);
    // fuel station (yellow) on the other side
    const fx = sp.x + c * 9 + sn * 6, fz = sp.z - sn * 9 + c * 6, fy = this.wor
    this.fuelPads = [{ x: fx, z: fz, r: 3.2 }];
    if (m.name === 'highway' && m.roadX) for (let zz = -3600; zz <= 3600; zz +=
    if (!this.fuelNode) {
      const pad = new Material({ name: 'fuelPad', color: [1, 0.82, 0.1, 0.6], r
      const face = new Material({ name: 'fuelSign', map: signTexture(['\u0648\u0642\u0641 \u0647\u0646\u0627'])
      const grey = new Material({ name: 'fuelPole', color: [0.5, 0.5, 0.52, 1],
      const mk = () => { const n = new Node('fuel'); const P = [], N = [], U =
        for (let i = 0; i <= 24; i++) { const a = i / 24 * Math.PI * 2; P.push(
        P.push(0, 0.06, 0); N.push(0, 1, 0); U.push(0, 0); for (let i = 0; i <
        const sign = new Node('fs'); sign.mesh = { prims: [makePrim([-1.4, 2.0,
        const pole = new Node('fp'); pole.mesh = { prims: [boxPrim(0.1, 2.2, 0.
        n.mesh = { prims: [makePrim(P, N, U, I, pad)] }; n.add(sign); n.add(pol
        this.fuelNode = new Node('fuels'); this.mkFuel = mk;
      }
      if (this.fuelNode.parent) this.fuelNode.parent.remove(this.fuelNode);
      this.fuelNode.children.length = 0;
      for (const f of this.fuelPads) { const n = this.mkFuel(); n.pos.set(f.x, th

```

```

    m.root.add(this.fuelNode);
  }
  this.npcs.forEach((n, i) => {
    const sp = m.npcs[i]; n.root.visible = !!sp; n.active = !!sp;
    if (sp) { const y = this.world.heightAt(sp[0], sp[1], 5); n.pos.set(sp[0],
    });
  }

buildCones() {
  const orange = new Material({ name: 'cone', color: [0.95, 0.35, 0.08, 1], rou
  const white = new Material({ name: 'coneStripe', color: [0.95, 0.95, 0.92, 1]
  const cone = coneMeshes(orange, white);
  this.cones = [];
  const spots = []; for (let i = 0; i < 17; i++) spots.push([0, 0]);
  for (const [x, z] of spots) {
    const n = new Node('cone'); n.mesh = cone; this.scene.add(n);
    this.cones.push({ node: n, pos: new V3(x, 0, z), home: new V3(x, 0, z), vel
  }
  this.syncCones();
}

syncCones() { for (const c of this.cones) { c.node.pos.copy(c.pos); c.node.rot.

buildSpectators() {
  this.npcs = [];
  const spots = [[0, 0, 0], [0, 0, 0], [0, 0, 0], [0, 0, 0], [0, 0, 0]];
  for (const [x, z, yaw] of spots) {
    const c = new Character(cloneCharacter(this.manSrc));
    const y = this.world.heightAt(x, z, 5);
    c.pos.set(x, y, z); c.yaw = yaw; c.home = c.pos.clone(); c.homeYaw = yaw;
    c.state = 'idle'; c.t = Math.random() * 10; c.stateT = 0; c.vel = new V3();
    this.scene.add(c.root); this.npcs.push(c);
  }
}

get car() { return this.cars[this.carKey]; }

// ----- modes -----
showroom(key) {
  // menu / garage view: car parked in the crossroads, slowly orbiting camera
  for (const k in this.cars) if (this.cars[k].node.parent) this.scene.remove(th
  if (this.hero && this.hero.root.parent) this.hero.root.parent.remove(this.her
  this.carKey = key;
  const car = this.car; this.scene.add(car.node);
  if (car.damage > 0 || car.deform.some(d => d.orig) || car.wheels.some(w => w.
  const sr = this.map.showroom; car.place(sr.x, this.world.heightAt(sr.x, sr.z,
  for (let i = 0; i < 60; i++) car.step(1 / 120, { throttle: 0, brake: 1, steer

```

```

    if (this.player.root.parent) this.player.root.parent.remove(this.player.root)
    this.mode = 'menu';
}

start() {
    const car = this.car;
    this.fx.skids.clear();
    this.clearDebris();
    car.repair(this.r);
    const sp = this.map.spawn; car.place(sp.x, this.world.heightAt(sp.x, sp.z, 5)
    this.mode = 'play';
    // player sits in the car
    this.enterSeatNow();
    this.score = 0; this.driftPts = 0;
    this.placeMapActors();
    this.camYawOff = 0;
    this.placeCameraBehind();
}

seatRoot(car) {
    const s = car.seatPos, k = car.def.driverScale || 1;
    return new V3(s.x, s.y + 0.1 - 0.915 * k - (car.seatDrop || 0), s.z + 0.04 +
}

// sit the driver low enough that the head stays under the roof lining (tall dr
fitHeadroom(car) {
    if (car.seatDrop !== undefined) return;
    car.seatDrop = 0;
    const p = this.player, hb = p.bones.head; if (!hb) return;
    const roof = car.def.interior ? car.def.interior.linerY : car.def.headroomY;
    car.node.updateWorld(null);
    const inv = M4.invert(M4.create(), car.node.world); const head = hb.worldPos(
    const k = car.def.driverScale || 1, top = head.y + 0.2 * k, lim = roof - 0.05
    if (top > lim) { car.seatDrop = Math.min(0.2, top - lim); p.root.pos.copy(thi
}

enterSeatNow() {
    const car = this.car, p = this.player;
    car.node.add(p.root);
    p.root.pos.copy(this.seatRoot(car)); p.root.rot.identity();
    const k = car.def.driverScale || 1; p.root.scl.set(k, k, k);
    p.applyPose(this.drivePose(car, p.poseDrive(0)));
    this.fitHeadroom(car); this.fitReach(car); p.applyPose(this.drivePose(car, p.
    this.playerMode = 'car'; car.doorAngle = 0; car.engineOn = true;
}

// car-local point -> character model space (root is a child of the car node)
toChar(p, carLocal) { const r = p.root; const k = r.scl.x; return carLocal.clon
// hands on the steering wheel rim (two-bone IK), following the wheel as it tur

```

```

drivePose(car, base, weight = 1) {
    const p = this.player; if (!car.steerCenter) return base;
    if (car.def.recline) { base = { ...base }; const pv = base.pelvis || [0, 0, 0]
    // each hand holds the rim and turns WITH the wheel; at the end of its reach
    // through the palm (no jumping, no letting go), and it drifts back to its ho
    const spin = car.wheelSpin();
    const H = this.hands && this.hands.car === car ? this.hands : (this.hands = {
    let ds = spin - H.spin; H.spin = spin; if (Math.abs(ds) > 2) ds = 0;
    const LIM = 0.7, relax = Math.abs(spin) < 0.3 ? 0.08 : 0;
    H.l = clamp((H.l - ds) * (1 - relax), -LIM, LIM); H.r = clamp((H.r - ds) * (1
    const L = this.toChar(p, this.gripPoint(car, 1.35 + H.l, 1)), Rr = this.toCha
    let pose = p.armIK(base, 'l', L, new V3(0.7, -1, -0.2));
    pose = p.armIK(pose, 'r', Rr, new V3(-0.7, -1, -0.2));
    return weight >= 1 ? pose : Character.blend(base, pose, weight);
}
gripPoint(car, ang, wrist = 0) {
    const R = (car.rimR || 0.185) + wrist * 0.075, tilt = new Q().setAxisAngle(ne
    return new V3(Math.sin(ang) * R, Math.cos(ang) * R, -0.025 - wrist * 0.035).a
}
// make sure the driver can actually reach the rim: slide the seat forward / pu
fitReach(car) {
    if (car.reachFitted || !car.steerCenter) return; car.reachFitted = true;
    const p = this.player, ub = p.bones.upperarm_l, lb = p.bones.lowerarm_l, hb =
    p.applyPose(p.poseDrive(0)); car.node.updateWorld(null);
    const S = ub.worldPos(new V3()), k = p.root.scl.x;
    const armLen = (p.rest.upperarm_l.wpos.dist(p.rest.lowerarm_l.wpos) + p.rest.
    const T = this.gripPoint(car, 1.0).applyM4(car.node.world);
    const need = S.dist(T) - 0.96 * armLen;
    if (need <= 0) return;
    const seatMove = Math.min(need * 0.55, 0.16), pull = Math.min(Math.max(0, nee
    car.seatShiftZ = seatMove;
    if (pull > 0 && car.steeringNode) { car.steeringNode.pos.z -= pull; car.steer
    p.root.pos.copy(this.seatRoot(car));
}
doorHandle(car, outside) {
    const L = car.doorLen || 1;
    const local = outside ? new V3(0.07, -0.12, -L + 0.14) : new V3(-0.1, -0.1, -
    return local.applyQ(new Q().setAxisAngle(new V3(0, 1, 0), -car.doorAngle)).ad
}

// ----- per-frame -----
// move the player (car, driver, camera) by a wrap offset: endless maps
shiftWorld(w) {
    const car = this.car, b = car.body, q = new Q().setAxisAngle(new V3(0, 1, 0),
    const piv = b.pos.clone();
    const mv = (v) => { v.sub(piv).applyQ(q).add(piv); v.x += w.dx; v.z += w.dz;

```

```

    b.q.premul(q); b.vel.applyQ(q); mv(b.pos); b.updateInertia();
    mv(this.camPos); mv(this.camLook); if (this.player) { mv(this.player.pos); }
    if (this.fx && this.fx.skids) this.fx.skids.clear();
    car.syncNode();
}
fixedStep(dt) {
    const car = this.car, inp = this.input;
    let ci = { throttle: 0, brake: 0, steer: 0, handbrake: true };
    if (this.playerMode === 'car') {
        ci = { throttle: inp.throttle, brake: inp.brake, steer: inp.steer, handbrake: inp.handbrake };
        if (this.drone) ci = { throttle: 0, brake: 0.3, steer: 0, handbrake: true };
        if (this.wantExit) { ci = { throttle: 0, brake: 1, steer: inp.steer, handbrake: true }; }
        else if (this.playerMode === 'seq' && this.seq && this.seq.type === 'exit') {
            ci = { throttle: 0, brake: 1, steer: 0, handbrake: true };
        }
    }
    const sm = this.surfaceUnder(car);
    ci.surfaceGrip = sm;
    ci.water = null;
    if (this.map.water) { const b = car.body.pos; for (const w of this.map.water)
    { // soft sand: how easily tyres dig in here
        const m = this.map, b = car.body.pos; let k = m.name === 'desert' ? 1 : m.n
        for (const z of m.deepSand || []) if (b.x > z.x0 && b.x < z.x1 && b.z > z.z)
            car.bogSand = k;
    }
    if (!this.inRoom) car.step(dt, ci);
    if (this.map.wrap && this.playerMode === 'car') { const b = car.body.pos, w =
    if (this.net) { this.net.stepBot(dt); this.net.collide(dt); }
    this.stepCones(dt);
}

surfaceUnder(car) {
    let sand = 0, n = 0;
    for (const w of car.wheels) if (w.onGround) { n++; if (w.tag === 3) sand++; }
    this.onSand = n && sand / n > 0.5;
    return 1;
}

update(dt) {
    this.time += dt; this.r.env.time = this.time;
    // night: headlights of every car in the scene light the road
    const L = this.r.lights || (this.r.lights = []); L.length = 0;
    if (this.night || this.weather === 'dusk') { if (this.car.node.parent) this.c
    const inp = this.input, car = this.car;
    if (this.mode !== 'play') { this.updateMenu(dt); return; }

    // one-shot actions
    if (inp.take('camera')) { const n = this.camCustom ? 5 : 4; this.camMode = (t
    if (inp.take('reset')) this.resetCar();

```

```

if (inp.take('repair')) { this.clearDebris(); this.car.repair(this.r); this.h
if (inp.take('horn')) { this.audio.horn(true); this.hornT = 0.5; }
if (inp.take('hornUp')) this.hornT = 0;
if (this.hornT !== undefined) { this.hornT -= dt; if (this.hornT <= 0 && !inp
if (inp.take('door')) this.onDoorButton();
if (inp.take('kick')) this.startKick();

// player
if (this.playerMode === 'car') {
    this.player.applyPose(this.drivePose(car, this.player.poseDrive(car.steer /
    if (this.wantExit && car.speed < 1.2) { this.wantExit = false; this.startEx
} else if (this.playerMode === 'seq') this.updateSeq(dt);
else this.updateFoot(dt);
// the chosen character (animated model) walks around when you're out of the
{ const useHero = this.charSel === 'hero' && this.playerMode === 'foot' && th
    this.player.root.visible = !useHero;
    if (useHero) { const H = this.hero, p = this.player; if (H.root.parent !==
        const sp = Math.hypot(p.vel.x, p.vel.z); H.t = sp > 0.3 ? H.t + dt * clam
        H.root.pos.copy(p.pos); H.root.rot.setAxisAngle(new V3(0, 1, 0), p.yaw);
    else if (this.hero && this.hero.root.parent && this.mode === 'play') this.h
// first-person: collapse the driver's head (ghutra) so it never blocks the v
{ const hb = this.player.bones.head; if (hb) { const k = this.camMode === 3 &

this.updateNPCs(dt);
this.handleImpacts(dt);
this.updateDebris(dt);
if (this.net) this.net.update(dt);
this.updateEffects(dt);
this.updateScore(dt);
this.updateCamera(dt);
this.updateAudio(dt);
this.updateSaher(dt);
if (car.flooded && !car.floodMsg) { car.floodMsg = true; this.hud.toast('2200
if (car.flooded && car.submerge < 0.02) { car.dryT = (car.dryT || 0) + dt; if
if (car.submerge > 0.05 && car.speed > 2) { const b = car.body; for (const w
if (car.cluster && this.playerMode !== 'foot') { this.clT = (this.clT || 0) +
this.r.shadowFocus.copy(this.playerMode === 'foot' ? this.player.pos : car.bo
// on foot next to a tyre -> the tyre-pressure button
this.nearWheel = -1;
if (this.playerMode === 'foot') { let best = 1.5; car.wheels.forEach((w, i) =
if (this.deflating && this.nearWheel >= 0) { const w = car.wheels[this.nearWh
// challenge target pillar
if (this.chal && this.chal.target) {
    const T = this.chal.target;
    if (!this.chalNode) { const mat = new Material({ name: 'chalGlow', color: [
    if (this.chalNode.parent !== this.scene) this.scene.add(this.chalNode);

```

```

    this.chalNode.pos.set(T.x, this.world.heightAt(T.x, T.z, 50) + 30, T.z);
    const b = car.body.pos; if (!this.chal.done && Math.hypot(b.x - T.x, b.z -
} else if (this.chalNode && this.chalNode.parent) this.chalNode.parent.remove
// fuel: burns with the throttle; park on a yellow pad to fill up
if (this.playerMode === 'car' && car.engineOn) { car.fuel = Math.max(0, (car.
if (this.fuelPads && this.playerMode === 'car') { const b = car.body.pos; con
    if (on && (car.fuel ?? 100) < 100) { car.fuel = Math.min(100, (car.fuel ??
// walk-in caravan: yellow spot behind the parked Shas / exit spot inside
if (this.playerMode === 'foot' && !this.roomBusy) {
    let pt = null;
    if (this.inRoom) pt = this.map.exit; else if (this.carKey === 'shas') { con
    if (pt) { if (!this.roomDot) { const m = new Material({ name: 'roomDot', co
        if (this.roomDot.parent !== this.scene) this.scene.add(this.roomDot); thi
        if (Math.hypot(this.player.pos.x - pt.x, this.player.pos.z - pt.z) < 0.9
    else if (this.roomDot && this.roomDot.parent) this.roomDot.parent.remove(th
} else if (this.roomDot && this.roomDot.parent && this.playerMode !== 'foot')
if (this.workshop && this.playerMode === 'car' && !this.drone) {
    const W = this.workshop, b = car.body.pos;
    if (Math.hypot(b.x - W.x, b.z - W.z) < W.r && Math.abs(car.speed) < 0.6) {
        const t0 = this.wsT || 0; this.wsT = t0 + dt;
        if (Math.floor(t0) !== Math.floor(this.wsT) && this.wsT < 10) this.hud.to
        if (t0 < 10 && this.wsT >= 10) { for (const w of car.wheels) { w.psi = 30
    } else this.wsT = 0;
}
if (!this.map._follow) { this.map._follow = []; this.map.root.traverse(n => {
for (const n of this.map._follow) n.pos.z = this.r.shadowFocus.z;
}

// ----- door sequences -----
doorPoints(car) {
    const H = car.doorHinge, L = car.doorLen || 1;
    return {
        A: new V3(H.x + 0.78, 0, H.z - L - 0.05),    // stand beside the door, near
        B: new V3(H.x + 0.28, 0, H.z - L * 0.5),    // in the doorway
        S: this.seatRoot(car),
        handle: new V3(H.x + 0.05, 1.05, H.z - L + 0.12),
    };
}

onDoorButton() {
    const car = this.car;
    if (this.playerMode === 'car') {
        if (car.speed > 1.2) { this.wantExit = true; this.hud.toast('السيارة للنزول')
        else this.startExit();
    } else if (this.playerMode === 'foot') {
        if (this.nearDoor()) this.startEnter();
        else this.hud.toast('قَرَب من باب السواق');
    }
}

```

```

    }
}
nearDoor() {
    const car = this.car; const pts = this.doorPoints(car);
    const wa = car.node.world; const A = pts.A.clone().applyM4(wa);
    const d = Math.hypot(A.x - this.player.pos.x, A.z - this.player.pos.z);
    const c = car.body.pos, dc = Math.hypot(c.x - this.player.pos.x, c.z - this.p
    return (d < 4.5 || dc < 5) && this.carUpright();
}
carUpright() { return this.car.up(new V3()).y > 0.6; }

startEnter() {
    const car = this.car, p = this.player;
    // convert player to car-local space
    const inv = M4.invert(M4.create(), car.node.world);
    const lp = p.pos.clone().applyM4(inv);
    const carYaw = this.carYaw();
    car.node.add(p.root);
    this.seq = { type: 'enter', stage: 'walk', t: 0, from: lp, yaw: angleWrap(p.y
    this.playerMode = 'seq';
}
// is there room to stand on this side of the car? (side 1 = driver/left, -1 =
sideFree(side) {
    const car = this.car; let blocked = false;
    for (const dz of [-0.2, -0.7]) {
        const lp = new V3(side * (car.def.halfWidth + 0.5), 0.95, car.doorHinge.z +
        const wp = car.localToWorld(lp, new V3());
        this.world.sphere(wp, 0.38, (nx, ny, nz, depth, px, py, pz, tag) => { if (t
        }
        return !blocked;
    }
}
startExit() {
    const car = this.car;
    let side = 1;
    if (!this.sideFree(1)) { if (this.sideFree(-1)) { side = -1; this.hud.toast('
    this.seq = { type: 'exit', stage: side > 0 ? 'open' : 'slide', t: 0, pts: thi
    this.playerMode = 'seq';
}
carYaw() { const f = this.car.forward(new V3()); return Math.atan2(f.x, f.z); }

updateSeq(dt) {
    const s = this.seq, car = this.car, p = this.player, P = s.pts;
    s.t += dt; p.t += dt;
    const floorY = 0.02; // local car ground
    const k = car.def.driverScale || 1;
    const setLocal = (pos, yaw, sc = 1) => { p.root.pos.copy(pos); p.root.rot.set

```

```

const Ayaw = -Math.PI / 2; // facing the car's side
const reachIK = (pose, side, target, w, pole) => w <= 0.001 ? pose : Character
if (s.type === 'enter') {
  if (s.stage === 'walk') {
    const to = P.A.clone(); to.y = floorY;
    // walk around the car (never through it) when coming from the passenger
    if (!s.path) {
      const hw = car.def.halfWidth + 0.75, zf = car.def.zMax + 0.7, zb = car.
      s.path = [f];
      if (f.x < hw - 0.3) {
        const viaFront = Math.abs(f.z - zf) < Math.abs(f.z - zb) || f.z > (zf
        const zc = viaFront ? zf : zb;
        if (f.x < -hw + 0.3 || Math.abs(f.x) < hw) { s.path.push(new V3(Math.
        }
        s.path.push(to);
        s.segL = []; s.pathL = 0; for (let i = 1; i < s.path.length; i++) { con
      }
      const dist = s.pathL;
      const dur = Math.max(0.35, dist / 1.6);
      const u = clamp(s.t / dur, 0, 1);
      let dd = (ease(u) * 0.3 + u * 0.7) * dist, seg = 0; while (seg < s.segL.1
      const pos = s.path[seg].clone().lerp(s.path[seg + 1], clamp(dd / (s.segL[
      const d = new V3().subVecs(s.path[seg + 1], s.path[seg]); d.y = 0;
      const walkYaw = Math.atan2(d.x, d.z);
      const yaw = u < 0.75 ? lerpAngle(s.yaw, walkYaw, clamp(s.t * 5, 0, 1)) :
      s.phase = (s.phase || 0) + dt * 7.2 * (1 - ease((u - 0.85) / 0.15));
      setLocal(pos, yaw);
      p.pelvisOffset.set(0, p.gaitBob(s.phase) * (1 - u), 0);
      p.applyPose(Character.blend(p.poseWalk(s.phase, 0), p.poseIdle(p.t), ease
      if (dist < 0.05 || u >= 1) { s.stage = 'open'; s.t = 0; p.pelvisOffset.se
    } else if (s.stage === 'open') {
      const u = clamp(s.t / 1.25, 0, 1);
      const H = this.doorHandle(car, true);
      const hy = Math.atan2(H.x - P.A.x, H.z - P.A.z);
      const yaw = lerpAngle(Ayaw, hy + 0.35, ease(u * 2.5));
      const pos = P.A.clone(); pos.y = floorY; pos.x += 0.18 * ease((u - 0.45)
      setLocal(pos, yaw);
      if (u > 0.3 && !s.opened) { s.opened = true; this.audio.door(true); }
      car.doorAngle = 1.15 * ease((u - 0.32) / 0.68);
      let pose = p.poseIdle(p.t); pose.spine_01 = [-0.1 * Math.sin(u * Math.PI)
      const w = u < 0.3 ? ease(u / 0.3) : u < 0.8 ? 1 : 1 - ease((u - 0.8) / 0.
      pose = reachIK(pose, 'r', this.doorHandle(car, true), w, new V3(-1, -0.6,
      p.applyPose(pose);
      if (u >= 1) { s.stage = 'in'; s.t = 0; }
    } else if (s.stage === 'in') {
      const u = clamp(s.t / 1.9, 0, 1);

```

```

const a = P.A.clone(); a.y = floorY; a.x += 0.18; a.z += 0.12;
const bb = P.B.clone(); bb.y = floorY + 0.2;
const S = P.S;
let pos;
if (u < 0.45) pos = a.clone().lerp(bb, ease(u / 0.45));
else pos = bb.clone().lerp(S, ease((u - 0.45) / 0.55));
pos.y += Math.sin(clamp(u / 0.6, 0, 1) * Math.PI) * 0.035;
const yaw = u < 0.35 ? lerpAngle(Ayaw + 0.2, Ayaw - 0.35, ease(u / 0.35))
setLocal(pos, yaw, lerp(1, k, ease((u - 0.4) / 0.6)));
const lift = Math.sin(clamp(u / 0.5, 0, 1) * Math.PI);
const stepLeg = { thigh_1: [-1.25 * lift, 0, 0.25], calf_1: [1.35 * lift,
const inside = 1 - clamp((pos.x - (car.def.halfWidth - 0.05)) / 0.4, 0, 1
let base = Character.blend(p.poseIdle(p.t), p.poseSit(), Math.max(ease((u
base = Character.add(base, stepLeg, 1 - ease((u - 0.45) / 0.3));
// left hand grabs the A-pillar / wheel while climbing in
base = reachIK(base, 'l', car.steerCenter ? car.steerCenter.clone().add(n
p.applyPose(base);
if (u >= 1) { s.stage = 'close'; s.t = 0; }
} else if (s.stage === 'close') {
const u = clamp(s.t / 0.9, 0, 1);
setLocal(P.S, 0, k);
car.doorAngle = 1.15 * (1 - ease((u - 0.25) / 0.55));
const reach = u < 0.25 ? ease(u / 0.25) : u < 0.8 ? 1 : 1 - ease((u - 0.8
let pose = p.poseSit(); pose.spine_01 = [0.1 * reach, 0.3 * reach, 0.05 *
pose = reachIK(pose, 'l', this.doorHandle(car, false), reach, new V3(0.4,
pose = Character.blend(pose, this.drivePose(car, p.poseDrive(0)), ease((u
p.applyPose(pose);
if (u >= 0.8 && !s.closed) { s.closed = true; this.audio.door(false); car
if (u >= 1) { this.playerMode = 'car'; this.seq = null; car.engineOn = tr
}
} else { // exit (mirrored to the passenger door when the driver side is bloc
const side = s.side || 1;
if (side < 0) {
const mirrorPose = (po) => { const o = {}; for (const kk in po) { const v
const mv = (v) => new V3(-v.x, v.y, v.z);
const setL0 = setLocal;
const setLocalM = (pos, yaw, sc = 1) => setL0(mv(pos), -yaw, sc);
const applyM = (po) => p.applyPose(mirrorPose(po));
if (s.stage === 'slide') { // shuffle across the bench to the passenger s
const u = clamp(s.t / 1.0, 0, 1);
const pos = P.S.clone().lerp(mv(P.S), ease(u)); pos.y += Math.sin(u * M
setLocal(pos, 0, k);
p.applyPose(p.poseSit());
if (u >= 1) { s.stage = 'open'; s.t = 0; }
return;
}
}

```

```

if (s.stage === 'open') {
    const u = clamp(s.t / 0.8, 0, 1); setLocalM(P.S, 0, k);
    const reach = u < 0.3 ? ease(u / 0.3) : 1 - ease((u - 0.6) / 0.4);
    const pose = p.poseSit(); pose.spine_01 = [0.05, 0.25 * reach, 0]; pose
    if (u > 0.25 && !s.opened) { s.opened = true; this.audio.door(true); }
    car.doorAngleR = 1.15 * ease((u - 0.25) / 0.75); applyM(pose);
    if (u >= 1) { s.stage = 'out'; s.t = 0; }
} else if (s.stage === 'out') {
    const u = clamp(s.t / 1.45, 0, 1);
    const S = P.S, bb = P.B.clone(); bb.y = 0.02 + 0.2; const a = new V3(P.
    let pos = u < 0.5 ? S.clone().lerp(bb, ease(u / 0.5)) : bb.clone().lerp
    pos.y += Math.sin(clamp((u - 0.3) / 0.5, 0, 1) * Math.PI) * 0.08;
    setLocalM(pos, lerpAngle(0, Math.PI / 2, ease(u / 0.6)), lerp(k, 1, eas
    const lift = Math.sin(clamp(u / 0.55, 0, 1) * Math.PI);
    const outside = clamp((pos.x - (car.def.halfWidth - 0.05)) / 0.4, 0, 1)
    let base = Character.blend(p.poseSit(), p.poseIdle(p.t), Math.min(ease(
    base = Character.add(base, { thigh_1: [-1.0 * lift, 0, 0.25], calf_1: [
    applyM(base);
    if (u >= 1) { s.stage = 'close'; s.t = 0; s.pos = a; }
} else if (s.stage === 'close') {
    const u = clamp(s.t / 1.0, 0, 1);
    const yaw = lerpAngle(Math.PI / 2, -Math.PI * 0.8, ease(u / 0.45));
    setLocalM(s.pos, yaw);
    car.doorAngleR = 1.15 * (1 - ease((u - 0.4) / 0.4));
    const reach = u < 0.3 ? 0 : u < 0.45 ? ease((u - 0.3) / 0.15) : u < 0.8
    const pose = p.poseIdle(p.t); pose.spine_01 = [-0.12 * reach, -0.2 * re
    applyM(pose);
    if (u >= 0.8 && !s.closed) { s.closed = true; car.doorAngleR = 0; this.
    if (u >= 1) this.finishExit(mv(s.pos), -yaw);
}
return;
}
if (s.stage === 'open') {
    const u = clamp(s.t / 0.8, 0, 1);
    setLocal(P.S, 0, k);
    const reach = u < 0.3 ? ease(u / 0.3) : 1 - ease((u - 0.6) / 0.4);
    let pose = p.poseSit(); pose.spine_01 = [0.05, 0.25 * reach, 0];
    if (u > 0.25 && !s.opened) { s.opened = true; this.audio.door(true); }
    car.doorAngle = 1.15 * ease((u - 0.25) / 0.75);
    pose = reachIK(pose, 'l', this.doorHandle(car, false), reach, new V3(0.4,
    p.applyPose(pose);
    if (u >= 1) { s.stage = 'out'; s.t = 0; }
} else if (s.stage === 'out') {
    const u = clamp(s.t / 1.45, 0, 1);
    const S = P.S, bb = P.B.clone(); bb.y = 0.02 + 0.2;
    const a = new V3(P.A.x + 0.1, 0.02, P.A.z + 0.35);

```

```

    let pos;
    if (u < 0.5) pos = S.clone().lerp(bb, ease(u / 0.5)); else pos = bb.clone
    pos.y += Math.sin(clamp((u - 0.3) / 0.5, 0, 1) * Math.PI) * 0.08;
    const yaw = lerpAngle(0, Math.PI / 2, ease(u / 0.6));
    setLocal(pos, yaw, lerp(k, 1, ease(u / 0.6)));
    const lift = Math.sin(clamp(u / 0.55, 0, 1) * Math.PI);
    const stepLeg = { thigh_l: [-1.0 * lift, 0, 0.25], calf_l: [1.1 * lift, 0
    const outside = clamp((pos.x - (car.def.halfWidth - 0.05)) / 0.4, 0, 1);
    let base = Character.blend(p.poseSit(), p.poseIdle(p.t), Math.min(ease((u
    base = Character.add(base, stepLeg, 1 - ease((u - 0.55) / 0.3)));
    p.applyPose(base);
    if (u >= 1) { s.stage = 'close'; s.t = 0; s.pos = a; }
  } else if (s.stage === 'close') {
    const u = clamp(s.t / 1.0, 0, 1);
    const turn = ease(u / 0.45);
    const yaw = lerpAngle(Math.PI / 2, -Math.PI * 0.8, turn);
    setLocal(s.pos, yaw);
    car.doorAngle = 1.15 * (1 - ease((u - 0.4) / 0.4));
    const reach = u < 0.3 ? 0 : u < 0.45 ? ease((u - 0.3) / 0.15) : u < 0.8 ?
    let pose = p.poseIdle(p.t); pose.spine_01 = [-0.12 * reach, -0.2 * reach,
    pose = reachIK(pose, 'r', this.doorHandle(car, true), reach, new V3(-1, -
    p.applyPose(pose);
    if (u >= 0.8 && !s.closed) { s.closed = true; car.doorAngle = 0; this.aud
    if (u >= 1) this.finishExit(s.pos, yaw);
  }
}
}
}
finishExit(localPos, localYaw) {
  const car = this.car, p = this.player;
  const wp = localPos.clone().applyM4(car.node.world);
  car.node.remove(p.root); this.scene.add(p.root); p.root.scl.set(1, 1, 1);
  p.pos.copy(wp); p.pos.y = this.world.heightAt(wp.x, wp.z, wp.y + 1.5);
  p.yaw = localYaw + this.carYaw();
  this.playerMode = 'foot'; this.seq = null;
  p.setTransform(p.pos, p.yaw);
  car.engineOn = true;
  this.camYawOff = 0;
}

// ----- on foot -----
updateFoot(dt) {
  const p = this.player, inp = this.input, car = this.car;
  p.t += dt;
  // camera-relative movement
  const camYaw = Math.atan2(this.camLook.x - this.camPos.x, this.camLook.z - th
  const mx = inp.moveX, my = inp.moveY; const mag = Math.min(1, Math.hypot(mx,

```

```

const run = mag > 0.85 || inp.keys.has('ShiftLeft');
const speed = this.kick ? 0 : mag < 0.08 ? 0 : run ? 5.2 : 1.7 * Math.max(0.5
let targetVel = new V3();
if (speed > 0) {
    const a = camYaw + Math.atan2(-mx, my); // joystick right -> screen right
    targetVel.set(Math.sin(a) * speed, 0, Math.cos(a) * speed);
    p.yaw = lerpAngle(p.yaw, a, 1 - Math.exp(-dt * 10));
}
p.vel.x = smooth(p.vel.x, targetVel.x, 10, dt); p.vel.z = smooth(p.vel.z, tar
p.pos.x += p.vel.x * dt; p.pos.z += p.vel.z * dt;
// walls
const c = new V3(p.pos.x, p.pos.y + 0.95, p.pos.z);
this.world.sphere(c, 0.33, (nx, ny, nz, depth) => { if (ny < 0.6) { p.pos.x +
this.pushOutOfCar(p.pos, 0.35);
// ground
const gy = this.world.heightAt(p.pos.x, p.pos.z, p.pos.y + 0.6);
if (gy > p.pos.y - 0.5) p.pos.y = smooth(p.pos.y, gy, 18, dt); else { p.vy =
const hs = Math.hypot(p.vel.x, p.vel.z);
p.walkBlend = smooth(p.walkBlend, hs > 0.15 ? 1 : 0, 7, dt);
p.runBlend = smooth(p.runBlend || 0, clamp((hs - 2.2) / 2.6, 0, 1), 5, dt);
const prevPh = p.phase;
// cadence: ~1.8 steps/s walking, ~2.8 steps/s running; stride grows with spe
p.phase += dt * Math.PI * lerp(1.7 + hs * 0.12, 2.6 + hs * 0.07, p.runBlend)
if (Math.floor(prevPh / Math.PI) !== Math.floor(p.phase / Math.PI) && hs > 0.
let pose = Character.blend(p.poseIdle(p.t), p.poseWalk(p.phase, p.runBlend),
p.pelvisOffset.set(0, p.gaitBob(p.phase, p.runBlend) * p.walkBlend, 0);
// lean into turns and with acceleration
const turnRate = angleWrap(p.yaw - (p.prevYaw ?? p.yaw)) / Math.max(dt, 1e-3)
p.lean = smooth(p.lean || 0, clamp(-turnRate * hs * 0.03, -0.25, 0.25), 6, dt
const acc = (hs - (p.prevHs ?? hs)) / Math.max(dt, 1e-3); p.prevHs = hs;
p.pitchLean = smooth(p.pitchLean || 0, clamp(acc * 0.02, -0.12, 0.15), 5, dt)
if (this.kick) {
    this.kick.t += dt; const u = this.kick.t / 0.8;
    pose = Character.blend(pose, p.poseKick(u), Math.min(1, u * 8, (1 - u) * 8)
    if (u > 0.42 && !this.kick.hit) { this.kick.hit = true; this.kickHit(); }
    if (u >= 1) this.kick = null;
}
p.applyPose(pose);
p.setTransform(p.pos, p.yaw, p.pitchLean, p.lean);
this.hud.setNearDoor(this.nearDoor());
}
pushOutOfCar(pos, rad) {
    const car = this.car; const d = car.def;
    const lp = car.worldToLocal(pos.clone().add(new V3(0, 0.9, 0)), new V3());
    const hx = d.halfWidth + rad, z0 = d.zMin - rad, z1 = d.zMax + rad;
    if (lp.y < -0.2 || lp.y > 2.3) return false;

```

```

    if (Math.abs(lp.x) < hx && lp.z > z0 && lp.z < z1) {
        const px = hx - Math.abs(lp.x), pz = Math.min(lp.z - z0, z1 - lp.z);
        if (px < pz) lp.x = Math.sign(lp.x || 1) * hx; else lp.z = (lp.z - z0 < z1
        const wp = car.localToWorld(lp, new V3()); pos.x = wp.x; pos.z = wp.z; retu
    }
    return false;
}
startKick() {
    if (this.playerMode !== 'foot' || this.kick) return;
    this.kick = { t: 0, hit: false }; this.audio.whoosh();
}
kickHit() {
    const p = this.player; const f = new V3(Math.sin(p.yaw), 0, Math.cos(p.yaw));
    const foot = p.pos.clone().addS(f, 0.85); foot.y += 0.7;
    let hit = false;
    for (const n of this.npcs) {
        if (!n.active || n.state === 'flying' || n.state === 'lying') continue;
        const d = new V3(n.pos.x - foot.x, 0, n.pos.z - foot.z);
        if (d.len() < 0.95) { this.knock(n, f.clone().scale(5.5).add(new V3(0, 2.2,
    }
    for (const c of this.cones) {
        if (c.pos.dist(new V3(foot.x, c.pos.y + 0.3, foot.z)) < 0.8) { c.vel.copy(f
    }
    const car = this.car; const lp = car.worldToLocal(foot, new V3());
    if (Math.abs(lp.x) < car.def.halfWidth + 0.15 && lp.z > car.def.zMin - 0.15 &
        car.dent(foot, f, 0.045, 0.35, this.r);
        car.body.applyImpulse(f.clone().scale(900), foot);
        this.fx.impactDust(foot, 2); hit = true;
        this.audio.thump(80, 0.25, 0.8);
    }
    if (hit) { this.audio.kick(); this.shake = Math.max(this.shake, 0.25); }
}

// ----- spectators -----
knock(n, vel, from) {
    n.state = 'flying'; n.stateT = 0; n.vel.copy(vel);
    n.yaw = Math.atan2(from.x - n.pos.x, from.z - n.pos.z); // face the hit, fall
    n.spin = (Math.random() - 0.5) * 2;
    this.audio.thump(100, 0.2, 0.7);
}
updateNPCs(dt) {
    const car = this.car; const cp = car.body.pos; const cv = car.body.vel;
    const drifting = Math.abs(car.driftAngle) > 0.35 && car.speed > 7;
    for (const n of this.npcs) {
        if (!n.active) continue;
        n.t += dt; n.stateT += dt;

```

```

let pose;

const toCar = new V3(cp.x - n.pos.x, 0, cp.z - n.pos.z); const dist = toCar
if (n.state === 'idle' || n.state === 'cheer') {
    // dodge a car coming at us
    const rel = -(cv.x * toCar.x + cv.z * toCar.z) / Math.max(dist, 0.1);
    if (dist < 10 && rel > 5 && dist / rel < 1.1) {
        const side = Math.sign((cv.x * toCar.z - cv.z * toCar.x) || 1);
        n.state = 'dodge'; n.stateT = 0; n.dodgeDir = new V3(cv.z, 0, -cv.x).no
    } else if (drifting && dist < 35) { if (n.state !== 'cheer') { n.state =
    else if (n.state === 'cheer' && n.stateT > 2.5) { n.state = 'idle'; n.sta
    // walk back home slowly if displaced
    const hd = new V3(n.home.x - n.pos.x, 0, n.home.z - n.pos.z);
    if (hd.len() > 0.6 && n.state === 'idle') {
        hd.norm(); n.pos.addS(hd, dt * 1.3); n.yaw = lerpAngle(n.yaw, Math.atan
        pose = n.poseWalk(n.phase, false);
    } else {
        n.yaw = lerpAngle(n.yaw, Math.atan2(toCar.x, toCar.z), 1 - Math.exp(-dt
        pose = n.state === 'cheer' ? n.poseCheer(n.t) : n.poseIdle(n.t);
        pose = n.headLookPose(pose, cp);
    }
} else if (n.state === 'dodge') {
    const u = n.stateT / 0.65;
    n.pos.addS(n.dodgeDir, dt * 5.5 * (1 - u));
    pose = n.poseStep(u, 1);
    if (u >= 1) { n.state = 'idle'; n.stateT = 0; }
} else if (n.state === 'flying') {
    n.vel.y -= 9.8 * dt; n.pos.addS(n.vel, dt);
    n.pitch = Math.max(-Math.PI / 2, n.pitch - dt * 4.5);
    n.yaw += n.spin * dt;
    const gy = this.world.heightAt(n.pos.x, n.pos.z, n.pos.y + 1);
    if (n.pos.y <= gy && n.vel.y < 0) {
        n.pos.y = gy;
        if (Math.abs(n.vel.y) > 3) { n.vel.y *= -0.25; n.vel.x *= 0.5; n.vel.z
        else { n.state = 'lying'; n.stateT = 0; n.vel.set(0, 0, 0); }
    }
    pose = n.poseFall(n.stateT);
} else if (n.state === 'lying') {
    n.pitch = -Math.PI / 2;
    pose = n.poseLying(n.t);
    if (n.stateT > 2.6) { n.state = 'getup'; n.stateT = 0; }
} else if (n.state === 'getup') {
    const u = n.stateT / 1.5;
    n.pitch = -Math.PI / 2 * (1 - ease(u * 1.3));
    pose = n.poseGetUp(u);
    if (u >= 1) { n.state = 'idle'; n.stateT = 0; n.pitch = 0; }
}

```

```

// collisions with the car body
if (n.state !== 'flying') {
  const center = new V3(n.pos.x, n.pos.y + 0.9, n.pos.z);
  const lp = car.worldToLocal(center, new V3());
  const d = car.def;
  if (Math.abs(lp.x) < d.halfWidth + 0.35 && lp.z > d.zMin - 0.35 && lp.z <
    const v = car.body.velAt(center, new V3());
    if (v.len() > 2.5 && n.state !== 'lying') {
      this.knock(n, v.clone().scale(0.85).add(new V3(0, 2.5 + v.len() * 0.1
      this.fx.impactDust(center, 4);
      car.body.vel.scale(0.985);
    } else this.pushOutOfCar(n.pos, 0.35);
  }
}

if (n.state !== 'flying' && n.state !== 'lying' && n.state !== 'getup') n.p
n.applyPose(pose || n.poseIdle(n.t));
const lift = n.state === 'lying' ? 0.12 : n.state === 'getup' ? 0.12 * (1 -
n.setTransform(new V3(n.pos.x, n.pos.y + lift, n.pos.z), n.yaw, n.pitch);
n.root.traverse(x => { x.castShadow = dist < 35; });
}

// player also gets knocked if run over? (just pushed)
}

// ----- cones -----
stepCones(dt) {
  const car = this.car; const b = car.body;
  for (const c of this.cones) {
    if (!c.node.visible) continue;
    // car contact
    if (c.pos.dist(b.pos) < 4.5) {
      const center = c.pos.clone(); center.y += 0.35;
      const lp = car.worldToLocal(center, new V3());
      const d = car.def;
      if (Math.abs(lp.x) < d.halfWidth + 0.2 && lp.z > d.zMin - 0.2 && lp.z < d
        const v = b.velAt(center, new V3());
        if (v.len() > 0.5) {
          c.vel.copy(v).scale(1.25); c.vel.y = 2 + v.len() * 0.25;
          c.ang.set((Math.random() - 0.5) * 14, (Math.random() - 0.5) * 8, (Mat
            if (!c.hitT || this.time - c.hitT > 0.4) { c.hitT = this.time; this.a
          }
        }
      }
    }
  }

  if (c.rest) continue;
  c.vel.y -= 9.8 * dt; c.pos.addS(c.vel, dt); c.q.integrate(c.ang, dt);
  const gy = this.world.heightAt(c.pos.x, c.pos.z, c.pos.y + 1);
  if (c.pos.y < gy) {

```

```

        c.pos.y = gy; if (c.vel.y < 0) c.vel.y *= -0.3; c.vel.x *= 0.9; c.vel.z *
        if (c.vel.len() < 0.4 && c.ang.len() < 0.6) { c.rest = true; }
    }
    c.node.pos.copy(c.pos); c.node.rot.copy(c.q);
}
}

// ----- impacts & effects -----
// wheels torn off in a crash roll/bounce away on their own
updateDebris(dt) {
    const car = this.car;
    if (car.detachEvents && car.detachEvents.length) {
        for (const ev of car.detachEvents) {
            const w = ev.w, n = w.node; if (!n) continue;
            const wp = car.localToWorld(new V3().copy(w.pos).add(w.dmgOff), new V3())
            const q = new Q().copy(car.body.q).mul(n.rot);
            if (n.parent) n.parent.remove(n);
            n.pos.copy(wp); n.rot.copy(q); this.scene.add(n);
            const out = new V3().copy(ev.dir).applyQ(car.body.q).scale(-1);
            const vel = car.body.velAt(wp, new V3()).addS(out, 3 + Math.random() * 3)
            (this.debris || (this.debris = [])).push({ node: n, vel, ang: new V3((Mat
            this.hud.toast('1000 , 'طار الكفر!');
        }
        car.detachEvents.length = 0;
    }
    if (!this.debris) return;
    for (const d of this.debris) {
        if (d.node.parent !== this.scene) continue;
        d.vel.y -= 9.81 * dt; d.node.pos.addS(d.vel, dt);
        const g = this.world.heightAt(d.node.pos.x, d.node.pos.z, d.node.pos.y + 1)
        if (d.node.pos.y - d.r * 0.55 < g) {
            d.node.pos.y = g + d.r * 0.55; if (d.vel.y < 0) d.vel.y = -d.vel.y * 0.35
            d.vel.x *= Math.exp(-dt * 0.8); d.vel.z *= Math.exp(-dt * 0.8); d.ang.sca
            if (Math.abs(d.vel.y) < 0.5 && d.vel.lenSq() < 0.3) d.vel.set(0, 0, 0);
        }
        const al = d.ang.len();
        if (al > 1e-3) { const dq = new Q().setAxisAngle(new V3().copy(d.ang).scale
    }
}
clearDebris() {
    for (const d of this.debris || []) { const car = this.cars[d.carKey]; if (d.n
        this.debris = [];
    }
}

handleImpacts(dt) {
    const car = this.car;

```

```

if (!car.impacts.length) return;
// strongest few distinct impacts this frame
const list = car.impacts.slice().sort((a, b) => b.speed - a.speed);
car.impacts.length = 0;
const picked = [];
for (const im of list) { if (picked.length >= 3) break; if (picked.every(p =>
const s = picked[0].speed;
if (s > 3 && this.time - this.lastDent > 0.1) {
    this.lastDent = this.time;
    for (const im of picked) {
        const si = im.speed; if (si < 3) continue;
        const amount = clamp((si - 2.2) * 0.062, 0.03, 1.05);
        const rad = clamp(0.65 + si * 0.055, 0.65, 2.2);
        car.dent(im.point, im.normal, amount, rad, this.r);
        // big hits crumple deeper into the car (a second, tighter, deeper fold)
        if (si > 11) car.dent(im.point.clone().addS(im.normal, 0.25), im.normal,
        if (si > 17) { car.dent(im.point.clone().addS(im.normal, 0.5), im.normal,
        if (si > 5) this.fx.sparkBurst(im.point, im.normal, si);
        this.fx.impactDust(im.point, si);
        // hits near a front wheel knock the steering out of line
        const lp = car.worldToLocal(im.point, new V3());
        if (lp.z > car.def.zMax - 1.4 && Math.abs(lp.x) > car.def.halfWidth - 0.4
    }
    const dmg = s < 4 ? 0 : (s - 3) * 4.6;
    car.damage = Math.min(100, car.damage + dmg);
    car.engineHealth = clamp(1 - Math.max(0, car.damage - 35) / 100, 0.35, 1);
    this.shake = Math.max(this.shake, clamp(s / 12, 0.15, 1.2));
    if (s > 14) this.hud.toast(car.damage > 80 ? '1200', '!حادث قوي!': 'ة تكسرت!');
    if (this.driftTime > 0.5) { this.hud.driftFail(); this.driftPts = 0; this.d
}
if (s > 2.5 && this.time - this.lastCrashSound > 0.15) { this.lastCrashSound
}

updateEffects(dt) {
    const car = this.car; const b = car.body;
    for (let i = 0; i < car.wheels.length; i++) {
        const w = car.wheels[i];
        if (!w.onGround) { this.fx.wheelTrail(car.key + i, w.contact, UP, 0); conti
        const side = new V3(1, 0, 0).applyQ(b.q);
        const onSand = w.tag === 3;
        if (onSand) {
            this.fx.dust(w.contact, b.vel, clamp(car.speed / 10, 0, 1) * 0.9 + w.skid
            if (car.speed > 12 || w.skid > 0.3) this.fx.dust(w.contact, b.vel, 0.6);
            this.fx.wheelTrail(car.key + i, w.contact, side, 0);
            this.fx.rutTrail(car.key + i, w.contact, side, car.speed > 0.8 ? 0.55 + w
        } else this.fx.rutTrail(car.key + i, w.contact, side, 0);
    }
}

```

```

        if (!onSand) {
            this.fx.wheelTrail(car.key + i, w.contact, side, w.skid, 0.27);
            this.fx.tyreSmoke(w.contact, b.vel, w.skid * (w.slipLong > 0.3 ? 1.2 : 1))
        }
    }
    if (car.damage > 55 && Math.random() < (car.damage - 50) / 60) {
        const hood = car.localToWorld(new V3((Math.random() - 0.5) * 0.6, 1.25, car
        this.fx.smoke.emit(hood, new V3(b.vel.x * 0.3, 1.2, b.vel.z * 0.3), { life:
    }
    // body scraping (e.g. sliding on the roof)
    if (car.scrape > 3) { const p = car.body.pos.clone(); p.y = Math.max(p.y - 0.
    this.fx.update(dt);
}

updateScore(dt) {
    const car = this.car;
    const a = Math.abs(car.driftAngle);
    const ok = this.playerMode === 'car' && car.grounded >= 2 && car.speed > 7 &&
    if (ok) {
        this.driftTime += dt; this.noDriftT = 0;
        this.driftCombo = 1 + Math.min(4, Math.floor(this.driftTime / 2));
        this.driftPts += car.speed * 3.6 * a * dt * 2.2 * this.driftCombo;
        this.hud.drift(Math.round(this.driftPts), this.driftCombo, a);
    } else if (this.driftTime > 0) {
        this.noDriftT += dt;
        if (this.noDriftT > 0.9) {
            if (this.driftPts > 50) { this.score += Math.round(this.driftPts); this.h
            else this.hud.drift(0, 1, 0, true);
            this.driftPts = 0; this.driftTime = 0; this.driftCombo = 1;
        }
    }
}

resetCar() {
    const car = this.car; car.flooded = false; car.floodMsg = false; car.engineOn
    const yaw = this.carYaw();
    car.place(p.x, this.world.heightAt(p.x, p.z, p.y + 3) + 0.3, p.z, yaw);
    if (this.playerMode === 'seq') { this.enterSeatNow(); this.seq = null; }
    this.hud.toast('رجعنا السيارة على كفرائها');
}

// ----- camera -----
placeCameraBehind() {
    const car = this.car; const f = car.forward(new V3());
    this.camPos.copy(car.body.pos).addS(f, -8).add(new V3(0, 3, 0));
    this.camLook.copy(car.body.pos);
}

```

```

}
// drone: free-flying photo camera (the car waits)
toggleDrone() {
  if (this.drone) { this.drone = null; this.hud.toast('900 , 'رجعنا للسيارة'); re
  const f = new V3().copy(this.camLook).sub(this.camPos); const yaw = Math.atan
  this.drone = { pos: this.camPos.clone().add(new V3(0, 1.5, 0)), yaw, pitch };
  this.hud.toast('3000 , 'اسحب الشاشة عشان تشوف');
}
droneCam(dt) {
  const inp = this.input, D = this.drone, cam = this.cam;
  if (inp.camDX || inp.camDY) { D.yaw -= inp.camDX * 0.005; D.pitch = clamp(D.p
  D.yaw -= (inp.steer || 0) * dt * 1.4;
  const dir = new V3(Math.sin(D.yaw) * Math.cos(D.pitch), Math.sin(D.pitch), Ma
  const sp = 14 * ((inp.throttle || 0) - (inp.brake || 0));
  D.pos.addS(dir, sp * dt);
  const g = this.world.heightAt(D.pos.x, D.pos.z, D.pos.y + 50); if (D.pos.y <
  const c = this.car.body.pos; const off = D.pos.clone().sub(c); if (off.len()
  this.camPos.copy(D.pos); this.camLook.copy(D.pos).add(dir);
  cam.pos.copy(this.camPos); cam.target.copy(this.camLook); cam.near = 0.15; ca
}
updateCamera(dt) {
  const inp = this.input, cam = this.cam, car = this.car;
  if (this.drone) { this.droneCam(dt); return; }
  // user orbit drag
  if (inp.camDX || inp.camDY) {
    const inCab = this.camMode === 3 && this.playerMode === 'car', sg = inCab ?
    this.camYawOff -= sg * inp.camDX * 0.006; this.camPitchOff = clamp(this.cam
    inp.camDX = 0; inp.camDY = 0; this.camIdle = 0;
  } else this.camIdle += dt;
  if (!inp.camDragging && this.camIdle > 1.6 && (this.playerMode === 'car' || t
    this.camYawOff *= Math.exp(-dt * 1.8); this.camPitchOff *= Math.exp(-dt * 1
  }
  let target, desired;
  if (this.playerMode === 'foot') {
    const p = this.player;
    target = p.pos.clone().add(new V3(0, 1.55, 0));
    const yaw = (this.footYaw ?? p.yaw);
    // follow behind the player when moving
    const hs = Math.hypot(p.vel.x, p.vel.z);
    if (this.footYaw === undefined) this.footYaw = p.yaw;
    if (hs > 0.5 && this.camIdle > 0.6) this.footYaw = lerpAngle(this.footYaw,
    const a = this.footYaw + this.camYawOff; const pitch = 0.28 + this.camPitch
    const dist = 4.2;
    desired = target.clone().add(new V3(-Math.sin(a) * Math.cos(pitch) * dist,
    { const dir = desired.clone().sub(target); const dl = dir.len(); dir.scale(
    this.camPos.lerp(desired, 1 - Math.exp(-dt * 8));
  }

```

```

    this.camLook.lerp(target, 1 - Math.exp(-dt * 12));
    this.clampCam(target);
} else {
    this.footYaw = undefined;
    const b = car.body; const f = car.forward(new V3()); f.y = 0; f.norm();
    const vflat = new V3(b.vel.x, 0, b.vel.z); const sp = vflat.len();
    let head = f.clone();
    if (sp > 4 && car.forwardSpeed > 0) { vflat.scale(1 / sp); head.lerp(vflat,
    if (car.reverse && car.forwardSpeed < -2) head = f.clone();
    const baseYaw = Math.atan2(head.x, head.z) + this.camYawOff;
    if (this.camMode === 2) { // hood cam
        const hp = car.localToWorld(new V3(0, 1.55, car.def.zMax - 1.9), new V3())
        const la = car.localToWorld(new V3(0, 1.45, car.def.zMax + 10), new V3())
        this.camPos.copy(hp); this.camLook.copy(la);
    } else if (this.camMode === 3 && this.playerMode === 'car') { // cockpit: d
        this.cockpitCam(dt);
    } else {
        const cp = this.chaseParams(this.camMode === 3 ? 0 : this.camMode);
        const dist = cp.dist + Math.min(sp, 40) * 0.035;
        const h = cp.h + this.camPitchOff * 4;
        target = b.pos.clone().add(new V3(0, 1.0, 0));
        desired = target.clone().add(new V3(-Math.sin(baseYaw) * dist, h, -Math.c
        // keep camera out of walls
        const dir = desired.clone().sub(target); const dl = dir.len(); dir.scale(
        const hit = this.world.raycast(target, dir, dl, {});
        if (hit) desired = target.clone().addS(dir, Math.max(0.3, hit.t - 0.4));
        const k = 1 - Math.exp(-dt * (this.camDragging ? 14 : 5));
        if (this.camPos.dist(desired) > 40) this.camPos.copy(desired);
        this.camPos.lerp(desired, k);
        this.camPos.y = Math.max(this.camPos.y, this.world.heightAt(this.camPos.x
        this.camLook.lerp(target, 1 - Math.exp(-dt * 14));
        this.clampCam(target);
    }
}
}
cam.pos.copy(this.camPos); cam.target.copy(this.camLook);
const inside = this.camMode === 3 && this.playerMode === 'car';
cam.near = inside ? 0.03 : 0.15;
if (inside) cam.up.copy(this.camUp); else cam.up.set(0, 1, 0);
if (this.shake > 0) {
    const s = this.shake * 0.25; cam.pos.add(new V3((Math.random() - 0.5) * s,
    this.shake = Math.max(0, this.shake - dt * 2.2);
}
const spd = this.playerMode === 'car' ? car.speed : 0;
cam.fov = smooth(cam.fov, ((inside ? 72 : 58) + Math.min(spd, 45) * (inside ?
}
// never let the (smoothed) camera end up behind a wall: re-cast from the targe

```

```

clampCam(target) {
    const dir = this.camPos.clone().sub(target); const dl = dir.len(); if (dl < 1
    const hit = this.world.raycast(target, dir, dl + 0.3, {});
    if (hit && hit.t < dl + 0.3) this.camPos.copy(target).addS(dir, Math.max(0.12
}
chaseParams(mode) {
    if (mode === 4 && this.camCustom) return this.camCustom;
    return mode === 1 ? { dist: 10, h: 3.4 } : { dist: 6.3, h: 2.2 };
}
// eyes of the seated driver; view follows the car's roll/pitch with a little l
cockpitCam(dt) {
    this.camYawOff = clamp(this.camYawOff, -1.25, 1.25); this.camPitchOff = clamp
    const car = this.car, p = this.player, hb = p.bones.head;
    // head position in car space (both matrices from the same frame, so no lag a
    if (!this._inv) this._inv = M4.create();
    M4.invert(this._inv, car.node.world);
    const hl = new V3(hb.world[12], hb.world[13], hb.world[14]).applyM4(this._inv
    if (!isFinite(hl.x) || hl.len() > 4) hl.set(car.seatPos.x, car.seatPos.y + 0.
    this.headLocal = this.headLocal ? this.headLocal.lerp(hl, 0.5) : hl;
    const f = car.forward(new V3()), up = new V3(0, 1, 0).applyQ(car.body.q), rig
    const b = car.body;
    // lateral / longitudinal g push the head (weight feel)
    const acc = this.prevVel ? b.vel.clone().sub(this.prevVel).scale(1 / Math.max
    this.prevVel = b.vel.clone();
    const lat = clamp(acc.dot(right) / 9.81, -1.5, 1.5), lon = clamp(acc.dot(f) /
    this.headSway = this.headSway || new V3();
    this.headSway.lerp(new V3(-lat * 0.045, 0, -lon * 0.03), 1 - Math.exp(-dt * 6
    const el = this.headLocal.clone().add(new V3(this.headSway.x, 0.0, 0.1 + this
    const eye = car.localToWorld(el, new V3());
    const yaw = this.camYawOff + car.steer * 0.12;
    const look = f.clone().scale(Math.cos(yaw)).addS(right, -Math.sin(yaw)).addS(
    this.camPos.copy(eye); this.camLook.copy(eye).addS(look, 10);
    this.camUp = this.camUp || new V3(0, 1, 0);
    this.camUp.lerp(up, 1 - Math.exp(-dt * 10)).norm();
}
// editor preview: fixed chase view behind the parked/paused car
editorCamera(params) {
    const car = this.car, cam = this.cam; const f = car.forward(new V3()); f.y =
    const t = car.body.pos.clone().add(new V3(0, 1.0, 0));
    if (params.mode === 2) { cam.pos.copy(car.localToWorld(new V3(0, 1.55, car.de
    else if (params.mode === 3) { cam.pos.copy(car.localToWorld(new V3(car.seatPo
    else { const p = params.mode === 4 ? params : this.chaseParams(params.mode);
    cam.up.set(0, 1, 0); cam.near = params.mode === 3 ? 0.03 : 0.15; cam.fov = 58
    this.r.shadowFocus.copy(car.body.pos);
}
updateSaher(dt) {

```

```

const S = this.map.saher; if (!S || this.playerMode !== 'car') return;
const car = this.car, kmh = Math.abs(car.forwardSpeed) * 3.6;
this.saherCool = Math.max(0, (this.saherCool || 0) - dt);
for (const s of S) {
    const dx = s.x - car.body.pos.x, dz = s.z - car.body.pos.z;
    if (dx * dx + dz * dz < 28 * 28 && kmh > 120 && !this.saherCool) {
        this.saherCool = 6; this.hud.flash && this.hud.flash();
        const fine = kmh > 170 ? 'مخالفة ٩٠٠' : kmh > 150 ? 'مخالفة ٣٠٠' : 'مخالفة ١٠٠';
        this.hud.toast('🚗! ساهر صُورَك ' + Math.round(kmh).toLocaleString('ar-SA'))
        if (this.audio.ctx) this.audio.burst({ freq: 3200, dur: 0.05, gain: 0.25 })
    }
}

// ---- character select: the chosen character stands in the showroom, playing
heroSetup() {
    if (this.hero || !this.assets.hero) return this.hero;
    const src = this.assets.hero, root = new Node('hero'); const inner = src.root
    inner.updateWorld(null); let y0 = 1e9, y1 = -1e9;
    let sk = null; inner.traverse(n => { if (n.skin) sk = n.skin; });
    if (sk) for (const j of sk.joints) { y0 = Math.min(y0, j.world[13]); y1 = Math.max(y1, j.world[13]); }
    const h = y1 - y0; const k = h > 0.1 && h < 50 ? 1.76 / h : 1; inner.scl.set(k, k, k);
    let hip = null; inner.traverse(n => { if (!hip && /Hip|Pelvis|hps/i.test(n.name)) hip = n; });
    this.hero = { root, inner, hip, base: inner.pos.clone(), anim: src.animations };
    if (hip) { playAnimation(this.hero.anim, 0); root.updateWorld(null); this.hero = this; }
    return this.hero;
}

// play the clip in place (cancel the root motion so the character walks on the ground)
heroPose(t) {
    const H = this.hero; playAnimation(H.anim, t); if (!H.hip) return;
    H.inner.pos.copy(H.base); H.root.updateWorld(null);
    const d = new V3(H.hip.world[12], 0, H.hip.world[14]).sub(H.root.pos).sub(H.hip.pos);
    H.inner.pos.x -= d.x; H.inner.pos.z -= d.z;
}

enterRoom() {
    const car = this.car; this.roomSave = { map: this.mapKey, pos: this.player.pos };
    this.inRoom = true; this.loadMap('room'); if (car.node.parent) car.node.parent.remove(car);
    const sp = this.map.spawn; this.player.pos.set(sp.x, this.world.heightAt(sp.x, sp.y), sp.y);
    this.camPos.copy(this.player.pos).add(new V3(0, 1.6, 2.5)); this.camLook.copy(this.camPos)
}

exitRoom() {
    const S = this.roomSave; this.inRoom = false; this.loadMap(S.map); this.scene.remove(this.player);
    this.player.pos.copy(S.pos).add(new V3(Math.sin(S.yaw + Math.PI) * 1.5, 0, Math.cos(S.yaw + Math.PI) * 1.5));
}

showChar(which) {

```

```

    this.charKey = which; this.mode = 'char';
    for (const k in this.cars) if (this.cars[k].node.parent) this.scene.remove(th
    const sr = this.map.showroom, y = this.world.heightAt(sr.x, sr.z, 5);
    if (this.hero && this.hero.root.parent) this.hero.root.parent.remove(this.her
    if (which === 'hero' && this.heroSetup()) { this.hero.root.pos.set(sr.x, y, s
    else { const p = this.player; if (p.root.parent) p.root.parent.remove(p.root)
    this.charPos = new V3(sr.x, y, sr.z);
  }
  updateChar(dt) {
    if (!this.menuHold) this.menuA = (this.menuA || 0) + dt * 0.25; const c = thi
    this.cam.pos.set(c.x + Math.sin(a) * 3.4, c.y + 1.45, c.z + Math.cos(a) * 3.4
    this.cam.up.set(0, 1, 0); this.cam.near = 0.1; this.cam.fov = 45 * Math.PI /
    if (this.charKey === 'hero' && this.hero) { this.hero.t += dt; this.heroPose(
    else { const p = this.player; p.t = (p.t || 0) + dt; p.applyPose(p.poseIdle(p
    this.audio.stopEngine('car');
  }
  updateMenu(dt) {
    if (this.mode === 'char') return this.updateChar(dt);
    const car = this.car;
    if (!this.menuHold) this.menuA = (this.menuA || 0) + dt * 0.12;
    const c = car.body.pos;
    const a = this.menuA;
    const d = this.mode === 'garage' ? 6.6 : 7.6;
    const gar = this.mode === 'garage';
    this.cam.pos.set(c.x + Math.sin(a) * d, c.y + (gar ? 1.5 : 1.4), c.z + Math.c
    this.cam.target.set(c.x, c.y + (gar ? -0.35 : 0.1), c.z);
    if (this.tuneView) { const rx = Math.cos(a), rz = -Math.sin(a); this.cam.targ
    this.cam.up.set(0, 1, 0); this.cam.near = 0.15;
    this.cam.fov = 45 * Math.PI / 180;
    this.r.shadowFocus.copy(c);
    for (const n of this.npcs) { if (!n.active) continue; n.t += dt; n.applyPose(
    if (this.player.root.parent) this.player.root.parent.remove(this.player.root)
    this.audio.stopEngine('car');
  }

  updateAudio(dt) {
    const car = this.car, a = this.audio;
    if (!a.ctx) return;
    const inCar = this.playerMode !== 'foot';
    const camDist = this.cam.pos.dist(car.body.pos);
    const vol = clamp(1.4 - camDist / 40, 0.15, 1) * (inCar ? 1 : 0.8);
    a.updateEngine('car', car.voice || car.def.engineVoice, car.rpm, car.engineOn
    let sk = 0; for (const w of car.wheels) if (w.onGround && w.tag !== 3) sk = M
    a.setSqueal(sk * vol, car.speed);
    a.setRoad(car.grounded ? car.speed * vol : 0);
    a.setScrape(car.scrape > 1.5 ? car.scrape : 0);
  }

```

```

}

// ----- thumbnails -----
// top-down map picture around the player, with the markers' pixel positions
renderMap(w, h, span) {
  const r = this.r, c = (this.playerMode === 'foot' ? this.player.pos : this.ca
  const t = this._mapTarget && this._mapTarget.w === w ? this._mapTarget : (thi
  const cam = new Camera(); cam.aspect = w / h; cam.fov = 2 * Math.atan(span /
  cam.pos.set(c.x, c.y + 900, c.z); cam.target.set(c.x, c.y, c.z); cam.up.set(0
  const oldFog = r.env.fog; r.env.fog = 0; const of = r.shadowFocus.clone(); r.
  r.render(this.roots(), cam, { target: t }); r.env.fog = oldFog; r.shadowFocus
  const img = r.readTarget(t);
  const proj = (x, y, z) => { const m = cam.vp; const cx = m[0] * x + m[4] * y
  return { img, proj };
}
thumbnail(key, w = 360, h = 220) {
  const car = this.cars[key]; const r = this.r;
  const t = this._thumbTarget || (this._thumbTarget = r.makeTarget(w * 2, h * 2
  const tmp = new Node('thumb'); const saved = car.node.parent; if (saved) save
  const pos = car.node.pos.clone(), rot = car.node.rot.clone();
  car.node.pos.set(0, 0, 0); car.node.rot.setAxisAngle(UP, 0.35);
  tmp.add(car.node); tmp.updateWorld();
  const cam = new Camera(); cam.fov = 34 * Math.PI / 180; cam.pos.set(4.9, 1.8,
  const oldFocus = r.shadowFocus.clone(); r.shadowFocus.set(0, 0, 0);
  const oldFog = r.env.fog; r.env.fog = 0;
  r.render([tmp], cam, { target: t });
  r.env.fog = oldFog; r.shadowFocus.copy(oldFocus);
  const big = r.readTarget(t);
  tmp.remove(car.node); car.node.pos.copy(pos); car.node.rot.copy(rot); if (sav
  const c = document.createElement('canvas'); c.width = w; c.height = h; c.get
  return c.toDataURL('image/png');
}

roots() { this.scene.updateWorld(); return [this.scene]; }
}

function lerpAngle(a, b, t) { return a + angleWrap(b - a) * clamp(t, 0, 1); }

function coneMeshes(orange, white) {
  // traffic cone: square base + tapered body with a white stripe
  const P = [], N = [], U = [], I = [];
  const seg = 16;
  const ring = (y, r) => { const b = P.length / 3; for (let i = 0; i <= seg; i++)
  const strip = (b0, b1) => { for (let i = 0; i < seg; i++) I.push(b0 + i, b1 + i
  const o = { P: [], N: [], U: [], I: [] };
  const r0 = ring(0.04, 0.17), r1 = ring(0.32, 0.105); strip(r0, r1);

```

```

const r2 = ring(0.44, 0.08), r3 = ring(0.7, 0.028); strip(r2, r3);
const body = makePrim(P, N, U, I, orange);
const P2 = [], N2 = [], U2 = [], I2 = [];
const ring2 = (y, r) => { const b = P2.length / 3; for (let i = 0; i <= seg; i+
const a0 = ring2(0.32, 0.105), a1 = ring2(0.44, 0.08); for (let i = 0; i < seg;
const stripe = makePrim(P2, N2, U2, I2, white);
const base = boxPrim(0.42, 0.04, 0.42, orange); for (let i = 1; i < base.attrs.
return { prims: [body, stripe, base] };
}

return { Game };
})();
__M['net'] = (() => {
// Online play: friends, friend requests, invitations and live sessions over the
// (everyone who has the game open right now). Each player publishes one presence
// { uid, name, sess, map, car, st:[...car state...], inv:[...], freq:[...], fa
// Friends are remembered on this phone (localStorage). Other players' cars are d
// interpolated, their crashes are replayed as dents on our copy of their car.
const { V3, Q, clamp, lerp } = __M['math'];
const { Car, CAR_DEFS } = __M['car'];
const { Character, cloneCharacter } = __M['character'];

const A1 = ['شاهين', 'نمر', 'برق', 'رعد', 'سهم', 'هزاع', 'عقاب', 'غيث', 'ليث'];
const A2 = ['ديرة', 'نجد', 'الشمال', 'الجنوب', 'البر', 'الخط', 'السرعة', 'التفحيط'];
const BOT_ID = 'bot-practice', BOT_NAME = 'خويك الوهمي';
function randomName() { const r = (a) => a[Math.floor(Math.random() * a.length)];

const load = (k, d) => { try { const v = localStorage.getItem(k); return v ? JSON
const save = (k, v) => { try { localStorage.setItem(k, JSON.stringify(v)); } catc
const r3 = (x) => Math.round(x * 1000) / 1000;

// deep copy of a car model with its own vertex buffers (so dents on a friend's c
function cloneGltf(g) {
const root = g.root.clone(true);
root.traverse(n => {
if (!n.mesh || !/^(body|door_l|door_r)$/.test(n.name)) return;
n.mesh = { ...n.mesh, prims: n.mesh.prims.map(p => ({ ...p, attrs: { ...p.att
});
return { root, json: g.json };
}

// ---- online without any account: every player is reachable by their NAME.
// The name maps to a fixed peer id on the free public PeerJS broker (only used t
// after that the game data flows directly phone-to-phone over WebRTC.
const PEERJS_URL = 'https://cdn.jsdelivr.net/npm/peerjs@1.5.4/dist/peerjs.min.js'

```

```

const ICE = [
  { urls: ['stun:stun1.l.google.com:19302', 'stun:stun1.l.google.com:19302', 'stun
    { urls: ['turn:openrelay.metered.ca:80', 'turn:openrelay.metered.ca:443', 'turn
];
const AR_DIGITS = '٠١٢٣٤٥٦٧٨٩', FA_DIGITS = '٠١٢٣٤٥٦٧٨٩';
function normName(n) {
  return String(n || '').replace(/\[s\u200B-\u200F\u00A0\]/g, '').replace(/[٩-٠]/
    .replace(/[١|٢|٣]/g, '1').replace(/[٤|٥|٦|٧|٨|٩]/g, '2').replace(/[١٠-١٩]/g
}
function peerIdFor(name) { // FNV-1a 64-bit of the normalised name -> short safe
  let h1 = 0x811c9dc5 | 0, h2 = 0x01000193 | 0; const t = normName(name);
  for (let i = 0; i < t.length; i++) { const c = t.charCodeAt(i); h1 = Math.imul(
  return 'hjl2-' + (h1 >>> 0).toString(36) + (h2 >>> 0).toString(36);
}
function loadPeerJs() {
  const get = () => window.Peer || (window.peerjs && window.peerjs.Peer);
  if (get()) return Promise.resolve(get());
  return new Promise((res, rej) => { const sc = document.createElement('script');
}

class Net {
  constructor(game) {
    this.game = game; this.peer = null; this.ok = false; this.status = 'off'; //
    this.name = load('hajwala.name', null) || randomName(); save('hajwala.name',
    this.uid = peerIdFor(this.name);
    this.friends = load('hajwala.friends2', []).map(f => ({ ...f, uid: peerIdFor(
    this.sentReq = new Set(load('hajwala.sentReq2', [])); // uids I asked
    this.ignored = new Set();
    this.conns = new Map(); // uid -> { conn, pres
    this.me = { v: 2, uid: this.uid, name: this.name, sess: null, map: null, wx:
    this.bot = null; this.botOn = false;
    this.dentSeq = 0; this.remotes = new Map(); this.onChange = () => { }; this.l
    this.lastPres = 0;
  }
  async init() {
    if (typeof RTCPeerConnection === 'undefined') { this.status = 'failed'; this.
    this.status = 'connecting'; this.onChange();
    try { await loadPeerJs(); } catch (e) { this.status = 'failed'; this.onChange
    this.startPeer();
    setInterval(() => this.housekeeping(), 8000);
  }
  startPeer() {
    try { if (this.peer) this.peer.destroy(); } catch (e) { }
    const P = window.Peer; const peer = this.peer = new P(this.uid, { debug: 0, c
    peer.on('open', () => { if (peer !== this.peer) return; this.ok = true; this.
    peer.on('connection', (c) => this.wire(c));

```

```

peer.on('disconnected', () => { if (peer !== this.peer || peer.destroyed) ret
peer.on('error', (err) => {
  const t = err && err.type;
  if (t === 'peer-unavailable') { const m = String(err.message || '').match(/
  if (t === 'unavailable-id') { this.ok = false; this.status = 'taken'; this.
  if (t === 'network' || t === 'server-error' || t === 'socket-error' || t ==
  });
}
}
wire(conn) {
  const uid = conn.peer;
  const old = this.conns.get(uid); if (old && old.conn !== conn && old.conn.ope
  const E = { conn, pres: old ? old.pres : null }; this.conns.set(uid, E);
  conn.on('open', () => { this.sendTo(E, this.me); const w = this.waiting && th
  conn.on('data', (d) => {
    if (!d || typeof d !== 'object' || d.uid !== uid) return;
    const prev = E.pres; E.pres = d; E.last = performance.now();
    if (!prev || prev.name !== d.name || prev.sess !== d.sess || prev.map !== d
    });
  const gone = () => { if (this.conns.get(uid) === E) { this.conns.delete(uid);
  conn.on('close', gone); conn.on('error', gone);
}
sendTo(E, o) { try { if (E.conn.open) E.conn.send(o); } catch (e) { } }
dial(uid) {
  if (!this.peer || !this.ok || uid === this.uid) return;
  const E = this.conns.get(uid); if (E && (E.conn.open || performance.now() - (
  const c = this.peer.connect(uid, { reliable: true, serialization: 'json' }));
}
dialFriends() { for (const f of this.friends) this.dial(f.uid); }
housekeeping() {
  if (!this.ok) return;
  this.dialFriends();
  // everyone playing in my session: make sure I'm linked to all of them (3+ pl
  for (const o of this.others()) if (o.sess && o.sess === this.me.sess) for (co
  for (const [uid, E] of this.conns) if (!E.conn.open && performance.now() - (E
  if (performance.now() - this.lastPres > 4000) this.push();
}
push() {
  this.lastPres = performance.now();
  this.me.mates = this.me.sess ? this.others().filter(o => o.sess === this.me.s
  for (const E of this.conns.values()) this.sendTo(E, this.me);
}
setName(n) {
  n = String(n || '').trim().slice(0, 24); if (!n || normName(n) === normName(t
  this.name = n; this.me.name = n; save('hajwala.name', n);
  this.uid = peerIdFor(n); this.me.uid = this.uid; for (const E of this.conns.v
  this.conns.clear(); this.ok = false; this.status = 'connecting'; this.onChang

```

```

}
others() {
  const out = [];
  for (const [uid, E] of this.conns) if (E.conn.open && E.pres) out.push({ peer
  return out;
}
online(uid) { if (uid === BOT_ID) return { uid: BOT_ID, name: BOT_NAME, sess: t
allFriends() { return [{ uid: BOT_ID, name: BOT_NAME, bot: true }, ...this.frie
isFriend(uid) { return this.friends.some(f => f.uid === uid); }
// ---- social
// find a player by exact name: resolves the player's presence, or null if nobo
find(name) {
  const uid = peerIdFor(name);
  if (uid === this.uid) return Promise.resolve('me');
  const have = this.online(uid); if (have) return Promise.resolve(have);
  if (!this.ok) return Promise.resolve(null);
  this.waiting = this.waiting || new Map();
  return new Promise(res => {
    const done = (ok) => { clearTimeout(t); if (!ok) return res(null); const t0
    const t = setTimeout(() => { if (this.waiting.get(uid) === done) { this.wai
    this.waiting.set(uid, done); const E = this.conns.get(uid); if (E && !E.con
  });
}
requestFriend(o) { this.sentReq.add(o.uid); save('hajwala.sentReq2', [...this.s
incomingRequests() { return this.others().filter(o => Array.isArray(o.freq) &&
accept(o) { this.addFriend(o); this.me.facc = [...new Set([...(this.me.facc ||
reject(o) { this.ignored.add(o.uid); this.onChange(); }
addFriend(o) { if (!this.isFriend(o.uid)) { this.friends.push({ uid: o.uid, nam
removeFriend(uid) { this.friends = this.friends.filter(f => f.uid !== uid); sav
invite(uid, map, wx, ch) {
  if (uid === BOT_ID) { const sess = this.me.sess || 's' + Math.random().toStri
  const sess = this.me.sess && this.me.map === map ? this.me.sess : 's' + Math.
  this.me.inv = [...(this.me.inv || [])].filter(i => i.to !== uid), { to: uid, s
  this.joinSession(sess, map, wx, ch); return sess;
}
incomingInvites() {
  const out = [];
  for (const o of this.others()) for (const i of o.inv || []) if (i.to === this
  return out;
}
// friends already playing somewhere I could drop into
joinable() { return this.others().filter(o => o.sess && o.sess !== this.me.sess
joinSession(sess, map, wx, ch) { if (this.me.sess !== sess) { this.me.cw = 0; t
leaveSession() { this.me.ch = null; this.me.cw = 0; this.me.cs = 0; this.me.ses
onPeers() {
  // friend accepted my request

```

```

    for (const o of this.others()) if (Array.isArray(o.facc) && o.facc.includes(t
    // new invitation toast
    for (const inv of this.incomingInvites()) { const k = inv.from.uid + inv.sess
    for (const f of this.friends) { const o = this.online(f.uid); if (o && o.name
    this.onChange());
}
// ---- live session
hookCar(car) { if (this.hooked === car) return; if (this.hooked) this.hooked.on
repaired() { this.me.rep = (this.me.rep || 0) + 1; this.me.dents = []; this.pus
update(dt) {
    this.updateBot(dt);
    if (!this.ok || !this.me.sess) { if (this.remotes.size) this.clearRemotes();
    const g = this.game, car = g.car, b = car.body;
    this.hookCar(car);
    const now = performance.now();
    if (now - this.lastSend > 50) {
        this.lastSend = now;
        const p = g.player; const foot = g.playerMode !== 'car';
        this.me.car = g.carKey; this.me.map = g.mapKey; this.me.tune = g.myTune ? g
        this.me.st = [r3(b.pos.x), r3(b.pos.y), r3(b.pos.z), r3(b.q.x), r3(b.q.y),
        this.push();
    }
    // remote players in my session
    const live = new Set();
    for (const o of this.others()) {
        if (o.sess !== this.me.sess || !o.st || !CAR_DEFS[o.car] || o.map !== g.map
        live.add(o.uid);
        let R = this.remotes.get(o.uid);
        if (!R || R.carKey !== o.car) { const isNew = !R; if (R) this.dropRemote(R)
        R.name = o.name;
        { const tj = JSON.stringify(o.tune || {}); if (tj !== R.tuneJ) { R.tuneJ =
        const st = o.st; if (!R.last || st[19] !== R.last[19]) { R.buf.push({ t: no
        // crashes and repairs
        if ((o.rep || 0) !== R.rep) { R.rep = o.rep || 0; R.car.repair(g.r); R.dent
        for (const d of o.dents || []) if (d.s > R.dentSeq) { R.dentSeq = d.s; cons
        for (const ev of R.car.detachEvents || []) if (ev.w.node) ev.w.node.visible
    }
    for (const [uid, R] of this.remotes) if (!live.has(uid)) { this.dropRemote(R)
    for (const R of this.remotes.values()) this.animateRemote(R, dt, now);
}
makeRemote(o) {
    const g = this.game, key = o.car;
    const pr = g.pristine[key]; const car = new Car(CAR_DEFS[key], cloneGltf({ ro
    car.key = key + '_net'; car.addPlates(g.assets.plate); car.snapshotNormals();
    g.scene.add(car.node);
    const man = new Character(cloneCharacter(g.manSrc)); g.scene.add(man.root);

```

```

    const label = document.createElement('div'); label.className = 'netlabel'; do
    const R = { uid: o.uid, carKey: key, car, man, label, buf: [], rep: o.rep ||
    // join late: skip the old dents already in the list, apply only new ones...
    return R;
}
dropRemote(R) { if (R.car.node.parent) R.car.node.parent.remove(R.car.node); if
clearRemotes() { for (const R of this.remotes.values()) this.dropRemote(R); thi
animateRemote(R, dt, now) {
    const g = this.game, car = R.car, b = car.body, buf = R.buf; if (!buf.length)
    // render ~110 ms in the past, interpolating between the two surrounding snap
    const rt = now - 110; let a = buf[0], c = buf[buf.length - 1];
    for (let i = 0; i < buf.length - 1; i++) if (buf[i].t <= rt && buf[i + 1].t >
    const u = c.t > a.t ? clamp((rt - a.t) / (c.t - a.t), 0, 1.3) : 1, A = a.st,
    b.pos.set(lerp(A[0], C[0], u), lerp(A[1], C[1], u), lerp(A[2], C[2], u));
    const qa = new Q(A[3], A[4], A[5], A[6]), qc = new Q(C[3], C[4], C[5], C[6]);
    b.vel.set(C[7], C[8], C[9]); car.steer = C[10]; car.doorAngle = C[17]; car.br
    const vF = b.vel.dot(car.forward(new V3()));
    for (const w of car.wheels) { w.steer = w.front ? -car.steer : 0; w.spin += v
    car.syncNode();
    // the driver: in the car, or walking around
    const foot = C[11] === 1, man = R.man;
    if (foot) {
        if (man.root.parent !== g.scene) { if (man.root.parent) man.root.parent.rem
        const pos = new V3(lerp(A[12], C[12], u), lerp(A[13], C[13], u), lerp(A[14]
        man.t += dt; R.phase = (R.phase || 0) + dt * 7.2 * clamp(C[16] / 1.6, 0, 1.
        man.applyPose(C[16] > 0.3 ? man.poseWalk(R.phase, clamp((C[16] - 2) / 3, 0,
        man.setTransform(pos, C[15])); man.root.visible = true;
    } else if (man.root.parent !== car.node) { // back in the car: sits in the dr
        if (man.root.parent) man.root.parent.remove(man.root);
        car.node.add(man.root); const k = car.def.driverScale || 1; man.root.scl.se
        man.applyPose(man.poseDrive(0)); man.root.visible = true;
    }
    // name tag above the car
    const head = foot ? new V3(C[12], C[13] + 2.1, C[14]) : b.pos.clone().add(new
    const m = g.cam.vp, cv = g.r.gl.canvas;
    const cx = m[0] * head.x + m[4] * head.y + m[8] * head.z + m[12], cy = m[1] *
    const s = cw > 0.1 ? { x: (cx / cw * 0.5 + 0.5) * cv.clientWidth, y: (0.5 - c
    if (s && Math.abs(cx / cw) < 1.2 && Math.abs(cy / cw) < 1.2) { R.label.style.
    else R.label.style.display = 'none';
}
pushLights(out) { for (const R of this.remotes.values()) R.car.pushLights(out);
setNight(on) { for (const R of this.remotes.values()) R.car.setNight(on); if (t
// ---- practice friend: an AI driver that follows you around (and crashes into
makeBot() {
    const g = this.game, keys = ['hilux', 'camry', 'changan', 'shas', 'avanza'],
    const pr = g.pristine[key];

```

```

const car = new Car(CAR_DEFS[key], cloneGltf({ root: { clone: () => { const r
car.key = key + '_bot'; car.addPlates(g.assets.plate); car.snapshotNormals();
const me = g.car, spot = me.localToWorld(new V3(3.4, 0, -4), new V3());
car.place(spot.x, g.world.heightAt(spot.x, spot.z, spot.y + 3) + 0.1, spot.z,
g.scene.add(car.node);
// a driver behind the wheel
const man = new Character(cloneCharacter(g.manSrc)); car.node.add(man.root);
const k = car.def.driverScale || 1; man.root.scl.set(k, k, k); man.root.pos.c
const label = document.createElement('div'); label.className = 'netlabel'; do
this.bot = { isBot: true, car, man, label, name: BOT_NAME, stuck: 0, rev: 0,
g.hud.toast(BOT_NAME + ' 2000 , 'دخّل الماب');
}
dropBot() { if (!this.bot) return; const B = this.bot; if (B.car.node.parent) B
updateBot(dt) {
const g = this.game;
if (!this.botOn || g.mode !== 'play') { if (this.bot) this.dropBot(); return;
if (!this.bot) { if (performance.now() > (this.botJoinAt || 0)) this.makeBot(
if (this.bot.car.world !== g.world) { this.dropBot(); return; }
const B = this.bot, car = B.car;
// name tag
const head = car.body.pos.clone().add(new V3(0, 2.0, 0)); const m = g.cam.vp,
const cx = m[0] * head.x + m[4] * head.y + m[8] * head.z + m[12], cy = m[1] *
if (cw > 0.1 && Math.abs(cx / cw) < 1.2 && Math.abs(cy / cw) < 1.2) { B.label
for (const ev of car.detachEvents || []) if (ev.w.node) ev.w.node.visible = f
}
// called from the fixed physics step: the AI drives and the car is simulated 1
stepBot(dt) {
const B = this.bot; if (!B) return;
const g = this.game, car = B.car, me = g.car;
const tgt = me.localToWorld(new V3(3.4, 0, -5 + Math.min(8, me.speed * 0.25))
const loc = car.worldToLocal(tgt, new V3()).sub(car.com), dist = Math.hypot(1
if (dist > 70 || car.body.pos.y < -20) { const y = g.world.heightAt(tgt.x, tg
const ang = Math.atan2(loc.x, Math.max(loc.z, 0.5));
let steer = clamp(-ang * 1.8, -1, 1);
const want = me.speed + clamp(loc.z * 0.45, -6, 9), v = car.forwardSpeed;
let throttle = clamp((want - v) * 0.35, 0, 1), brake = v > want + 1.5 ? clamp
if (dist < 3 && me.speed < 1) { throttle = 0; brake = 1; }
// unstick: reverse out when pushing against something
if (throttle > 0.4 && Math.abs(v) < 0.4) B.stuck += dt; else B.stuck = 0;
if (B.stuck > 1.5) { B.rev = 1.2; B.stuck = 0; }
if (B.rev > 0) { B.rev -= dt; throttle = 0; brake = 1; steer = -steer; }
// back on its wheels if it rolled over
if (car.up(new V3()).y < 0.3) { B.flip += dt; if (B.flip > 2.5) { const p = c
car.step(dt, { throttle, brake, steer, handbrake: false, surfaceGrip: 1, wate
// rubber band: when it falls behind it gets a push so it keeps up with you
if (loc.z > 10 && car.grounded >= 2 && Math.abs(ang) < 0.5) car.body.vel.addS

```

```

    // its own crashes dent it
    if (car.impact.length) {
        let best = null; for (const im of car.impact) if (!best || im.speed > best
            if (best.speed > 3 && g.time - (B.lastDent || 0) > 0.1) { B.lastDent = g.ti
        }
    }
    // my car bumps into friends' cars (each side resolves its own car; together it
    collide(dt) {
        const list = [...this.remotes.values()]; if (this.bot) list.push(this.bot);
        if (!list.length) return;
        const my = this.game.car, b = my.body;
        for (const R of list) {
            const o = R.car, ob = o.body;
            if (b.pos.dist(ob.pos) > 9) continue;
            for (const s of my.spheres) {
                const c1 = new V3().copy(s).applyQ(b.q).add(b.pos);
                for (const t of o.spheres) {
                    const c2 = new V3().copy(t).applyQ(ob.q).add(ob.pos);
                    const d = c1.dist(c2), rr = s.r + t.r; if (d >= rr || d < 1e-4) continu
                    const n = c1.clone().sub(c2).scale(1 / d), depth = rr - d;
                    b.pos.addS(n, Math.min(depth * 0.5, 0.05));
                    const cp = c2.clone().addS(n, t.r);
                    const rel = b.velAt(cp, new V3()).sub(ob.vel); const vn = rel.dot(n);
                    if (vn < 0) {
                        const k = b.invMassAt(cp, n) + (R.isBot ? ob.invMassAt(cp, n) : 1 / o
                        const j = -(1.15) * vn / k; b.applyImpulse(n.clone().scale(j), cp);
                        if (R.isBot) { ob.applyImpulse(n.clone().scale(-j), cp); ob.pos.addS(
                        if (-vn > 2.5) my.impact.push({ point: cp, normal: n.clone(), speed:
                    }
                }
            }
        }
    }
}

return { BOT_ID, randomName, normName, peerIdFor, Net };
})();

__M['main'] = (() => {
const { Renderer } = __M['renderer'];
const { parseGLB } = __M['gltf'];
const { V3, Q, clamp } = __M['math'];
const { CAR_DEFS } = __M['car'];
const { FX } = __M['fx'];
const { Audio } = __M['audio'];
const { Game } = __M['game'];
const { Net } = __M['net'];

```

```

const TUNE = __M['tune'];
window.__tuneMod = TUNE;
const { Input, bindSteering, bindHold, bindTap, bindJoystick, bindCameraDrag, bin

const $ = (id) => document.getElementById(id);

// ----- icons (SVG) -----
const ICON = {
  car: '<svg viewBox="0 0 32 32" fill="none" stroke="currentColor" stroke-width="
  sound: '<svg viewBox="0 0 32 32" fill="none" stroke="#ecdcbc" stroke-width="2.2
  mute: '<svg viewBox="0 0 32 32" fill="none" stroke="#ecdcbc" stroke-width="2.2"
  gear: '<svg viewBox="0 0 32 32" fill="none" stroke="#ecdcbc" stroke-width="2.2"
  back: '<svg viewBox="0 0 32 32" fill="none" stroke="#ecdcbc" stroke-width="2.6"
  menu: '<svg viewBox="0 0 32 32" fill="none" stroke="#ecdcbc" stroke-width="2.6"
  camera: '<svg viewBox="0 0 32 32" fill="none" stroke="#ecdcbc" stroke-width="2.
  wrench: '<svg viewBox="0 0 32 32" fill="none" stroke="#ecdcbc" stroke-width="2.
  reset: '<svg viewBox="0 0 32 32" fill="none" stroke="#ecdcbc" stroke-width="2.2
  horn: '<svg viewBox="0 0 32 32" fill="none" stroke="#ecdcbc" stroke-width="2.2"
  door: '<svg viewBox="0 0 32 32" fill="none" stroke="#ecdcbc" stroke-width="2.2"
  enter: '<svg viewBox="0 0 32 32" fill="none" stroke="#ecdcbc" stroke-width="2.2
  kick: '<svg viewBox="0 0 32 32" fill="none" stroke="#ecdcbc" stroke-width="2.2"
  gfx: '<svg viewBox="0 0 32 32" fill="none" stroke="#ecdcbc" stroke-width="2.2"
  steer: '<svg viewBox="0 0 32 32" fill="none" stroke="#ecdcbc" stroke-width="2.2
  move: '<svg viewBox="0 0 32 32" fill="none" stroke="#ecdcbc" stroke-width="2.2"
  close: '<svg viewBox="0 0 32 32" fill="none" stroke="#ecdcbc" stroke-width="2.6
  handbrake: '<svg viewBox="0 0 48 48" fill="none" stroke="#ecdcbc" stroke-width=
};

const CAM_ART = {
  near: '<svg viewBox="0 0 64 40" fill="none" stroke="#ecdcbc" stroke-width="2"><
  far: '<svg viewBox="0 0 64 40" fill="none" stroke="#ecdcbc" stroke-width="2"><r
  hood: '<svg viewBox="0 0 64 40" fill="none" stroke="#ecdcbc" stroke-width="2"><
  cockpit: '<svg viewBox="0 0 64 40" fill="none" stroke="#ecdcbc" stroke-width="2
  custom: '<svg viewBox="0 0 64 40" fill="none" stroke="#ecdcbc" stroke-width="2"
};

const MAP_ART = {
  city: '<svg class="art" viewBox="0 0 180 100"><rect width="180" height="100" fi
  highway: '<svg class="art" viewBox="0 0 180 100"><rect width="180" height="100"
  riyadh: '<svg class="art" viewBox="0 0 180 100"><defs><linearGradient id="rsk"
  test: '<svg class="art" viewBox="0 0 180 100"><rect width="180" height="100" fi
  desert: '<svg class="art" viewBox="0 0 180 100"><defs><linearGradient id="dsk"
};

function pedalsSVG(w, h, grooves, kind) {
  const r = 12; let g = '';
  for (let i = 0; i < grooves; i++) { const y = 16 + i * (h - 34) / (grooves - 1)

```

```

const top = kind === 'gas' ? '#9ea3a8' : '#a9a49b';
return `

```

```

    <path d="M24 104 L68 98 L72 120 L28 124 Z M176 104 L132 98 L128 120 L172 124
    <rect x="70" y="86" width="60" height="48" rx="12" fill="url(#rim)" stroke="#
// modern sedan (Changan): three slim spokes with silver accents
return defs + `<circle cx="100" cy="100" r="84" fill="none" stroke="url(#lea)"
    <path d="M24 102 L70 100 L72 114 L26 116 Z M176 102 L130 100 L128 114 L174 11
    <path d="M30 108 L68 107 M170 108 L132 107 M100 132 L100 166" stroke="url(#si
}
// round analog speedometer + tachometer arc (real speed, 0-220 km/h)
function speedoSVG() {
    let t = '';
    for (let v = 0; v <= 220; v += 10) {
        const a = (-225 + v / 220 * 270) * Math.PI / 180, big = v % 20 === 0;
        const r1 = big ? 70 : 74, r2 = 82;
        t += `<line x1="${100 + Math.cos(a) * r1}" y1="${100 + Math.sin(a) * r1}" x2=
        if (v % 40 === 0) t += `<text x="${100 + Math.cos(a) * 58}" y="${100 + Math.s
    }
    return `<defs><radialGradient id="dial" cx=".5" cy=".45" r=".6"><stop offset="0
    <circle cx="100" cy="100" r="96" fill="url(#dial)" stroke="#57524a" stroke-widt
    <path id="rpmArc" d="" fill="none" stroke="#f5a524" stroke-width="5" stroke-lin
    ${t}
    <text x="100" y="136" fill="#ecdcbc" font-size="30" font-weight="700" text-anch
    <text x="100" y="152" fill="#b9a888" font-size="11" text-anchor="middle" font-f
    <text x="100" y="80" fill="#f5a524" font-size="20" font-weight="700" text-anch
    <rect id="dmgBar" x="70" y="164" width="0" height="4" rx="2" fill="#d4412f"/>
    <g id="needle"><path d="M97 100 L100 26 L103 100 Z" fill="#d4412f"/><circle cx=
    }

// ----- HUD -----
class HUD {
    constructor() { this.tt = null; this.driftHide = null; }
    toast(msg, ms = 1800) { const t = $('toast'); t.textContent = msg; t.classList.
    setNearDoor(on) { $('enterBtn').classList.toggle('hot', !!on); }
    drift(pts, combo, a, cancel) {
        const b = $('driftBox');
        if (cancel) { b.classList.remove('on'); return; }
        clearTimeout(this.driftHide);
        b.classList.remove('fail'); b.classList.add('on');
        $('driftLbl').textContent = a > 1.1 ? 'تفحيط خطير!' : a > 0.7 ? 'زحلقه' : '!
        $('driftPts').textContent = pts.toLocaleString('ar-SA'); $('driftX').textCont
    }
    driftBank(pts) {
        $('driftLbl').textContent = 'انحسبت!'; $('driftPts').textContent = '+' + pts.
        clearTimeout(this.driftHide); this.driftHide = setTimeout(() => $('driftBox')
    }
    driftFail() {
        const b = $('driftBox'); b.classList.add('fail', 'on'); $('driftLbl').textCon

```

```

        clearTimeout(this.driftHide); this.driftHide = setTimeout(() => b.classList.r
    }
}

// ----- asset loading -----
async function fetchParts(parts, onBytes) {
    const bufs = [];
    for (const url of parts) {
        const res = await fetch(url);
        if (!res.ok) throw new Error('تعذر تحميل ' + url + ' (' + res.status + ')');
        if (res.body && res.body.getReader) {
            const rd = res.body.getReader(); const chunks = []; let n = 0;
            for (;;) { const { done, value } = await rd.read(); if (done) break; chunks
            const out = new Uint8Array(n); let o = 0; for (const c of chunks) { out.set
            } else { const b = new Uint8Array(await res.arrayBuffer()); onBytes(b.length)
        }
    }
    if (bufs.length === 1) return bufs[0].buffer.slice(bufs[0].byteOffset, bufs[0].
    const total = bufs.reduce((a, b) => a + b.length, 0); const out = new Uint8Arra
    return out.buffer;
}

async function boot() {
    const MANIFEST = window.__MANIFEST__;
    const hud = new HUD();
    const canvas = $('view');
    let renderer;
    try { renderer = new Renderer(canvas); }
    catch (e) { $('loadErr').hidden = false; $('loadErr').textContent = 'ذك ما يدعم';
    const isMobile = /iPhone|iPad|Android|Mobile/i.test(navigator.userAgent) || (na
    const QUALITY = {
        high: { pr: isMobile ? 1.6 : 2, shadow: 2048, range: 55 },
        mid: { pr: isMobile ? 1.25 : 1.5, shadow: 2048, range: 45 },
        low: { pr: 1, shadow: 1024, range: 35 },
    };
    let quality = 'high', resScale = 1;
    try { resScale = +(localStorage.getItem('hajwala.res') || 1) || 1; } catch (e)
    try { quality = localStorage.getItem('hajwala.q') || (isMobile ? 'mid' : 'high'
    const applyQuality = (q) => {
        quality = q; const Q = QUALITY[q]; renderer.pixelRatio = Math.min(window.devi
        if (renderer.shadowSize !== Q.shadow) { renderer.shadowSize = Q.shadow; rende
        try { localStorage.setItem('hajwala.q', q); localStorage.setItem('hajwala.res
    };
    applyQuality(quality);

    // loading slideshow with pictures from the game
    let shotTimer = null;

```

```

if (MANIFEST.shots && MANIFEST.shots.length) {
  const box = $('shots'); const imgs = MANIFEST.shots.map((u, i) => { const im
  let k = 0; shotTimer = setInterval(() => { imgs[k].classList.remove('on'); k
  }
  // load everything
  const total = MANIFEST.total; let got = 0;
  const bar = $('loadBar'), txt = $('loadText');
  const onBytes = (n) => { got += n; const p = Math.min(1, got / total); bar.styl
  const assets = {};
  try {
    const entries = Object.entries(MANIFEST.files).filter(([k]) => !(MANIFEST.laz
    const bufs = await Promise.all(entries.map([k, parts]) => fetchParts(parts,
    txt.textContent = '...نجهز الجرافيكس';
    for (let i = 0; i < entries.length; i++) assets[entries[i][0]] = await parseG
  } catch (e) {
    console.error(e); $('loadErr').hidden = false; $('loadErr').textContent = 'يل
  }

  const audio = new Audio();
  const fx = new FX(renderer);
  const input = new Input();
  const game = new Game(renderer, audio, fx, assets, input, hud);
  window.__game = game;
  try { const w = localStorage.getItem('hajwala.night'); if (w && w !== 'day') ga

  // ----- UI wiring -----
  $('carsBtn').innerHTML = ICON.car + '<span>السيارات</span>';
  $('soundBtn').innerHTML = ICON.sound; $('settingsBtn').innerHTML = ICON.gear; $
  $('pauseBtn').innerHTML = ICON.menu; $('resetBtn').innerHTML = ICON.reset; $('h
  $('doorBtn').innerHTML = ICON.door; $('enterBtn').innerHTML = ICON.enter; $('ki
  $('gas').innerHTML = pedalsSVG(78, 150, 7, 'gas'); $('brake').innerHTML = pedals
  $('wheel').innerHTML = wheelsSVG(CAR_DEFS.shas.wheelStyle);
  $('speedo').innerHTML = speedoSVG();
  const arrowIcon = (d) => `<svg viewBox="0 0 32 32"><path d="$${d} > 0 ? 'M11 6l12
  $('stL').innerHTML = arrowIcon(-1); $('stR').innerHTML = arrowIcon(1);

  let muted = false;
  try { muted = localStorage.getItem('hajwala.mute') === '1'; } catch (e) { }
  const setMute = (m) => { muted = m; audio.setMuted(m); $('soundBtn').innerHTML
  setMute(muted);
  const unlockAudio = () => { audio.init(); audio.setMuted(muted); };
  window.addEventListener('pointerdown', unlockAudio, { capture: true });
  window.addEventListener('keydown', unlockAudio, { capture: true });

  const steering = bindSteering($('wheel'), input);
  try { input.pedalFull = localStorage.getItem('hajwala.pedal') === 'full'; } cat

```

```

const gasP = bindPedal($('gas'), input, 'touchGas'), brakeP = bindPedal($('brake'), input, 'touchBrake');
const arrows = bindArrows($('stL'), $('stR'), input);
// never leave a pedal or the wheel stuck when the app loses focus or a touch g
const releaseAll = () => { gasP.release(); brakeP.release(); arrows.release();
document.addEventListener('visibilitychange', releaseAll); window.addEventListener('blur', releaseAll);
window.addEventListener('touchend', (e) => { if (!e.touches.length) releaseAll(); });
bindHold($('handbrake'), () => { input.touchHB = true; }, () => { input.touchHB = false; });
bindHold($('kickBtn'), () => input.pressed.add('kick'));
bindTap($('doorBtn'), () => input.pressed.add('door'));
bindTap($('enterBtn'), () => input.pressed.add('door'));
bindTap($('resetBtn'), () => input.pressed.add('reset'));
bindTap($('repairBtn'), () => input.pressed.add('repair'));
bindTap($('camBtn2'), () => input.pressed.add('camera'));
bindTap($('droneBtn'), () => { const on = game.toggleDrone(); $('droneBtn').className = on ? 'active' : ''; });
bindTap($('shotBtn'), () => { game.snap = true; });
bindTap($('moveCarBtn'), () => { // drone: drop the car under the drone
  const D = game.drone; if (!D) return; const car = game.car; const f = new V3(
    D.pos.x + f.x * 6, D.pos.z + f.z * 6, 0); const g = new V3(
    car.place.x, car.place.y + 0.4, car.place.z + D.yaw); for (const w of car.wheels) { w.spinVel = 0;
    hud.toast('1200', '👍 نقلنا السيارة');
  });
};
// tyre pressure (on foot next to a wheel)
let hiss = null;
const hissOn = (on) => {
  const ctx = audio.ctx; if (!ctx) return;
  if (on && !hiss) { const len = ctx.sampleRate; const buf = ctx.createBuffer(1, len, ctx.sampleRate);
    const src = ctx.createBufferSource(); src.buffer = buf; src.loop = true; connect(src, f, g, ctx.destination); src.start(); hiss = src;
  } else if (!on && hiss) { try { hiss.src.stop(); } catch (e) {} hiss = null; }
};
const tyreUI = () => {
  const near = screen === 'play' && game.playerMode === 'foot' && game.nearWheel;
  $('tyreBtn').hidden = !near || !$('tyrePanel').hidden;
  if (!$('tyrePanel').hidden) { if (!near) { $('tyrePanel').hidden = true; game.moveCarBtn.hidden = !game.drone; }
};
setInterval(tyreUI, 120);
bindTap($('tyreBtn'), () => { $('tyrePanel').hidden = false; tyreUI(); });
bindTap($('tyreClose'), () => { $('tyrePanel').hidden = true; game.deflating = true;
const b = $('deflateBtn'); const on = (e) => { e.preventDefault(); game.deflating = true;
  b.addEventListener('pointerdown', on); for (const t of ['pointerup', 'pointercancel']) { b.addEventListener(t, () => { game.deflating = false; });
bindHold($('hornBtn'), () => { audio.horn(true); }, () => { audio.horn(false); });
bindJoystick($('joy'), $('joyKnob'), input);
bindCameraDrag(canvas, input);

const show = (id, on) => { $(id).hidden = !on; };

```

```

let selected = 'shas';
try { const s = localStorage.getItem('hajwala.car'); if (s && CAR_DEFS[s]) sele

// ----- persisted settings -----
const load = (k, d) => { try { const v = localStorage.getItem(k); return v ? JS
const save = (k, v) => { try { localStorage.setItem(k, JSON.stringify(v)); } ca
const nf = (x, d = 1) => Number(x).toLocaleString('ar-SA', { minimumFractionDig

const steerCfg = Object.assign({ sens: 5, ret: 7, curve: 1, size: 1, mode: 'whe
const applySteer = () => {
  STEER_OPTS.max = 3 * Math.PI * clamp(1.4 - (steerCfg.sens - 1) * 0.1, 0.4, 1.
  $('wheelWrap').style.width = $('wheelWrap').style.height = `calc(min(42vh, 21
  input.useArrows = steerCfg.mode === 'arrows';
  $('wheelWrap').hidden = input.useArrows; $('arrows').hidden = !input.useArrow
  save('hajwala.steer2', steerCfg);
};
applySteer();
const camCfg = Object.assign({ mode: 0, custom: null }, load('hajwala.cam', {}))
let savedCams = load('hajwala.cams', []); if (!Array.isArray(savedCams)) savedC
game.camMode = camCfg.mode; game.camCustom = camCfg.custom;
if (game.camMode === 4 && !game.camCustom) game.camMode = 0;
const saveCam = () => { camCfg.mode = game.camMode; camCfg.custom = game.camCus

// ----- garage: cars + maps as a compact bottom strip -----
const specs = Object.fromEntries(Object.entries(CAR_DEFS).map(([k, d]) => [k, d
const WEATHERS = [['day', 'نهار'], ['night', 'ليل'], ['cloudy', 'غيوم'], ['suns
const MAPS = { riyadh: { name: 'الرياض', sp: 'برج المملكة والفيصلية والراجحي'
let mapSel = load('hajwala.map', 'city'); if (!MAPS[mapSel]) mapSel = 'city';
let mapBusy = false;
const LAZY_FOR = { riyadh: ['riyadh', 'mall'], hero: ['hero'], caravan: ['carav
const ensureMapAssets = async (key, onP) => {
  for (const k of LAZY_FOR[key] || []) {
    if (assets[k]) continue;
    const total = MANIFEST.lazy[k]; let got = 0;
    const buf = await fetchParts(MANIFEST.files[k], (n) => { got += n; onP(Math
    assets[k] = await parseGLB(buf);
  }
};
const strip = $('carStrip'), mstrip = $('mapStrip');
for (const key of Object.keys(CAR_DEFS)) {
  const card = document.createElement('button'); card.className = 'card'; card.
  const img = document.createElement('img'); img.alt = CAR_DEFS[key].name;
  try { img.src = game.thumbnail(key); } catch (e) { console.warn(e); }
  card.append(img);
  const nm = document.createElement('div'); nm.className = 'nm'; nm.textContent
  card.addEventListener('click', () => { selected = key; try { localStorage.set

```

```

    strip.append(card);
}
// thumbnails again once textures/shaders are ready (on some phones the first p
const reThumb = () => { for (const img of strip.querySelectorAll('img')) { cons
setTimeout(reThumb, 2500); setTimeout(reThumb, 8000);
for (const key of Object.keys(MAPS)) {
    const card = document.createElement('button'); card.className = 'card'; card.
    card.innerHTML = MAP_ART[key] + `<div class="nm">${MAPS[key].name}</div>`;
    card.addEventListener('click', async () => {
        if (mapSel === key || mapBusy) return;
        mapBusy = true;
        try {
            await ensureMapAssets(key, (p) => { $('gSpec').textContent = `نحْمَلْ ${MAPS
            $('gSpec').textContent = '...نجهز الخريطة';
            await new Promise(r => setTimeout(r, 40));
            game.loadMap(key); mapSel = key; save('hajwala.map', key);
            game.showroom(selected); game.mode = 'garage';
        } catch (e) { console.error(e); hud.toast('جَرَبْ مرة ثانية، الخريطة، جَرَبْ مرة ثانية');
        mapBusy = false; refreshCards();
    });
    mstrip.append(card);
}
let gTab = 'cars';
const refreshCards = () => {
    strip.querySelectorAll('.card').forEach(c => c.classList.toggle('sel', c.data
    mstrip.querySelectorAll('.card').forEach(c => c.classList.toggle('sel', c.dat
    $('gTitle').textContent = gTab === 'cars' ? 'اختار الماب' : 'اختار سيارتك'; $
    strip.hidden = gTab !== 'cars'; mstrip.hidden = gTab !== 'maps';
    $('gName').textContent = gTab === 'cars' ? CAR_DEFS[selected].name : MAPS[map
    $('gSpec').textContent = gTab === 'cars' ? specs[selected] || '' : MAPS[mapSe
};
if (mapSel !== 'city') { if (LAZY_FOR[mapSel]) { const want = mapSel; mapSel =

let screen = 'menu';
const dropDrone = () => { game.drone = null; $('shotBtn').hidden = true; $('dro
const goMenu = () => { game.chal = null; game.tuneView = false; show('tune', fa
const goGarage = (tab) => { dropDrone(); if (game.net) game.net.leaveSession();
const goPlay = () => {
    game.tuneView = false; show('tune', false); show('chars', false); screen = 'p
    $('wheel').innerHTML = wheelSVG(CAR_DEFS[selected].wheelStyle);
    game.showroom(selected); game.start();
    hud.toast(mapSel === 'highway' ? 'دوس بنزين وتوكل على الله', '2200', 'ر فوق ١٢٠!');
    try { navigator.wakeLock && navigator.wakeLock.request('screen').catch(() =>
};
hud.flash = () => { const f = $('flash'); f.classList.add('on'); requestAnimati
$('carsBtn').addEventListener('click', () => { garageFrom = 'menu'; goGarage('c

```

```

$('backBtn').addEventListener('click', () => { if (gTab === 'maps' && garageFrom !== 'menu') {
    let garageFrom = 'menu';
    $('startBtn').addEventListener('click', () => { garageFrom = 'menu'; goGarage('maps');
    $('startBtn2').addEventListener('click', () => { if (gTab === 'cars') { garageFrom = 'cars'; goGarage('cars');
    $('soundBtn').addEventListener('click', () => setMute(!muted));
    const pauseGame = () => { if (screen === 'play') { game.paused = true; show('pause');
    $('pauseBtn').addEventListener('click', pauseGame);
    $('resumeBtn').addEventListener('click', () => { show('pause', false); game.paused = false;
    $('toGarageBtn').addEventListener('click', () => { garageFrom = 'menu'; goGarage('menu');

// ----- settings screen: categories on the side, options in the centre -----
let setCat = 'gfx', setFrom = null;
const CATS = { gfx: ['الجغرافيا', ICON.gfx], drive: ['التوجيه', ICON.move], steering: ['التحكم', ICON.steering],
document.querySelectorAll('#setNav button').forEach(b => { b.innerHTML = CATS[b.getAttribute('cat')];
$('setClose').innerHTML = ICON.back;
const openSettings = (cat) => { setFrom = screen === 'play' ? 'pause' : 'menu';
const closeSettings = () => { show('settings', false); if (setFrom === 'pause') {
    $('settingsBtn').addEventListener('click', () => openSettings());
    $('toSettingsBtn').addEventListener('click', () => openSettings());

// ----- online: search by name / invitations / friends / requests -----
const net = new Net(game); game.net = net;
const CHALS = [['none', 'بس نلعب'], ['drift', 'تحدي التفحيط'], ['race', 'سباق'], ['rac
let pickWx = null;
let onTab = 'search', onFrom = null, pickFor = null, pickMap = null, searchRes = null;
const el = (tag, cls, txt) => { const e = document.createElement(tag); if (cls) e.className = cls; e.textContent = txt;
const btn = (txt, fn, ghost) => { const b = el('button', 'sbtn' + (ghost ? 'ghost' : ''), txt); b.addEventListener('click', fn);
// host: someone already in this session whose car we should spawn beside (not necessarily the same as the one we are currently following)
const hostOf = (sess) => net.others().find(o => o.sess === sess && Array.isArray(o.car));
const playSession = async (map, wx, guest, ch) => {
    try {
        if (!assets[map] && LAZY_FOR[map]) { hud.toast('1500', 'نحمل الخريطة...'); await loadAssets();
        if (game.mapKey !== map) { game.loadMap(map); mapSel = map; save('hajwala.map');
    } catch (e) { hud.toast('ما قدرنا نحمل الخريطة'); return; }
        if (wx) game.setWeather(wx);
        show('online', false); show('menu', false); show('garage', false); show('pause', false);
        const keep = net.me.sess, keepMap = net.me.map; goPlay(); net.joinSession(keep);
        const h = hostOf(keep);
        if (h && game.car) { // park next to the friend, 4.5 m to their right, facing them
            const st = h.st, q = new Q(st[3], st[4], st[5], st[6]), right = new V3(-1, 0, 0);
            const x = st[0] + right.x * 4.6, z = st[2] + right.z * 4.6, y = game.world.y;
            game.car.place(x, y + 0.15, z, Math.atan2(right.x, right.z));
        }
        else if (guest && game.car) { // host not visible yet: still never on the same map
            const b = game.car.body, r = game.car.localToWorld(new V3(-5, 0, 0), new V3(0, 0, 1));
            game.car.place(r.x, y + 0.15, r.z, Math.atan2(game.car.forward(new V3(1, 0, 0)).x, r.z));
        }
    }
}

```

```

    }
};
const refreshBadges = () => {
    const ni = net.ok ? net.incomingInvites().length : 0, nr = net.ok ? net.incom
    const set = (b, n) => { if (!b) return; b.hidden = !n; b.textContent = n.toLo
    set($('onBadge'), ni + nr);
    const bs = document.querySelectorAll('#onNav .badge'); set(bs[0], ni); set(bs
    $('myName').textContent = '👤 ' + net.name; $('onName').textContent = net.nam
};
const statusLine = () => {
    const st = net.status;
    if (st === 'on') return null;
    if (st === 'taken') return 'مستخدم من لاعب ثاني. اضغط «غير» فوق واختر اسم ثاني';
    if (st === 'failed') return 'اللعبة من الرابط في المتصفح (سفاري أو فوكل كروم)';
    return 'نتصل بالأنلاين... تأكد إن أنت شغال';
};
const mapPicker = (onGo) => {
    const wrap = el('div', 'step');
    wrap.append(el('h4', '', 'اختار الماب'));
    const box = el('div', 'maps');
    for (const k of Object.keys(MAPS)) { const b = btn(MAPS[k].name, () => { pick
    wrap.append(box);
    if (pickMap) {
        wrap.append(el('h4', '', 'اختار الجو'));
        const wb = el('div', 'maps');
        for (const [k, n] of WEATHERS) wb.append(btn(n, () => { pickWx = k; renderO
        wrap.append(wb);
        if (pickWx) {
            wrap.append(el('h4', '', 'تبي تحدي؟'));
            const cb = el('div', 'maps');
            for (const [k, n] of CHALS) cb.append(btn(n, () => { const m = pickMap, w
            wrap.append(cb);
        }
    }
    return wrap;
};
const renderOnline = () => {
    refreshBadges();
    if ($('online').hidden) return;
    document.querySelectorAll('#onNav button').forEach(b => b.classList.toggle('o
    const main = $('onMain'); main.innerHTML = '';
    const sl = statusLine(); if (sl) { const d = el('div', 'empty', sl); main.app
    const orow = (o, extra, sub) => { const r = el('div', 'row'); const d = el('s
    if (onTab === 'search') {
        main.append(el('div', 'empty', 'لعبته (جنب 👤)، وخويك لازم يكون فاتح اللعبة'
        const inp = document.createElement('input'); inp.placeholder = 'اسم خويك';

```

```

const go = btn(searching ? 'ابحث' : 'ندور...', async () => {
  const q = inp.value.trim(); if (!q || searching) return; renderOnline.q =
  const r = await net.find(q); searching = false; searchRes = r === 'me' ?
});
inp.addEventListener('keydown', (e) => { if (e.key === 'Enter') { e.preventDefault
const row = el('div', 'btns'); row.append(go); main.append(inp, row);
if (searchRes === 'none') main.append(el('div', 'empty', 'خويك فاتح اللعبة
else if (searchRes === 'me') main.append(el('div', 'empty', 'هذا اسمك أنت 🤖
else if (searchRes) {
  const o = net.online(searchRes.uid) || searchRes;
  const act = net.isFriend(o.uid) ? [btn('ادعُه', () => { onTab = 'friends';
  main.append(orrow(o, act, o.sess ? 'متصل' : 'يلعب الحين')));
}
} else if (onTab === 'inv') {
  const inv = net.incomingInvites(), jn = net.joinable();
  if (!inv.length && !jn.length) main.append(el('div', 'empty', 'معه على طول
  for (const i of inv) main.append(orrow(i.from, [btn('ادخل معه', () => { net.
  for (const o of jn) if (!inv.some(i => i.from.uid === o.uid)) main.append(o
} else if (onTab === 'friends') {
  for (const f of net.allFriends()) {
    const on = net.online(f.uid);
    const acts = [];
    if (on) acts.push(btn('ادعُه', () => { pickFor = pickFor === f.uid ? null
    if (on && on.sess && on.sess !== net.me.sess && !f.bot) acts.push(btn('عه
    if (!on) acts.push(el('span', 'sub', 'مو متصل'));
    if (!f.bot) acts.push(btn('حذف', () => net.removeFriend(f.uid), true));
    main.append(orrow(f, acts, f.bot ? 'يلحقك وين ما رحت ويصدم معك' :
    if (pickFor === f.uid) main.append(mapPicker((m, wx, ch) => { net.invite(
  }
  if (net.friends.length === 0) main.append(el('div', 'empty', '« واكتب اسمه
} else {
  const rq = net.incomingRequests();
  if (!rq.length) main.append(el('div', 'empty', 'ما فيه طلبات صداقة.'));
  for (const o of rq) main.append(orrow(o, [btn('قبول', () => net.accept(o)),
}
});
net.onChange = () => { const ae = document.activeElement; if (ae && ae.tagName
const openOnline = (tab) => { onFrom = screen === 'play' ? 'pause' : 'menu'; if
$('onClose').innerHTML = ICON.back;
$('onClose').addEventListener('click', () => { show('online', false); if (onFro
document.querySelectorAll('#onNav button').forEach(b => b.addEventListener('cli
$('onlineBtn').addEventListener('click', () => openOnline());
$('toOnlineBtn').addEventListener('click', () => openOnline());
const rename = () => { const n = prompt('يشوفه أصدقاؤك):', net.name)
$('myName').addEventListener('click', rename); $('onRename').addEventListener('
refreshBadges(); net.init().then(refreshBadges);

```

```

// the page must never zoom or slide (iOS ignores user-scalable=no for pinch un
for (const ev of ['gesturestart', 'gesturechange', 'gestureend']) document.addE
document.addEventListener('dblclick', (e) => e.preventDefault(), { passive: fal
window.addEventListener('scroll', () => { if (window.scrollX || window.scrollY)
document.addEventListener('focusout', () => setTimeout(() => window.scrollTo(0,

$('setClose').addEventListener('click', closeSettings);
// full screen (where the browser allows it inside the page) so a small browser
{ const fb = $('fsBtn'), de = document.documentElement, can = document.fullscre
  if (fb && can) { fb.hidden = false; fb.innerHTML = '<svg viewBox="0 0 24 24"
    fb.addEventListener('click', () => { try { if (document.fullscreenElement |

const seg = (items, cur, onPick) => {
  const d = document.createElement('div'); d.className = 'seg';
  for (const [v, label] of items) { const b = document.createElement('button');
  return d;
};

const opt = (title, valText, body) => {
  const o = document.createElement('div'); o.className = 'opt';
  o.innerHTML = `<div class="row"><h3>${title}</h3>${valText !== null ? `<b>${v
};

const slider = (min, max, step, val, onInput, fmt, valEl) => {
  const r = document.createElement('input'); r.type = 'range'; r.min = min; r.m
  const paint = () => r.style.setProperty('--p', ((r.value - min) / (max - min)
  r.addEventListener('input', () => { paint(); onInput(+r.value); if (valEl())
  return r;
};

const renderSettings = () => {
  document.querySelectorAll('#setNav button').forEach(b => b.classList.toggle('
  const m = $('setMain'); m.innerHTML = '';
  if (setCat === 'gfx') {
    m.append(opt('جودة الجرافيكس', null, seg([[ 'high', 'عالي'], [ 'mid', 'متوسط'
    const rs = opt('دقة الشاشة', Math.round(resScale * 100).toLocaleString('ar-
    m.append(rs);
    { const box = el('div'); box.style.cssText = 'display:flex;flex-direction:c
    m.append(opt('الدواسة', null, seg([[ 'full', 'كامل'], [ 'analog', 'بكيفي']]),
    m.append(opt('الجو', null, seg(WEATHERS, game.weather || (game.night ? 'nig
    const n = document.createElement('div'); n.className = 'note'; n.textConten
  } else if (setCat === 'drive') {
    m.append(opt('طريقة التوجيه', null, seg([[ 'wheel', 'دركسون'], [ 'arrows', 'ر
    const lb = document.createElement('button'); lb.className = 'btn primary wi
    const n = document.createElement('div'); n.className = 'note'; n.textConten
  } else if (setCat === 'steer') {
    const s1 = opt('حساسية الدركسون', steerCfg.sens.toLocaleString('ar-SA'), s1
    const s2 = opt('رجعة الدركسون للنص', null, seg([[3.5, 'بطيئة'], [7, 'عادية'
    const s3 = opt('الاستجابة', null, seg([[1, 'مباشرة'], [1.6, 'ناعمة بالنص'],

```

```

const s4 = opt('الدركسون', null, seg([[0.8, 'مغير'], [وسط, 1]], [كبير'
m.append(s1, s2, s3, s4);
const n = document.createElement('div'); n.className = 'note'; n.textContent
} else {
const grid = document.createElement('div'); grid.className = 'camgrid';
const presets = [[0, 'قريبة', 'near', 'السيارة'], [بعيدة, 1, 'far', '
if (game.camCustom) presets.push([4, 'كاميرتك', 'custom', nf(game.camCustom
for (const [v, label, art, sub] of presets) {
const b = document.createElement('button'); b.innerHTML = CAM_ART[art] +
b.classList.toggle('on', game.camMode === v); b.addEventListener('click',
}
m.append(opt('النوع الكاميرا', null, grid));
const eb = document.createElement('button'); eb.className = 'btn primary wi
m.append(eb);
if (savedCams.length) { const ab = document.createElement('button'); ab.cla
}
};

// ----- button layout editor -----
let layout = load('hajwala.layout', {}); if (typeof layout !== 'object' || !lay
const layEls = () => [...document.querySelectorAll('#hud [data-lay]')];
const applyLayout = () => {
for (const el of layEls()) {
const p = layout[el.dataset.lay];
if (p) { el.style.left = (p.x * window.innerWidth).toFixed(0) + 'px'; el.st
else { el.style.left = el.style.top = el.style.right = el.style.bottom = ''
}
}
};
applyLayout(); window.addEventListener('resize', applyLayout);
let layBack = null;
function openLayout() {
layBack = { hud: $('hud').classList.contains('on'), menu: !$('menu').hidden,
show('settings', false); show('menu', false);
$('hud').className = 'layer on mode-car layout';
input.layoutMode = window.__layoutMode = true; show('layoutBar', true);
}
const closeLayout = () => {
input.layoutMode = window.__layoutMode = false; show('layoutBar', false);
$('hud').className = layBack.cls; if (!layBack.hud) $('hud').classList.remove
show('settings', true); renderSettings();
};
{ let drag = null;
$('hud').addEventListener('pointerdown', (e) => {
if (!input.layoutMode) return; const el = e.target.closest('[data-lay]'); i
const r = el.getBoundingClientRect(); drag = { el, id: e.pointerId, dx: e.c
try { $('hud').setPointerCapture(e.pointerId); } catch (x) { } e.preventDefault

```

```

    }, true);
    $('hud').addEventListener('pointermove', (e) => {
        if (!drag || e.pointerId !== drag.id) return; const el = drag.el; const r =
        const x = clamp(e.clientX - drag.dx, 0, window.innerWidth - r.width), y = c
        layout[el.dataset.lay] = { x: x / window.innerWidth, y: y / window.innerHei
    });
    $('hud').addEventListener('pointerup', () => { drag = null; });
}
$('laySave').addEventListener('click', () => { save('hajwala.layout', layout);
$('layReset').addEventListener('click', () => { layout = {}; save('hajwala.layo

// ----- custom camera editor -----
const ce = { open: false, cur: { dist: 6.3, h: 2.2 }, start: null, old: null, c
const ceSync = () => {
    $('ceDist').value = ce.cur.dist; $('ceH').value = ce.cur.h;
    for (const id of ['ceDist', 'ceH']) { const r = $(id); r.style.setProperty('-
    const show2 = ce.comparing ? (ce.old.mode === 4 || ce.old.mode <= 1 ? game.ch
    $('ceVal').innerHTML = (show2 ? nf(show2.dist) : '-') + '<small> p</small>';
};
function openEditor() {
    ce.old = { mode: game.camMode };
    const base = game.camMode === 4 || game.camMode <= 1 ? game.chaseParams(game.
    ce.cur = { dist: base.dist, h: base.h }; ce.start = { ...ce.cur };
    ce.open = true; ce.comparing = false; ce.hudWas = $('hud').classList.contains
    show('settings', false); show('menu', false); show('garage', false); $('hud')
}
const closeEditor = () => {
    ce.open = false; show('camEdit', false); show('camList', false);
    if (ce.hudWas) $('hud').classList.add('on'); else if (screen === 'menu') show
    show('settings', true); renderSettings();
};
$('ceClose').innerHTML = ICON.close; $('ceClose').addEventListener('click', clo
$('ceDist').addEventListener('input', () => { ce.cur.dist = +$('ceDist').value;
$('ceH').addEventListener('input', () => { ce.cur.h = +$('ceH').value; ceSync()
const stepD = (d) => { ce.cur.dist = clamp(Math.round((ce.cur.dist + d) * 10) /
$('ceMinus').addEventListener('click', () => stepD(-0.5)); $('cePlus').addEvent
$('ceSave').addEventListener('click', () => {
    const c = { dist: ce.cur.dist, h: ce.cur.h };
    if (!savedCams.some(s => Math.abs(s.dist - c.dist) < 0.05 && Math.abs(s.h - c
    game.camCustom = c; game.camMode = 4; saveCam(); closeEditor();
});
bindHold($('ceCmp'), () => { ce.comparing = true; $('camEdit').classList.add('c
$('ceRevert').addEventListener('click', () => { ce.cur = { ...ce.start }; ceSyn
const openCamList = (inEditor) => {
    const box = $('camItems'); box.innerHTML = '';
    if (!savedCams.length) box.innerHTML = '<div class="note" style="text-align:c

```

```

savedCams.forEach((c, i) => {
  const it = document.createElement('div'); it.className = 'camItem';
  it.innerHTML = `<div class="ت">مفاس ${ (i + 1).toLocaleString('ar-SA')}<small
const use = document.createElement('button'); use.className = 'use'; use.te
use.addEventListener('click', () => {
  if (ce.open) { ce.cur = { dist: c.dist, h: c.h }; ceSync(); closeCamList(
  else { game.camCustom = { dist: c.dist, h: c.h }; game.camMode = 4; saveC
});
const del = document.createElement('button'); del.textContent = 'احذف';
del.addEventListener('click', () => { savedCams.splice(i, 1); save('hajwala
it.append(use, del); box.append(it);
});
show('camList', true);
};
$('ceAll').addEventListener('click', () => { show('camEdit', false); openCamLis
const closeCamList = () => { show('camList', false); if (ce.open) show('camEdit
$('camListClose').addEventListener('click', closeCamList);

document.addEventListener('gesturestart', e => e.preventDefault());
document.addEventListener('contextmenu', e => e.preventDefault());

$('loading').hidden = true; if (shotTimer) clearInterval(shotTimer);
goMenu();

// ----- main loop -----
let last = performance.now(), acc = 0; const STEP = 1 / 120;
let fpsT = 0, fpsN = 0, hudKmh = 0;
const hudEl = $('hud');
const manual = /manual/.test(location.search);
const frame = (now, fixedDt, noRender) => {
  if (!manual) requestAnimationFrame(frame);
  let dt = fixedDt || Math.min(0.05, (now - last) / 1000); last = now;
  input.update(dt);
  steering.tick(dt); arrows.tick(dt); if (input.keyActive && !input.steerHeld)
  if (input.take('pause') && screen === 'play' && !ce.open && $('settings').hid
  if (ce.open) {
    game.editorCamera(ce.comparing ? (ce.old.mode === 4 ? { mode: 4, ...game.ch
  } else if (screen === 'play' && !game.paused) {
    acc += dt; let n = 0;
    while (acc >= STEP && n < 8) { game.fixedStep(STEP); acc -= STEP; n++; }
    if (n >= 8) acc = 0;
    game.update(dt);
    // HUD
    const car = game.car;
    hudEl.className = 'layer on mode-' + game.playerMode;
    const kmh = Math.abs(car.forwardSpeed) * 3.6;

```

```

hudKmh += (kmh - hudKmh) * Math.min(1, dt * 8);
$('spd').textContent = Math.round(kmh).toLocaleString('ar-SA');
$('needle').setAttribute('transform', `rotate(${(-135 + clamp(hudKmh, 0, 22
$('gear').textContent = game.playerMode !== 'car' ? 'P' : car.reverse ? 'R'
{ const f = clamp(car.rpm / car.def.redline, 0, 1); const a0 = -225 * Math.
  $('rpmArc').setAttribute('d', f < 0.01 ? '' : `M${100 + Math.cos(a0) * R}
$('dmgBar').setAttribute('width', (car.damage * 0.6).toFixed(0));
$('score').textContent = game.score.toLocaleString('ar-SA');
if (game.camMode !== camCfg.mode) saveCam();
} else if (screen !== 'play') {
  game.update(dt);
}
if (!noRender) renderer.render(game.roots(), game.cam);
if (game.snap && !noRender) { game.snap = false; // photo from the drone: sha
  try { renderer.gl.canvas.toBlob(async (b) => { if (!b) return; const f = ne
    try { if (navigator.canShare && navigator.canShare({ files: [f] })) { awa
      const a = document.createElement('a'); a.href = URL.createObjectURL(b); a
// adaptive resolution on slow devices
fpsT += dt; fpsN++;
if (fpsT > 3) {
  const fps = fpsN / fpsT; fpsT = 0; fpsN = 0;
  if (!manual && fps < 24 && renderer.pixelRatio > 0.8) renderer.pixelRatio =
    window.__fps = fps;
}
};
if (!manual) requestAnimationFrame(frame);
window.__tick = (n = 1, dt = 1 / 30) => { for (let i = 0; i < n; i++) frame(per

// ===== challenges with a friend =====
const hashStr = (t) => { let h = 2166136261; for (const c of String(t)) h = Mat
const setupChal = (ch, sess) => {
  game.chal = null; $('chal').hidden = true; if (!ch || ch === 'none') return;
  const sp = game.map.spawn; const C = { type: ch, t0: performance.now(), score
  if (ch === 'race' || ch === 'point') {
    const a = ch === 'race' ? sp.yaw : hashStr(sess) * Math.PI * 2, dist = ch =
    let x = sp.x + Math.sin(a) * dist, z = sp.z + Math.cos(a) * dist;
    if (game.map.roadX && ch === 'race') x = game.map.roadX(z) - 4;
    C.target = { x, z };
  }
  game.chal = C; $('chal').hidden = false;
  hud.toast(ch === 'drift' ? 'أكثر يفوز' : 'تحدي التفحيط: ٩٠ ثانية، اللي يجمع نقاط أكثر يفوز');
};
game.myTune = () => { const T = tunes[selected] || {}; return { color: T.color,
game.onChalDone = () => { hud.toast('2000 , 🏆 وصلت!'); };
setInterval(() => {
  const C = game.chal; if (!C || screen !== 'play') { if (!C) $('chal').hidden

```

```

const mates = net.others().filter(o => o.sess && o.sess === net.me.sess);
const fmt = (n) => Math.round(n).toLocaleString('ar-SA');
if (C.type === 'drift') {
  const left = Math.max(0, 90 - (performance.now() - C.t0) / 1000); if (left
  const best = mates.reduce((a, o) => Math.max(a, o.cs || 0), 0);
  $('chal').textContent = left > 0 ? `⌚ ${fmt(left)} أنت ⬇️ ${fmt(C.score)}`
} else {
  const b = game.car.body.pos, d = Math.hypot(b.x - C.target.x, b.z - C.targe
  const winner = mates.find(o => o.cw && (!net.me.cw || o.cw < net.me.cw));
  if (net.me.cw && !winner) $('chal').textContent = '🏆 وصلت أول';
  else if (winner) $('chal').textContent = '🏁 ' + (winner.name || 'خويك') +
  else $('chal').textContent = '🏎️ العمود الأصفر ⬇️ ' + fmt(d) + 'م';
}
}, 250);

// ===== tuning (التعديل) =====
const ENGINES = { datsum: [4, 6], gmcold: [8], caprice: [6, 8], hilux07: [4, 6]
const PAINTS = [['#f4f4f2', 'أبيض'], ['#أسود', '101012'], ['#b9bbbe', 'فضي'], [
const hex = (h) => [parseInt(h.slice(1, 3), 16) / 255, parseInt(h.slice(3, 5),
const tunes = load('hajwala.tune', {});
const tuneOf = (k) => (tunes[k] || (tunes[k] = {}));
const saveTune = (k) => { save('hajwala.tune', tunes); try { game.cars[k].apply
for (const k of Object.keys(CAR_DEFS)) if (tunes[k] && game.cars[k]) try { game
const sprays = {}; // car -> list of [p, n, color, size] (kept while the game i
let tTab = 'parts', paintMode = 'color', paintPart = 'all', sprayColor = PAINTS
const tpanel = $('tPanel');
const chip = (txt, on, fn, lock) => { const b = el('button', 'chip2' + (on ? '
const renderTune = () => {
  const k = selected, T = tuneOf(k), d = CAR_DEFS[k], car = game.cars[k];
  document.querySelectorAll('#tTabs button').forEach(b => b.classList.toggle('o
  tpanel.innerHTML = ''; $('sprayHint').hidden = !(tTab === 'paint' && paintMod
  tpanel.append(el('h4', '', d.name + ' ⬇️ الفلوس: ∞'));
  if (tTab === 'parts') {
    tpanel.append(el('div', 'note', 'اضغط على القطعة عشان تفكّها أو ترجّعها'));
    const { PART_NAMES } = window.__tuneMod; car && window.__tuneMod.splitParts
    const row = el('div', 'row2'); T.hide = T.hide || {};
    for (const [pk, nm] of Object.entries(PART_NAMES)) { if (!car.parts || !car
    tpanel.append(row, chip('رجع كل القطع', false, () => { T.hide = {}; saveTun
  } else if (tTab === 'engine') {
    const base = (d.engineVoice && d.engineVoice.cyl) || 4, ok = ENGINES[k] ||
    tpanel.append(el('div', 'note', 'اختار المكيّنة. الأقوى تزيد العزم والسرعة'));
    const row = el('div', 'row2');
    for (const c of [4, 6, 8]) { const lock = !ok.includes(c); row.append(chip(
    tpanel.append(row);
    tpanel.append(el('h4', '', 'تيربو'));
    const r2 = el('div', 'row2'); r2.append(chip('بدون', !T.turbo, () => { T.tu

```

```

const tq = Math.round((d.torque || 300) * ((car && car.tuneTorque) || 1));
} else if (tTab === 'paint') {
const r0 = el('div', 'row2'); r0.append(chip('🎨 لون', paintMode === 'color'
const sw = (onPick, curHex) => { const r = el('div', 'row2'); for (const [h
if (paintMode === 'color') {
tpanel.append(el('h4', '', 'وش تلون؟'));
const { PART_NAMES } = window.__tuneMod; car && window.__tuneMod.splitPar
const r1 = el('div', 'row2'); r1.append(chip('السيارة كاملة', paintPart =
for (const [pk, nm] of Object.entries(PART_NAMES)) { if (pk === 'glass' |
tpanel.append(r1, el('h4', '', 'اللون'));
const curHex = paintPart === 'all' ? T.colorHex : (T.partsHex || {})[pain
tpanel.append(sw((h) => { if (paintPart === 'all') { T.colorHex = h; T.co
tpanel.append(chip('🔄 رجّع الألوان الأصلية', false, () => { delete T.color;
} else {
tpanel.append(el('div', 'note', 'وبعدين اضغط «بخ» واسحب على السيارة'
const r3 = el('div', 'row2'); r3.append(chip(sprayCam ? '📷 الكاميرا'
tpanel.append(el('h4', '', 'لون البخاخ'), sw((h) => { sprayColor = h; ren
const sl = document.createElement('input'); sl.type = 'range'; sl.min = 0
tpanel.append(el('h4', '', 'حجم البخ'), sl, chip('🧼 امسح البخ', false, (
}
} else if (tTab === 'stance') {
tpanel.append(el('div', 'note', 'ية. الأمامي فوق والخلفي تحت = وزنبة التفحيط'
const slider2 = (label, key) => { const w = el('div'); const lab = el('h4',
tpanel.append(slider2('الكفرات الأمامية', 'hf'), slider2('الكفرات الخلفية',
const r = el('div', 'row2'); r.append(chip('قدام فوق)', false,
} else if (tTab === 'power') {
renderPower(tpanel, k);
}
};
const renderPower = (box, k) => {
const T = tuneOf(k);
box.append(el('h4', '', 'الدبل'));
const r1 = el('div', 'row2'); for (const [v, n] of [['off', 'بدون دبل (2H)'],
box.append(el('h4', '', 'الدفع'));
const r2 = el('div', 'row2'); for (const [v, n] of [['', 'الأصلي'], ['rwd', 'ي
box.append(el('h4', '', 'الدفلوك'));
const r3 = el('div', 'row2'); for (const [v, n] of [['', '(مفتوح)'], ['r
box.append(el('div', 'note', 'الدفلوك يخلّي الكفرين يدقّون مع بعض لو واحد رفع
});
let sprayCam = false;
const openTune = () => { show('menu', false); screen = 'tune'; game.showroom(se
$('tBack').innerHTML = ICON.back; $('tBack').addEventListener('click', () => {
document.querySelectorAll('#tTabs button').forEach(b => b.addEventListener('cli
$('tuneBtn').addEventListener('click', openTune);
// drag to orbit the car / spray on it
{ const cv = renderer.gl.canvas; let drag = null, spraying = false;

```

```

const ray = (e) => { const r = cv.getBoundingClientRect(); const x = (e.clie
  const un = (z) => { const X = m[0] * x + m[4] * y + m[8] * z + m[12], Y = m
    const a = un(-1), b = un(1); return [a, b.sub(a).norm()]; };
const doSpray = (e) => { const car = game.car; const [o, d] = ray(e); const h
cv.addEventListener('pointerdown', (e) => { if (screen !== 'tune' && screen !
cv.addEventListener('pointermove', (e) => { if (spraying) { doSpray(e); retur
const up = () => { drag = null; spraying = false; }; cv.addEventListener('poi
}

// ===== character (الشخصية) =====
let charSel = load('hajwala.char', 'base'); game.charSel = charSel;
const CHARS = [['base', 'الأساسية', '👤'], ['hero', 'الجديدة (أنميشن)', '🏃']];
const renderChars = () => { const st = $('charStrip'); st.innerHTML = '';
  for (const [k, n, ic] of CHARS) { const c = el('button', 'card' + (charSel ==
const openChars = async () => { show('menu', false); screen = 'chars'; show('ch
$('cBack').innerHTML = ICON.back; $('cBack').addEventListener('click', () => {
$('charBtn').addEventListener('click', openChars);
if (charSel === 'hero') ensureMapAssets('hero', () => { }).catch(() => { });

// ===== caravan (walk in from the Shas) =====
game.onRoom = async (enter) => {
  const f = $('fade'); f.style.opacity = 1; await new Promise(r => setTimeout(r
  try { if (enter) { if (!assets.caravan) { hud.toast('1500', 'نفتح الكرفان...'); a
    catch (e) { console.warn(e); hud.toast('الكرفان نفتح'); }
  await new Promise(r => setTimeout(r, 250)); f.style.opacity = 0; setTimeout((
});

// ===== map + fuel HUD =====
$('mapBtn').innerHTML = '<svg viewBox="0 0 32 32" fill="none" stroke="#ecdcbc"
$('mapClose').innerHTML = ICON.back;
const drawMap = () => {
  const cv = $('mapCanvas'), W = 900, H = 520; cv.width = W; cv.height = H; con
  const { img, proj } = game.renderMap(W, H, 320); g2.drawImage(img, 0, 0);
  const dot = (x, z, col, r, label) => { const [px, py] = proj(x, game.world.he
  if (game.workshop) dot(game.workshop.x, game.workshop.z, '#9b40f2', 12, 'ورشة'
  for (const f of game.fuelPads || []) dot(f.x, f.z, '#f5c518', 11, 'بنزين');
  if (game.chal && game.chal.target) dot(game.chal.target.x, game.chal.target.z
  for (const o of net.others()) if (o.st && o.map === game.mapKey) dot(o.st[0],
  if (net.bot) dot(net.bot.car.body.pos.x, net.bot.car.body.pos.z, '#ff8c1a', 1
  const me = game.playerMode === 'foot' ? game.player.pos : game.car.body.pos;
};
bindTap($('mapBtn'), () => { try { drawMap(); show('mapView', true); } catch (e
$('mapClose').addEventListener('click', () => show('mapView', false));
setInterval(() => { const f = game.car ? (game.car.fuel ?? 100) : 100; $('fuelF

window.__ui = { ensure: ensureMapAssets, assets, select: k => { selected = k; }

```

```
    if (manual) frame(performance.now(), 1 / 30);  
    window.ready = true;  
}  
  
boot();  
  
return {  };  
})();
```